

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRUNG TÂM NGHIÊN CỨU VÀ ĐÀO TẠO THIẾT KẾ VI MẠCH

VERILOG HDL

VÀ ỨNG DỤNG

Nguyễn Hùng Quân

Hầu Nguyên Thanh Hoàng

Nguyễn Phú Quốc

Đỗ Ngọc Quỳnh

Trần Kiên Cường

Lê Phúc

Nguyễn Duy Thái



NHÀ XUẤT BẢN ĐẠI HỌC QUỐC GIA
THÀNH PHỐ HỒ CHÍ MINH - 2014

**Đây là sản phẩm được hình thành từ
“Chương trình Phát triển Công nghiệp
Vĩ mạch Thành phố Hồ Chí Minh
giai đoạn 2013-2020”**

LỜI NÓI ĐẦU

Cuốn sách này hướng dẫn sử dụng ngôn ngữ Verilog HDL kèm theo nhiều ví dụ, theo đó người đọc có thể ứng dụng ngay vào các thiết kế cụ thể. Bên cạnh những hướng dẫn cơ bản về Verilog HDL, các ví dụ được phân tích một cách chi tiết, được minh họa bằng sơ đồ mạch và netlist sau tổng hợp, giúp người đọc thấy được mối tương quan giữa ngôn ngữ Verilog HDL và mạch số do nó tạo ra.

Verilog HDL, được viết đầy đủ là Verilog Hardware Description Language, là ngôn ngữ mô tả phần cứng thông dụng trên thế giới hiện nay. Ngôn ngữ này được sử dụng để mô tả hành vi (tuần tự hoặc đồng thời) hay cấu trúc của một mô hình mạch số. Mô hình này hoặc có thể tổng hợp được thành mạch số, hoặc chỉ có thể được sử dụng cho việc mô phỏng (không tổng hợp được). Cuốn sách này không trình bày mọi vấn đề về ngôn ngữ Verilog HDL, mà chỉ tập trung vào phần ngôn ngữ tổng hợp được, những phần liên quan đến mô phỏng sẽ được lược bỏ hoặc được làm đơn giản hóa.

Trong cuốn sách này, các ví dụ và ứng dụng mẫu được thiết kế đa dạng từ cơ bản đến nâng cao. Những ví dụ và ứng dụng mẫu này sẽ được phân tích ở nhiều khía cạnh khác nhau để người đọc hiểu được bằng cách nào mà ngôn ngữ Verilog HDL có thể chuyển hóa những đoạn code thuần túy thành các mạch số. Từ đó, người đọc có thể tự nghiệm ra cách để thể hiện mạch số từ ý tưởng thông qua ngôn ngữ Verilog HDL. Ngoài ra, cuốn sách cũng phân tích những trường hợp có thể gây ra sự sai biệt về chức năng giữa code mô tả và mạch số thực tế được tạo ra, đồng thời đề ra phương pháp phòng tránh.

Netlist sau tổng hợp được trình bày trong sách được thực hiện bằng phần mềm tổng hợp Design Compiler. Dạng sóng mô phỏng được thực hiện bằng phần mềm kiểm tra mô phỏng VCS. Đây là những công cụ thiết kế vi mạch chuyên nghiệp, là lựa chọn hàng đầu của những công ty, tổ chức hoạt động trong lĩnh vực này. Bên cạnh hai

phần mềm kể trên còn có nhiều phần mềm tổng hợp và mô phỏng mạch khác, trong đó một số phần mềm có cung cấp phiên bản miễn phí phục vụ cho việc học tập như Quatus II của hãng Altera, ModelSim của hãng Mentor Graphics, Xilinx ISE của hãng Xilinx...

Cuốn sách này được thực hiện nhằm mục đích phục vụ cho các sinh viên ngành vi mạch, ngành điện tử, ngành công nghệ thông tin, những người mới biết đến ngôn ngữ mô tả phần cứng, các bạn sẽ dễ dàng hiểu được những kiến thức cơ bản về ngôn ngữ Verilog HDL và sẽ nhanh chóng áp dụng được những kiến thức này thông qua các ví dụ cụ thể. Những kỹ sư thiết kế vi mạch cũng có thể tìm thấy ở đây nhiều kiến thức bổ ích, thực tế, những kinh nghiệm đã được nhóm tác giả tích lũy trong quá trình làm việc.

Nội dung chính được trình bày trong năm chương. Chương 1 giới thiệu tổng quan về Verilog HDL. Chương 2 trình bày các khái niệm cơ bản như cấu trúc module, loại dữ liệu, mảng, biểu thức,... Nội dung quan trọng nhất của cuốn sách nằm ở chương 3. Chương 3 mô tả và phân tích các loại phát biểu của ngôn ngữ Verilog HDL như: phát biểu gán, phát biểu always, phát biểu if/else, phát biểu case, định nghĩa và gọi function, phát biểu generate,... Kèm theo đó, các ví dụ được phân tích kỹ lưỡng và được minh họa bằng sơ đồ mạch, netlist sau tổng hợp. Chương 4, chương 5 thể hiện các mô hình và ứng dụng mẫu có tính ứng dụng trong thực tế.

Chúng tôi muốn bày tỏ lòng biết ơn sâu sắc tới ông Ngô Đức Hoàng, Giám đốc Trung tâm Nghiên cứu và Đào tạo Thiết kế Vi mạch, người đã khuyến khích, động viên và tạo điều kiện để chúng tôi thực hiện cuốn sách này. Xin chân thành cảm ơn gia đình, bạn bè, quý đồng nghiệp đã hết lòng giúp đỡ, động viên chúng tôi trong suốt thời gian qua.

Hy vọng cuốn sách này sẽ mang đến cho quý bạn đọc những kiến thức bổ ích. Chúng tôi rất mong nhận được nhiều ý kiến đóng góp giúp cuốn sách ngày càng tốt hơn. Mọi phản hồi xin gửi về một trong những địa chỉ email sau:

quan.nguyenhung@icdrec.edu.vn

hoang.haunguyenthanh@icdrec.edu.vn

quoc.nguyenphu@icdrec.edu.vn

quynh.dongoc@icdrec.edu.vn

cuong.trankien@icdrec.edu.vn

phuc.le@icdrec.edu.vn

thai.nguyenduy@icdrec.edu.vn

Nhóm tác giả

Mục Lục

LỜI NÓI ĐẦU	iii
Mục Lục	vi
THUẬT NGỮ VÀ TỪ VIẾT TẮT	x
1 TỔNG QUAN	1
1.1 Mở đầu	2
1.2 Ngôn ngữ mô tả phần cứng	2
1.2.1 Định nghĩa	2
1.2.2 Lịch sử ra đời và phát triển của Verilog HDL	3
1.2.3 Xu hướng sử dụng Verilog HDL trên thế giới	3
2 KHÁI NIỆM CƠ BẢN	5
2.1 Kiểu dữ liệu	6
2.1.1 Các giá trị có thể sử dụng	6
2.1.2 Các kiểu dữ liệu	6
2.2 Scalar và vector	8
2.3 Mảng	9
2.3.1 Định nghĩa	9
2.3.2 Phân loại	9
2.4 Biểu thức	13
2.4.1 Toán hạng	13
2.4.2 Toán tử	20
2.4.3 Độ ưu tiên của các toán tử	36
2.5 Các loại phép gán	36
2.5.1 Phép gán thủ tục	36
2.5.2 Phép gán liên tục	37
2.6 Mạch tổ hợp	37
2.7 Mạch tuần tự	39

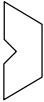
2.7.1	Khái niệm	39
2.7.2	Latch và Flip-Flop	41
3	VERILOG HDL KHẢ TỔNG HỢP	45
3.1	Cấu trúc always	46
3.1.1	Cú pháp always	46
3.1.2	Phép gán trong một khối always	47
3.1.3	Cách sử dụng always	50
3.1.4	Reset trong mạch tuần tự	55
3.2	Phép gán assign	59
3.2.1	Cú pháp assign	59
3.2.2	Cách sử dụng assign	60
3.3	Phát biểu if	64
3.3.1	Cú pháp phát biểu if	64
3.3.2	Cách sử dụng phát biểu if	65
3.4	Phát biểu case	73
3.4.1	Cú pháp phát biểu case	73
3.4.2	Cách sử dụng phát biểu case	76
3.4.3	Những lưu ý khi sử dụng phát biểu case	89
3.5	Function	93
3.5.1	Giới thiệu	93
3.5.2	Khai báo function	94
3.5.3	Gọi function	96
3.6	Các tham số và chỉ dẫn tổng hợp	98
3.6.1	Parameter	98
3.6.2	Localparam	107
3.6.3	Define	108
3.7	Phát biểu generate	113
3.7.1	Cú pháp phát biểu generate	113
3.7.2	Cách sử dụng phát biểu generate	114
3.7.3	Lưu ý khi sử dụng phát biểu generate	122
3.8	Module	127
3.8.1	Định nghĩa module	127

3.8.2	Cú pháp module	128
3.8.3	Cách sử dụng module	128
3.8.4	Gọi module	134
4	PHƯƠNG PHÁP MÔ TẢ MẠCH	150
4.1	Mạch tổ hợp	151
4.1.1	Phương pháp mô tả mạch tổ hợp	151
4.1.2	Lưu ý khi mô tả mạch tổ hợp	160
4.2	Mạch tuần tự	168
4.2.1	Phương pháp mô tả mạch tuần tự	168
4.2.2	Lưu ý khi mô tả mạch tuần tự	171
4.3	Máy trạng thái	182
4.3.1	Mô hình máy trạng thái	182
4.3.2	Phương pháp mô tả máy trạng thái	184
4.3.3	Mã hóa trạng thái	196
5	ỨNG DỤNG VERILOG HDL	199
5.1	Mạch chuyển song song sang nối tiếp	200
5.1.1	Đặt vấn đề	200
5.1.2	Phân tích vấn đề	200
5.1.3	Mô tả RTL code	202
5.2	Mạch chuyển nối tiếp sang song song	202
5.2.1	Đặt vấn đề	202
5.2.2	Phân tích vấn đề	203
5.2.3	Mô tả RTL code	205
5.3	Giải mã hex sang mã LED bảy đoạn	206
5.3.1	Đặt vấn đề	206
5.3.2	Phân tích vấn đề	207
5.3.3	Mô tả RTL code	208
5.4	Mạch so sánh và tìm giá trị lớn nhất	209
5.4.1	Đặt vấn đề	209
5.4.2	Phân tích vấn đề	209
5.4.3	Mô tả RTL code	210
5.5	Mạch chia tần số	211

5.5.1	Đặt vấn đề	211
5.5.2	Phân tích và thiết kế	212
5.5.3	Mô tả RTL code	213
5.6	Mạch điều khiển đèn giao thông	213
5.6.1	Đặt vấn đề	213
5.6.2	Phân tích vấn đề	215
5.6.3	Mô tả RTL code	216
5.7	MUX one-hot tổng quát	218
5.7.1	Đặt vấn đề	218
5.7.2	Phân tích vấn đề	218
5.7.3	Mô tả RTL code	222
5.8	FIFO đồng bộ	223
5.8.1	Đặt vấn đề	223
5.8.2	Phân tích vấn đề	224
5.8.3	Mô tả RTL code	227
5.9	LIFO đồng bộ	229
5.9.1	Đặt vấn đề	229
5.9.2	Phân tích vấn đề	231
5.9.3	Mô tả RTL code	232
5.10	Truy xuất bộ nhớ NVM	234
5.10.1	Đặt vấn đề	234
5.10.2	Phân tích vấn đề	236
5.10.3	Mô tả RTL code	239
Tài liệu tham khảo		241

THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Chú thích
Bus	Là một tín hiệu có nhiều hơn một bit
Don't care	Được sử dụng với ý nghĩa "không quan tâm" đến giá trị đang mô tả là 0, 1, x hay z
Khả tổng hợp	Trình tổng hợp có thể hiểu và biên dịch RTL code thành mạch số tương ứng
Logic cell	Thành phần logic của thư viện tổng hợp (cổng logic, Flip-Flop, AOI, ...)
Netlist sau tổng hợp	Sơ đồ kết nối các logic cell sau khi tổng hợp RTL code
Reset	Khởi tạo giá trị ban đầu hoặc thiết lập lại giá trị cho Flip-Flop, thanh ghi
RTL code	Code viết bằng ngôn ngữ mô tả phần cứng
Scalar	Tín hiệu kiểu net hoặc reg có độ rộng một bit
Sensitivity list	Danh sách độ nhạy sử dụng cùng với khối always
Thanh ghi	Một hoặc một nhóm Flip-Flop
Thư viện tổng hợp	Thư viện chứa các logic cell được sử dụng để tạo ra mạch số
Tích các tổng	Phương trình mà vế trái có dạng tích (AND) các tổ hợp tín hiệu làm cho giá trị vế trái bằng 0, viết tắt là POS. Mỗi tổ hợp tín hiệu là tổng (OR) của các tín hiệu thành phần.
Tín hiệu	Danh từ chung chỉ các đường kết nối trong mạch số. Tín hiệu có thể là ngõ vào, ngõ ra, hai chiều hoặc kết nối nội bộ trong mạch. Tín hiệu có thể có độ rộng một bit, nhiều bit. Tín hiệu có thể bắt nguồn từ thanh ghi hoặc mạch tổ hợp.
Tổng các tích	Phương trình mà vế trái có dạng tổng (OR) các tổ hợp tín hiệu làm cho giá trị vế trái bằng 1, viết tắt là SOP. Mỗi tổ hợp tín hiệu là tích (AND) của các tín hiệu thành phần.
Trình tổng hợp	Phần mềm thực thi biên dịch RTL code thành netlist sau tổng hợp dựa trên thư viện tổng hợp
Vector	Tín hiệu kiểu net hoặc reg có độ rộng nhiều hơn một bit
Xung clock	Tín hiệu xung nhịp đồng bộ của Flip-Flop

Ký hiệu hoặc viết tắt	Chú thích
ASIC	Application Specific Integrated Circuit
DEMUX	De-multiplexer
FIFO	First In First Out
FSM	Finite State Machine
HDL	Hardware Description Language
LHS	Left Hand Side
LIFO	Last In First Out
MUX	Multiplexer
NVM	Non-Volatile Memory
POS	Product Of Sums
RHS	Righ Hand Side
RTL	Register Transfer Level
SOP	Sum Of Products
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit
	Flip-Flop hoặc thanh ghi
	Bộ cộng hoặc trừ
	Bộ so sánh bằng, lớn hơn, nhỏ hơn, lớn hơn hoặc bằng,... tùy vào ký hiệu tương ứng bên trong
	Bộ dịch

1

TỔNG QUAN

Nội dung chương này giới thiệu sơ lược về ngôn ngữ mô tả phần cứng cùng với lịch sử ra đời và phát triển của Verilog HDL. Đồng thời, chương này cũng trình bày xu hướng sử dụng của hai ngôn ngữ mô tả phần cứng phổ biến hiện nay là Verilog HDL và VHDL.

1.1 Mở đầu

Trước đây, thiết kế số sử dụng phương pháp thủ công là nhập sơ đồ mạch điện. Do phương pháp cũ này có nhiều bất lợi nên hiện nay đã và đang được nhanh chóng thay thế bằng các phương pháp mới là sử dụng các ngôn ngữ mô tả phần cứng HDL cùng với phần mềm tổng hợp.

Một số thuận lợi của ngôn ngữ mô tả phần cứng là:

- Rút ngắn thời gian thiết kế và giảm thời gian đưa sản phẩm ra thị trường
- Cho phép tái sử dụng thiết kế
- Dễ dàng thay đổi và chỉnh sửa thiết kế
- Dễ dàng thử nghiệm các kiến trúc và công nghệ khác nhau
- Cho phép sử dụng phần mềm tổng hợp để kiểm tra diện tích và định thời của mạch
- Dễ quản lý những mạch lớn và phức tạp
- Dễ dàng kiểm tra thiết kế hơn

Hai ngôn ngữ mô tả phần cứng đã được IEEE chuẩn hóa là VHDL và Verilog HDL.

1.2 Ngôn ngữ mô tả phần cứng

1.2.1 Định nghĩa

Ngôn ngữ mô tả phần cứng là ngôn ngữ thuộc lớp ngôn ngữ máy tính, dùng để miêu tả cấu trúc và hoạt động, hành vi cũng như mô phỏng và kiểm tra hoạt động của vi mạch.

Ngôn ngữ máy tính được mô hình hóa theo một quá trình tuần tự để phát triển một giải thuật theo từng bước. Trong khi đó, mạch số cần có những đặc tính như sự kết nối giữa các phần, thực thi đồng thời và khái niệm định thời và trễ truyền. Những đặc tính này không thể diễn tả bởi ngôn ngữ lập trình truyền thống. Từ đó, thực tế đặt ra yêu cầu cần phải có một ngôn ngữ mới - ngôn ngữ mô tả phần cứng.

1.2.2 Lịch sử ra đời và phát triển của Verilog HDL

Năm 1981, Phil Moorby, nhân viên của công ty Gateway đề xuất ngôn ngữ mô tả phần cứng GenRad và trình mô phỏng HILO.

Năm 1983, Gateway phát hành ngôn ngữ mô tả phần cứng Verilog HDL cùng với trình mô phỏng Verilog.

Năm 1985, trình mô phỏng Verilog XL được ra đời.

Năm 1990, sau khi mua lại Gateway, Cadence giới thiệu Verilog HDL ra cộng đồng.

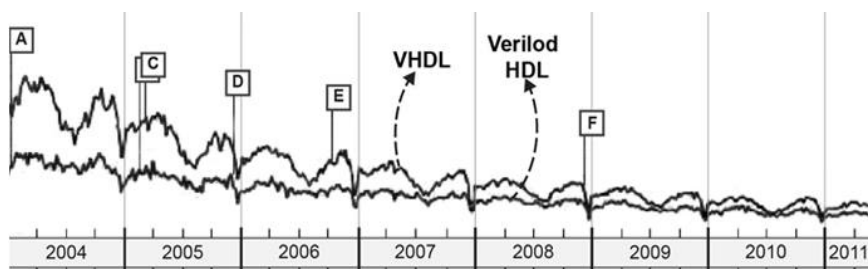
Năm 1995, Verilog HDL được xem xét lại và được công nhận bởi IEEE, trở thành chuẩn IEEE Std 1364-1995 và thường được gọi Verilog 1995.

Năm 2001, Verilog HDL được công bố chỉnh sửa lần thứ nhất bởi IEEE trở thành chuẩn IEEE Std 1364-2001 và thường được gọi Verilog 2001.

Năm 2006, Verilog được công bố chỉnh sửa lần thứ hai bởi IEEE trở thành chuẩn IEEE Std 1364-2005 và thường được gọi Verilog 2005.

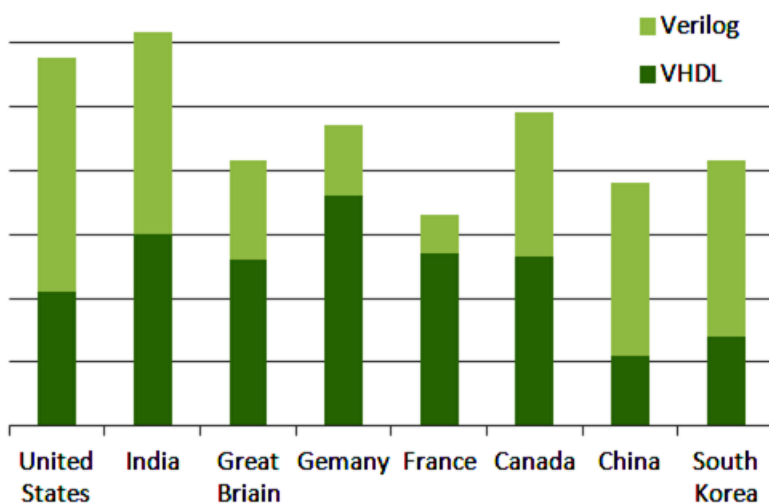
1.2.3 Xu hướng sử dụng Verilog HDL trên thế giới

Từ năm 2004 đến nay thời gian, số lượng tìm kiếm VHDL nhiều hơn so với Verilog HDL. Từ năm 2004 đến 2007, VHDL được tìm kiếm nhiều đáng kể hơn so với Verilog HDL. Tuy nhiên, từ năm 2007 đến 2011, số lượng tìm kiếm VHDL giảm đi dù vẫn hơn Verilog.



Hình 1-1. Mức độ tìm kiếm Verilog HDL và VHDL theo Google Trend

Trong khi đó, số lượng tìm kiếm được tạo ra bởi những người trên khắp thế giới cho cả hai Verilog HDL và VHDL là gần tương đương trong khoảng thời gian từ năm 2010 đến thời điểm hiện tại.



Hình 1-2. Mức độ tìm kiếm VHDL và Verilog HDL ở các quốc gia từ 7/2013 đến 7/2014

Hình trên cho thấy, Ấn Độ và Mỹ có lượng truy vấn tìm kiếm lớn nhất. Ở Ấn Độ, lượng truy vấn tìm kiếm VHDL và Verilog HDL là tương đối ngang nhau. Ở Mỹ, Trung Quốc và Hàn Quốc, Verilog HDL phổ biến hơn VHDL. Tuy nhiên, ở Đức và Pháp thì VHDL lại phổ biến hơn Verilog HDL.

Nhìn chung, VHDL và Verilog HDL là những ngôn ngữ có khả năng tương đương nhau. Việc chọn lựa ngôn ngữ nào là phụ thuộc lựa chọn, và quyết định của người dùng.

2

KHÁI NIỆM CƠ BẢN

Nội dung chương này cung cấp những khái niệm cơ bản nhất cho bạn đọc mới bắt đầu tiếp xúc với Verilog HDL như các kiểu dữ liệu, khai báo tín hiệu, scalar, vector, mảng, biểu thức, toán hạng và các loại phép gán. Đồng thời, khái niệm về hai loại mạch cơ bản là mạch tổ hợp và mạch tuần tự cũng được trình bày trong chương này

2.1 Kiểu dữ liệu

2.1.1 Các giá trị có thể sử dụng

Verilog HDL có các giá trị dữ liệu 0, 1, x, z và bên dưới là mô tả các giá trị này.

- 0 – biểu diễn mức logic 0 hay điều kiện là *false*.
- 1 – biểu diễn mức logic 1 hay điều kiện là *true*.
- x – biểu diễn mức logic chưa xác định.
- z – biểu diễn tổng trở cao.

2.1.2 Các kiểu dữ liệu

2.1.2.1 Kiểu dữ liệu net

Cú pháp:

```
net_type <signed> <range> net_name_1, <net_name_2>, ...;
```

Mô tả:

- *net_type*: khai báo bằng một trong các từ khóa **supply0**, **supply1**, **tri**, **triand**, **trior**, **tri0**, **tri1**, **wire**, **wand**, **wor**, **triereg**.
- *signed*: chỉ ra tín hiệu có dấu hay không. Mặc định là không dấu.
- *range*: độ rộng bit của tín hiệu.
- *net_name_1*, *net_name_2*, ...: tên các tín hiệu **net**.

Kiểu dữ liệu **net** được dùng để mô hình hóa kết nối vật lý như kết nối tín hiệu, cổng, module,... Kiểu dữ liệu này có giá trị của tín hiệu lái. Nếu một **net** không có tín hiệu lái thì chúng sẽ có giá trị z. Ngoài ra, chúng không lưu trữ dữ liệu ngoài trừ khai báo **triereg**. Khai báo **triereg** là khai báo không tổng hợp được và chỉ dùng cho mô phỏng.

Các khai báo có thể tổng hợp được là: **supply0**, **supply1**, **tri**, **triand**, **trior**, **tri0**, **tri1**, **wire**, **wand** và **wor**. Trong đó, khai báo **wire** thường được dùng phổ biến.

Ví dụ 2-1

```
wire [7:0] y;  
tri tristate_buffer;  
wand sig_1;
```

2.1.2.2 Kiểu dữ liệu *variable*

Cú pháp:

```
variable_type <signed> <range> var_name_1,<var_name_2>  
,...;
```

Mô tả:

- *variable_type*: khai báo một trong từ các khóa sau **reg**, **integer**, **time**, **real**, **realtime**.
- *range*: độ rộng bit của tín hiệu.
- *var_name_1*, *var_name_2*,...: tên các tín hiệu có kiểu dữ liệu **variable**.

Kiểu dữ liệu **variable** là kiểu dữ liệu lưu trữ giá trị đã gán cho đến khi được gán một giá trị mới. Tuy nhiên, phụ thuộc vào đoạn RTL code, kiểu dữ liệu **variable** có thể không tạo ra thành phần nhớ. Kiểu dữ liệu **variable** hỗ trợ biến có dấu và biến không dấu. Trong đó, kiểu dữ liệu có thể tổng hợp được là: **reg** và **integer** còn kiểu dữ liệu không thể tổng hợp được là: **real** và **realtime**. Từ đây trở đi, nội dung sách chỉ tập trung trình bày những khai báo khả tổng hợp của kiểu dữ liệu **variable** là **reg** và **integer**.

Ví dụ 2-2

```
reg [15:0] a, b;  
integer c;
```

Tín hiệu *sig_a* và *sig_b* được khai báo với độ rộng 16 bit. Tín hiệu *sig_c* có được khai báo với từ khóa là **integer** nên độ rộng ngầm hiểu là 32 bit.

2.2 Scalar và vector

Scalar là tín hiệu có độ rộng một bit có kiểu dữ liệu là **net** hoặc có khai báo là **reg**. Vector là tín hiệu có độ rộng nhiều hơn một bit có kiểu dữ liệu là **net** hoặc kiểu dữ liệu là **variable** có khai báo là **reg**.

Cú pháp khai báo scalar:

```
type signal_name_1, <signal_name_2>,...;
```

Cú pháp khai báo vector:

```
type <signed> [index_1:index_2] signal_name_1,  
<signal_name_2>,...;
```

Mô tả:

- *type*: khai báo kiểu dữ liệu của tín hiệu.
- *signed*: chỉ ra tín hiệu có dấu hay không. Mặc định là không dấu nếu không khai báo.
- *index_1*: là hằng số bên trái của dãy địa chỉ.
- *index_2*: là hằng số bên phải của dãy địa chỉ. *index_1* và *index_2* có thể nhận giá trị dương, âm và 0. Giá trị *index_2* có thể lớn hơn, bằng hoặc nhỏ hơn *index_1*.
- *signal_name_1*, *signal_name_2*: khai báo tên tín hiệu.

Ví dụ 2-3

Khai báo scalar

```
reg a; // a scalar reg
```

Ví dụ 2-4

Khai báo một vector được tạo thành từ bốn bit *v[3]*, *v[2]*, *v[1]* và *v[0]*.

```
reg [3:0] v;
```

Ví dụ 2-5

```
wire [7:0] bus;  
assign bus = 8'h0F;
```

Tín hiệu *bus* được khai báo là một vector có độ rộng tám bit, có *index1* lớn hơn *index2*. Cụ thể, LSB của tín hiệu *bus* là bit 0 còn MSB của tín hiệu là bit 7. Như vậy, *bus[0]* sẽ có giá trị 1'b1 và *bus[7]* có giá trị 1'b0.

Ví dụ 2-6

```
wire [0:7] bus;  
assign bus = 8'h0F;
```

Tín hiệu *bus* được khai báo là một vector có độ rộng tám bit, có *index1* nhỏ hơn *index2*. Cụ thể, LSB của tín hiệu *bus* là bit 7 còn MSB của tín hiệu là bit 0. Như vậy, *bus[0]* sẽ có giá trị 1'b0 và *bus[7]* có giá trị 1'b1.

Ví dụ 2-7

```
wire [-3:4] bus;  
assign bus = 8'h0F;
```

Tín hiệu *bus* được khai báo là một vector có độ rộng bốn bit, có *index1* có giá trị âm là -3, *index2* mang số dương là 4. Cụ thể, MSB của tín hiệu *bus* là bit -3 còn LSB của tín hiệu là bit 4. Như vậy, *bus[-3]* sẽ có giá trị 1'b0 và *bus[4]* có giá trị 1'b1.

Chúng ta không nên sử dụng giá trị *index1* hay *index2* là số âm vì trình tổng hợp chỉ giữ độ rộng bit và tự động điều chỉnh các *index1* và *index 2* thành các số không âm. Trên thực tế, chúng ta thường khai báo *index1*, *index2* là các số không âm và *index1* lớn hơn *index2*.

2.3 Mảng

2.3.1 Định nghĩa

Mảng là tập hợp các phần tử có cùng kiểu dữ liệu.

2.3.2 Phân loại

2.3.2.1 Mảng một chiều một bit

Cú pháp:

```
type <signed> signal_name [index_1:index_2];
```

Mô tả:

- *type*: kiểu dữ liệu **net** hoặc kiểu dữ liệu **variable** có khai báo với từ khóa **reg**, **integer**.
- *signed*: chỉ ra tín hiệu có dấu hay không. Mặc định là không dấu nếu không khai báo.
- *signal_name*: khai báo tên tín hiệu.
- *index_1* và *index_2* là những biểu thức hằng. Biểu thức hằng có thể là hằng số, parameter, define, hoặc là một hàm hằng nào đó. Kích thước mảng lớn nhất có thể khai báo là 2^{24} (16.777.216).

Ví dụ 2-8

Khai báo một mảng tên là *sig_a* có tám phần tử, mỗi phần tử là 1 bit.

```
reg sig_a[0:7];
```

Ví dụ 2-9

Khai báo một mảng số nguyên có 64 phần tử

```
integer inta[63:0];
```

Lưu ý, mảng một chiều một bit có một số điểm cơ bản khác hoàn toàn với vector. Thứ nhất, vector có thể được gán giá trị chỉ với một câu lệnh duy nhất trong khi đó để truy xuất vào một phần tử của mảng thì phải thông qua index. Thứ hai, vector có thể được khai báo trong danh sách port nhưng mảng thì không. Thứ ba, các toán tử bit-wise, toán tử số học có thể sử dụng trực tiếp đối với vector nhưng không thể áp dụng trực tiếp với mảng.

Ví dụ 2-10

```
reg [7:0] vect_a; // 8-bit vector  
reg mem_a [0:7]; // 1D array of 1 bit  
reg [7:0] vect_b; // 8-bit vector  
reg mem_b [0:7]; // 1D array of 1 bit  
reg [7:0] vect_c; // 8-bit vector  
reg mem_c [0:7]; // 1D array of 1 bit
```

Cú pháp bên dưới là hợp lệ do việc gán giá trị lên vector là được phép.

```
vect_a = 0; // CORRECT syntax
```

Cú pháp bên dưới không hợp lệ do gán trực tiếp giá trị lên một mảng.

```
mem_a = 0; // INCORRECT syntax
```

Cú pháp bên dưới là hợp lệ do truy xuất mảng thông qua chỉ số index.

```
mem_a[1] = 0; // CORRECT syntax
```

Ví dụ 2-11

Cú pháp bên dưới là không hợp lệ do thực hiện toán toán tử bit-wise, toán tử số học và toán tử logic trên mảng.

```
mem_c = mem_a + mem_b; // INCORRECT syntax
mem_c = mem_a - mem_b; // INCORRECT syntax
mem_c = mem_a & mem_b; // INCORRECT syntax
mem_c = mem_a | mem_b; // INCORRECT syntax
mem_c = mem_a ^ mem_b; // INCORRECT syntax
mem_c = mem_a && mem_b; // INCORRECT syntax
```

Cú pháp bên dưới là hợp lệ do thực hiện toán toán tử bit-wise, toán tử số học trên vector.

```
vect_c = vect_a + vect_b; // CORRECT syntax
vect_c = vect_a - vect_b; // CORRECT syntax
vect_c = vect_a & vect_b; // CORRECT syntax
vect_c = vect_a | vect_b; // CORRECT syntax
vect_c = vect_a ^ vect_b; // CORRECT syntax
vect_c = vect_a && vect_b; // CORRECT syntax
```

2.3.2.2 Mảng một chiều nhiều bit

Cú pháp:

```
type <signed> <range> signal_name [index_1:index_2];
```

Mô tả:

- *type*: kiểu dữ liệu **net** hoặc kiểu dữ liệu **variable** có khai báo với từ khóa **reg**, **integer**.
- *signed*: chỉ ra tín hiệu có dấu hay không. Mặc định là không dấu nếu không khai báo.
- *signal_name*: khai báo tên tín hiệu
- *range*: độ rộng bit của tín hiệu có dạng *[idx1:idx2]*. Nếu như đã khai báo tín hiệu là **integer** thì không được phép khai báo

range. Kích thước *range* lớn nhất có thể được khai báo là 2^{16} (65.536) bit.

- *index_1* và *index_2* là những biểu thức hằng. Biểu thức hằng có thể là hằng số, parameter, define, hoặc là một hàm hằng nào đó. Kích thước mảng lớn nhất có thể khai báo là 2^{24} (16.777.216).

Mảng một chiều thường dùng để mô hình hóa bộ nhớ.

Ví dụ 2-12

Khai báo một mảng có tên là *mem_a* có 256 phần tử, mỗi phần tử là một vector 8-bit.

```
reg [7:0] mem_a[0:255];
```

Truy xuất vào vector đầu tiên của mảng *mem_a*:

```
mem_a[0]
```

Truy xuất vào vector cuối cùng của mảng *mem_a*:

```
mem_a[255]
```

Truy xuất vào bit 2 trong vector cuối cùng của mảng *mem_a*:

```
mem_a[255][2]
```

Gán giá trị 8'hFF vào vector thứ ba của mảng *mem_a*:

```
mem_a[2] = 8'hFF;
```

Gán giá trị 1'b1 vào bit 2 trong vector cuối cùng của mảng *mem_a*:

```
mem_a[255][2] = 1'b1;
```

2.3.2.3 Mảng nhiều chiều

Cú pháp:

```
type <signed> <range> array_name [index_1:index_2]...  
[index_i:index_j];
```

Mô tả:

- *type*: kiểu dữ liệu **net** hoặc kiểu dữ liệu **variable** có khai báo với từ khóa **reg**, **integer**.
- *array_name*: khai báo tên mảng.

- *signed*: chỉ ra tín hiệu có dấu hay không. Mặc định là không dấu nếu không khai báo.
- *range*: độ rộng bit của tín hiệu có dạng $[idx1:idx2]$. Nếu như đã khai báo tín hiệu là **integer** thì không được phép khai báo *range*. Kích thước *range* lớn nhất có thể được khai báo là 2^{16} (65.536) bit.
- $[index_1:index_2] \dots [index_i:index_j]$: khai báo dãy địa chỉ ở mỗi chiều của mảng. Trong đó, $index_1$, $index_2$, $index_i$, $index_j$ phải là biểu thức hằng.

Ví dụ 2-13

Khai báo một mảng hai chiều

```
wire [7:0] abiter [1:0][3:0]; //2D array of vector
```

Truy xuất vào một vector của mảng *abiter*

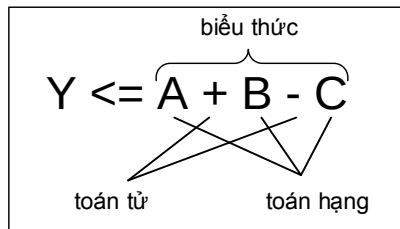
```
abiter [0][2];
```

Truy xuất vào bit 2 trong một vector của mảng *abiter*

```
abiter [0][2][2];
```

2.4 Biểu thức

Một biểu thức là một cấu trúc kết hợp các toán hạng và toán tử để cho ra kết quả mong muốn. Trong phép gán $A = B$ hoặc $A \leq B$ thì A được gọi là LHS, B được gọi là RHS.



Hình 2-1. Biểu thức gồm có toán hạng và toán tử

2.4.1 Toán hạng

Các đối tượng dữ liệu tạo thành các toán hạng của một biểu thức và giá trị của chúng được sử dụng bởi những toán tử trong biểu thức.

Bảng 2-1. Các toán hạng của Verilog

Toán hạng	Khai báo	Ghi chú
Hằng số		Khả tổng hợp
Tín hiệu có kiểu dữ liệu net	supply0, supply1, tri, triand, trior, tri0, tri1, wire, wand, wor	Khả tổng hợp
	trireg	Không tổng hợp được
Tín hiệu có kiểu dữ liệu variable	reg, integer	Khả tổng hợp
	real, time, realtime	Không tổng hợp được
bit-select của một tín hiệu kiểu dữ liệu variable	reg, integer	Khả tổng hợp
	time	Không tổng hợp được
part-select của một tín hiệu kiểu dữ liệu net	supply0, supply1, tri, triand, trior, tri0, tri1, wire, wand, wor	Khả tổng hợp
	trireg	Không tổng hợp được
part-select của một tín hiệu kiểu dữ liệu variable	reg, integer	Khả tổng hợp
	time	Không tổng hợp được
Phần tử của mảng		Khả tổng hợp
Lời gọi hàm		Khả tổng hợp

2.4.1.1 Toán hạng hằng

Một toán hạng hằng là một toán hạng có giá trị không đổi. Có hai loại toán hạng là chuỗi và hằng số. Chuỗi là một mảng của các ký tự được đặt trong dấu ngoặc kép (“ ”). Hằng số là những số có thể được chỉ rõ ở dạng nhị phân, bát phân, thập phân hoặc thập lục phân.

Cú pháp:

```
<size>' <signed> <radix> value
```

Mô tả:

- *size*: số bit nhị phân mà số đó bao gồm, mặc định là 32 bits.
- *'*: dấu nháy đơn đóng vai trò dấu cách.
- *signed*: chỉ ra số đó có dấu hay không. Ký tự “s” hoặc “S” có thể được sử dụng. Mặc định là không dấu.

- *radix*: chỉ hệ cơ số. Mặc định là hệ thập phân
 - “b” hoặc “B” : hệ nhị phân
 - “o” hoặc “O” : hệ bát phân
 - “h” hoặc “H” : hệ thập lục phân
 - “d” hoặc “D” : hệ thập phân
- *value*: những giá trị có thể được gán tùy theo hệ cơ số *radix*

Bảng 2-2. Bảng các ký tự hợp lệ của các hằng số

Định dạng	Tiếp đầu ngữ	Những ký tự hợp lệ dùng trong value
binary	'b hoặc 'B	• 0 hoặc 1 • x hoặc X • z hoặc Z hoặc ? • Dấu gạch dưới “_”
octal	'o hoặc 'O	• Từ 0 đến 7 • x hoặc X • z hoặc Z hoặc ? • Dấu gạch dưới “_”
decimal	'd hoặc 'D	• Từ 0 đến 9 • Dấu gạch dưới “_”
hexadecimal	'h hoặc 'H	• Từ 0 đến 9 • Từ a đến f • Từ A đến F • x hoặc X • z hoặc Z hoặc ? • Dấu gạch dưới “_”

Ví dụ 2-14

Số 314159 có thể được biểu diễn bởi các hệ cơ số khác nhau:

- *19'b1001100101100101111*: hằng số nhị phân dài 19 bit.
- *19'b100_1100_1011_0010_1111*: hằng số nhị phân dễ đọc hơn do dùng dấu gạch dưới biểu diễn.
- *19'o1145457*: hằng số bát phân.
- *19'h4CB2F*: hằng số thập lục phân.
- *19'd314159*: hằng số thập phân.

- 'd314159: hằng số thập phân với độ rộng không khai báo. Do đó, độ rộng mặc định là 32 bit.

Ví dụ 2-15

Chiều dài bit nhỏ hơn chiều dài tối thiểu để biểu diễn số.

```
assign sig_a[15:0] = 8'hFFFF;
```

Sau khi mô phỏng, ta thấy *sig_a* được gán giá trị là 'h00FF, chứ không phải là giá trị 'hFFFF.

Ví dụ 2-16

Gán một chuỗi là “Hello world” vào tín hiệu *str*

```
wire      [8*11-1:0] str;
assign str = "Hello world";
```

Chuỗi “Hello world” có 11 ký tự ASCII, mỗi ký tự có kích thước tám bit. Do đó, ta cần một tín hiệu có độ rộng ít nhất $11 \times 8 = 88$ bit để chứa đủ chuỗi đó. Tín hiệu *str* sẽ được gán giá trị mã ASCII tương ứng với chuỗi “Hello world”. Cụ thể, *str* sẽ có giá trị hex như sau:

```
88'h48_65_6C_6C_6F_20_77_6F_72_6C_64
```

2.4.1.2 Toán hạng định danh

Một định danh thường được dùng để cung cấp tên cho một đối tượng dữ liệu sao cho ta có thể tham chiếu dễ dàng đối tượng dữ liệu này trong một mô hình HDL. Chẳng hạn, tên của một tín hiệu là một định danh. Định danh là loại toán hạng được sử dụng phổ biến. Các định danh của Verilog bao gồm chữ số, dấu gạch dưới “_” và dấu dollar “\$”. Dấu \$ không được phép là chữ cái đầu tiên của tên đối tượng dữ liệu.

Ví dụ 2-17

```
module IDENTIFIERS (A, B, C, D, E, Y1, Y2);
input A, B, C, D;
input [7:0] E;
output Y1, Y2;
reg F, Y1, Y2;
```

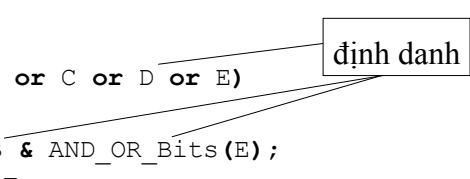
định danh

```

function AND_OR_Bits;
input [7:0] A;
begin
    AND_OR_Bits = (A[7] & A[6] & A[5] & A[4])
    & (A[3] | A[2] | A[1] | A[0]);
end
endfunction

always @(A or B or C or D or E)
begin
    F = A & B & AND_OR_Bits(E);
    Y2 = D | F;
end
endmodule

```



2.4.1.3 Toán hạng gọi hàm

Toán hạng gọi hàm là lời gọi hàm nằm trong một biểu thức. Giá trị duy nhất trả về từ một hàm là giá trị của toán hạng sử dụng trong biểu thức. Ở ví dụ 2-17, định nghĩa hàm là:

```

function AND_OR_Bits;
input [7:0] A;
begin
    AND_OR_Bits = (A[7] & A[6] & A[5] & A[4])
    & (A[3] | A[2] | A[1] | A[0]);
end

```

Lời gọi hàm là:

```
F = A & B & AND_OR_Bits(E);
```

2.4.1.4 Toán hạng bit-select

Toán hạng bit-select dùng để chỉ ra một bit của một vector kiểu **net** hay kiểu **variable** có khai báo **reg** hoặc **integer**. Nếu bit-select này ngoài tầm của địa chỉ hoặc bit-select mang giá trị x hoặc z thì giá trị trả về sẽ là x . bit-select sẽ ra kết quả không dấu bất chấp toán hạng đó có dấu hay không.

Cú pháp:

```
signal_name[index_expr]
```

Mô tả:

- *signal_name*: Khai báo tên tín hiệu.
- *index_expr*: là hằng số hoặc là một biểu thức.

Ví dụ 2-18

Cho một thanh ghi 8-bit, được khởi tạo giá trị là 4 và thực hiện bit-select ở bit 2 trên tín hiệu *sig_x*.

```
wire sig_y;  
wire [7:0] sig_x = 8'b0000_0100;  
assign sig_y = sig_x[2]; // bit-select
```

Do bit 2 nằm trong tầm của tín hiệu *sig_x* (từ bit 0 đến bit 7) nên sau khi thực hiện bit-select ở bit 2 trên tín hiệu thì tín hiệu *sig_y* sẽ được gán giá trị xác định là *1'b1*.

Ví dụ 2-19

Cho một thanh ghi 8-bit, được khởi tạo giá trị là 4 và thực hiện bit-select ở bit 8 trên tín hiệu *sig_x*.

```
wire sig_y;  
wire [7:0] sig_x = 8'b0000_0100;  
assign sig_y = sig_x[8]; // bit-select
```

Do bit 8 nằm ngoài tầm của tín hiệu *sig_x* (từ bit 0 đến bit 7) nên sau khi thực hiện bit-select ở bit 8 trên tín hiệu thì tín hiệu *sig_y* sẽ được gán giá trị là *1'bx*.

2.4.1.5 Toán hạng part-select

Toán hạng part-select dùng để chỉ một nhóm các bit liên tục nhau của một vector kiểu **net** hay kiểu **variable** có khai báo **reg** hoặc **integer**.

Nếu part-select một phần ra khỏi biên giới hạn của địa chỉ thì khi đọc sẽ trả về x cho những bit nào ra nằm khỏi tầm địa chỉ của tín hiệu. Việc ghi thì chỉ ảnh hưởng đối với những bit nằm trong tầm địa chỉ của tín hiệu. Một part-select sẽ ra kết quả không dấu bất chấp toán hạng đó có dấu hay không, thậm chí là part-select cho toàn bộ vector.

Cú pháp:

```
signal_name[msb_expr:lsb_expr]
```

Mô tả:

- *signal_name*: Khai báo tên tín hiệu.
- *msb_expr*, *lsb_expr*: là một biểu thức hằng.

Ví dụ 2-20

Cho một tín hiệu *sig_a* có độ rộng tám bit và được khởi tạo giá trị là *8'b1101_0100*.

```
wire [7:0] sig_a = 8'b1101_0100;
wire [3:0] sig_b;
wire [4:0] sig_c;
wire [3:0] sig_d;
assign sig_b = sig_a[3:0]; // part-select
assign sig_c = sig_a[5:1]; // part-select
assign sig_d = sig_a[9:6]; // part-select
```

Tín hiệu *sig_b* lấy nhóm bit từ bit 0 đến bit 3 của tín hiệu *sig_a*. Do đó, tín hiệu *sig_b* sẽ được gán giá trị *4'b0100*. Tín hiệu *sig_c* lấy nhóm bit từ bit 1 đến bit 5 của tín hiệu *sig_a*. Do đó, tín hiệu *sig_b* sẽ được gán giá trị *5'b01010*. Tín hiệu *sig_d* lấy nhóm bit từ bit 6 đến bit 9 của tín hiệu *sig_a*. Tuy nhiên, do bit 8 và bit 9 nằm ngoài tầm của tín hiệu của tín hiệu *sig_a* nên hai bit này sẽ được gán giá trị x. Do đó, tín hiệu *sig_b* sẽ được gán giá trị *4'bxx11*.

Ví dụ 2-21

Ứng dụng cho việc hoán đổi bit.

```
assign sig_y[2:0] = sig_a[0:2]; // swap bit
```

Những bit cao của tín hiệu *sig_y* sẽ nhận giá trị của những bit thấp của tín hiệu *sig_a* và những bit thấp của tín hiệu *sig_y* sẽ được gán giá trị của những bit cao trong tín hiệu *sig_a*. Cụ thể, *sig_y[2] = sig_a[0]*, *sig_y[1] = sig_a[1]* và *sig_y[0] = sig_a[2]*.

Ví dụ 2-22

Lấy một phần của tín hiệu.

```
wire [7:0] sig_a;
assign sig_y[8:6] = sig_a[2:0];
```

Bit 8, bit 7 và bit 6 của tín hiệu *sig_y* được gán giá trị của ba bit thấp của tín hiệu *sig_a*.

Ví dụ 2-23

```
assign sig_y[5:4] = {sig_a[5] & sig_b[5], sig_a[4] | sig_b[4]};
```

Bit *sig_y[5]* là kết quả phép AND bit-wise của bit thứ năm của tín hiệu *sig_a* và bit thứ năm của tín hiệu *sig_b*. Bit *sig_y[4]* là kết quả phép OR bit-wise của bit thứ tư của tín hiệu *sig_a* và bit thứ tư của tín hiệu *sig_b*.

2.4.2 Toán tử

Các toán tử thực hiện phép toán trên một hoặc nhiều toán hạng trong một biểu thức theo chức năng mong muốn của người thiết kế.

2.4.2.1 Toán tử số học

Toán tử số học là những toán tử dùng để thực hiện các phép tính toán cơ bản của số học. Nếu bất kỳ bit nào trong toán hạng có giá trị *x* hay *z* thì toàn bộ kết quả sẽ là giá trị *x*. Khi sử dụng toán tử số học thì bit dấu sẽ tự động mở rộng cho những tín hiệu có dấu.

Bảng 2-3. Liệt kê các toán tử số học

Kí hiệu	Mô tả	Ghi chú
+	Cộng	Khả tổng hợp
-	Trừ	Khả tổng hợp
*	Nhân	Khả tổng hợp
/	Chia	Khả tổng hợp
**	Mũ	Khả tổng hợp trong một số các trường hợp sau: - Hai toán hạng đều là biểu thức hằng - Một trong hai toán hạng là tín hiệu thì - cơ số là mũ của 2 hoặc - số mũ là 2
%	Phép chia lấy dư	Khả tổng hợp

Ví dụ 2-24

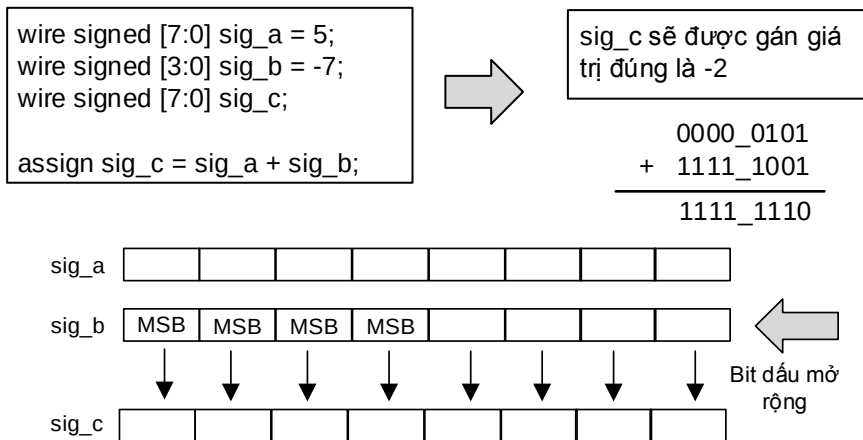
```

wire signed [4:0] sig_a, sig_b, sig_y4, sig_y5;
wire signed [5:0] sig_y1, sig_y2;
wire signed [8:0] sig_y3;
wire signed [16:0] sig_y6;
assign sig_y1 = sig_a + sig_b;
assign sig_y2 = sig_a - sig_b;
assign sig_y3 = sig_a * sig_b;
assign sig_y4 = sig_a / sig_b;
assign sig_y5 = sig_a % sig_b;
assign sig_y6 = 2 ** sig_b;

```

Ví dụ 2-25

Giả sử tín hiệu *sig_a* có giá trị là 5, *sig_b* có giá trị là -7. Do độ rộng tín hiệu *sig_a* bằng với độ rộng của tín hiệu *sig_c* nên tín hiệu *sig_a* không cần mở rộng dấu và biểu diễn dạng nhị phân là 8'b0000_0101. Tín hiệu *sig_b* là tín hiệu có dấu, được mở rộng bit dấu có giá trị là 1 nên biểu diễn nhị phân của tín hiệu *sig_b* là 8'b1111_1001.



Hình 2-2. Mở rộng bit dấu khi thực thi toán hạng số học

Ví dụ 2-26

Mặc dù *sig_b* được khai báo là tín hiệu có dấu nhưng toán tử part-select lại được sử dụng trên *sig_b*. Điều này dẫn đến *sig_b* không được mở rộng bit dấu có giá trị là 1, tức là *sig_b* có biểu diễn nhị phân là 8'b0000_1001 và dẫn đến kết quả tính toán bị sai.

```

wire signed [7:0] sig_a = 5;
wire signed [3:0] sig_b = -7;
wire signed [7:0] sig_c;

assign sig_c = sig_a + sig_b[3:0];

```



sig_c sẽ được gán trị là 14, chứ không phải giá trị đúng là -2

$$\begin{array}{r}
 0000_0101 \\
 + \quad 0000_1001 \\
 \hline
 0000_1110
 \end{array}$$

2.4.2.2 Toán tử quan hệ

Toán tử quan hệ là toán tử so sánh hai toán hạng và kết quả trả là *true* có giá trị là 1 hay *false* có giá trị là 0. Nếu như có bit nào trong hai toán hạng là x hay z thì kết quả so sánh là giá trị x.

Bảng 2-4. Liệt kê các toán tử quan hệ

Kí hiệu	Mô tả	Ghi chú
>	Lớn hơn	Khả tổng hợp
<	Nhỏ hơn	Khả tổng hợp
>=	Lớn hơn hoặc bằng	Khả tổng hợp
<=	Nhỏ hơn hoặc bằng	Khả tổng hợp

Ví dụ 2-27

```

wire [7:0] sig_a = 8'h7C;
wire [7:0] sig_b = 8'hF;
assign sig_y = (sig_a > sig_b);

```

Tín hiệu *sig_a* là tín hiệu không dấu có giá trị là 124 lớn hơn giá trị tín hiệu *sig_b* cũng là tín hiệu không dấu là 15 nên tín hiệu *sig_y* sẽ được gán giá trị là 1.

Ví dụ 2-28

```

wire signed [7:0] sig_a = 8'h7C;
wire signed [7:0] sig_b = 8'hF;
assign sig_y = (sig_a > sig_b);

```

Tín hiệu *sig_a* là tín hiệu có dấu và có giá trị là 124 lớn hơn giá trị tín hiệu *sig_b* cũng là tín hiệu có dấu là 15 nên *sig_y* sẽ được gán giá trị 1.

Ví dụ 2-29

```
wire signed [7:0] sig_a = 8'hFC;  
wire signed [7:0] sig_b = 8'hF;  
assign sig_y = (sig_a > sig_b);
```

Tín hiệu *sig_y* sẽ được gán giá trị là 0 do giá trị của *sig_a* là -4 nhỏ hơn giá trị tín hiệu *sig_b* là 15.

Ví dụ 2-30

```
wire [3:0] sig_a = 4'b0101;  
wire [3:0] sig_b = 4'b001x;  
assign sig_y = (sig_a > sig_b);
```

Tín hiệu *sig_y* sẽ được gán giá trị là x vì *sig_b[0]* mang giá trị x.

2.4.2.3 Toán tử so sánh

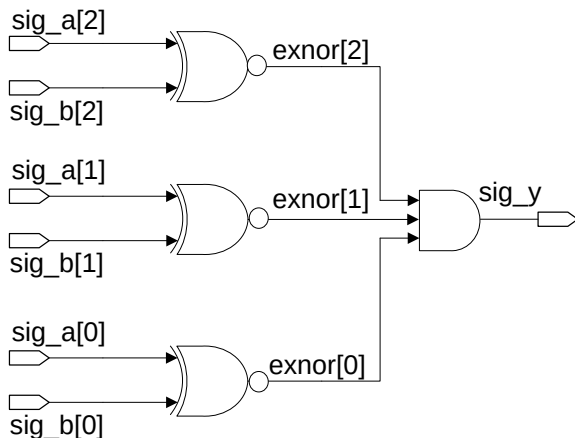
Toán tử so sánh bằng “==” và toán tử so sánh không bằng “!=” sẽ so sánh từng bit tương ứng của hai toán hạng. Kết quả của toán tử so sánh bằng là một giá trị là 1 nếu hai toán hạng có giá trị bằng nhau hoặc là 0 nếu hai toán hạng có giá trị khác nhau. Ngược lại, đối với toán tử so sánh không bằng. Nếu một trong hai toán hạng có chứa x hoặc z thì kết quả so sánh có giá trị x.

Bảng 2-5. Các toán tử so sánh

Kí hiệu	Mô tả
==	So sánh bằng
!=	So sánh không bằng

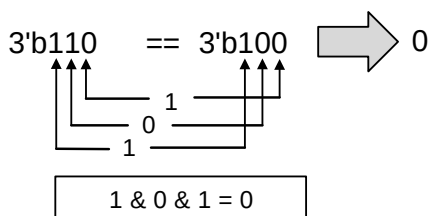
Ví dụ 2-31

```
wire [2:0] sig_a = 3'b110;  
wire [2:0] sig_b = 3'b100;  
wire sig_y;  
assign sig_y = (sig_a == sig_b);
```



Hình 2-3. Netlist sau tổng hợp của ví dụ 2-31

Kết quả so sánh giữa $sig_a[0] = 0$ và $sig_b[0] = 0$ là $exor[0] = 1$. Kết quả so sánh giữa $sig_a[1] = 1$ và $sig_b[1] = 0$ là $exor[1] = 0$. Kết quả so sánh giữa $sig_a[2] = 1$ và $sig_b[2] = 1$ là $exor[2] = 1$. Thực hiện phép AND bit-wise các bit $exor[0]$, $exor[1]$, $exor[2]$, ta có giá trị $sig_y = 0$.



Ví dụ 2-32

```

wire [3:0] sig_a = 4'b01x0;
wire [3:0] sig_b = 4'b01x0;
wire sig_y;
assign sig_y = (sig_a == sig_b);

```

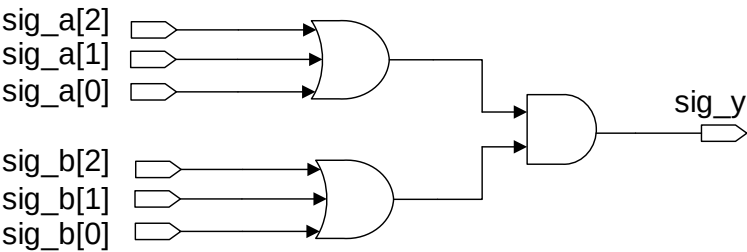
Kết quả so sánh giữa $sig_a[0] = 0$ và $sig_b[0] = 0$ là 1. Kết quả so sánh giữa $sig_a[1] = x$ và $sig_b[1] = x$ là x. Kết quả so sánh giữa $sig_a[2] = 1$ và $sig_b[2] = 1$ là 1. Kết quả so sánh giữa $sig_a[3] = 0$ và $sig_b[3] = 0$ là 1. Thực hiện phép AND bit-wise các kết quả so sánh từng bit ta có giá trị sig_y là x.

Bảng 2-6. Các toán tử logic

Kí hiệu	Mô tả
!	bù logic
&&	AND logic
	OR logic

Ví dụ 2-34

```
wire [2:0] sig_a = 4'b1x0;  
wire [2:0] sig_b = 4'b00x;  
wire sig_y;  
assign sig_y = (sig_a && sig_b);
```



Hình 2-4. Netlist sau tổng hợp của ví dụ 2-34

Trước khi thực hiện phép AND logic giữa *sig_a* và *sig_b*, ta cần phải đánh giá từng toán hạng. Để đánh giá toán hạng *sig_a*, ta thực hiện OR bit-wise các bit *sig_a[2]*, *sig_a[1]* và *sig_a[0]*. Kết quả là *sig_a* được có giá trị là 1. Tương tự, để đánh giá toán hạng *sig_b*, ta thực hiện OR bit-wise các bit *sig_b[2]*, *sig_b[1]* và *sig_b[0]*. Do vậy, *sig_b* có giá trị là x. Kết quả là *sig_y* có giá trị là x.

2.4.2.2 Toán tử bit-wise

Toán tử bit-wise là toán tử thực hiện việc tính toán trên từng bit tương ứng của các toán hạng. Ngoại lệ duy nhất là toán tử phủ định “~”, toán tử này lấy bù từng bit của toán hạng đơn theo sau.

Bảng 2-7. AND bit-wise

&	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

Bảng 2-8. OR bit-wise

 	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

Bảng 2-9. XOR bit-wise

^	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

Bảng 2-10. XNOR bit-wise

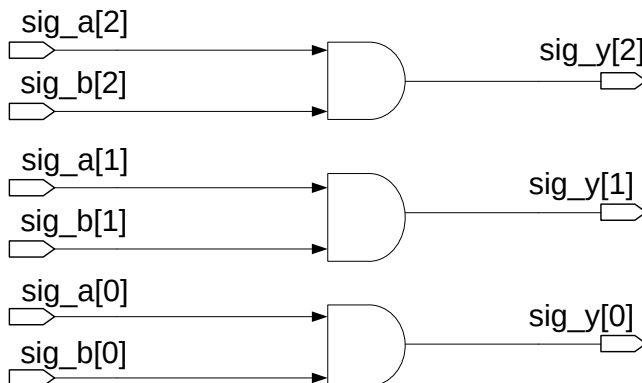
~^	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

Bảng 2-11. NOT bit-wise

~	0	1	x	z
	1	0	x	x

Ví dụ 2-35

```
wire [2:0] sig_a, sig_b, sig_y;
assign sig_y = sig_a & sig_b; // AND bit-wise
```



Hình 2-5. Netlist sau tổng hợp của ví dụ 2-35

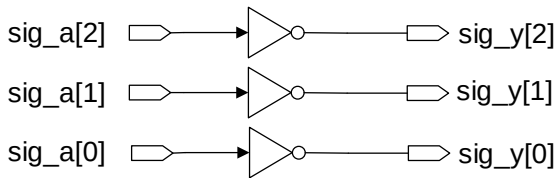
Giả sử tín hiệu *sig_a* được gán giá trị 3'b110, tín hiệu *sig_b* được gán giá trị 3'b101. Toán tử & sẽ thực hiện AND từng bit tương ứng. Cụ thể như sau:

- $\text{sig_y}[2] = \text{sig_a}[2] \& \text{sig_b}[2]$ tương ứng là 1'b1 & 1'b1 = 1'b1
- $\text{sig_y}[1] = \text{sig_a}[1] \& \text{sig_b}[1]$ tương ứng là 1'b1 & 1'b0 = 1'b0
- $\text{sig_y}[0] = \text{sig_a}[0] \& \text{sig_b}[0]$ tương ứng là 1'b0 & 1'b1 = 1'b0

Kết quả *sig_y* được gán giá trị là 3'b100.

Ví dụ 2-36

```
wire [2:0] sig_a, sig_y;
assign sig_y = ~sig_a; // NOT bit-wise
```



Hình 2-6. Netlist sau tổng hợp của ví dụ 2-36

Giả sử tín hiệu *sig_a* được gán giá trị 3'b110. Toán tử phủ định bit-wise sẽ thực hiện đảo logic cho từng bit của *sig_y*. Cụ thể như sau:

- *sig_y*[2] là phủ định bit-wise cho tín hiệu *sig_a*[2]. Do *sig_a*[2] có giá trị là 1'b1 nên *sig_y*[2] có giá trị là 1'b0.
- *sig_y*[1] là phủ định bit-wise cho tín hiệu *sig_a*[1]. Do *sig_a*[1] có giá trị là 1'b1 nên *sig_y*[1] có giá trị là 1'b0.
- *sig_y*[0] là phủ định bit-wise cho tín hiệu *sig_a*[0]. Do *sig_a*[0] có giá trị là 1'b0 nên *sig_y*[0] có giá trị là 1'b1.

2.4.2.3 Toán tử dịch bit logic

Toán tử dịch bit logic là toán tử dịch bit toán hạng bên trái bởi một số lượng bit được chỉ ở toán hạng bên phải. Các toán tử dịch bit yêu cầu hai toán hạng. Toán hạng trước toán tử là toán hạng chứa dữ liệu được dịch bit còn toán hạng sau toán tử là toán hạng chứa số thao tác dịch bit được thực hiện (nghĩa là số bit được dịch). Nếu như toán hạng bên vế phải có chứa giá trị x hoặc z thì kết quả là x.

Bảng 2-12. Các toán tử dịch bit logic

Kí hiệu	Mô tả	Ghi chú
>>	Dịch phải logic	Khả tổng hợp
<<	Dịch trái logic	Khả tổng hợp

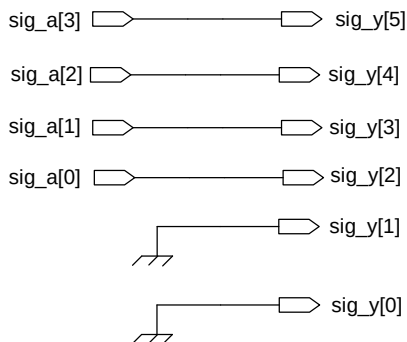
Ví dụ 2-37

Nếu dịch bit bởi một hằng số thì netlist tổng hợp chỉ là nối dây, không có cổng logic được tạo ra.

```

wire [3:0] sig_a;
wire [5:0] sig_y;
assign sig_y = sig_a << 2;

```



Hình 2-7. Netlist sau tổng hợp của ví dụ 2-37

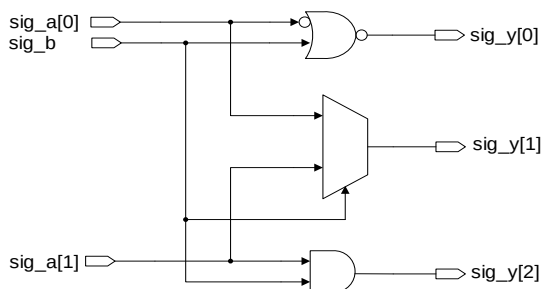
Ví dụ 2-38

Dịch bit bởi một tín hiệu có giá trị thay đổi.

```

wire [1:0] sig_a;
wire sig_b;
wire [2:0] sig_y;
assign sig_y = sig_a << sig_b;

```



Hình 2-8. Netlist sau tổng hợp của ví dụ 2-38

2.4.2.4 Toán tử dịch bit số học

Toán tử dịch bit phải số học “>>>” sẽ điền những bit bỏ trống là 0 nếu kết quả của phép toán là không dấu. Nó sẽ điền vào những bit bỏ trống là giá trị MSB của toán hạng bên trái nếu kết quả là có dấu. Nếu toán hạng bên phải mang giá trị *x* hay *z* thì kết quả trả về là *x*. Toán hạng bên phải luôn được đổi xử là số không dấu và không ảnh hưởng gì đến kết quả mở rộng dấu. Việc mở rộng dấu trong kết quả được quyết định bởi toán hạng bên trái và số dư của biểu thức.

Bảng 2-13. Các toán tử dịch bit số học

Kí hiệu	Mô tả	Ghi chú
>>>	Dịch phải số học	Khả tổng hợp
<<<	Dịch trái số học	Khả tổng hợp

Ví dụ 2-39

```
wire [3:0] start = 4'b1000;
wire [3:0] result;
assign result = (start >>> 2);
```

Tín hiệu *result* được gán giá trị *4'b0010* do giá trị *4'b1000* của tín hiệu *start* được dịch phải 2 bit nhưng không được mở rộng dấu do toán hạng bên phải *start* là tín hiệu không dấu.

Ví dụ 2-40

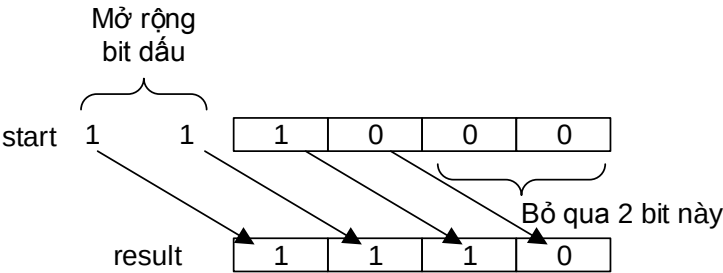
```
wire [3:0] start = 4'b1000;
wire signed [3:0] result;
assign result = (start >>> 2);
```

Tín hiệu *result* được gán giá trị *4'b0010* do giá trị *4'b1000* của tín hiệu *start* được dịch phải 2 bit nhưng không được mở rộng dấu do toán hạng bên phải *start* là tín hiệu không dấu.

Ví dụ 2-41

```
wire signed [3:0] start = 4'b1000;
wire [3:0] result;
assign result = (start >>> 2);
```

Tín hiệu *result* được gán giá trị 4'b1110 do giá trị 4'b1000 của tín hiệu *start* được dịch phải 2 bit và được mở rộng dấu do toán hạng bên phải *start* là tín hiệu có dấu.



Hình 2-9. Minh họa dịch bit logic bởi một hằng số

Ví dụ 2-42

```

wire signed [3:0] start = 4'b1000;
wire signed [3:0] result;
assign result = (start >>> 2);

```

Tín hiệu *result* được gán giá trị 4'b1110 do giá trị 4'b1000 của tín hiệu *start* được dịch phải 2 bit và được mở rộng dấu do toán hạng bên phải *start* là tín hiệu có dấu.

2.4.2.5 Toán tử rút gọn

Toán tử rút gọn là toán tử nhận toán hạng nhiều bit, thực hiện thao tác trên từng bit của toán hạng, trả về kết quả đơn bit.

Bảng 2-14. Liệt kê toán tử rút gọn

Kí hiệu	Mô tả	Ghi chú
&	AND rút gọn	Khả tổng hợp
	OR rút gọn	Khả tổng hợp
~&	NAND rút gọn	Khả tổng hợp
~	NOR rút gọn	Khả tổng hợp
^	XOR rút gọn	Khả tổng hợp
^~ hoặc ^~	XNOR rút gọn	Khả tổng hợp

Ví dụ 2-43

```
wire [3:0] sig_a = 4'b1000;  
wire [3:0] sig_y;  
assign sig_y = &sig_a;
```

Toán tử & sẽ thực hiện việc AND bit-wise từng bit. Cụ thể, tín hiệu $sig_y = sig_a[3] \& sig_a[2] \& sig_a[1] \& sig_a[0] = 1'b1 \& 1'b0 \& 1'b0 \& 1'b0 = 1'b0$.

2.4.2.6 Toán tử nối và toán tử sao chép

Toán tử nối “{ }” kết hợp các bit của hai hay nhiều đối tượng dữ liệu. Các đối tượng này có thể đơn bit (scalar) hoặc nhiều bit (dạng vector). Nhiều phép nối có thể được thực hiện với một tiền tố hằng và được gọi là sao chép. Tiền tố hằng không được phép mang giá trị 0, x hay z. Kết quả của biểu thức sau khi thực hiện là không dấu bất chấp toán hạng có dấu hay không.

Những hằng số không có kích thước sẽ không được phép dùng trong toán tử nối bởi vì kích thước của mỗi toán hạng trong phép nối cần được xác định để có thể tính toán được kích thước của phép nối.

Ví dụ 2-44

```
wire [3:0] sig_a;  
wire [31:0] sig_y;  
assign sig_y = {32,sig_a}; //INCORRECT SYNTAX
```

Phép nối trên là không hợp lệ do hằng số 32 không được qui định kích thước cụ thể.

Ví dụ 2-45

```
sig_a[31:0] = {1'b1, {0{1'b0}}} }; //INCORRECT SYNTAX  
sig_a[31:0] = {1'b1, {1'bz{1'b0}}} }; //INCORRECT SYNTAX  
sig_a[31:0] = {1'b1, {1'bx{1'b0}}} }; //INCORRECT SYNTAX
```

Phép nối trên là không hợp lệ do tiền tố hằng mang giá trị 0, x và z.

Ví dụ 2-46

{4{a}} tương đương với {a, a, a, a}.

$\{3\{a, b\}\}$ tương đương với $\{a, b, a, b, a, b\}$.

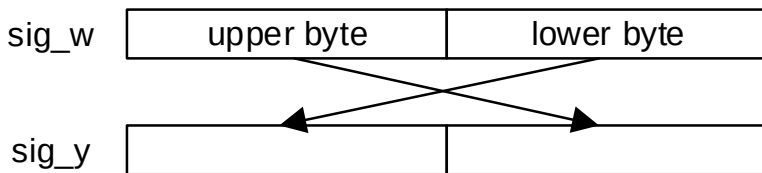
$\{c, \{3\{a, b\}\}\}$ tương đương với $\{c, a, b, a, b, a, b\}$.

Ví dụ 2-47

```
wire [15:0] sig_y, sig_w;  
assign sig_y = { sig_w[7:0], sig_w[15:8]}
```

Dòng code ở trên tương đương với dòng code bên dưới

```
assign {sig_y[7:0], sig_y[15:8]} = {sig_w[15:0]};
```



Hình 2-10. Swap bytes

Byte thấp của tín hiệu *sig_y* được gán byte cao của tín hiệu *sig_w* và byte cao của tín hiệu *sig_y* được gán byte thấp của tín hiệu *sig_w*.

Ví dụ 2-48

Xoay phải 1 bit

```
assign next_sig_w[15:0] = {sig_w[0], sig_w[15:1]}; //  
bit right rotation
```

Bit *next_sig_w[15]* được gán giá trị *sig_w[0]* và *next_sig_w[14:0]* sẽ được gán giá trị *sig_w[15:1]*.

Ví dụ 2-49

Xoay trái 1 bit

```
assign next_sig_w[15:0] = {sig_w[14:0], sig_w[15]}; //  
left bit rotation
```

Vector *next_sig_w[15:1]* được gán giá trị *sig_w[14:0]* và bit *next_sig_w[0]* sẽ được gán giá trị *sig_w[15]*.

2.4.2.7 Toán tử điều kiện

Cú pháp:

```
<expression_1>?<expression_2>:<expression_3>
```

Mô tả:

- Nếu như biểu thức điều kiện *expression_1* có giá trị *true* (1) thì *expression_2* sẽ được đánh giá và được sử dụng như là kết quả.
- Nếu như biểu thức điều kiện *expression_1* có giá trị *false* (0) thì *expression_3* sẽ được đánh giá và được sử dụng như là kết quả.
- Nếu như biểu thức *expression_1* có giá trị giá trị *x* hay là *z* thì cả hai *expression_2* và *expression_3* sẽ được đánh giá và kết quả của chúng sẽ được kết hợp theo từng bit theo bảng bên dưới.
- Nếu độ rộng bit của *expression_2* và *expression_3* khác nhau thì biểu thức có độ rộng bit ngắn hơn sẽ được điền 0 để cho hai biểu thức có cùng độ rộng bit với nhau

Bảng 2-15. Đánh giá từng bit giữa hai toán hạng trong toán tử điều kiện

?:	0	1	x	z
0	0	x	x	x
1	x	1	x	x
x	x	x	x	x
z	x	x	x	x

Ví dụ 2-50

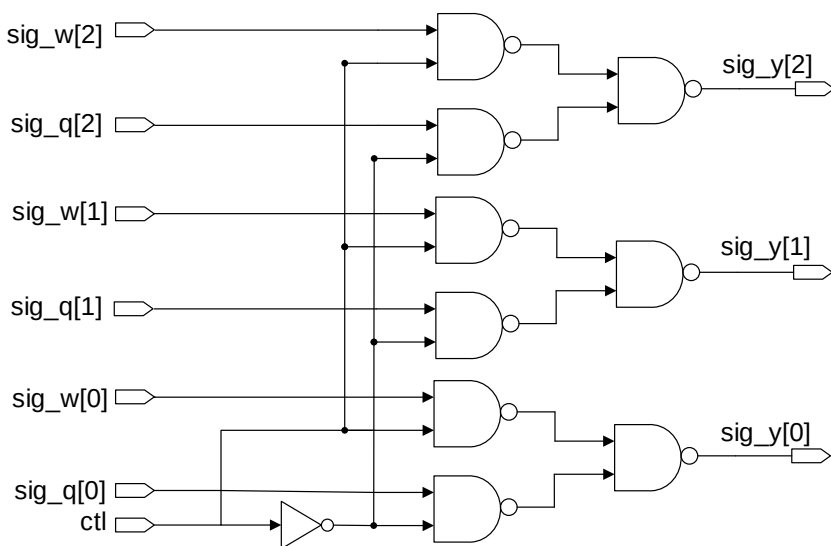
So sánh giữa toán tử điều kiện và phát biểu **if**.

Bên dưới là minh họa đoạn code dùng phát biểu **if**.

```
if (ctl == 1'b1)    sig_y = sig_w;  
else               sig_y = sig_q;
```

Bên dưới là minh họa đoạn code dùng toán tử điều kiện.

```
assign sig_y[2:0] = (ctl==1'b1)? sig_w[2:0]: sig_q[2:0];
```

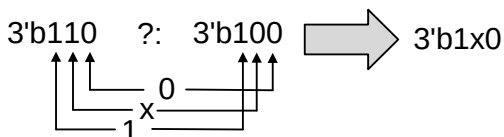


Hình 2-11. Netlist sau tổng hợp của ví dụ 2-50

Giả sử sig_w có giá trị 3'b110 và sig_q có giá trị 3'b100.

Bảng 2-16. So sánh giữa toán tử điều kiện và phát biểu if

ctl Biểu thức	1'b1	1'b0	1'bx	
	Mô phỏng mức RTL và mức cổng	Mô phỏng mức RTL và mức cổng	Mô phỏng mức RTL	Mô phỏng mức cổng
if (ctl == 1'b1) sig_y = sig_w; else sig_y = sig_q;	3'b110	3'b100	3'b100	3'bx0
sig_y = (ctl == 1'b1) ? sig_w : sig_q;	3'b110	3'b100	3'b1x0	3'bx0



Hình 2-12. Kết quả của toán tử điều kiện ở mức RTL khi ctl có giá trị x

Theo bảng phân tích ở trên, việc sử dụng phát biểu **if** và toán tử điều kiện đều có thể tạo nên sự khác nhau khi mô phỏng RTL và mô phỏng ở mức cổng.

2.4.3 Độ ưu tiên của các toán tử

Những toán tử cùng mức ưu tiên sẽ được gom thành một nhóm. Những toán tử có cùng mức ưu tiên thì sẽ được thực hiện theo thứ tự từ trái sang phải. Lưu ý, dấu + và dấu - ở nhóm 1 trong bảng dưới là toán tử số học còn dấu + và dấu - ở nhóm 3 là dấu dương và dấu âm của toán hạng.

+ - ! ~ (nhóm 1)	Độ ưu tiên cao nhất ↓ Độ ưu tiên thấp nhất
* / %	
+ - (nhóm 3)	
<< >>	
< <= > >=	
== != === !==	
& ~&	
^ ^~	
~	
&&	
?:	

Hình 2-13. Độ ưu tiên của các toán tử

2.5 Các loại phép gán

Phép gán có hai loại:

- Phép gán thủ tục
 - Phép gán blocking
 - Phép gán non-blocking
- Phép gán liên tục

2.5.1 Phép gán thủ tục

Phép gán thủ tục là phép gán cập nhật giá trị của biến dựa theo kết quả của điều kiện trong lưu đồ thực thi. Các biến giữ giá trị của phép gán cho đến khi một phép gán thủ tục tiếp theo cập nhật lên biến đó.

LHS có kiểu dữ liệu là **variable**. Trong đó, các khai báo khả tổng hợp là **reg** hoặc **integer**. RHS có thể là bất kì kiểu dữ liệu nào mà biểu thức của nó có thể đánh giá được giá trị.

Phát gán thủ tục có hai loại:

- Phép gán blocking là phép gán mà dòng kế tiếp chỉ được thực thi khi dòng trước đó được thực thi xong.
- Phép gán non-blocking là phép gán thực thi dòng kế tiếp không cần đợi dòng trước đó thực thi xong.

2.5.2 Phép gán liên tục

Phép gán liên tục là phép gán sẽ được thực thi bất cứ khi nào RHS thay đổi. LHS phải có kiểu dữ liệu là **net**.

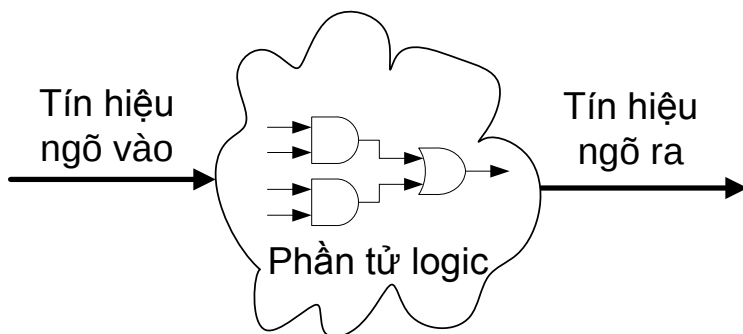
Ví dụ 2-51

```
assign sig_y = (sel == 1'b1) ? sig_a : sig_b;
```

Bất cứ khi nào tín hiệu *sig_a*, *sig_b* hay *sel* thay đổi giá trị thì phép gán cho tín hiệu *sig_y* sẽ được thực thi.

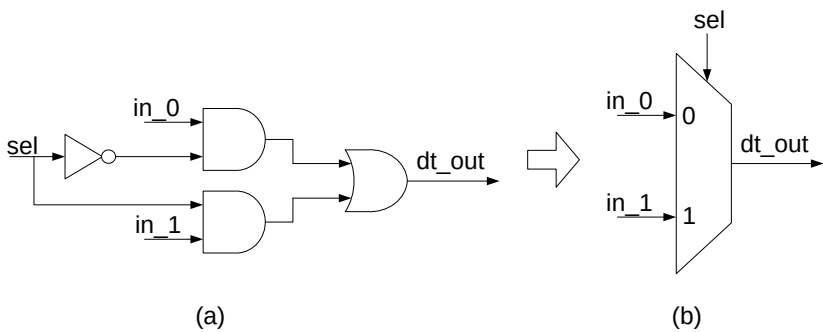
2.6 Mạch tổ hợp

Mạch tổ hợp là mạch có trạng thái các ngõ ra chỉ phụ thuộc vào trạng thái hiện tại của các ngõ vào.



Hình 2-14. Mô hình mạch tổ hợp

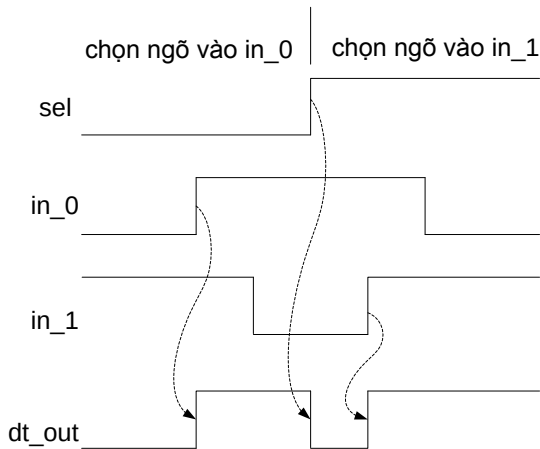
Một mạch tổ hợp gồm các tín hiệu ngõ vào được kết nối với nhau bằng các phần tử logic để tạo ra mạch chức năng điều khiển các tín hiệu ngõ ra.



Hình 2-15. Ví dụ về mạch tổ hợp

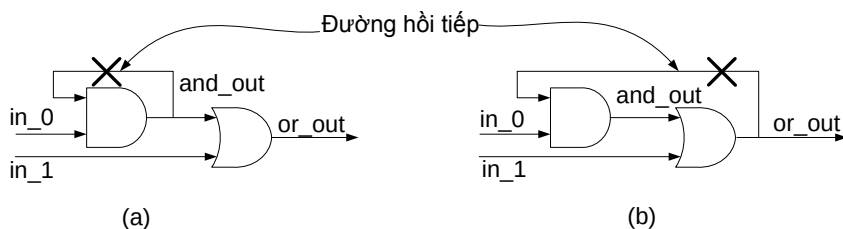
Trong mạch tổ hợp hình 2-15, các ngõ vào *sel*, *in_0* và *in_1* được kết nối với nhau bằng một cổng NOT, hai cổng AND và một cổng OR để điều khiển ngõ ra *dt_out*. Mạch này thực hiện chức năng kết nối ngõ vào *in_0* đến ngõ ra *dt_out* khi *sel* bằng 0 và kết nối ngõ vào *in_1* đến ngõ ra *dt_out* khi *sel* bằng 1.

Khi các ngõ vào thay đổi trạng thái, ngõ ra *dt_out* sẽ cập nhật ngay trạng thái mới nếu bỏ qua độ trễ lan truyền trên các phần tử logic. Trạng thái mới có thể giống hoặc khác với trạng thái trước đó tùy thuộc giá trị ngõ vào và chức năng mạch tổ hợp.



Hình 2-16. Chuyển trạng thái ở ngõ ra *dt_out* trong mạch tổ hợp

Mạch tổ hợp không phụ thuộc vào trạng thái trước đó của mạch nên nó không chứa các đường hồi tiếp.



Hình 2-17. Mạch có đường hồi tiếp

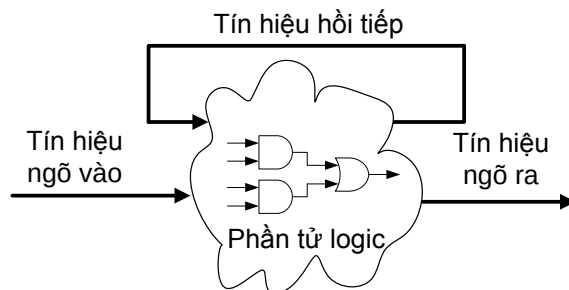
Mạch trong hình 2-17 chứa các đường hồi tiếp làm cho ngõ ra *or_out* không chỉ phụ thuộc vào các ngõ vào *in_0*, *in_1* mà còn phụ thuộc vào trạng thái của tín hiệu nội *and_out* (hình a) hoặc phụ thuộc vào chính trạng thái của ngõ ra *or_out* (hình b). Mạch chỉ chứa các phần tử logic như NOT, AND, OR, XOR, ... nhưng có đường hồi tiếp thường được gọi là mạch tổ hợp có hồi tiếp.

Ngõ ra mạch tổ hợp không phụ thuộc vào trạng thái trước đó nên nó sẽ không lưu giữ trạng thái trước đó và không chứa các phần tử nhớ như Latch, Flip-Flop, RAM, ...

2.7 Mạch tuần tự

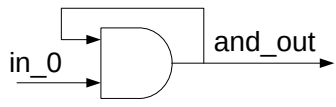
2.7.1 Khái niệm

Mạch tuần tự là mạch có trạng thái các ngõ ra phụ thuộc vào giá trị các ngõ vào hiện tại và trạng thái trước đó của mạch.



Hình 2-18. Mô hình mạch tuần tự

Mạch tuần tự bao gồm các thành phần cơ bản như mạch tổ hợp và có thêm thành phần đường hồi tiếp. Đường hồi tiếp này thể hiện sự phụ thuộc vào trạng thái trước đó của mạch.

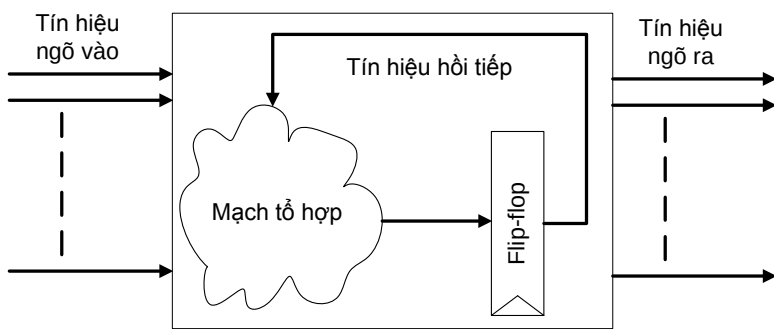


Hình 2-19. Ví dụ đơn giản về mạch tuần tự

Trong hình 2-19, giả sử ban đầu, ngõ vào *in_0* bằng 1, giá trị ngõ ra *and_out* bằng 1. Khi *in_0* chuyển sang giá trị 0, ngõ ra *and_out* sẽ nhanh chóng ổn định bằng 0. Khi ngõ ra *and_out* bằng 0, ngõ vào *in_0* lúc này bằng 0 hay 1 cũng không làm thay đổi giá trị ngõ ra *and_out*. Ngõ ra *and_out* không chỉ phụ thuộc vào ngõ vào *in_0* mà còn phụ thuộc vào trạng thái của chính *and_out* trước đó.

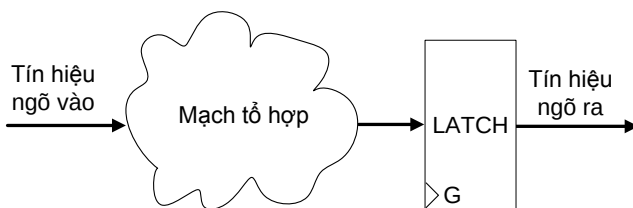
Mô hình mạch tuần tự trình bày trên đây chính là mạch tổ hợp có hồi tiếp nhưng mô hình này ít được sử dụng trong thiết kế mạch số vì việc tính toán và bảo đảm định thời cho mạch phức tạp. Mô hình mạch tuần tự thường được sử dụng là mạch có chứa D Flip-Flop. Mô hình mạch này sẽ được trình bày trong sách và bao gồm các thành phần cơ bản như sau:

- Tín hiệu ngõ vào
- Tín hiệu hồi tiếp từ ngõ ra của Flip-Flop (có thể có hoặc không)
- Tín hiệu ngõ ra
- Mạch tổ hợp (có thể có hoặc không)
- Flip-Flop



Hình 2-20. Mô hình mạch tuần tự được trình bày trong sách

Mạch chứa Latch cũng là dạng mạch tuần tự. Tuy Verilog HDL và các trình tổng hợp hỗ trợ đầy đủ việc tạo Latch nhưng mạch chứa Latch chỉ nên dùng trong những thiết kế thật sự cần thiết với những ràng buộc thích hợp khi tổng hợp để đảm bảo định thì và chức năng của mạch. Cuốn sách này sẽ không trình bày về việc sử dụng Latch trong thiết kế mạch tuần tự mà chỉ trình bày về việc sử dụng Verilog HDL để tránh tạo Latch không mong muốn.



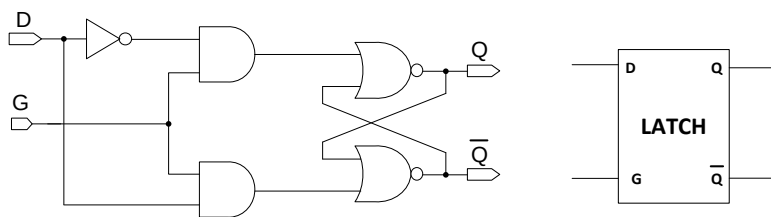
Hình 2-21. Mô hình mạch chứa D Latch

2.7.2 Latch và Flip-Flop

Latch và Flip-Flop là các thành phần lưu trữ thông tin cơ bản. Mỗi Latch hoặc Flip-Flop lưu trữ được một bit thông tin. Ngõ ra của Latch và Flip-Flop chỉ cập nhật giá trị mới khi tín hiệu cho phép nó tích cực. Latch và Flip-Flop có bốn loại cơ bản là SR, JK, T và D nhưng loại D được sử dụng phổ biến hơn các loại khác. Nhiều thư viện tổng hợp mạch số chỉ hỗ trợ loại D Latch và D Flip-Flop nên cuốn sách này chỉ trình bày về loại D và được gọi tắt là Latch và Flip-Flop.

2.7.2.1 Latch

Latch hoạt động theo mức logic của tín hiệu cho phép nên thường có đáp ứng nhanh hơn so với Flip-Flop đồng bộ. Tuy nhiên, sử dụng Latch trong thiết kế vi mạch số sẽ kém an toàn hơn sử dụng Flip-Flop vì khó quản lý định thời.

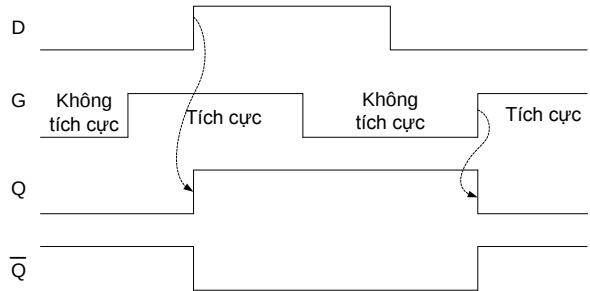


Hình 2-22. Sơ đồ logic và ký hiệu của D Latch

Bảng 2-17. Hoạt động của D Latch

G	D	Q	Q _{next}	/Q _{next}
0	x	0	0	1
0	x	1	1	0
1	0	x	0	1
1	1	x	1	0

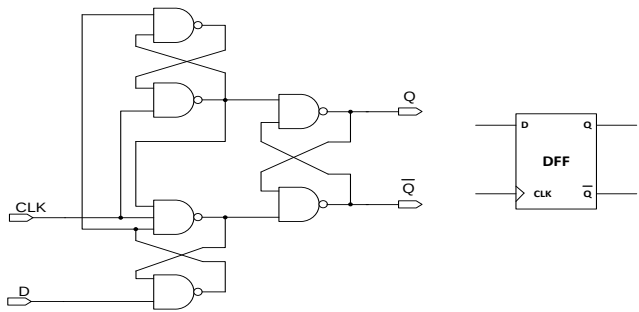
Với D Latch, khi tín hiệu cho phép *G* tích cực, giá trị ngõ ra *Q* sẽ thay đổi theo giá trị ngõ vào *D*. Khi tín hiệu cho phép *G* không tích cực, giá trị ngõ ra *Q* sẽ giữ ổn định giá trị trước đó và không phụ thuộc vào sự thay đổi của ngõ vào *D*.



Hình 2-23. Hoạt động của D Latch

2.7.2.2 Flip-Flop đồng bộ

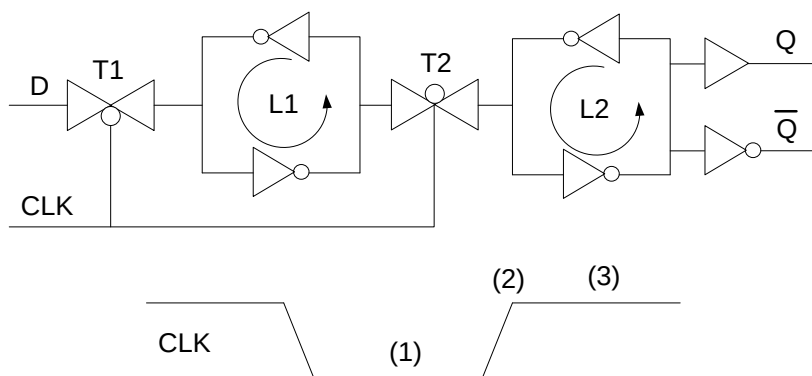
Khác với Latch, Flip-Flop hoạt động theo cạnh tích cực của tín hiệu cho phép. Cạnh tích cực là cạnh cho phép cho ngõ ra của Flip-Flop cập nhật giá trị mới. Cạnh tích cực có thể là cạnh lên, khi tín hiệu chuyển từ mức logic thấp lên mức logic cao, hoặc cạnh xuống, khi tín hiệu chuyển từ mức logic cao xuống mức logic thấp. Tín hiệu này thường được gọi là xung clock.



Hình 2-24. Sơ đồ logic và ký hiệu của D Flip-Flop

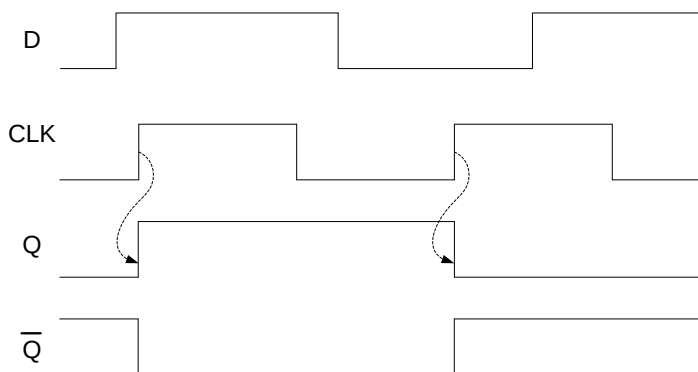
Hoạt động của D Flip-Flop được phân tích tại ba vị trí đặc trưng là mức thấp, cạnh tích cực và mức cao của xung clock, ký hiệu là CLK , trên sơ đồ nguyên lý hình 2-25.

- Tại vị trí (1), CLK mức thấp. Cổng T1 mở và cổng T2 đóng. Giá trị ngõ vào D sẽ đưa vào trong Flip-Flop và lặp ở mạch lặp L1. Ngõ ra Q giữ giá trị ổn định trước đó bởi mạch lặp L2.
- Tại vị trí (2), CLK chuyển từ mức thấp lên mức cao. Cổng T1 đóng và cổng T2 mở. Giá trị lặp trong mạch L1 chuyển sang mạch lặp L2. Giá trị ngõ vào D đã được chuyển tới ngõ ra Q và ngõ ra Q cập nhật giá trị mới.
- Tại vị trí (3), CLK ổn định mức cao. Cổng T1 đóng và cổng T2 mở. Ngõ ra Q ổn định theo giá trị mới bởi mạch lặp L2. Nếu giá trị ngõ vào D thay đổi thì cũng không ảnh hưởng đến ngõ ra Q vì cổng T1 đã đóng.



Hình 2-25. Sơ đồ cấu trúc của D Flip-Flop

Giá trị ngõ ra Q của D Flip-Flop được thay đổi theo giá trị ngõ vào D tại cạnh tích cực. Sau cạnh xung clock này, giá trị ngõ ra Q được giữ ổn định cho đến cạnh tích cực tiếp theo mà không phụ thuộc vào sự thay đổi của ngõ vào D .



Hình 2-26. Hoạt động của D Flip-Flop

3

VERILOG HDL KHẢ TỔNG HỢP

Chương này trình bày cách sử dụng ngôn ngữ Verilog HDL. Nội dung tập trung mô tả và phân tích các loại phát biểu như: phát biểu gán, phát biểu **always**, phát biểu **if**, phát biểu **case**, định nghĩa và gọi **function**, phát biểu **generate**,... Mỗi loại phát biểu được chia làm ba mục nhỏ: mô tả cú pháp, mô tả cách sử dụng, các vấn đề cần lưu ý. Kèm theo đó là các ví dụ được phân tích kỹ lưỡng và được minh họa bằng sơ đồ mạch, netlist sau tổng hợp.

3.1 Cấu trúc **always**

3.1.1 Cú pháp **always**

```
always @ (event_expression) begin  
    <statements>  
end
```

Mô tả:

- **@**: ký hiệu đặc tả các sự kiện điều khiển
- **event_expression**: các sự kiện điều khiển
- **begin - end**: các từ khóa bắt đầu và kết thúc, dùng để gom nhiều phát biểu trong một khối **always**, nếu chỉ có một phát biểu trong khối thì có thể có từ khóa này hoặc không.
- **statements**: các phát biểu trong khối **always**.

Ví dụ 3-1

```
always @ (posedge clk) begin  
    q1 <= in;  
    out <= q2;  
end
```

Đoạn code ví dụ 3-1 thực hiện phép gán cho biến *q1* và *out*. Khối **always** này hoạt động bằng cạnh lên của tín hiệu *clk*.

Ví dụ 3-2

```
always @ (in1 or in2 or in3) begin  
    out1 = in1 | in2;  
    out2 = out1 & in3;  
end
```

Đoạn code ví dụ 3-2 thực hiện phép gán cho biến *out1* và *out2*. Khối **always** này hoạt động bằng các sự kiện chuyển mức ở ngõ vào *in1*, *in2* và *in3*.

Khối bao gồm một hay nhiều phát biểu giới hạn bởi **begin** và **end** gọi là khối tuần tự. Một khối tuần tự hoạt động theo nguyên tắc như sau:

- Các phát biểu trong khối được thực thi tuần tự từ trên xuống.
- Độ trễ của phát biểu trước đó ảnh hưởng đến thời gian xử lý của phát biểu sau.
- Sự kiện tác động xảy ra kết thúc ảnh hưởng vào khối **always** sau khi phát biểu cuối cùng thực thi.

3.1.2 Phép gán trong một khối **always**

Các phép gán trong **always** được sử dụng để cập nhật giá trị cho biến **reg**, biến **integer** là các biến có thể tổng hợp được và một số kiểu biến khác chỉ dùng cho mô phỏng. Giá trị của biến được giữ cho đến khi bất kỳ thành phần nào trong danh sách sự kiện tham chiếu tích cực thì nó sẽ được cập nhật giá trị.

Phần biến bên trái của biểu thức trong khối **always** thông thường có thể là một trong những kiểu sau:

- Một biến **reg**, **integer** là các biến tổng hợp được
- Một phần trong một vector
- Một phần trong một mảng
- Kết hợp của các kiểu trên

Phần gán bên phải là một biểu thức hoặc một **function**.

Biến **reg** và **integer** dùng để mô hình hóa cho thanh ghi hoặc Latch hoặc mạch tổ hợp. Ngoài ra, biến **integer** còn là biến đa chức năng được dùng cho các thao tác đếm.

Ví dụ 3-3

```
integer i;
reg [7:0] out;
//
always @ (posedge clk) begin
    for (i=0; i<8, i=i+1) begin
        out[i] <= in[i];
    end
end
```

ví dụ 3-3 mô tả phép gán ngõ vào $in[7:0]$ cho ngõ ra $out[7:0]$ bằng cách sử dụng biến đếm i . Vòng *for* lúc này tương đương với phép gán $out[7:0] \leq in[7:0]$.

Có hai kiểu gán cơ bản trong khối **always** đó là phép gán non-blocking và phép gán blocking.

3.1.2.1 Phép gán blocking

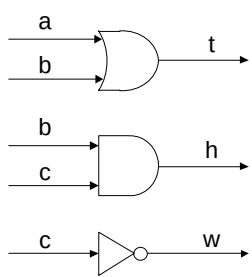
Đối với phép gán blocking, các biểu thức tính toán và phép gán được thực thi ngay khi biểu thức có kết quả. Phép gán blocking sử dụng ký hiệu “=” để mô tả phép gán.

Ví dụ 3-4

```
always @ (a or b or c) begin
    t = a | b;
    h = b & c;
    w = ~c;
end
```

Trong ví dụ 3-4, ba phát biểu trong nhóm **begin ... end** sẽ được thực thi theo thứ tự như sau:

- Tính toán $(a | b)$ và gán cho t
- Tính toán $(b \& c)$ và gán cho h
- Tính toán $(\sim c)$ và gán cho w



Hình 3-1. Mạch nguyên lý cho ví dụ 3-4

Phép gán blocking thường sử dụng để mô tả mạch tổ hợp dùng khối **always**.

3.1.2.2 Phép gán non-blocking

Phép gán non-blocking cho phép gán theo một kế hoạch đã xác định và chỉ thực hiện gán một lần khi tất cả các biểu thức bên phải đã tính toán xong. Phép gán non-blocking sử dụng ký hiệu “ $\leq$ ” để mô tả phép gán.

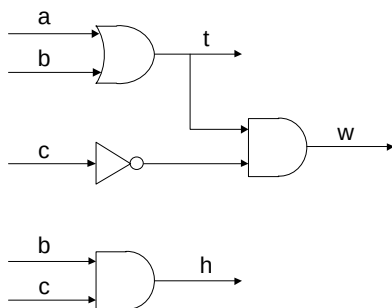
Ví dụ 3-5

```
always @ (a or b or c) begin
    t <= a | b;
    h <= b & c;
    w <= ~c & t;
end
```



Các biểu thức trong ví dụ 3-5 được tính toán nhưng chưa gán cho biến tương ứng mà chúng chỉ thực hiện gán khi tất cả các biểu thức bên phải đã hoàn thành. Ba phát biểu trong nhóm **begin ... end** sẽ được thực thi theo thứ tự như sau:

- Tính toán $(a | b)$ nhưng chưa gán cho t
- Tính toán $(b \& c)$ nhưng chưa gán cho h
- Tính toán $(\sim c \& t)$ nhưng chưa gán cho w (t là giá trị cũ)
- Gán đồng thời các giá trị mới cho ba biến t , h và w



Hình 3-2. Mạch nguyên lý cho ví dụ 3-5

Phép gán non-blocking thường sử dụng để mô tả mạch tuần tự dùng khối **always**. Ví dụ 3-5 được đưa ra chỉ để phân tích hành vi của phép gán non-blocking, trường hợp này nên thay phép gán non-blocking bằng phép gán blocking.

3.1.3 Cách sử dụng always

Cấu trúc **always** thường dùng để tạo ra mạch tuần tự, mạch tổ hợp hay mạch Latch và một số tác vụ dùng trong mô phỏng. Phần này sẽ tập trung trình bày các sự kiện điều khiển và cách sử dụng **always** trong mô tả mạch tổ hợp cũng như mạch tuần tự.

3.1.3.1 Các sự kiện điều khiển

Quá trình xử lý của một phép gán thủ tục trong khối **always** được đồng bộ với một sự thay đổi giá trị trên sự kiện đã được khai báo trong danh sách các sự kiện điều khiển. Giá trị thay đổi này được sử dụng để kích hoạt xử lý phép gán.

Một kiểu sự kiện khác được định nghĩa dựa trên hướng chuyển của dây tín hiệu hay biến khai báo, đó là kiểu hướng về 1 (**posedge**) hoặc kiểu hướng về 0 (**negedge**).

- Một sự kiện hướng về 0 **negedge** được phát hiện bằng sự kiện chuyển từ mức 1 về x, z hoặc 0 và từ x, z về 0.
- Một sự kiện hướng về 1 **posedge** được phát hiện bằng sự kiện chuyển từ mức 0 đến x, z hoặc 1 và từ x, z đến 1.

Ví dụ 3-6

Một số kiểu sự kiện điều khiển

```
// normal event control
@ (a, b) c = a & b;
// event towards to the value 1
@ (posedge a) e <= b & d;
// event towards to the value 0
@ (negedge a) f <= b & d;
```

Trong ví dụ 3-6, biến *c* sẽ cập nhật giá trị khi *a* thay đổi giá trị, *e* cập nhật giá trị tại cạnh lên của *a* và *f* cập nhật giá trị tại cạnh xuống của *a*.

Ví dụ 3-7

```
always @ (posedge clock) begin
    reg_a <= data_in;
end
```

Trong ví dụ 3-7, phép gán *data_in* gán cho biến thanh ghi *reg_a* sẽ cập nhật giá trị cho *reg_a* mỗi khi có cạnh lên xung clock và sự kiện cạnh lên của xung clock được khai báo bằng từ khóa **posedge**.

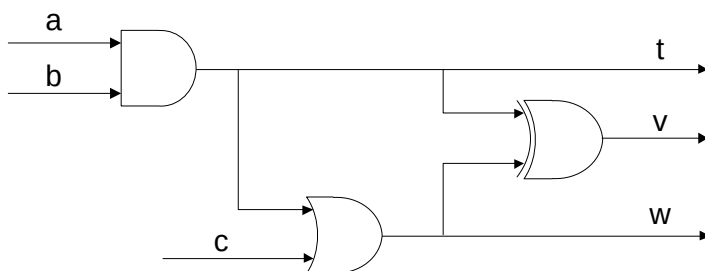
Verilog HDL sử dụng từ khóa **or** để gom và phân biệt các sự kiện điều khiển khi có nhiều sự kiện điều khiển ảnh hưởng đến quá trình cập nhật giá trị cho các biến trong khối **always**, và nhóm các sự kiện này được gọi là danh sách độ nhạy.

Ví dụ 3-8

Nhóm sự kiện điều khiển *a*, *b* và *c* được khai báo cách nhau bằng từ khóa **or**

```
always @ (a or b or c) begin
    t = a & b;
    w = t | c;
    v = w ^ t;
end
```

Trong ví dụ 3-8, các phát biểu được thực hiện và gán cho biến khi có ít nhất một trong các sự kiện tham chiếu *a*, *b*, hoặc *c* thay đổi giá trị.



Hình 3-3. Mạch nguyên lý của ví dụ 3-8

Từ khóa **or** có thể được thay thế bằng dấu “,”. Và khi tất cả các ngõ vào đều là các sự kiện điều khiển thì ta nên thay danh sách độ nhạy bằng ký hiệu “*” để tránh khai báo thiếu sót. Khai báo thiếu sự kiện điều khiển không gây ra lỗi cú pháp nhưng có thể tạo khác biệt giữa mô phỏng mức RTL và mô phỏng mức cổng.

Ví dụ 3-9

Khai báo danh sách độ nhạy không có **or**

```
always @ (a, b, c) begin  
    t = a & b;  
    w = t | c;  
    v = w ^ t;  
end
```

Hoặc khai báo danh sách độ nhạy rút gọn

```
always @ (*) begin // sensitivity list: a, b and c  
    t = a & b;  
    w = t | c;  
    v = w ^ t;  
end
```

Trong ví dụ 3-9, ký hiệu “*” thay thế cho danh sách gồm ba tín hiệu điều khiển *a*, *b* và *c*. Biến *t* và *w* được cập nhật giá trị khi *a* hoặc *b* hoặc *c* thay đổi. Tuy nhiên, khi *t* và *w* thay đổi thì *v* lại không được cập nhật giá trị mới. Để danh sách độ nhạy có chứa cả tín hiệu nội, ta khai báo như ví dụ sau:

Ví dụ 3-10

```
always @* begin // sensitivity list: a, b, c, t and w  
    t = a & b;  
    w = t | c;  
    v = w ^ t;  
end
```

Trong ví dụ 3-10, danh sách độ nhạy bao gồm cả *a*, *b*, *c*, *t* và *w*. Để tránh trường hợp lỗi mô phỏng khi biến nội không có trong danh sách độ nhạy như ví dụ 3-9 thì ta nên viết một khối **always** chỉ cho một ngõ ra như ví dụ sau:

Ví dụ 3-11

```
always @ (*) begin  
    t = a & b;  
end  
//
```



```

always @ (*) begin
    w = t | c;
end
//
always @ (*) begin
    v = w ^ t;
end

```

3.1.3.2 Always mô tả mạch tổ hợp

Mạch tổ hợp được tạo ra khi sử dụng cấu trúc **always** với *event_expression* là sự kiện thay đổi mức tín hiệu, đồng thời tất cả các trường hợp trong các phát biểu có điều kiện bên trong khối **always** đều được gán giá trị.

Cấu trúc **always** thuận lợi khi sử dụng mô tả các mạch tổ hợp phức tạp. Khi sử dụng **always** mô tả mạch tổ hợp, sử dụng phép gán blocking cho tất cả các phép gán vì phép gán blocking phù hợp với hành vi của mạch tổ hợp.

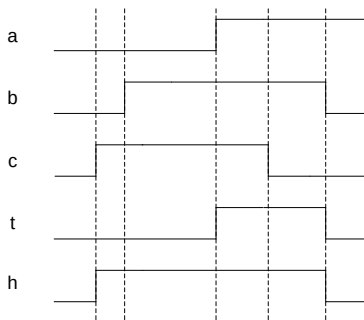
Ví dụ 3-12

```

reg t;
reg h;
always @ (*) begin
    t = a & b;
    h = t | c;
end

```

Khối **always** ở ví dụ 3-12 thực hiện tính toán từng biểu thức bên phải và gán ngay lập tức cho biến bên trái khi tính toán xong. Dạng sóng hoạt động của mạch trên được mô tả như sau nếu bỏ qua độ trễ của các cổng logic.



Hình 3-4. Dạng sóng hoạt động của mạch tổ hợp ở ví dụ 3-12

Quan sát dạng sóng ở hình trên, ta có thể thấy các ngõ ra t , h thay đổi ngay lập tức theo chức năng của mạch ngay khi ngõ vào thay đổi.

Sử dụng **always** mô tả mạch tổ hợp có thể tạo ra Latch. Latch sẽ được tạo ra khi có ít nhất một trường hợp trong các phát biểu có điều kiện bên trong khối **always** không được gán giá trị. Các trường hợp tạo Latch và ảnh hưởng của Latch trong mạch sẽ được phân tích kỹ trong các phần sau.

3.1.3.3 Always mô tả mạch tuần tự

Mạch tuần tự được tạo ra khi sử dụng cấu trúc **always** với *event_expression* là sự kiện cạnh xung clock. Xung clock dùng cho khối **always** mô tả mạch tuần tự được khai báo bằng từ khóa **posedge** khi sử dụng cho cạnh lên và **negedge** cho trường hợp sử dụng cạnh xuống. Một tín hiệu điều khiển theo cạnh được xem là xung clock khi nó không sử dụng như dữ liệu hay không tổ hợp với bất kỳ tín hiệu nào khác trong khối **always**.

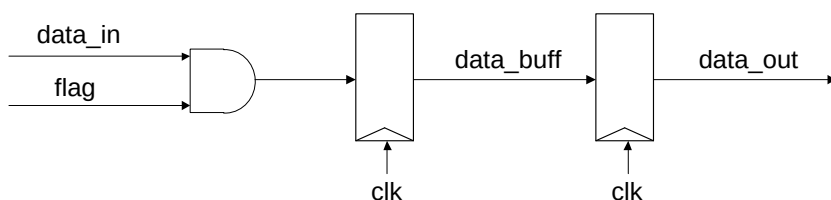
Một khối **always** không nên thiết kế với nhiều cạnh clock hay nhiều tín hiệu clock. Trong mô tả mạch tuần tự, dùng phép gán non-blocking cho tất cả các phép gán vì phép gán non-blocking phù hợp với hành vi của mạch tuần tự.

Ví dụ 3-13

```
reg data_buff;  
reg data_out;  
//  
always @ (posedge clk) begin  
    data_buff <= data_in & flag;  
    data_out <= data_buff;  
end
```

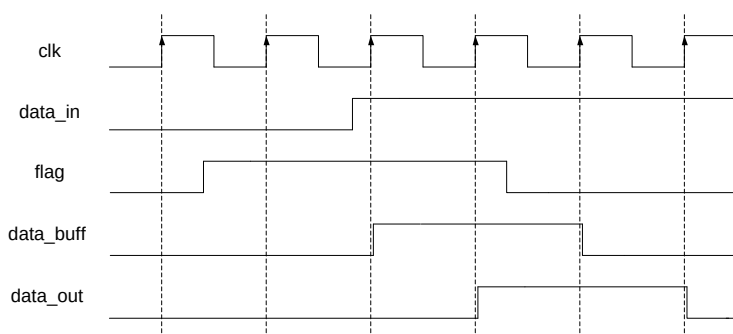
Trong ví dụ 3-13, biểu thức *data_in & flag* sẽ được tính toán nhưng chỉ được gán cho biến *data_buff* khi xuất hiện cạnh lên xung clock, và tại thời điểm này, giá trị trước đó của *data_buff* sẽ được cập nhật cho ngõ ra *data_out*.

Mạch nguyên lý của ví dụ trên như sau:



Hình 3-5. Mạch nguyên lý cho ví dụ 3-13

Dạng sóng hoạt động của mạch trên được mô tả như sau:



Hình 3-6. Dạng sóng hoạt động của mạch mô tả trong ví dụ 3-13

Ở hình 3-6, thanh ghi *data_buff* và *data_out* chỉ cập nhật giá trị mới tại cạnh lên của xung clock, mặc dù ngõ vào thay đổi tại bất kỳ thời điểm nào.

3.1.4 Reset trong mạch tuần tự

Tín hiệu reset được dùng để thiết lập giá trị cho các thanh ghi. Có hai loại reset sau:

- Reset đồng bộ: tín hiệu reset tác động thanh ghi khi có hai điều kiện là cạnh xung clock và là mức tích cực của tín hiệu reset.
- Reset bất đồng bộ: tín hiệu reset tác động thanh ghi tại hai trường hợp là cạnh tích cực xung reset hoặc cạnh xung clock với mức tích cực của tín hiệu reset.

Reset bất đồng bộ sẽ thiết lập giá trị thanh ghi ngay khi có cạnh tích cực của xung reset, do đó giá trị thanh ghi thường được thiết lập sớm hơn reset đồng bộ. Tuy nhiên, thiết kế reset bất đồng bộ rủi ro hơn

thiết kế đồng bộ, nên các tín hiệu thiết lập bất đồng bộ thường được đồng bộ với xung clock hệ thống để chống nhiễu.

Tên gọi reset đồng bộ hay bất đồng bộ tùy thuộc vào cách mô tả reset trong khối always.

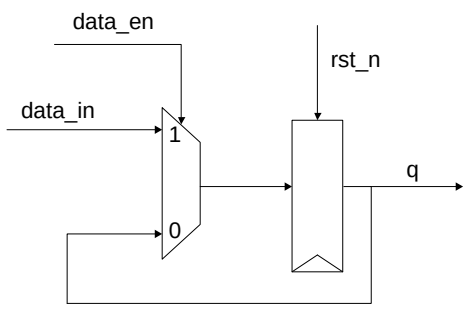
3.1.4.1 Reset đồng bộ

Trong Verilog HDL, reset đồng bộ được mô tả như một tín hiệu điều khiển bình thường nhưng nó thường được thiết kế ưu tiên cao nhất trong nhóm các tín hiệu điều khiển. Reset đồng bộ dùng để đặt lại giá trị cho một hay một nhóm thanh ghi và tác động lên thanh ghi theo mức tại cạnh tích cực của xung clock.

Ví dụ 3-14

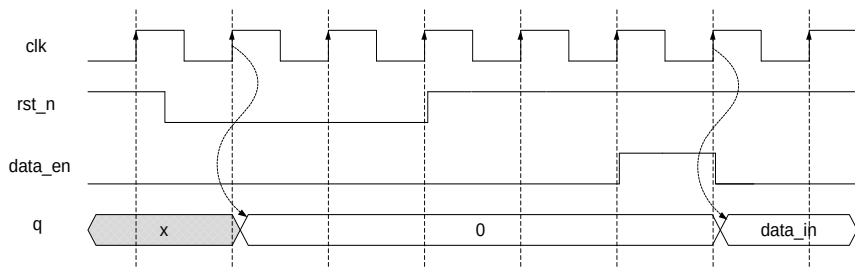
```
always @ (posedge clk) begin
  if (rst_n == 1'b0)
    q <= 1'b0;
  else begin
    if (data_en == 1'b1)
      q <= data_in;
    else
      q <= q;
  end
end
```

Trong ví dụ trên, tại mỗi cạnh lên xung clock, tín hiệu *rst_n* sẽ được tham chiếu. Khi *rst_n* bằng 0 tại cạnh lên xung clock thì Flip-Flop *q* bị reset về 0. Khi mức 0 của tín hiệu reset kéo dài thì Flip-Flop sẽ giữ giá trị reset cho đến khi tín hiệu này thôi tích cực.



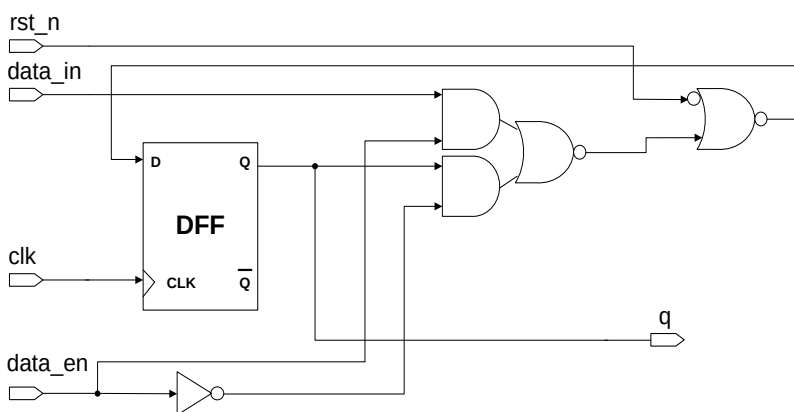
Hình 3-7. Mạch nguyên lý của ví dụ 3-14

Dạng sóng của mạch mô tả như sau:



Hình 3-8. Dạng sóng mô tả reset đồng bộ

Quan sát dạng sóng, ta thấy Flip-Flop *q* bị reset tại các cạnh xung clock *clk* mà tại đó tín hiệu *rst_n* bằng 0. Đoạn code trên khi tổng hợp sẽ cho netlist như sau:



Hình 3-9. Netlist sau tổng hợp cho ví dụ 3-14

3.1.4.2 Reset bất đồng bộ

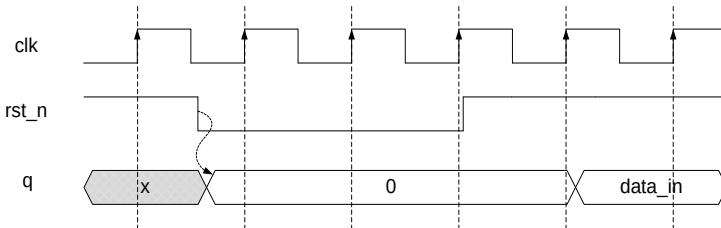
Tín hiệu reset bất đồng bộ thường chỉ dùng để khởi tạo giá trị cho Flip-Flop. Thời điểm giá trị của Flip-Flop được khởi tạo không phụ thuộc vào xung clock, chúng bị reset ngay tại cạnh tích cực của xung reset và giá trị reset của Flip-Flop được giữ cho đến khi tín hiệu reset thôi tích cực.

Trong Verilog HDL, reset bất đồng bộ được khai báo bằng các từ khóa **posedge**, **negedge** trong danh sách độ nhạy và được chỉ ra mức tích cực trong nội dung khối **always**.

Ví dụ 3-15

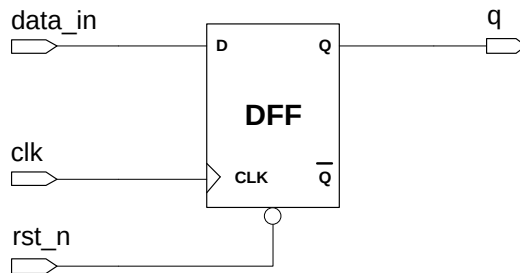
```
always @ (posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 1'b0;
    else
        q <= data_in;
end
```

Ở ví dụ 3-15, tín hiệu reset bất đồng bộ *rst_n* được khai báo bằng từ khóa **negedge** và mức tích cực chỉ trong khối **always** là mức thấp. Khi cạnh tích cực của reset xảy ra, Flip-Flop bị reset ngay lập tức và tiếp tục giữ giá trị reset khi mức 0 của tín hiệu *rst_n* vẫn còn. Tác động của mức 0 tín hiệu *rst_n* ảnh hưởng tại các cạnh lên xung clock *clk*. Dạng sóng của mạch mô tả như sau:



Hình 3-10. Dạng sóng mô tả reset bất đồng bộ

Nếu reset bất đồng bộ tác động cạnh xuống, khai báo bằng từ khóa **negedge**, thì mức tích cực của nó trong khối **always** phải là mức thấp. Ngược lại, nếu khai báo *rst_n* là **posedge** thì mức tích cực phải là 1. Nếu không thực hiện đúng, mạch sẽ không tổng hợp được. Đoạn code ví dụ 3-15 khi tổng hợp sẽ cho netlist như sau:



Hình 3-11. Netlist sau tổng hợp của ví dụ 3-15

3.2 Phép gán assign

3.2.1 Cú pháp assign

Một phép gán liên tục bắt đầu bằng từ khóa **assign** và cấu trúc của một phép gán liên tục hay phép gán **assign** được mô tả như sau:

```
assign net = expression;
```

Mô tả:

- **assign**: từ khóa hàm gán liên tục
- **net**: biến được gán, thường khai báo dưới các kiểu sau:
 - o **net** (vector hoặc scalar)
 - o bit-select của một vector kiểu **net**
 - o part-select của một vector kiểu **net**
 - o kết hợp các kiểu phía trên
- *expression* : là một biểu thức hoặc một **function**.

Phép gán liên tục là hàm cơ bản được sử dụng để lái giá trị cho các biến **net**. Phép gán liên tục là hàm tích cực, tức là biến bên trái sẽ thay đổi ngay khi biểu thức bên phải thay đổi giá trị.

Ví dụ 3-16

Các kiểu biến trong phép gán **assign**

```
// net type
assign out = a & b;
// vector net type
assign addr[15:0] = data[15:0] | en[15:0];
// indexed part select net type
assign addr[8+:16] = data[15:8] | en[7:0];
// net type and vector net type
assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in;
```

Verilog HDL cho phép khai báo một tín hiệu kiểu **net** và đồng thời trực tiếp gán cho chúng. Ở ví dụ sau, hai cách viết là tương đương:

Ví dụ 3-17

```
// standard
wire out;
assign out = in1 & in2;

// shorthand
wire out = in1 & in2;
```

3.2.2 Cách sử dụng assign

Verilog HDL cung cấp nhiều cách thức để mô tả mạch tổ hợp như: **always**, **assign**, **function**,... Tùy thuộc vào quy mô từng mạch mà mỗi cách thức có ưu nhược điểm khác nhau. Hàm gán **assign** thường rất thuận tiện để mô tả mạch quy mô nhỏ ở mức cổng. Cấu trúc **always** thường được dùng để mô tả mạch phức tạp hơn. Bảng sau sẽ so sánh ưu nhược điểm khi sử dụng **assign** và **always** để mô tả mạch tổ hợp.

Bảng 3-1. So sánh assign và always

	Gán assign	Cấu trúc always
Ưu điểm	• Không tạo chốt • Phân biệt rõ ràng mạch tổ hợp và mạch tuần tự	• Có thể mô tả được ở mức hành vi, mã ngắn gọn, dễ kiểm tra và gỡ lỗi
Nhược điểm	• Không báo lỗi cú pháp khi không khai báo biến một bit • Chỉ có một ngõ ra. Muốn có nhiều ngõ ra phải gom nhóm tín hiệu	• Có thể tạo chốt • Danh sách độ nhạy thiếu có thể gây sai lệch giữa mô phỏng RTL và mô phỏng mức cổng sau khi tổng hợp

Khi có nhiều ngõ ra, ta phải gom chúng lại thành một bus trước khi dùng chúng với hàm **assign**.

Ví dụ 3-18

```
wire [1:0] bus_1;
wire [3:0] bus_2;
//
assign bus_1[1:0] = num[1:0];
assign bus_2[2:0] = in[2:0];
```


Tương đương với cách viết sau:

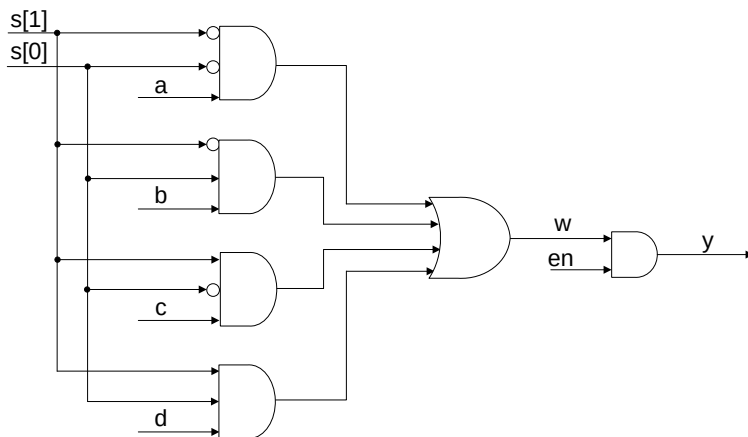
```
wire [1:0] bus_1;  
wire [3:0] bus_2;  
//  
assign {bus_1[1:0], bus_2[2:0]} = {num[1:0], in[2:0]};
```

Ví dụ 3-19

Viết bộ MUX 4-1 bằng phép gán liên tục

```
module MUX41 (a, b, c, d, en, s, y);  
  input a, b, c, d, en;  
  input [1:0] s;  
  //  
  output wire y;  
  wire w;  
  //  
  assign w = (a & ~s[1] & ~s[0]) |  
             (b & ~s[1] & s[0]) |  
             (c & s[1] & ~s[0]) |  
             (d & s[1] & s[0]);  
  assign y = w & en;  
endmodule
```

Đoạn code trên mô tả một mạch MUX 4-1 có ngõ ra được cho phép hay không bằng tín hiệu *en*. Các điều kiện chọn của MUX là *s[1:0]* và hoạt động của MUX ví dụ 3-19 được mô tả chi tiết trong bảng 3-2. Mạch nguyên lý của MUX 4-1 ở ví dụ 3-19 như sau:

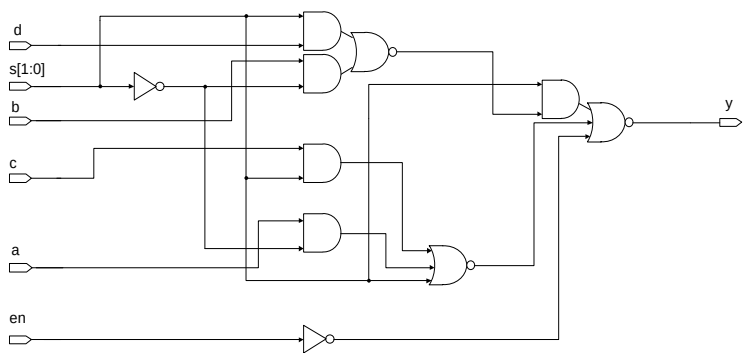


Hình 3-12. Mạch nguyên lý của ví dụ 3-19

Hoạt động của mạch trên được mô tả trong bảng sự thật như sau:

Bảng 3-2. Bảng sự thật của MUX 4-1 ở ví dụ 3-19

en	s[1]	s[0]	w	y
0	x	x	x	0
1	0	0	a	a
1	0	1	b	b
1	1	0	c	c
1	1	1	d	d



Hình 3-13. Netlist sau tổng hợp của ví dụ 3-19

Ví dụ 3-20 cho thấy kiểu dữ liệu của các toán hạng và biến có thể là một trong các kiểu sau: **integer**, **real**, **net**, **reg**, kiểu mảng số, mảng tín hiệu, mảng bộ nhớ hoặc một vài kiểu khác.

Ví dụ 3-20

```
// integer operands
integer sum, unit;
assign sum = unit * 16;
// real operands
real a, b, c;
assign c = a + b;
// part-select operands
wire [15:0] in1, in2;
wire [7:0] out;
assign out[7:0] = in1[7:0] | in2[7:0];
end
```

Các toán tử được sử dụng trong hàm gán **assign** bao gồm các phép tính được thống kê theo mức ưu tiên từ cao xuống thấp như sau:

Bảng 3-3. Ví dụ gán với các toán tử

Toán tử	Ví dụ
! ~	assign net_a = ~net_b ;
* / %	assign result[7:0] = data1[3:0] * data2[3:0];
+ -	assign sum[4:0] = data1[3:0] + data2[3:0];
<< >>	assign result[4:0] = result[4:0] >> 2;
< <= > >=	assign net_a = (data1[1:0] >= data2[1:0]);
== != === !==	assign net_a = (data1[1:0] != data2[1:0]);
&	assign net_a = net_b & net_c;
^ ^^	assign net_a = net_b ^ net_c;
	assign net_a = net_b net_c;
&&	assign net_a = net_b && net_c;
	assign net_a = net_b net_c;
?:	assign result = (a[1:0] > b[1:0]) ? in1 : in1;

Một số toán tử thực hiện trên vector có thể viết gọn để giảm kích thước code.

Ví dụ 3-21

```
// Usage of a reduction operator
reg [4:0] a;
reg out;
assign out = & a;
assign out = | a;
assign out = ^ a;
```

Tương đương với cách viết:

```
// standard
reg [4:0] a;
reg out;
assign out = a[0] & a[1] & a[2] & a[3] & a[4];
assign out = a[0] | a[1] | a[2] | a[3] | a[4];
assign out = a[0] ^ a[1] ^ a[2] ^ a[3] ^ a[4];
```

3.3 Phát biểu if

3.3.1 Cú pháp phát biểu if

```
if (expression_1) <statement_1>  
else if (expression_2) <statement_2>  
else <statement_3>
```

Mô tả:

Ba dòng cú pháp trong phát biểu if được thực hiện theo thứ tự ưu tiên từ trên xuống dưới

- *expression*: biểu thức điều kiện sẽ trả về một giá trị logic là *true* hoặc *false*
- *statement*: phát biểu
- Dòng cú pháp 1: nếu giá trị trả về của *expression_1* là *true* thì *statement_1* xảy ra, không xét đến các dòng cú pháp tiếp theo; nếu giá trị trả về của *expression_1* là *false* thì thực hiện dòng cú pháp tiếp theo
- Dòng cú pháp 2: nếu giá trị trả về của *expression_2* là *true* thì *statement_2* xảy ra, không xét đến các dòng cú pháp tiếp theo; nếu giá trị trả về của *expression_1* là *false* thì thực hiện dòng cú pháp tiếp theo; trong phát biểu **if**, dòng cú pháp 2 có thể không xuất hiện, hoặc xuất hiện một lần, hoặc được lặp lại nhiều lần
- Dòng cú pháp 3: khi tất cả các *expression* ở trên đều không đúng thì *statement_3* xảy ra; trong cú pháp của phát biểu **if**, vế này có thể không xuất hiện hoặc xuất hiện một lần

Ví dụ 3-22

Trường hợp không có **else if** và **else**

```
if (number[2:0] > 3) result[7:0] = IN1[7:0];
```

Ví dụ 3-23

Trường hợp có **else**, không có **else if**

```
if (signal[1:0] == 2'b01) result[3:0] = IN1[3:0];  
else result[3:0] = IN2[3:0];
```

Ví dụ 3-24

Trường hợp đầy đủ có cả **else if** và **else**

```
if (select[0])      result[31:0] = IN1[31:0];
else if (select[1]) result[31:0] = IN2[31:0];
else if (select[2]) result[31:0] = IN3[31:0];
else if (select[3]) result[31:0] = IN4[31:0];
else               result[31:0] = IN5[31:0];
```

3.3.2 Cách sử dụng phát biểu if

Thông thường, phát biểu **if** được sử dụng trong khối **always** để tạo ra mạch tuần tự, mạch tổ hợp hay mạch Latch. Ngoài ra, phát biểu **if** còn được sử dụng trong **function** và **generate**. Phần này tập trung trình bày cách sử dụng phát biểu **if** với cấu trúc **always**, các cách sử dụng phát biểu **if** khác sẽ được trình bày trong các phần **function** và **generate**.

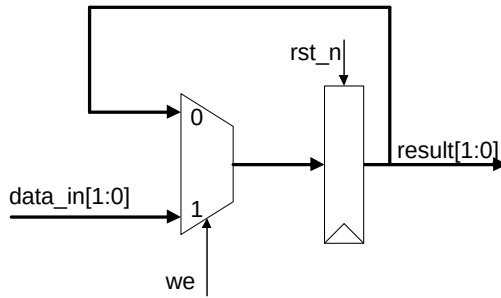
3.3.2.1 Phát biểu if tạo mạch tuần tự

Như đã trình bày trong phần **always**, sử dụng cấu trúc **always** với *event_expression* là sự kiện cạnh xung clock sẽ tạo nên mạch tuần tự. Thêm vào đó, đối với các phát biểu bên trong khối **always**, các phép gán dạng non-blocking có đặc tính phù hợp cho mô tả mạch tuần tự. Phần này sẽ trình bày các ví dụ mô tả mạch tuần tự có sử dụng phát biểu **if**.

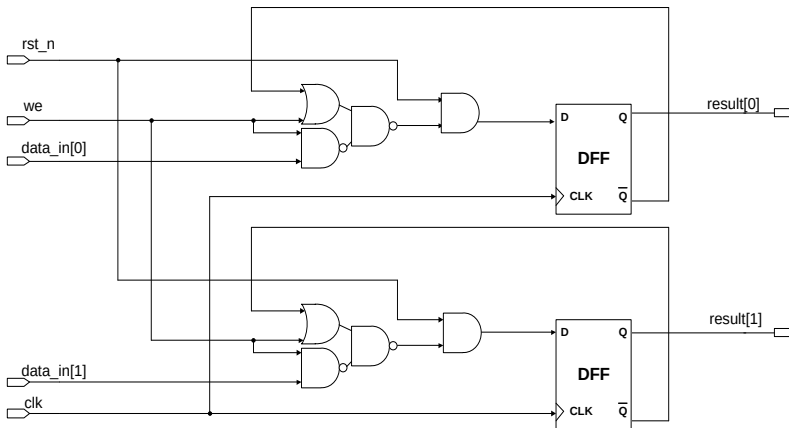
Ví dụ 3-25

```
always @(posedge clk) begin
    if (!rst_n) result[1:0] <= 2'b00;
    else if (we) result[1:0] <= data_in[1:0];
end
```

Về nguyên lý, mạch được tạo ra gồm một mạch tổ hợp và một thanh ghi. Mạch tổ hợp là một bộ MUX 2-1, tín hiệu chọn MUX là *we*. Tín hiệu chọn *we* sẽ chọn giữa *data_in[1:0]* và giá trị hiện tại của thanh ghi *result_in[1:0]*. Ngõ ra của mạch tổ hợp này được đưa vào thanh ghi *result[1:0]*. Tín hiệu reset đồng bộ *rst_n* tích cực mức thấp, giá trị reset của thanh ghi sẽ là 2'b00. Mạch hoạt động ở cạnh lên xung *clk*.



Hình 3-14. Mạch nguyên lý của ví dụ 3-25



Hình 3-15. Netlist sau tổng hợp của ví dụ 3-25

Ví dụ 3-26

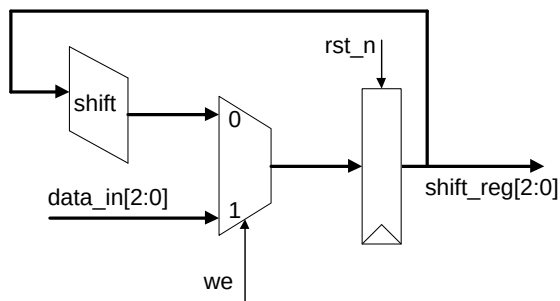
```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) shift_reg[2:0] <= 3'b000;
    else if (we) shift_reg[2:0] <= data_in[2:0];
    else begin
        shift_reg[0] <= 1'b0;
        shift_reg[1] <= shift_reg[0];
        shift_reg[2] <= shift_reg[1];
    end
end

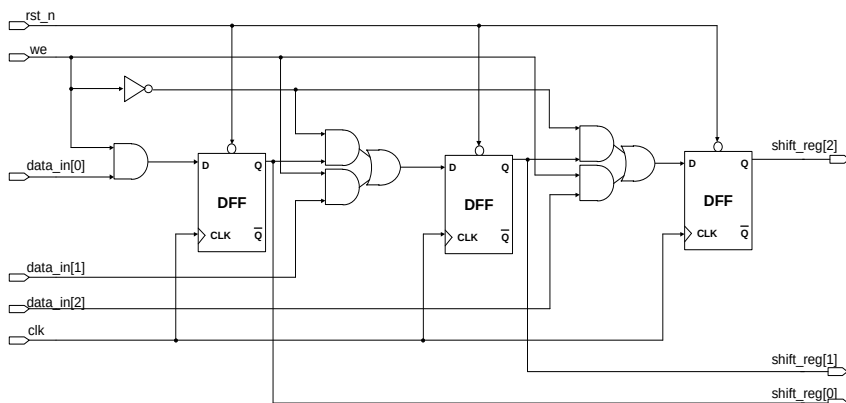
```

Trong ví dụ này, mạch tổ hợp kết hợp với thanh ghi được tạo ra. Mạch tổ hợp gồm một khối shift và một bộ MUX 2-1, tín hiệu chọn mux là we. Tín hiệu chọn we sẽ chọn giữa *data_in[2:0]* và giá trị thanh ghi *shift_reg[2:0]* sau khi dịch. Ngõ ra của mạch tổ hợp này

được đưa vào thanh ghi *shift_reg[2:0]*. Tín hiệu reset bất đồng bộ *rst_n* tích cực mức thấp, giá trị reset của thanh ghi sẽ là 3'b000. Mạch hoạt động ở cạnh lên xung *clk*.



Hình 3-16. Mạch nguyên lý của ví dụ 3-26



Hình 3-17. Netlist sau tổng hợp của ví dụ 3-26

Ví dụ 3-27

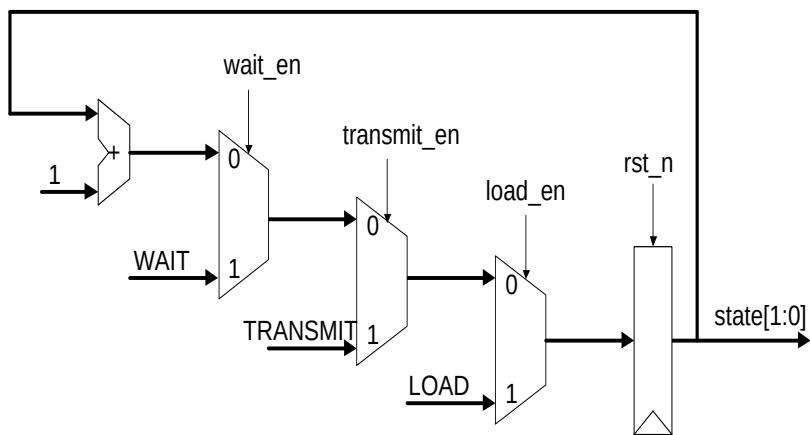
```

parameter LOAD      = 2'b01;
parameter TRANSMIT  = 2'b10;
parameter WAIT      = 2'b11;

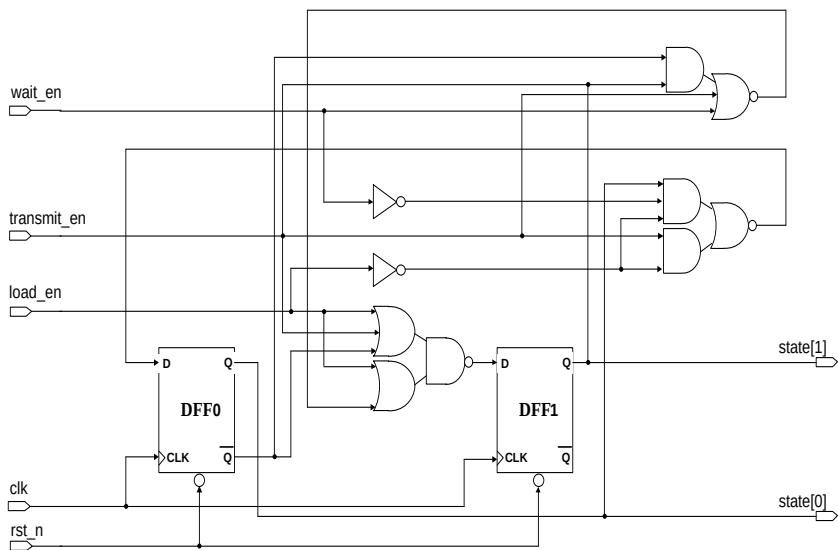
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        state[1:0] <= 2'b00;
    else if(load_en)
        state[1:0] <= LOAD;
    else if(transmit_en)
        state[1:0] <= TRANSMIT;
    else if(wait_en)
        state[1:0] <= WAIT;
    else
        state[1:0] <= state[1:0] +
1'b1;
end

```

Một thanh ghi có giá trị thay đổi theo điều kiện ngõ vào được tạo ra. Các giá trị gán cho thanh ghi *state[1:0]* đã được khai báo trước dùng khai báo parameter. Điều kiện ngõ vào là *load_en*, *transmit_en* và *wait_en*. Nếu tất cả các điều kiện ngõ vào đều không tích cực, giá trị thanh ghi *state[1:0]* sẽ tự động tăng 1. Tín hiệu reset bất đồng bộ *rst_n* tích cực mức thấp. Mạch hoạt động ở cạnh lên xung *clk*.



Hình 3-18. Mạch nguyên lý của ví dụ 3-27



Hình 3-19. Netlist sau tổng hợp của ví dụ 3-27

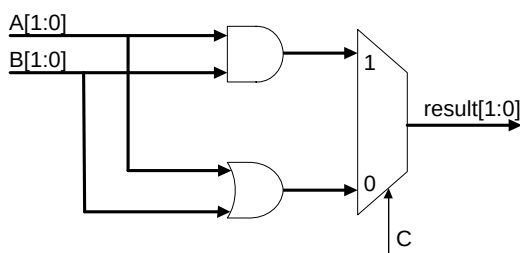
3.3.2.2 Phát biểu if trong mạch tổ hợp

Khác với mô tả mạch tuần tự, mạch tổ hợp sẽ được tạo ra khi sử dụng cấu trúc **always** với *event_expression* là sự kiện thay đổi mức tín hiệu, đồng thời tất cả các trường hợp trong các phát biểu có điều kiện bên trong khối **always** đều được gán giá trị. Ngoài ra, các phép gán sử dụng trong trường hợp này là các phép gán dạng blocking. Phần này sẽ trình bày các ví dụ mô tả mạch tổ hợp có sử dụng phát biểu **if**.

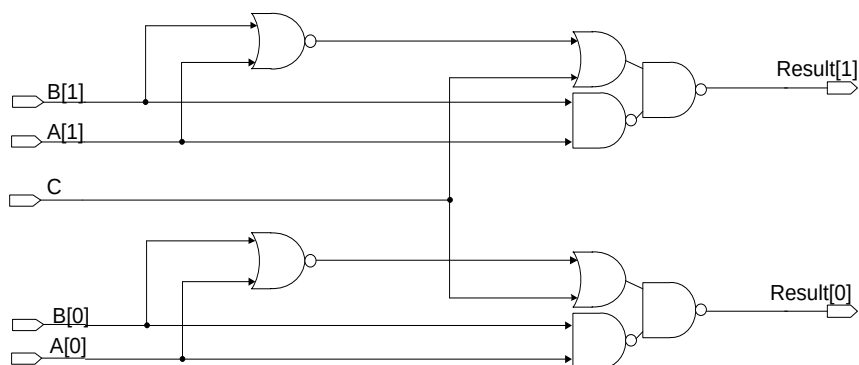
Ví dụ 3-28

```
always @(C or A[1:0] or B[1:0]) begin  
  if (C) result[1:0] = A[1:0] & B[1:0];  
  else   result[1:0] = A[1:0] | B[1:0];  
end
```

Mạch tạo ra sẽ có một bộ MUX 2-1 với ngõ vào 1 là $A \& B$, ngõ vào 2 là $A | B$, tín hiệu chọn là C.



Hình 3-20. Mạch nguyên lý của ví dụ 3-28



Hình 3-21. Netlist sau tổng hợp của ví dụ 3-28

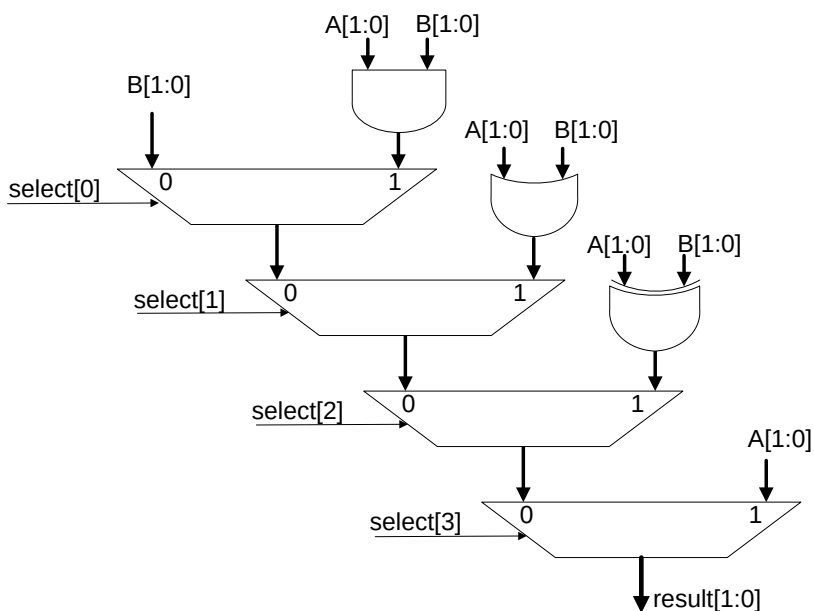
Ví dụ 3-29

```

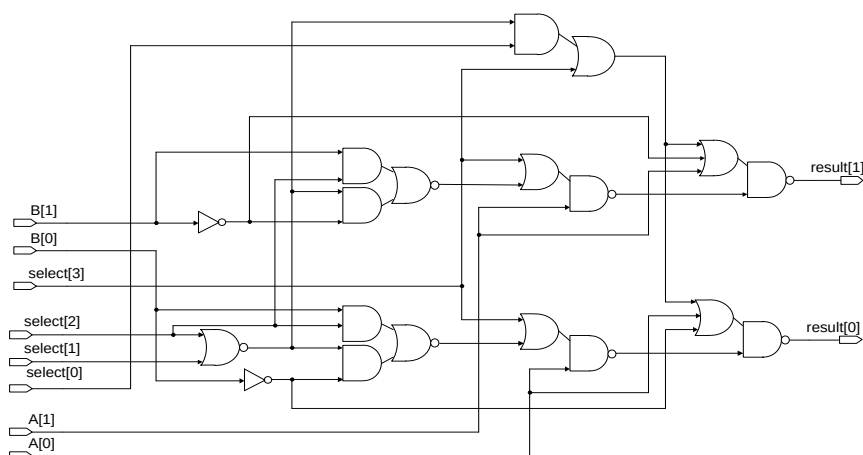
always @(select[3:0] or A[1:0] or B[1:0]) begin
    if (select[3]) result[1:0] = A[1:0];
    else if (select[2]) result[1:0] = A[1:0] ^ B[1:0];
    else if (select[1]) result[1:0] = A[1:0] | B[1:0];
    else if (select[0]) result[1:0] = A[1:0] & B[1:0];
    else result[1:0] = B[1:0];
end

```

select[3], *select*[2], *select*[1], *select*[0] sẽ là tín hiệu chọn cho bốn bộ MUX 2-1 được sắp xếp theo thứ tự tương ứng. Ngõ vào của các bộ MUX này là những biến đổi logic khác nhau giữa A[1:0] và B[1:0].



Hình 3-22. Mạch nguyên lý của ví dụ 3-29



Hình 3-23. Netlist sau tổng hợp của ví dụ 3-29

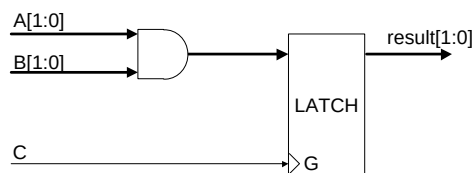
3.3.2.3 Phát biểu if tạo Latch

Cũng giống như mô tả tạo mạch tổ hợp, phát biểu **if** tạo Latch cũng sử dụng cấu trúc **always** với *event_expression* là sự kiện thay đổi mức tín hiệu, các phép gán bên trong khối **always** có dạng blocking. Khác nhau ở chỗ, Latch sẽ được tạo ra khi có ít nhất một trường hợp trong các phát biểu có điều kiện bên trong khối **always** không được gán giá trị. Phần này sẽ trình bày các ví dụ mô tả mạch tạo Latch có sử dụng phát biểu **if**.

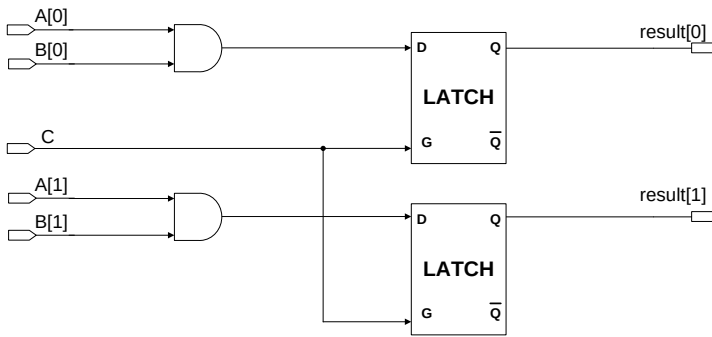
Ví dụ 3-30

```
always @(C or A[1:0] or B[1:0]) begin
    if (C) result[1:0] = A[1:0] & B[1:0];
end
```

Một mạch tổ hợp nhỏ kết hợp với Latch được tạo ra. Mạch tổ hợp là mạch AND giữa $A[1:0]$ và $B[1:0]$. Tín hiệu C kết nối vào cổng G , là cổng đóng mở của các Latch.



Hình 3-24. Mạch nguyên lý của ví dụ 3-30



Hình 3-25. Netlist sau tổng hợp của ví dụ 3-30

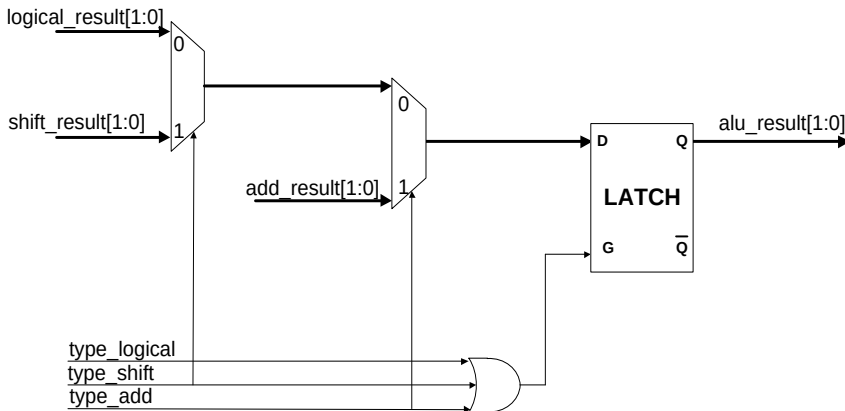
Ví dụ 3-31

```

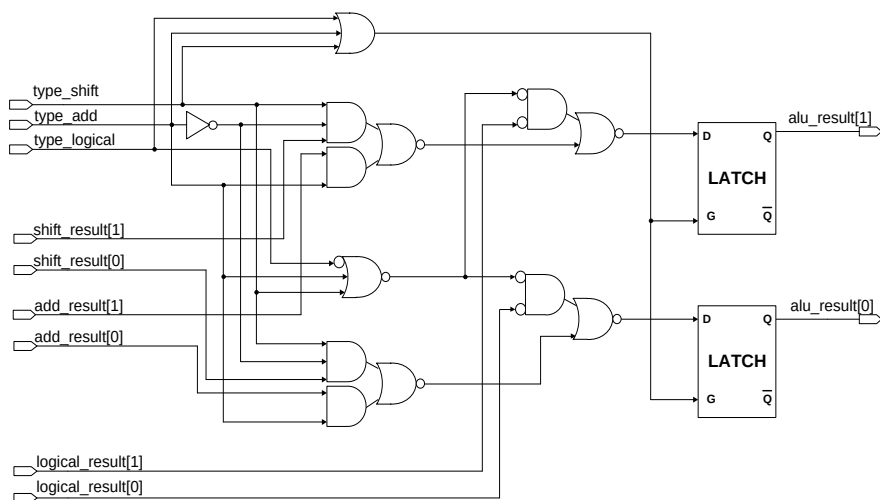
always @(*) begin
  if (type_add)
    alu_result[1:0] = add_result[1:0];
  else if (type_shift)
    alu_result[1:0] = shift_result[1:0];
  else if (type_logical)
    alu_result[1:0] = logical_result[1:0];
end

```

Trong ví dụ này, mạch sẽ gồm Latch kết hợp với hai bộ MUX 2-1 để chọn lựa các ngõ vào theo thứ tự ưu tiên như sau: *add_result[1:0]*, *shift_result[1:0]*, *logical_result[1:0]*. Tín hiệu đóng mở cổng được lấy từ tổ hợp OR của ba tín hiệu điều kiện *type_shift*, *type_add*, *type_logical*.



Hình 3-26. Mạch nguyên lý của ví dụ 3-31



Hình 3-27. Netlist sau tổng hợp của ví dụ 3-31

3.4 Phát biểu case

Phát biểu case¹ trong Verilog HDL cho phép người sử dụng kiểm tra nhiều điều kiện cùng lúc. Verilog hỗ trợ ba loại phát biểu case khác nhau là **case**, **casex** và **casez**.

3.4.1 Cú pháp phát biểu case

Cú pháp **case**:

```
case (expression)
  case_item_0 <,case_item_1, ...>: <statement>
  case_item_2 <,case_item_3, ...>: <statement>
  default: statement
endcase
```

Cú pháp **casex**:

```
casex (expression)
  case_item_0 <,case_item_1, ...>: <statement>
  case_item_2 <,case_item_3, ...>: <statement>
  default: statement
endcase
```

¹ Trong tài liệu này, chúng tôi sử dụng thuật ngữ “phát biểu case” để thay thế cho cả **case**, **casex** và **casez**. Người đọc cần phân biệt case với phát biểu case.

Cú pháp **casez**:

```
casez (expression)
    case_item_0 <,case_item_1, ...>: <statement>
    case_item_2 <,case_item_3, ...>: <statement>
    default: statement
endcase
```

Mô tả:

Cả ba loại của phát biểu case đều có chung một cú pháp, chỉ có một điểm khác duy nhất là từ khóa **case**, **casex** hay **casez**.

- *expression* là một diễn tả được dùng để làm điều kiện ngõ vào.
- *case_item_0*, *case_item_1*, *case_item_2*, *case_item_3*, ... được gọi là các *case_item* trong danh sách điều kiện. Chúng chính là các giá trị có thể xảy ra của *expression*.
- *statement* là phát biểu dùng để mô tả chức năng của mạch khi *case_item* được thỏa. Một hoặc nhiều *case_item* cùng với *statement* tạo thành một nhánh của phát biểu case. Phát biểu case có thể có một hoặc nhiều nhánh khác nhau. Trong mỗi nhánh, *statement* có thể có hoặc không có tùy thuộc vào chức năng của mạch thiết kế.
- **default** là giá trị mặc định, bao gồm tất cả các giá trị của *expression* chưa được liệt kê trong danh sách điều kiện. Có thể có hoặc không có **default**. Tuy nhiên, nếu đã có **default** thì *statement* ở nhánh này không được để khuyết.
- Trong phát biểu case, thứ tự thực hiện từ trên xuống. Theo thứ tự này, nếu *case_item* xuất hiện trước và được thỏa mãn thì *statement* ở đó được thực hiện mà không quan tâm đến các trường hợp khác.

Ví dụ 3-32

Đoạn code bên dưới là ví dụ cho **case** có hai *case_item*, hai nhánh và có **default**.

```
case (sel[1:0])
    2'b00: dout = a;
    2'b01: dout = b;
    default: dout = c;
endcase
```

Ví dụ 3-33

Đây là ví dụ cho trường hợp **case** có ba *case_item*, hai nhánh và có **default**. Nhánh đầu tiên chỉ có một *case_item*, nhánh thứ hai có hai *case_item*.

```
case (sel[1:0])
  2'b00: dout = a & b;
  2'b01, 2'b10: dout = b | c;
  default: dout = c;
endcase
```

Ví dụ 3-34

Ví dụ này minh họa cho trường hợp **case** có bốn *case_item*, bốn nhánh và không có **default**.

```
case (pc_sel[1:0])
  2'b00: pc_next[15:0] = pc[15:0] + 1'b1;;
  2'b01: pc_next[15:0] = {stack_hi[7:0],
                        stack_lo[7:0]};
  2'b10: pc_next[15:0] = branch_target[15:0]
  2'b11: pc_next[15:0] = pc[15:0];
endcase
```

Ví dụ 3-35

Đoạn code minh họa cho **case** có bảy *case_item*, ba nhánh và có **default**.

```
case (contrl[2:0])
  3'b001: pri = req[0];
  3'b010, 3'b011: pri = req[1];
  3'b100, 3'b101, 3'b110, 3'b111: pri = req[2];
  default: pri = 0;
endcase
```

Ví dụ 3-36

Đoạn code trong ví dụ 3-35 được viết lại với **casex** có ba *case_item*, ba nhánh và có **default**.

```
casex (contrl[2:0])
  3'b001: pri = req[0];
  3'b01x: pri = req[1];
  3'b1xx: pri = req[2];
  default: pri = 0;
endcase
```

Ví dụ 3-37

Đoạn code trong ví dụ 3-35 được viết lại với **casez** có ba *case_item*, ba nhánh và có **default**.

```
casez (sel)
  2'b00: dout = a;
  2'b01: dout = b;
  2'b1z: dout = c;
  default: dout = 0;
endcase
```

Ví dụ 3-38

Ví dụ này minh họa cho trường hợp **casez** có bốn *case_item*, bốn nhánh, không có **default** và *statement* bị khuyết ở một nhánh.

```
casez ({update, incr, dec})
  3'b1zz: buffer[7:0] <= new_data[7:0];
  3'b01z: buffer[7:0] <= buffer[7:0] + 1'b1;
  3'b001: buffer[7:0] <= buffer[7:0] - 1'b1;
  3'b000: ; //no statement
endcase
```

Trong ví dụ này, *statement* ở nhánh cuối cùng được để trống nên nếu điều kiện ở nhánh này được thỏa thì giá trị tín hiệu ngõ ra sẽ không thay đổi.

3.4.2 Cách sử dụng phát biểu case

3.4.2.1 Sử dụng don't care trong phát biểu case

Trong Verilog HDL, *don't care* là một thuật ngữ dùng để chỉ giá trị một tín hiệu khi người thiết kế không muốn quan tâm đến giá trị của tín hiệu này.

Trong **case**, *x* và *z* (hay *?*) không được coi là *don't care* mà chúng được coi là các giá trị xác định tương tự như 0 và 1.

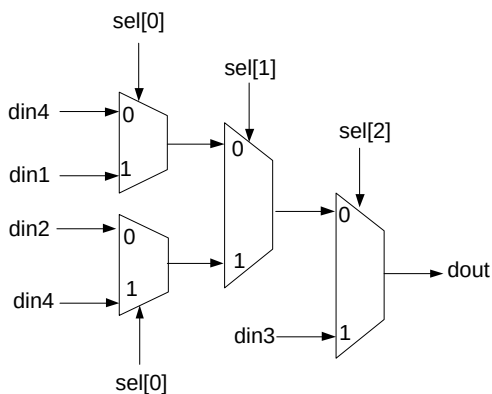
Ví dụ 3-39

Giả sử để mô tả chức năng cho mạch có sơ đồ nguyên lý như hình 3-28, đoạn RTL code được viết như bên dưới.


```

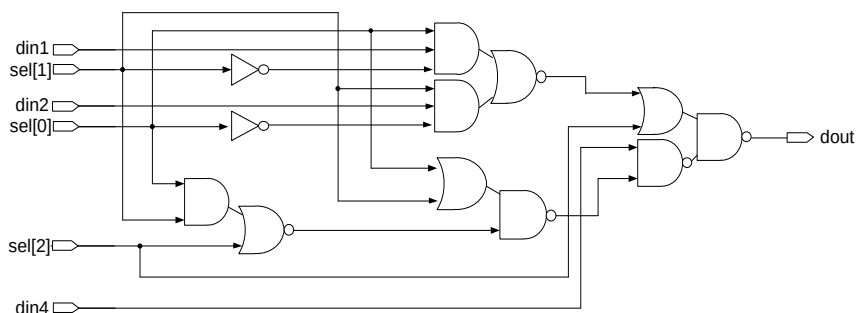
always @ (*) begin
case (sel[2:0])
  3'b001: dout = din1;
  3'b010: dout = din2;
  3'b1??: dout = din3;
  default: dout = din4;
endcase
end

```



Hình 3-28. Mạch nguyên lý của ví dụ 3-39

Ý tưởng của đoạn code trên là chỉ cần *sel[2]* có giá trị bằng 1 thì gán *dout* bằng *din3*. Tuy nhiên, do **case** chỉ hiểu ? là z, nên 3'b1?? được hiểu là 3'b1zz. Nếu *sel[2:0]* có giá trị là 3'b100, 3'b101, 3'b110, 3'b111, 3'b1x0, 3'b1x1, 3'b10x, 3'b11x, 3'b1z0, 3'b1z1, 3'b10z, 3'b11z hay 3'b1xx thì nó được xếp vào trường hợp **default**. Điều này làm cho netlist tổng hợp được không đúng theo mong muốn của người thiết kế như trong hình 3-29.

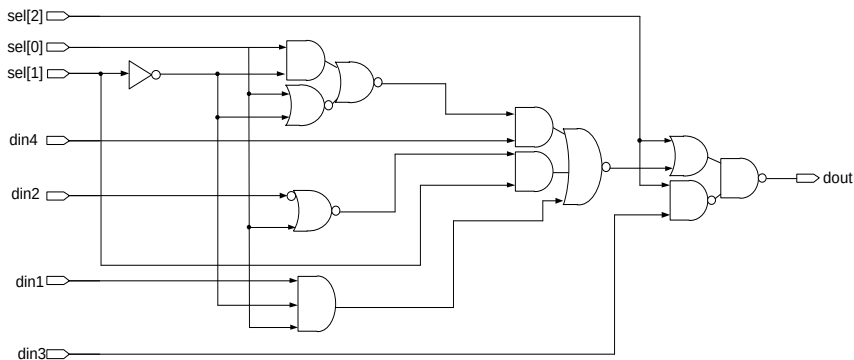


Hình 3-29. Netlist sau tổng hợp của ví dụ 3-32

Để có được mạch tổng hợp như mong muốn, ta phải viết lại đoạn code trên bằng cách liệt kê tất cả các trường hợp có thể xảy ra của *sel[2:0]*, như đoạn code dưới đây.

Ví dụ 3-40

```
always @ (*) begin
  case (sel[2:0])
    3'b001: dout = din1;
    3'b010: dout = din2;
    3'b100,3'b101,3'b110,3'b111: dout = din3;
    default: dout = din4;
  endcase
end
```



Hình 3-30. Netlist mong muốn của ví dụ 3-33

Khác với **case**, trong **casex**, các giá trị *x*, *z* và *?* được hiểu là *don't care*. Hay nói cách khác, **casex** có hỗ trợ *don't care*.

Đoạn code trong ví dụ 3-33 được viết lại như ví dụ bên dưới.

Ví dụ 3-41

```
always @ (*) begin
  casex (sel[2:0])
    3'b001: dout = din1;
    3'b010: dout = din2;
    3'b1xx: dout = din3; //Or 3'b1?? Or 3'b1zz
    default: dout = din4;
  endcase
end
```

Ở đây, điều kiện *3'b1xx* dùng để thay thế cho tất cả các giá trị của *sel* có *sel[2]* là 1. Do đó, ta không cần phải liệt kê tất cả các trường hợp có thể xảy ra như trong phát biểu **case**. Điều này giúp cho code trở nên ngắn gọn và dễ đọc. Đoạn code này cho ta cùng một netlist tổng hợp như trong hình 3-30.

Tương tự với **casex**, **casez** cũng hỗ trợ *don't care*. Tuy nhiên, chỉ có *z* (hay ?) được coi là *don't care*. Đoạn code trong ví dụ 3-33 được viết lại với **casez** cho cùng một netlist sau tổng hợp như hình 3-30.

Ví dụ 3-42

```
always @ (*) begin
    casez (sel[2:0])
        3'b001: dout = din1;
        3'b010: dout = din2;
        3'b1zz: dout = din3; //Or 3'b1??
        default: dout = din4;
    endcase
end
```

Điều kiện *3'b1zz* trong ví dụ trên đại diện cho tất cả các giá trị của *sel* có *sel[2]* là 1. Do đó, điều này giúp cho code được ngắn gọn tương tự như sử dụng **casex**.

Tuy nhiên, nếu đoạn code trên được viết lại như sau như ví dụ 3-43 thì sẽ có vấn đề xảy ra.

Ví dụ 3-43

```
always @ (*) begin
    casez (sel)
        3'b001: dout = din1;
        3'b010: dout = din2;
        3'b1xx: dout = din3;
        default: dout = din4;
    endcase
end
```

Trong trường hợp này, *3'b1xx* chỉ được hiểu là một trường hợp cụ thể là *3'b1xx* mà không bao hàm thêm bất kỳ giá trị nào khác nữa. Giả sử, nếu điều kiện ngõ vào là *3'b100* thì nó sẽ được xếp vào trường hợp **default**, vì giá trị này không được tìm thấy trong danh sách điều kiện. Điều đó có nghĩa là chức năng của mạch không còn đúng như ý tưởng

thiết kế ban đầu. Do đó, khi sử dụng **casez** chúng ta phải lưu ý giá trị *x* không được coi là *don't care*.

3.4.2.2 Phát biểu case trong mạch tuần tự

Tương tự như phát biểu **if**, phát biểu **case** cũng có thể được dùng để mô tả mạch tuần tự. Phần này sẽ trình bày các ví dụ mô tả mạch tuần tự bằng cách sử dụng phát biểu **case**.

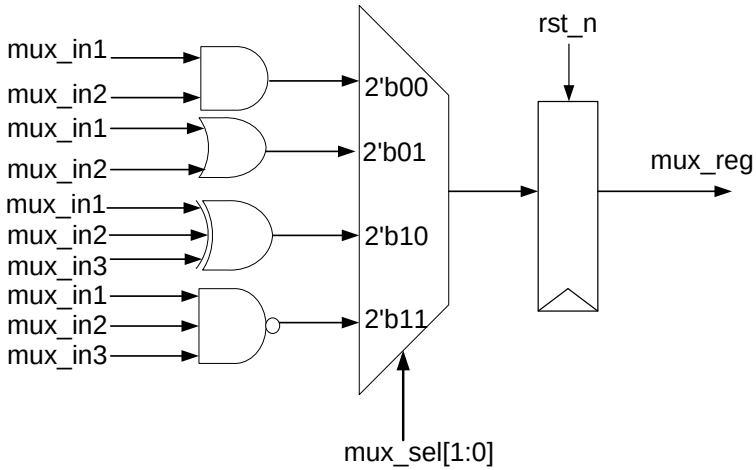
Ví dụ 3-44

```
always @ (posedge clk) begin
    if (!rst_n) mux_reg <= 1'b0;
    else begin
        case (mux_sel[1:0])
            2'b00: mux_reg <= in1 & in2;
            2'b01: mux_reg <= in1 | in2;
            2'b10: mux_reg <= in1 ^ in2 ^ in3;
            2'b11: mux_reg <= ~(in1 & in2 & in3);
        endcase
    end
end
```

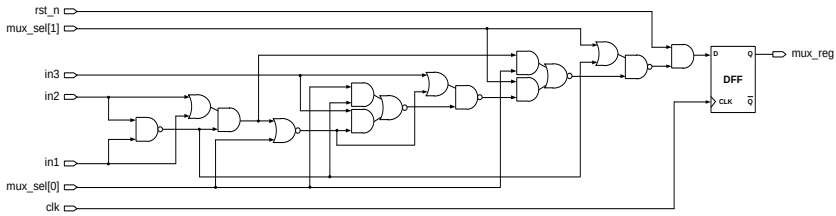
Theo mô tả của đoạn code trên, mạch hoạt động theo cạnh lên xung clock *clk*. Khi tín hiệu reset đồng bộ ở mức thấp, giá trị của *mux_reg* được xóa về 0. Ngược lại, tại mỗi cạnh lên xung clock, mạch hoạt động như một mạch lựa chọn theo nguyên tắc sau:

- Khi tín hiệu điều khiển *mux_sel[1:0]* bằng 2'b00 thì giá trị của biểu thức *in1 & in2* sẽ cập nhật cho *mux_reg*.
- Khi *mux_sel[1:0]* bằng 2'b01 thì giá trị sẽ cập nhật cho *mux_reg* sẽ là kết quả của biểu thức *in1 | in2*.
- Khi *mux_sel[1:0]* bằng 2'b10 thì giá trị của *mux_reg* sẽ cập nhật kết quả phép XOR của *in1*, *in2* và *in3*.
- Cuối cùng, nếu *mux_sel[1:0]* bằng 2'b11, giá trị cập nhật của *mux_reg* sẽ là kết quả phép NAND của *in1*, *in2* và *in3*.

Đoạn code trên mô tả hoạt động cho sơ đồ nguyên lý ở hình 3-31.



Hình 3-31. Mạch nguyên lý của ví dụ 3-44

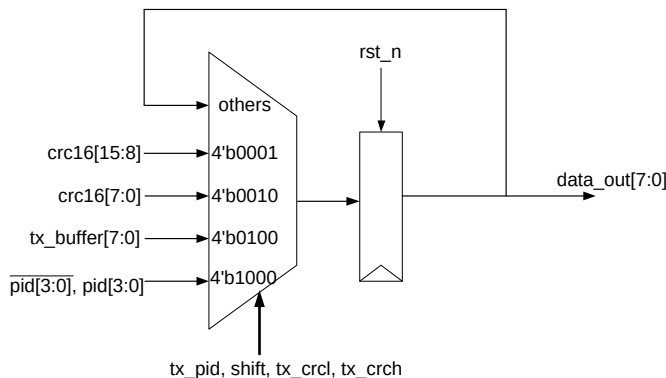


Hình 3-32. Netlist sau tổng hợp của ví dụ 3-44

Ví dụ 3-45

Giả sử ta có một mạch hoạt động theo nguyên lý như hình 3-33. Trong mạch này, thanh ghi *data_out[7:0]* hoạt động theo cạnh lên xung clock *clk* và tín hiệu reset bất đồng bộ *rst_n*. Các tín hiệu điều khiển *tx_pid*, *shift*, *tx_crcl* và *tx_crch* tích cực không đồng thời với nhau. Nếu *tx_pid* tích cực mức cao, bốn bit cao của *data_out[7:0]* sẽ cập nhật giá trị mới từ giá trị đảo của *pid[3:0]* và bốn bit thấp sẽ lấy giá trị trực tiếp từ *pid[3:0]*. Tín hiệu *shift* dùng để cho phép cập nhật giá trị của *tx_buffer[7:0]* vào thanh ghi *data_out[7:0]*. Tín hiệu *crc16[15:0]* được chia thành hai nhóm, tám bit cao và tám bit thấp, để nạp vào *data_out[7:0]* theo hai đợt. Tám bit thấp *crc16[7:0]* sẽ được nạp vào thanh ghi khi tín hiệu *tx_crcl* tích cực. Ngược lại, các bit cao *crc16[15:8]* sẽ cập nhật cho thanh ghi khi *tx_crch* tích cực. Nếu

không có tín hiệu điều khiển nào tích cực thì thanh ghi sẽ giữ nguyên giá trị trước đó.



Hình 3-33. Mạch nguyên lý của ví dụ 3-45

Để mô tả hoạt động cho mạch trên, ta có thể sử dụng **case** như đoạn RTL code dưới đây.

```
always @ (posedge clk or negedge rst_n) begin
    if (!rst_n) data_out[7:0] <= 8'b0;
    else begin
        case ({tx_pid, shift, tx_crcl, tx_crch})
            4'b1000: data_out[7:0] <= {~pid[3:0], pid[3:0]};
            4'b0100: data_out[7:0] <= tx_buffer[7:0];
            4'b0010: data_out[7:0] <= crc16[7:0];
            4'b0001: data_out[7:0] <= crc16 [15:8];
        endcase
    end
end
```

Mặt khác, ta cũng có thể viết lại đoạn code trên bằng cách sử dụng **casex** hay **casez**. Đoạn code sau đây là một ví dụ cho **casez**.

```
always @ (posedge clk or negedge rst_n) begin
    if (!rst_n) data_out[7:0] <= 8'b0;
    else begin
        casex ({tx_pid, shift, tx_crcl, tx_crch})
            4'b1??? : data_out[7:0] <= {~pid[3:0], pid[3:0]};
            4'b01?? : data_out[7:0] <= tx_buffer[7:0];
            4'b001? : data_out[7:0] <= crc16[7:0];
            4'b0001 : data_out[7:0] <= crc16 [15:8];
        endcase
    end
end
```

```

        endcase
    end
end

```

Các tín hiệu điều khiển không tích cực đồng thời nên chúng được coi là điều kiện one-hot. Do đó, ta có thể mô tả mạch trên như đoạn code sau đây.

```

always @ (posedge clk or negedge rst_n) begin
    if (!rst_n) data_out[7:0] <= 8'b0;
    else begin
        case (1'b1)
            tx_pid: data_out[7:0] <= {~pid[3:0], pid[3:0]};
            shift: data_out[7:0] <= tx_buffer[7:0];
            tx_crcl: data_out[7:0] <= crc16[7:0];
            tx_crch: data_out[7:0] <= crc16 [15:8];
        endcase
    end
end
end

```

3.4.2.3 Phát biểu case trong mạch tổ hợp

Phát biểu case có thể được sử dụng để mô tả mạch tổ hợp trong **always** và **function**. Tuy nhiên, để đơn giản, phần này chỉ đưa ra các ví dụ cơ bản có sử dụng phát biểu case trong **always** nhằm mục đích minh hoạt một số cách viết RTL code từ một sơ đồ nguyên lý đã có.

Ví dụ 3-46

```

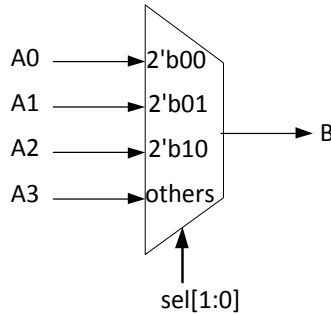
always @ (*) begin
    case(sel[1:0])
        2'b00: B = A0;
        2'b01: B = A1;
        2'b10: B = A2;
        default: B = A3;
    endcase
end

```

Đoạn code trên mô tả hoạt động cho bộ MUX 4-1 như hình 3-34. Mạch này thực hiện chức năng lựa chọn các ngõ vào A0, A1, A2 và A3 để gán cho ngõ ra B tùy theo giá trị của tín hiệu điều khiển sel[1:0].

- Khi sel[1:0] = 2'b00, B được gán bởi A0.
- Khi sel[1:0] = 2'b01, B được gán bởi A1.

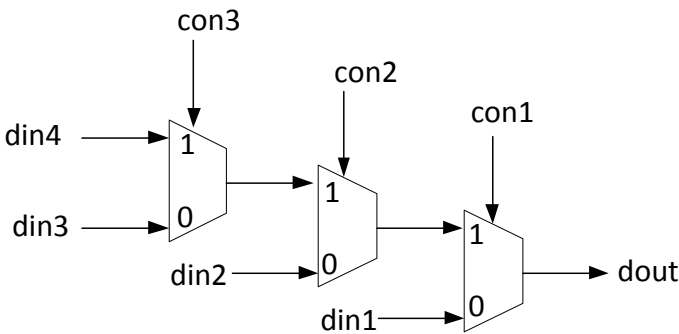
- Khi $sel[1:0] = 2'b10$, B được gán bởi A2.
- Trường hợp còn lại, B được gán bởi A3.



Hình 3-34. Mạch nguyên lý của ví dụ 3-46

Ví dụ 3-47

Giả sử ta có mạch nguyên lý như hình 3-35. Theo mạch này, khi $con1$ bằng 0 thì $dout$ được gán bởi $din1$. Khi $con2$ bằng 0 thì $din2$ sẽ được gán cho $dout$. Tương tự, khi $con3$ bằng 0 thì $din3$ sẽ gán cho $dout$. Ngược lại, $dout$ được gán bởi $din4$.



Hình 3-35. Mạch nguyên lý của ví dụ 3-47

Mạch trên có thể được mô tả bằng cách sử dụng **case** như đoạn RTL code dưới đây.

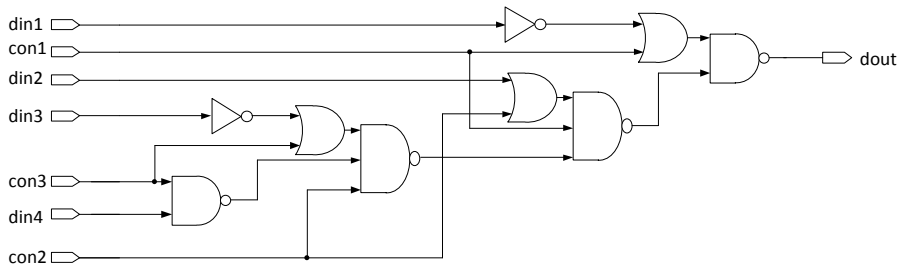
```
always @ (*) begin
    case ({con1, con2, con3})
        3'b000, 3'b001, 3'b010, 3'b011: dout = din1;
        3'b100, 3'b101: dout = din2;
        3'b110: dout = din3;
```



```

    3'b111: dout = din4;
    default: dout = 1'b0;
endcase
end

```



Hình 3-36. Netlist sau tổng hợp của ví dụ 3-47

Hoặc, để RTL ngắn gọn và dễ đọc, đoạn code trên được viết lại với **casez** như sau:

```

always @ (*) begin
    casez({con1, con2, con3})
        3'b0???: dout = din1;
        3'b10?: dout = din2;
        3'b110: dout = din3;
        3'b111: dout = din4;
        default: dout = 1'b0;
    endcase
end

```

Ví dụ 3-48

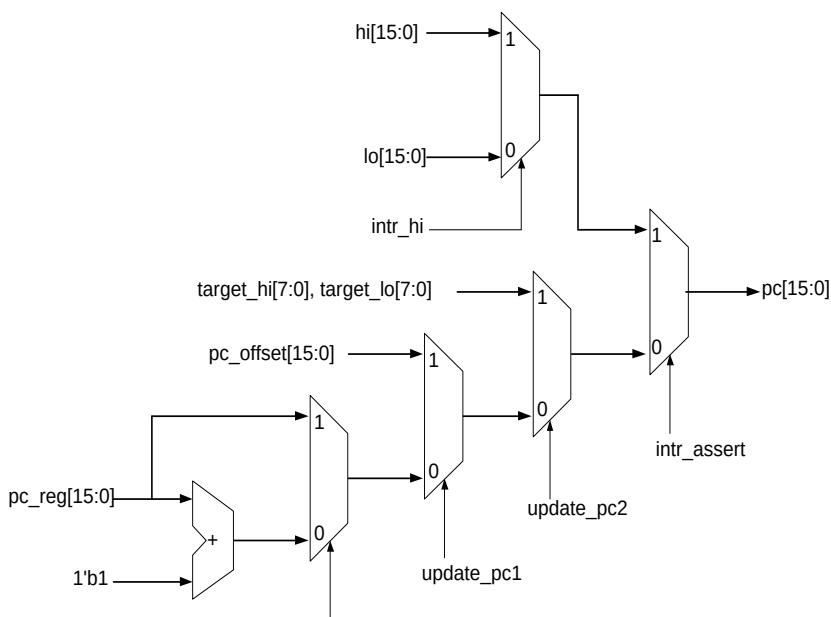
```

always @ (*) begin
    casez ({intr_assert, update_pc2, update_pc1, stall})
        4'b1???: pc[15:0] = (intr_hi)? hi[15:0]:lo[15:0];
        4'b01??: pc[15:0] = {target_hi[7:0],
                               target_lo[7:0]};
        4'b001?: pc[15:0] = pc_offset[15:0];
        4'b0001: pc[15:0] = pc_reg[15:0];
        default: pc[15:0] = pc_reg[15:0] + 1'b1;
    endcase
end

```

Đoạn RTL code trong ví dụ này mô tả hoạt động của mạch tính toán *pc[15:0]*. Việc xác định giá trị của *pc[15:0]* được thực hiện theo thứ tự dưới đây:

- Khi tín hiệu *intr_assert* ở mức cao, giá trị của *pc[15:0]* sẽ được lựa chọn từ *hi[15:0]* và *lo[15:0]* thông qua tín hiệu *intr_hi*. Nếu *intr_hi* bằng 1, *hi[15:0]* sẽ được chọn và ngược lại.
- Khi *update_pc2* ở mức cao, *target_hi[7:0]* được gán cho *pc[15:8]* và *target_lo[7:0]* gán cho *pc[7:0]*.
- Nếu *update_pc1* bằng 1, *pc[15:0]* sẽ được gán bởi *pc_offset[15:0]*.
- Tương tự, khi *stall* bằng 1, *pc_reg[15:0]* sẽ được gán cho *pc[15:0]*.
- Cuối cùng, nếu không có tín hiệu nào ở trên bằng 1, giá trị của *pc[15:0]* được gán bởi giá trị *pc_reg[15:0]* + 1.



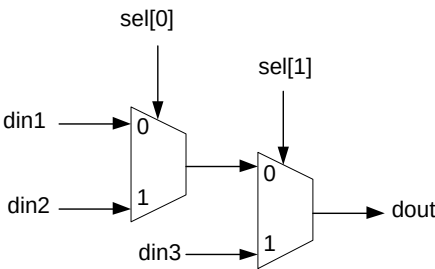
Hình 3-37. Mạch nguyên lý của ví dụ 3-48

3.4.2.4 So sánh case, casex và casez

Như đã phân tích trong mục 3.4.2.1, sử dụng **case** bị hạn chế trong việc mô tả mạch lựa chọn có độ ưu tiên. Khi đó, sử dụng **casex** và **casez** sẽ tốt hơn vì nó giúp code ngắn gọn và dễ đọc. Tuy nhiên, giữa **casex** và **casez** thì loại nào sẽ là sự lựa chọn tốt hơn? Để trả lời câu hỏi này, chúng ta cùng phân tích ví dụ sau.

Ví dụ 3-49

Giả sử chúng ta có hai đoạn code cùng mô tả chức năng của mạch bên dưới:



Hình 3-38. Mạch nguyên lý của ví dụ 3-49

Đoạn code sử dụng **casex**:

```
casex (sel[1:0])
    2'b00: dout = din1;
    2'b01: dout = din2;
    2'b1x: dout = din3;
    default: dout = 0;
endcase
```

Kết quả mô phỏng trước và sau khi tổng hợp của đoạn code trên được thể hiện ở bảng bên dưới.

Bảng 3-4. Kết quả mô phỏng trước và sau tổng hợp khi sử dụng casex

sel[1:0]	dout trước tổng hợp	dout sau tổng hợp
00	din1	din1
01	din2	din2
10	din3	din3
11	din3	din3
0x	din1 (không phải giá trị default)	x
0z	din1 (không phải giá trị default)	x
x0	din1 (không phải giá trị default)	x
z0	din1 (không phải giá trị default)	x
1x	din3	din3
1z	din3	din3

sel[1:0]	dout trước tổng hợp	dout sau tổng hợp
x1	din2 (không phải giá trị default)	x
z1	din2 (không phải giá trị default)	x
xx	din1 (không phải giá trị default)	x
zz	din1 (không phải giá trị default)	x

Đoạn code sử dụng **casez**:

```
casez (sel[1:0])
  2'b00: dout = din1;
  2'b01: dout = din2;
  2'b1z: dout = din3;
  default: dout = 0;
endcase
```

Bảng 3-5. Kết quả mô phỏng trước và sau tổng hợp khi sử dụng casez

sel[1:0]	dout trước tổng hợp	dout sau tổng hợp
00	din1	din1
01	din2	din2
10	din3	din3
11	din3	din3
0x	0 (giá trị default)	x
0z	din1 (không phải giá trị default)	x
x0	0 (giá trị default)	x
z0	din1 (không phải giá trị default)	x
1x	din3	din3
1z	din3	din3
x1	0 (giá trị default)	x
z1	din2 (không phải giá trị default)	x
xx	0 (giá trị default)	x
zz	din1 (không phải giá trị default)	x

Dựa vào bảng trên, ta thấy netlist sau tổng hợp của **casex** và **casez** cho kết quả mô phỏng hoàn toàn giống nhau. Đối với kết quả trước tổng hợp, nếu tất cả các điều kiện ngõ vào là x hoặc z thì **casex** sẽ thực hiện phép gán cho điều kiện đầu tiên thay vì điều kiện **default**. Điều này làm cho kết quả mô phỏng trước tổng hợp không đúng theo ý tưởng thiết kế ban đầu. Tuy nhiên, trong thiết kế vi mạch số, không thiết kế đúng nào tạo ra tín hiệu có giá trị z ngoài ý muốn. Do đó, trong phần này, chúng ta chỉ xét đến ảnh hưởng của giá trị x mà thôi. Cũng chính vì điều đó, **casez** cho ta được kết quả như mong đợi.

Do vậy, trong một số trường hợp phải sử dụng don't care trong phát biểu **case**, chúng ta nên dùng **casez** thay vì **casex**.

3.4.3 Những lưu ý khi sử dụng phát biểu case

Phát biểu **case** và phát biểu **if** nên được sử dụng một cách linh hoạt để rút gọn RTL code và tối ưu mạch nếu có thể. Một số trình tổng hợp khuyến cáo chúng ta nên không nên sử dụng số lượng tín hiệu đầu vào và danh sách điều kiện case quá lớn, vì điều đó có thể làm cho kết quả tổng hợp không được tối ưu.

Ví dụ 3-50

Giả sử ta có RTL mô tả chức năng của *dout* như đoạn code dưới đây.

```
casez (sel[15:0])
    16'b1zzz_zzzz_1010_1010: dout = 4'b0001;
    16'b1zzz_zzzz_1010_1011: dout = 4'b0010;
    16'b1zzz_zzzz_0010_1110: dout = 4'b1011;
    16'b1zzz_zzzz_1010_1111: dout = 4'b0100;
    16'b1zzz_zzzz_0010_1000: dout = 4'b0101;
    16'b1zzz_zzzz_0010_1010: dout = 4'b1110;
    default: dout = 4'b0000;
endcase
```

Trong đoạn code trên, người thiết kế không quan tâm đến các tín hiệu *sel[14:8]*, mà chỉ sử dụng các bit *sel[7:0]* để giải mã khi *sel[15]* bằng 1'b1. Do đó, đoạn code trên có thể được viết lại như sau:

```
if sel[15] = 1'b1 begin
    casez (sel[7:0])
        8'b1010_1010: dout = 4'b0001;
```

```

8'b1010_1011: dout = 4'b0010;
8'b0010_1110: dout = 4'b1011;
8'b1010_1111: dout = 4'b0100;
8'b0010_1000: dout = 4'b0101;
8'b0010_1010: dout = 4'b1110;
default: dout = 4'b0000;
endcase
end
else dout = 4'b0000;

```

Khi sử dụng phát biểu case trong mô tả mạch lựa chọn có điều kiện ưu tiên, danh sách điều kiện không nên trùng lặp lẫn nhau.

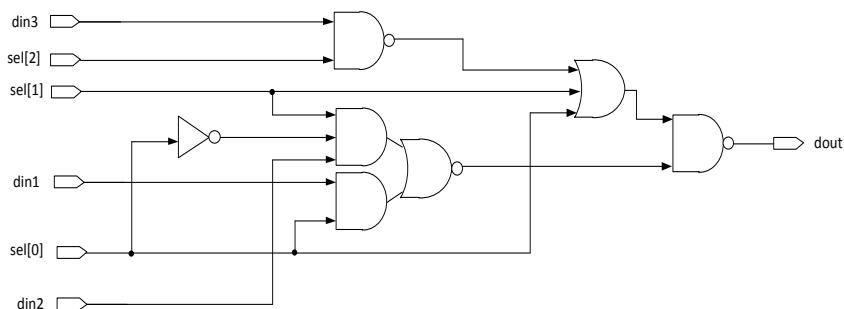
Ví dụ 3-51

```

always @ (*) begin
    casez (sel[2:0])
        3'b??1: dout = din1;
        3'b?1?: dout = din2;
        3'b1??: dout = din3;
        default: dout = 0;
    endcase
end

```

Trong ví dụ này, người thiết kế tạo muốn ra mạch chọn lựa có độ ưu tiên giảm dần theo thứ tự sau: *sel[0]*, *sel[1]*, *sel[2]*. Đoạn code trên hoàn toàn tổng hợp được và kết quả đúng như ý tưởng ban đầu của người thiết kế. Tuy nhiên, các điều kiện lại bị trùng lặp lẫn nhau. Giả sử vì một lý do nào đó, điều kiện ở các nhánh case được sắp xếp không đúng thứ tự như mong muốn. Khi đó, kết quả thu được hoàn toàn khác với ý tưởng ban đầu. Như vậy, trong trường hợp này, mạch tạo ra phụ thuộc hoàn toàn vào thứ tự của các điều kiện trong RTL code.



Hình 3-39. Netlist sau tổng hợp của ví dụ 3-51

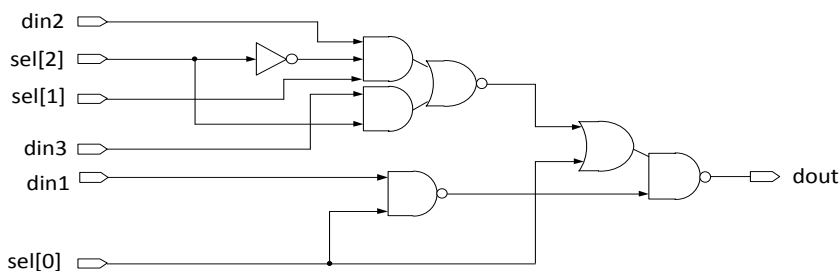
Ví dụ 3-52

```

always @ (*) begin
  casez (sel[2:0])
    3'b??1: dout = din1;
    3'b1??: dout = din3;
    3'b?1?: dout = din2;
    default: dout = 0;
  endcase
end

```

Đoạn code này được viết lại từ đoạn code trong ví dụ 3-51 với thứ tự của các điều kiện bị thay đổi. Netlist sau tổng hợp của ví dụ này có chức năng hoàn toàn khác với chức năng của netlist ở hình 3-39.



Hình 3-40. Netlist sau tổng hợp của ví dụ 3-52

Đề thứ tự của các điều kiện trong RTL không ảnh hưởng đến chức năng của mạch, đoạn code với của ví dụ 3-51 nên được viết lại sao cho không còn sự trùng lặp mà vẫn giữ được chức năng mong muốn.

```

casez (sel[2:0])
  3'b??1: dout = din1;
  3'b?10: dout = din2;

```

```

3'b100: dout = din3;
default: dout = 0;
endcase

```

Đặc biệt, trong mô tả mạch tổ hợp, nếu các danh sách điều kiện trong phát biểu case không đầy đủ, chúng ta sử dụng **default** để tránh tạo Latch ngoài ý muốn.

Ví dụ 3-53

Mạch tạo Latch do không có **default**.

```

always @ (*) begin
    case(sel[1:0])
        2'b10: D[3:0] = A[3:0] + 1'b1;
        2'b01: D[3:0] = A[3:0] + B[3:0];
    endcase
end

```

Phát biểu case ở trên chỉ liệt kê các điều kiện 2'b10 và 2'b01, các điều kiện còn lại không được xét đến, 2'b00 và 2'b11 chẳng hạn. Do vậy, Latch sẽ được tạo ra.

Để tránh tạo Latch không mong muốn, đoạn code trên được viết lại bằng cách bổ sung trường hợp **default**.

```

always @ (*) begin
    case(sel[1:0])
        2'b10: D[3:0] = A[3:0] + 1'b1;
        2'b01: D[3:0] = A[3:0] + B[3:0];
        default: D[3:0] = 4'b0;
    endcase
end

```

Ngoài ra, để tránh tạo Latch không mong muốn, ta có thể thực hiện gán giá trị mặc định của tín hiệu ngõ vào trước khi sử dụng phát biểu case.

Ví dụ 3-54

Trong ví dụ này, giá trị mặc định của *dout* được gán trước khi sử dụng phát biểu case nên trường hợp **default** có thể lược bỏ mà không tạo Latch.


```

always @ (*) begin
    D[3:0] = 4'b0;    //default assignment
    case(sel[1:0])
        2'b10: D[3:0] = A[3:0] + 1'b1;
        2'b01: D[3:0] = A[3:0] + B[3:0];
        //default: dout = 0;    //default can be omitted
    endcase
end

```

Điều cuối cùng, chúng ta cần lưu ý khi gán giá trị x cho ngõ ra trong điều kiện **default**. Khi làm như vậy, trình tổng hợp tự động lựa chọn giá trị hợp lý để gán cho ngõ ra nhằm tối ưu mạch theo cách tốt nhất có thể. Tuy nhiên, việc này có thể làm cho trình tổng hợp sinh ra các kết quả khác nhau cho những lần tổng hợp khác nhau và kết quả mô phỏng trước và sau khi tổng hợp cũng có thể khác nhau. Việc này sẽ gây khó khăn cho việc gỡ lỗi sau tổng hợp. Do đó, người thiết kế nên cân nhắc lựa giữa việc tối ưu thiết kế hay dễ dàng gỡ lỗi sau tổng hợp.

Ví dụ 3-55

```

casez(sel[1:0])
    2'b10: dout = 4'b1010;
    2'b01: dout = 4'b1011;
    ...
    default: dout = 4'bxxxx;
endcase

```

3.5 Function

3.5.1 Giới thiệu

Function là một hàm chức năng trả về một giá trị tương ứng tùy theo các giá trị ngõ vào và chức năng được mô tả. **Function** rất tiện lợi trong việc sử dụng lại những đoạn RTL code. Trong mô tả Verilog HDL khả năng tổng hợp, một phép gọi **function** là một phần của một phát biểu và sẽ tạo ra một mạch tổ hợp.

Sử dụng **function** có thể được chia ra làm hai giai đoạn:

- Khai báo **function**
- Gọi **function**

Khai báo **function** là quy định chức năng cho function. Gọi **function** là sử dụng lại **function**, được thực hiện bằng cách gọi tên **function** đó trong các phép gán. Về cơ bản, việc khai báo và gọi **function** có thể được thực hiện trong cùng module. Ngoài ra, nếu function được gọi nhiều lần trong nhiều module khác nhau, việc khai báo **function** có thể được thực hiện trong một module bất kỳ hoặc trong một file định nghĩa. Việc gọi **function** có thể được thực hiện ở bất cứ đâu bên trong các module hoặc ngay trong các file định nghĩa.

Tất cả các biến được khai báo bên trong **function** là các biến cục bộ, nó sẽ không gây ảnh hưởng đến các biến ở các module mà nó được gọi.

3.5.2 Khai báo function

3.5.2.1 Cú pháp khai báo function

```
function <signed> <type_or_range> function_name;  
<parameter_declaration>  
<input_declaration>  
<reg_declaration>  
<integer_declaration>  
<statement>  
endfunction
```

Mô tả:

- *signed*: quy định kiểu dữ liệu của kết quả **function** là có dấu hoặc không dấu
- *type_or_range*: quy định kiểu dữ liệu hoặc độ rộng bit
- *parameter_declaration*: khai báo **parameter**
- *input_declaration*: khai báo ngõ vào
- *reg_declaration*: khai báo biến **reg**
- *integer_declaration*: khai báo biến **integer**
- *statement*: các phát biểu

Ghi chú: Ngoài cách khai báo như trên, *input_declaration* có thể được khai báo theo cách khác như sau:

```
function <signed> <type_or_range>  
function_name (<input_declaration> );  
...  
Endfunction
```

3.5.2.2 Cách sử dụng khai báo function

Ví dụ 3-56

```
function [1:0] aoi;  
input [1:0] A;  
input [1:0] B;  
input [1:0] C;  
aoi[1:0] = ~(A[1:0] & B[1:0]) | C[1:0]); // <statement>  
endfunction
```

function *aoi* có ba ngõ vào *A[1:0]*, *B[1:0]*, *C[1:0]*; ngõ ra là *aoi[1:0]*, độ rộng ngõ ra 2 bit, tên ngõ ra cũng chính là tên của **function**. Kết quả của **function** *aoi* được thể hiện trong phép gán ở phần *statement*.

Ví dụ 3-57

```
function [1:0] aoi(input [1:0] A, input [1:0] B,  
input [1:0] C);  
aoi[1:0] = ~(A[1:0] & B[1:0]) | C[1:0]); // <statement>  
endfunction
```

Ví dụ này là làm lại ví dụ 3-56 với cách khai báo *input_declaration* khác.

Ví dụ 3-58

```
function [6:0] plru_algorithm;  
input [7:0] way;  
input [6:0] plru;  
input [6:0] plru_new;  
reg [6:0] plru_sel;  
integer i;  
begin  
  
    case (way[7:0])  
        8'b00000001: plru_sel[6:0] = 7'b0101110;  
        8'b00000010: plru_sel[6:0] = 7'b0101110;  
        8'b00000100: plru_sel[6:0] = 7'b0101101;  
        8'b00001000: plru_sel[6:0] = 7'b0101101;  
        8'b00010000: plru_sel[6:0] = 7'b0011011;  
        8'b00100000: plru_sel[6:0] = 7'b0011011;  
        8'b01000000: plru_sel[6:0] = 7'b0010111;  
        8'b10000000: plru_sel[6:0] = 7'b0010111;  
        default: plru_sel[6:0] = 7'b0000000;  
    endcase  
endfunction
```

```

for (i=0;i<=6;i=i+1) plru_algorithm[i] =
    plru_sel[i] ? plru[i] : plru_new[i];

end
endfunction

```

function *plru_algorithm* có ba ngõ vào là *way[7:0]*, *plru[6:0]*, *plru_new[6:0]*. Ngõ ra là *plru_algorithm[6:0]*, độ rộng 6 bit, tên ngõ ra chính là tên **function**. Các biến cục bộ gồm có *plru_sel[6:0]*, *i*. Các biến cục bộ dùng để lưu trữ các giá trị tạm thời được sử dụng trong quá trình tính toán kết quả cho function. Chức năng của function này là một mạch tổ hợp, đầu tiên là giải mã ngõ vào *way[7:0]* thành *plru_sel[6:0]*, sau đó từng bit của *plru_sel[6:0]* sẽ được dùng làm tín hiệu chọn cho từng bit của hai ngõ vào *plru[6:0]* và *plru_new[6:0]*.

3.5.3 Gọi function

3.5.3.1 Cú pháp gọi function

`function_name (<input_list>)`

Mô tả:

Gọi **function** được sử dụng như một toán hạng trong một biểu thức. Khi đó, tên của **function** được gọi ra, đồng thời tất cả các giá trị ngõ vào của **function** phải được chỉ định rõ ở phần *input_list*

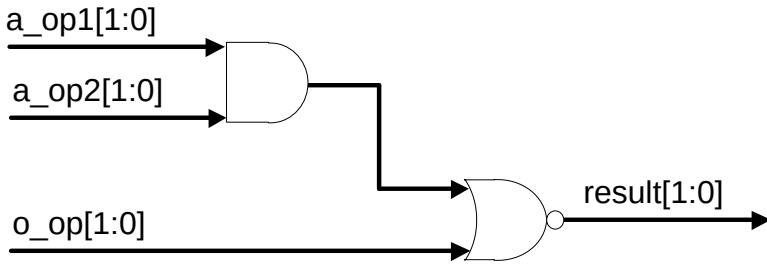
- *function_name*: tên của function
- *input_list*: danh sách ngõ vào, theo thứ tự input trong định nghĩa function

3.5.3.2 Cách sử dụng gọi function

Ví dụ 3-59

```
assign result[1:0]=aoi(a_op1[1:0],a_op2[1:0],o_op[1:0]);
```

Mạch tạo ra có chức năng như đã được khai báo ở **function** *aoi*, ngõ vào của mạch là những tín hiệu trong *input_list* *a_op1[1:0]*, *a_op2[1:0]*, *o_op[1:0]*, ngõ ra của mạch là *result[1:0]*.

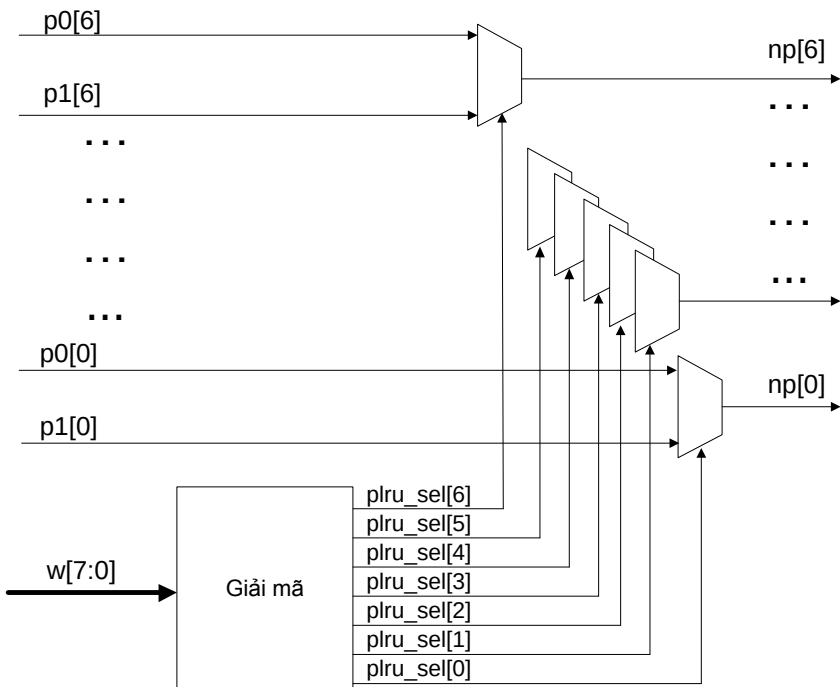


Hình 3-41. Mạch nguyên lý của ví dụ 3-59

Ví dụ 3-60

assign np[6:0] = plru_algorithm(w[7:0],p0[6:0],p1[6:0]);

Mạch tạo ra có chức năng như đã được khai báo ở **function** *plru_algorithm*, ngõ vào của mạch là những tín hiệu trong input_list *w[7:0]*, *p0[6:0]*, *p1[6:0]*, ngõ ra của mạch là *np[6:0]*.

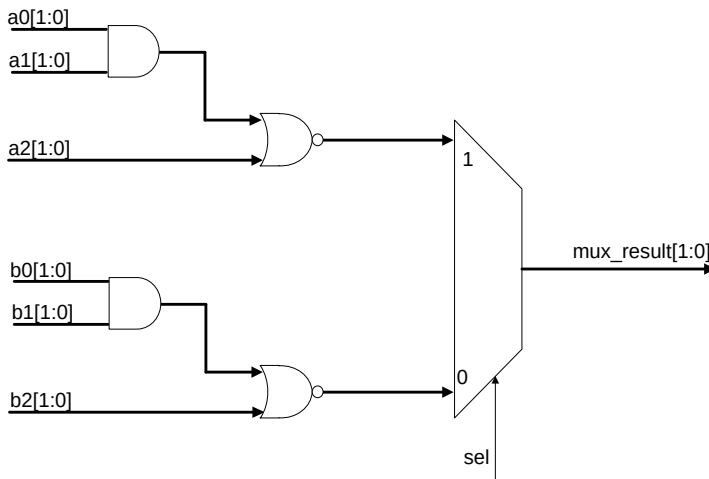


Hình 3-42. Mạch nguyên lý của ví dụ 3-60

Ví dụ 3-61

```
assign mux_result = sel ? aoi(a0,a1,a2) : aoi(b0,b1,b2);
```

Trong ví dụ này, **function** *aoi* được gọi hai lần, do đó sẽ có hai mạch chức năng của **function** *aoi* được tạo ra. Ngõ ra của hai mạch này được đưa vào một bộ MUX 2-1, tín hiệu chọn cho bộ MUX này là tín hiệu *sel*.



Hình 3-43. Mạch nguyên lý của ví dụ 3-61

3.6 Các tham số và chỉ dẫn tổng hợp

3.6.1 Parameter

Parameter dùng để khai báo một hằng số trong Verilog HDL. Hằng số này có thể gán lại được bằng nhiều cách khác nhau.

3.6.1.1 Cú pháp khai báo parameter

```
parameter <signed> <range> name_1 = constant_1
                                <, name_2 = constant_2>
                                <, name_3 = constant_3,...>;
```

Mô tả:

- **signed** là từ khóa dùng để cho biết parameter được khai báo kiểu có dấu hay không dấu.

- *range* dùng để khai báo độ rộng của **parameter**. Việc khai báo *range* có thể có hoặc không. Nếu *range* không được khai báo thì **parameter** sẽ lấy độ rộng của giá trị được gán. Nếu giá trị này cũng không được khai báo độ rộng thì **parameter** sẽ có độ rộng mặc định là 32.
- *name_1, name_2, name_3, ...* là tên của các **parameter** cần khai báo. Việc có hay không *name_2, name_3,...* là tùy thuộc vào cách người dùng khai báo. Chúng chỉ xuất hiện khi nhiều **parameter** được khai báo cùng lúc. Các **parameter** trong các module khác nhau có thể được trùng tên với nhau.
- *constant_1, constant_2, constant_3, ...* là giá trị của các **parameter**. Các giá trị này có là một số, một **define**, hoặc lấy từ kết quả trả về của một hàm, một phép tính,...

Ví dụ 3-62

Các trường hợp sau đây minh họa cho trường hợp giá trị gán cho **parameter** là một số.

```
parameter      [3:0] PARAM1 = 1;
parameter      [3:0] PARAM2 = 4'b0010;
parameter      [3:0] PARAM3 = 4'h3;
parameter signed [7:0] PARAM4 = -4;
parameter      [3:0] PARAM5 = 4'b0101,
                  PARAM6 = 4'd6,
                  PARAM7 = 7;
parameter      PARAM1 = 8;
```

Ví dụ 3-63

Ví dụ sau đây minh họa cho trường hợp **parameter** được gán từ một **parameter** khác hoặc một giá trị của **define**.

```
`define DATA_WIDTH      32
parameter DWIDTH = `DATA_WIDTH;
parameter BW1      = 32;
parameter BW2      = BW1;
```

Ví dụ 3-64

```
parameter BLOCK_SIZE = 32;
parameter DWIDTH      = 8;
```

```
parameter BSIZE = 2;
parameter BNUM = (BLOCK_SIZE / DWIDTH) * BSIZE;
```

Trong ví dụ này, *BNUM* được gán từ kết quả biểu thức của các hằng số khác.

Ví dụ 3-65

Parameter có thể được gán bởi giá trị hằng của một **function**.

```
parameter BLOCK = 64;
parameter WIDTH = log2(BLOCK);

function integer log2;
  input integer value;
  for (log2=-1; value>0; log2=log2+1)
    value = value>>1;
endfunction
```

3.6.1.2 Các trường hợp sử dụng parameter

Một trong những ứng dụng phổ biến của **parameter** là dùng để đặt tên cho các trạng thái khi mô tả máy trạng thái. Việc sử dụng **parameter** trong máy trạng thái làm cho code trở nên gọn nhẹ, dễ đọc và dễ gỡ lỗi.

Ví dụ 3-66

```
parameter IDLE = 2'b00;
parameter READY = 2'b01;
parameter RECEIVE = 2'b10;
parameter COMPLETE = 2'b11;
```

Đặc biệt, đối với các thiết kế cấu hình được và có khả năng sử dụng lại nhiều lần, người ta sử dụng **parameter** trong việc khai báo các thông số cấu hình cho thiết kế đó. Chỉ cần thay đổi giá trị của các **parameter**, ta sẽ có một thiết kế mới tương ứng với thông số vừa cấu hình. Các thông số cấu hình thông thường là độ rộng đường dữ liệu, đường địa chỉ; số lượng và kích thước các phân tử trong thiết kế,...

Ví dụ 3-67

```
parameter DWIDTH = 8;
parameter AWIDTH = 16;
```



```
parameter    MASTER_NUM    = 8;  
parameter    FIFO_DEPT     = 3;
```

Ngoài ra, **parameter** còn được sử dụng kết hợp với **generate** để tạo ra RTL mà người dùng có thể lựa chọn tính năng của một thiết kế.

Ví dụ 3-68

```
module param_exam (...);  
  
parameter    OR_GATE = 1;  
...  
generate  
    if (OR_GATE == 1)  
        assign result = a | b;  
    else  
        assign result = a & b;  
endgenerate  
endmodule
```

Trong ví dụ này, để cấu hình thiết kế có chức năng của cổng OR, người sử dụng gán cho parameter *OR_GATE* giá trị 1. Ngược lại, mạch được tạo ra là cổng AND.

Thông thường, chúng ta sử dụng **define** để thực hiện việc này. Tuy nhiên, trong một số trường hợp **define** không thể dùng được, chúng ta phải sử dụng **parameter** để thay thế. Điều này sẽ được bàn kỹ hơn ở mục 3.6.3.

3.6.1.3 Cách thay đổi giá trị parameter

Parameter là một hằng số có thể gán lại giá trị khi sử dụng thiết kế. Tuy nhiên, nó không thay đổi được trong thời gian thực, mà chỉ thay đổi và có tác dụng trong quá trình tổng hợp. Trong quá trình sử dụng thiết kế có sẵn, **parameter** có thể được gán lại theo nhiều cách khác nhau.

Cách đầu tiên được nhắc đến là sử dụng **defparam**, như ví dụ bên dưới.

Ví dụ 3-69

```
module sub_module (...);
parameter PORT_NUM = 4;
parameter PORT_WIDTH = 8;
...
endmodule

module top_module (...);
...
//Assign new value to parameters of sub_module_inst0
defparam sub_module_inst0.PORT_NUM = 8;
defparam sub_module_inst0.PORT_WIDTH =16;
//Assign new value to parameters of sub_module_inst1
defparam sub_module_inst1.PORT_NUM = 2;
defparam sub_module_inst1.PORT_WIDTH =8;

sub_module sub_module_inst0 (...);
sub_module sub_module_inst1 (...);

endmodule
```

Lợi điểm của **defparam** là nó cho phép thay đổi giá trị của parameter một cách dễ dàng. Ví dụ như, nó có thể thay đổi giá trị một parameter của bất kỳ module nào có cấp độ thấp hơn module sử dụng **defparam** mà không nhất thiết phải ở cấp liền kề.

Ví dụ 3-70

```
//top module
module top (...);
...
defparam sub_inst.sub_sub_inst.DW = 16;
sub sub_inst (...);
...
endmodule

//sub module
module sub (...);
...
sub_sub sub_sub_inst (...);
...
endmodule

//sub_sub module
```

```

module sub_sub (...);
...
parameter DW = 8;
...
endmodule

```

Trong ví dụ này, **defparam** ở module *top* có thể thay đổi parameter của module thấp hơn hai cấp là *sub_sub_inst*.

Tuy nhiên, đây vừa là lợi điểm vừa là nhược điểm của **defparam**. Bởi việc cho phép thay đổi **parameter** một cách tự do làm cho người sử dụng dễ bị nhầm lẫn và sai sót khi viết code. Khi một thiết kế với số lượng file code và dòng code rất lớn thì những sai sót này làm cho việc gỡ lỗi gặp rất nhiều khó khăn. Điều này được minh họa trong ví dụ sau.

Ví dụ 3-71

```

module two_registers (q, d, clk, rst_n);
parameter SIZE = 16;
input          clk;
input          rst_n;
input  [SIZE-1:0] d;
output [SIZE-1:0] q;

wire  [SIZE-1:0] dx;
defparam reg1.SIZE=16;
register reg1 (.clk(clk),
              .rst_n(rst_n),
              .d(dx),
              .q(q)
              );
defparam reg2.SIZE=16;
register reg2 (.clk(clk),
              .rst_n(rst_n),
              .d(dx),
              .q(q)
              );
defparam reg2.SIZE=4; // Mistake!
endmodule

```

Giả sử, việc gán lại giá trị của parameter *SIZE* lần cuối cùng là do sai sót khi viết code. Điều này dẫn đến toàn bộ thiết kế bị sai không đáng có. Do đó, một số cụ công cụ thiết kế số khuyến cáo chúng ta không

nên sử dụng **defparam**. Chính vì vậy, trong tài liệu này chúng ta không đề cập thêm về **defparam**.

Thông thường, ta sử dụng cách khác là truyền giá trị cho parameter trong quá trình gọi module con. Bản thân cách truyền parameter này cũng có hai cách thực hiện khác nhau. Cách thứ nhất là truyền parameter theo thứ tự khai báo.

Ví dụ 3-72

```
//top module
module top_module (...);
...
sub_module    # (8, 16) sub_module_inst (...);
endmodule

//sub module
module sub_module (...);
parameter PORT_NUM = 4;
parameter PORT_WIDTH = 8;
...
endmodule
```

Trong ví dụ trên, *PORT_NUM* và *PORT_WIDTH* được ngầm hiểu gán lại các giá trị lần lượt là 8 và 16. Điều này rõ ràng không tốt vì nó dễ xảy ra sai sót và gây khó khăn cho quá trình gỡ lỗi. Do đó, một cách khác tốt hơn thường được sử dụng là truyền parameter vào module con theo tên gọi. Cách này giúp cho code được rõ ràng, tường minh, dễ đọc và dễ gỡ lỗi khi cần.

Ví dụ 3-73

```
//top module
module top_module (...);
...
sub_module # (.PORT_NUM(8), .PORT_WIDTH(16))
sub_module_inst (...);
endmodule
```

3.6.1.4 Những vấn đề cần lưu ý khi sử dụng parameter

Parameter nên được viết sao cho dễ phân biệt với tên tín hiệu. Thông thường, người ta chọn cách viết in hoa. Ngoài ra, parameter phải nên ngắn gọn, gọi nhớ, dễ đọc, dễ hiểu. Mục đích của những điều này làm cho code được rõ ràng và điều quan trọng là giúp

chúng ta dễ dàng gỡ lỗi. Để thấy rõ điều này, chúng ta phân tích ví dụ sau.

Ví dụ 3-74

```
module rx_control ( ... );
....
//
//parameter
//
parameter    idle = 2'b00;
parameter    ready = 2'b01;
parameter    receive = 2'b10;
parameter    complete = 2'b11;

reg [1:0]    state;
reg [1:0]    next_state;
//
//fsm
//
always @ (*) begin
    next_state = current_state;
    casez
        idle: if (...) next_state = ready;
        ready: if (...) next_state = receive;
        receive: if (...) next_state = complete;
        complete: if (...) next_state = idle;
    endcase
end

always @ (posedge clk) begin
    if (reset!) current_state <= idle;
    else current_state <= next_state;
end

endmodule
```

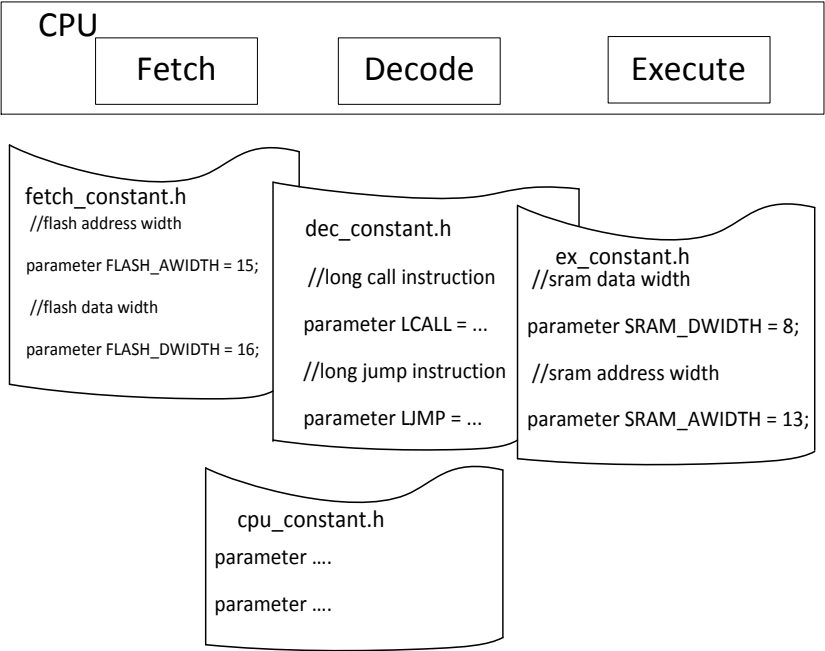
Trong ví dụ này, tất cả các parameter đều là chữ thường nên chúng ta không phân biệt được đó là parameter hay là tên tín hiệu. Điều này làm code không được tường minh và gây khó khăn cho việc gỡ lỗi.

Các parameter có thể viết trực tiếp trong mỗi module nếu chúng không được sử dụng ở bất kỳ module nào khác và chúng không phải là thông số cấu hình của toàn bộ thiết kế. Tuy nhiên, nếu các parameter sử dụng chung cho nhiều module, chúng ta nên viết chúng

trong cùng một file sau đó dùng **include** để gọi các parameter vào để sử dụng. Trong trường hợp một thiết kế quá lớn, để dễ quản lý, chúng ta nên phân chia các parameter theo từng khối chức năng và đặt chúng vào các file khác nhau tương ứng cho mỗi khối. Các parameter sử dụng chung cho toàn bộ thiết kế thì đặt ở một file riêng.

Giả sử ta có một thiết kế lớn CPU được chia thành ba khối chính là Fetch, Decode và Execute. Mỗi khối đều có một file chứa parameter riêng như sau:

- File “fetch_constant.h” chứa các parameter của khối Fetch
- File “dec_constant.h” chứa các parameter của khối Decode
- File “ex_constant.h” chứa các parameter của khối Execute
- Các parameter sử dụng chung cho toàn bộ thiết kế chúng ta đặt trong file “cpu_constant.h”



Hình 3-44. Tổ chức các file chứa parameter

Để gọi các parameter vào trong một module, chúng ta sử dụng **include** như ví dụ sau:

Ví dụ 3-75

```
module decode ();  
`include "decode_constant.h"  
...  
endmodule
```

Lưu ý, dòng lệnh **include** phải nằm trong module.

3.6.2 Localparam

Localparam dùng để khai báo hằng số không gán lại được khi gọi thiết kế đã có vào sử dụng. Thông thường, người thiết kế không muốn người dùng gán lại giá trị cho các thông số này nhằm tránh trường hợp giá trị được gán không phù hợp.

Cú pháp khai báo localparam như sau:

```
localparam <signed> <range> name_1 = constant_1  
                <, name_2 = constant_2>  
                <, name_3 = constant_3, ...>;
```

Mô tả:

Cú pháp khai báo **localparam** hoàn toàn giống với **parameter**, chỉ có từ khóa được thay đổi thành **localparam**.

Ví dụ 3-76

```
parameter ADDR_WIDTH = 8;  
localparam MEM_SIZE = 1 << ADDR_WIDTH;
```

Do *MEM_SIZE* là một thông số được tính toán từ parameter *ADDR_WIDTH* nên mỗi giá trị của *ADDR_WIDTH* cho ra duy nhất một giá trị của *MEM_SIZE*. Nếu *MEM_SIZE* là parameter, khi đó người dùng có thể gán lại parameter này với một giá trị khác. Trong trường hợp giá trị được gán không phù hợp có thể dẫn đến sai chức năng thiết kế.

Tương tự như parameter, localparam có thể dùng để đặt tên trạng thái của một FSM. Đặc biệt, localparam thường được sử dụng để khai báo cáo thông số nội của một module mà người thiết kế không cho người dùng gán lại. Các localparam có thể được gán giá trị từ một số, một **define**, hoặc từ kết quả trả về của một hàm, một phép tính,...

Ví dụ 3-77

```
//localparam value is a number
localparam LOCAL_PRAM1 = 2'b10;

//localparam value is from a define
`define DEFINE_VALUE 8
localparam [3:0] LOCAL_PRAM2 = `DEFINE_VALUE;

//localparam value is from a function
parameter NUM1 = 3'd5;
parameter NUM2 = 3'd2;
parameter NUM3 = 3'd4;
localparam MAX = maximum(NUM1, NUM2, NUM3);

function [2:0] maximum;
input [2:0] a0;
input [2:0] a1;
input [2:0] a2;

reg [2:0] a_array[2:0];
integer i;
begin
    a_array[0] = a0;
    a_array[1] = a1;
    a_array[2] = a2;
    maximum = 3'b0;
    for (i = 0; i<3; i = i +1) begin
        if (maximum > a_array[i]) maximum = maximum;
        else maximum = a_array[i];
    end
end
endfunction
```

Localparam cũng có thể được khai báo từ một file khác, sau đó gọi vào bằng **include** tương tự như parameter. Việc gọi các **localparam** phải được thực hiện bên trong **module**.

3.6.3 Define

Define là một chỉ dẫn của Verilog HDL dùng trong việc định nghĩa các macro. Một **define** có thể sử dụng ở bất kỳ vị trí nào trong file Verilog HDL mà không bị bắt buộc phải nằm trong **module** như **parameter** hay **localparam**. Tuy nhiên, do tất cả các **define** được gọi

vào một lần khi tổng hợp nên Verilog HDL không cho phép các **define** được đặt trùng tên.

3.6.3.1 Cú pháp khai báo define

```
`define name <macro_text>
```

Mô tả:

- *name* là tên cần khai báo của define.
- *macro_text* là chuỗi ký tự sẽ được thay thế khi *name* được gọi. Chuỗi ký tự này có thể có hoặc không.

Ví dụ 3-78

```
`define MSB          16
`define VALUE1      8'b0
`define VALUE1      8'd9
`define VALUE3      `VALUE1 + `VALUE2
wire [`MSB - 1: 0] DO;
```

3.6.3.2 Cách sử dụng define

Define được dùng để định nghĩa một chức năng của thiết kế có tồn tại hay không.

Ví dụ 3-79

```
`define AND_GATE
...
`ifdef AND_GATE
assign c = a & b;
`endif
```

Trong ví dụ trên, nếu dòng *`define AND_GATE* được định nghĩa thì mạch AND của *a* và *b* sẽ được tạo ra. Ngược lại, trình tổng hợp sẽ bỏ qua đoạn code này.

Để hủy bỏ một **define** đã định nghĩa trước đó, chúng ta sử dụng **`undef**.

Ví dụ 3-80

```
`define AND
`ifdef OR
    `undef AND
`endif
```

Trong ví dụ trên, giả sử người thiết kế mong muốn code của họ có thể linh hoạt lựa chọn mạch tạo ra là *AND* hay *OR*. Tuy nhiên, nếu người sử dụng khai báo không đúng, tức là khai báo cả *AND* và *OR*, thì mạch *OR* sẽ được ưu tiên cao hơn. Kết quả là chỉ có mạch *OR* được tạo ra thay vì cả hai.

Ngoài ra, chúng ta còn sử dụng kết hợp các phát biểu điều kiện của **define** như **`ifdef**, **`ifndef**, **`else** và **`endif** để có thể sử dụng **define** một cách linh hoạt. Phát biểu **`ifdef** và **`ifndef** dùng để kiểm tra một define đã có khai báo hay chưa. Phát biểu **`ifdef** trả về kết quả *true* nếu define đã được khai báo và *false* nếu chưa khai báo. Trong khi đó, **`ifndef** cho kết quả ngược lại. Nếu điều kiện đúng thì đoạn code tiếp theo sẽ được loại bỏ cho đến khi gặp **`else** (nếu có) để chuyển sang thực hiện cho điều kiện ngược lại hoặc khi gặp **`endif** để kết thúc quá trình kiểm tra.

Ví dụ 3-81

```
module define_exam (...);
...
`ifdef OR      //If OR is defined
assign result = a | b;
`else         //OR is defined
assign result = a & b;
`endif

`ifndef OR    //If OR is not defined
assign xor_result = a ^ b;
`endif

endmodule
```

Các define phải mang tính toàn cục, tức là nó chỉ cho phép người sử dụng cấu hình một lần cho toàn bộ thiết kế mà không được phép gán lại ở bất cứ chỗ nào khác. Chẳng hạn, chúng ta không thể sử dụng module *define_exam* nhiều lần trong cùng một thiết kế với define *OR*

thay đổi. Trong trường hợp này, ta phải sử dụng parameter để thay thế cho define (xem mục 3.6.1.2).

Define còn được dùng để khai báo hằng số tương tự như parameter.

Ví dụ 3-82

```
module sub_module ();  
  `define PORT_NUM 4  
  ....  
endmodule
```

Tuy nhiên, **define** không nên được sử dụng với mục đích này mà nên sử dụng **parameter**. Bởi vì, nếu *sub_module* được sử dụng lại nhiều lần trong cùng một thiết kế với các giá trị *PORT_NUM* khác nhau, khi đó chúng ta không có cách nào để gán lại giá trị cho **define** này.

Tương tự như parameter và localparam, các define cũng nên đặt ở một file riêng để dễ dàng thay đổi khi cần thiết. Để gọi các define này, chúng ta sử dụng lệnh **include**. Lệnh này phải được gọi trước khi sử dụng các define. Do đó, nó thường được đặt bên ngoài **module**. Trong một số trường hợp, người viết code vẫn có thể đặt nó bên trong **module** nhưng với điều kiện là phải trước khi sử dụng các **define**.

Ví dụ 3-83

```
module add_sub (a, b, c);  
  `include "add_sub_define.h" //include before using  
  input a;  
  input b;  
  output a;  
  `ifdef ADD  
    assign c = a + b;  
  `else  
    assign c = a - b;  
  `endif  
endmodule
```

Ở ví dụ này, các define chỉ được dùng trong module nên lệnh **include** được phép đặt ở vị trí này. Tuy nhiên, trong ví dụ dưới đây, việc sử dụng **include** bên trong module là hoàn toàn không thể.

Ví dụ 3-84

```
module add_sub (  
    a,  
    b,  
    `ifdef ADD  
        add_result  
    `else  
        sub_result  
    `endif  
);  
`include "add_sub_define.h" //Error!  
input a;  
input b;  
`ifdef ADD  
output add_result;  
`else  
output sub_result;  
`endif  
`ifdef ADD  
assign add_result = a + b;  
`else  
assign sub_result = a - b;  
`endif  
endmodule
```

Trong ví dụ này, define *ADD* được sử dụng trước khi gọi vào nên trình biên dịch sẽ báo lỗi. Do đó, đoạn code trên phải được viết lại như sau:

```
`include "add_sub_define.h" //include before using  
module add_sub (  
    a,  
    b,  
    `ifdef ADD  
        add_result  
    `else  
        sub_result  
    `endif  
);  
input a;  
input b;  
`ifdef ADD  
output add_result;  
`else  
output sub_result;  
`endif  
`ifdef ADD  
assign add_result = a + b;
```

```
`else
assign sub_result = a - b;
`endif
```

```
endmodule
```

Tóm lại, dù define được sử dụng ở đâu, người viết code cũng nên đặt lệnh **include** ở bên ngoài module để dễ quản lý và tránh những sai sót không đáng có.

3.7 Phát biểu generate

3.7.1 Cú pháp phát biểu generate

Cú pháp *generate-loop*

```
genvar genvar_name;
generate
  for (genvar_assignment; expression; genvar_assignment)
    begin: for_name
      <statement>
    end
endgenerate
```

Cú pháp *generate-conditional*

```
generate
  if (expression_1)           <statement_1>
  else if (expression_2)      <statement_2>
  else                        <statement_3>
endgenerate
```

Cú pháp *generate-case*

```
generate
  case (expression)
    case_item_0 <,case_item_1, ...>: <statement_1>
    case_item_2 <,case_item_3, ...>: <statement_2>
    default:                                <statement_3>
  endcase
endgenerate
```

Mô tả:

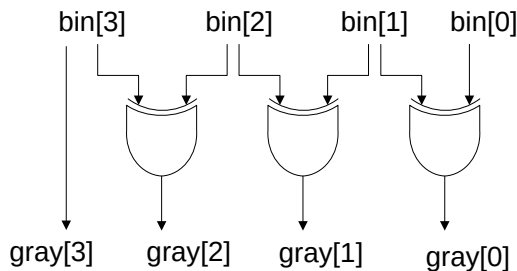
- *genvar_name* là tên biến chạy của sử dụng trong *generate-loop*. Biến **genvar** có các ràng buộc sử dụng như sau:

- Biến **genvar** phải được khai báo trong module sử dụng nó và có thể nằm ngoài hoặc nằm bên trong cặp từ khóa **generate-endgenerate**.
- Biến **genvar** không được gán giá trị là x, z và số âm.
- Một biến **genvar** không được sử dụng chung cho các vòng lặp *for* lồng nhau.
- *genvar_assignment* là các biểu thức hằng số gán giá trị cho biến **genvar**.
- *expression* là biểu thức điều kiện.
- *case_item* là giá trị có thể xảy ra của điều kiện trong *generate-case*.
- *statement* là các phát biểu tạo mạch số như **assign**, **always**, gọi **module**, ...
- *for_name* là tên vòng lặp **for**. Tên này bắt buộc phải có khi sử dụng *generate-loop*.
- Đối với *generate-conditional* và *generate-case*, nếu *statement* gồm nhiều phát biểu khác nhau thì cặp từ khóa **begin-end** sẽ được sử dụng.

3.7.2 Cách sử dụng phát biểu generate

3.7.2.1 generate-loop

generate-loop thường được sử dụng để mô tả những mạch có cấu trúc giống nhau và có mối liên hệ theo quy luật. Xét bộ chuyển từ mã binary sang mã Gray cho bốn bit.



Hình 3-45. Mạch nguyên lý chuyển đổi bốn bit mã binary sang mã Gray

Mạch nguyên lý trên được mô tả bằng nhiều biểu thức gán như đoạn RTL sau đây:

Ví dụ 3-85

```
assign gray[0] = bin[0] ^ bin[1];
assign gray[1] = bin[1] ^ bin[2];
assign gray[2] = bin[2] ^ bin[3];
assign gray[3] = bin[3] ^ 1'b0 ;
```

Ba phép gán đầu tiên trong ví dụ 3-85 có chung quy luật và có thể được mô tả bằng *generate-loop*.

Ví dụ 3-86

```
generate
genvar i;
  for (i=0; i<=2; i=i+1) begin : gen_convert
    assign gray[i] = bin[i] ^ bin[i+1];
  end
  assign gray[3] = bin[3] ^ 1'b0; //=bin[3]
endgenerate
```

Xét thiết kế có một mảng gồm nhiều phần tử và mỗi phần tử được lái bởi một vector các tín hiệu điều khiển. Số lượng các phần tử của mảng có thể thay đổi theo tham số cấu hình. Với số lượng trường hợp của tham số là 2 hoặc 4 thì có thể viết RTL code cho từng trường hợp của tham số. Nhưng nếu số lượng trường hợp quá lớn thì phải có cách viết tổng quát để dễ dàng thay đổi theo tham số cấu hình.

Theo hình 3-46, các *valid_bit* được điều khiển bởi các tín hiệu *set_bit* và *clear_bit* khác nhau, nên viết riêng cho mỗi bit.

Ví dụ 3-87

```
//internal registers and wires
reg [2**N-1:0] valid_bit; // valid bits
reg [2**N-1:0] set_bit; // set signals to valid bits
reg [2**N-1:0] clear_bit; // clear signals to valid
bits

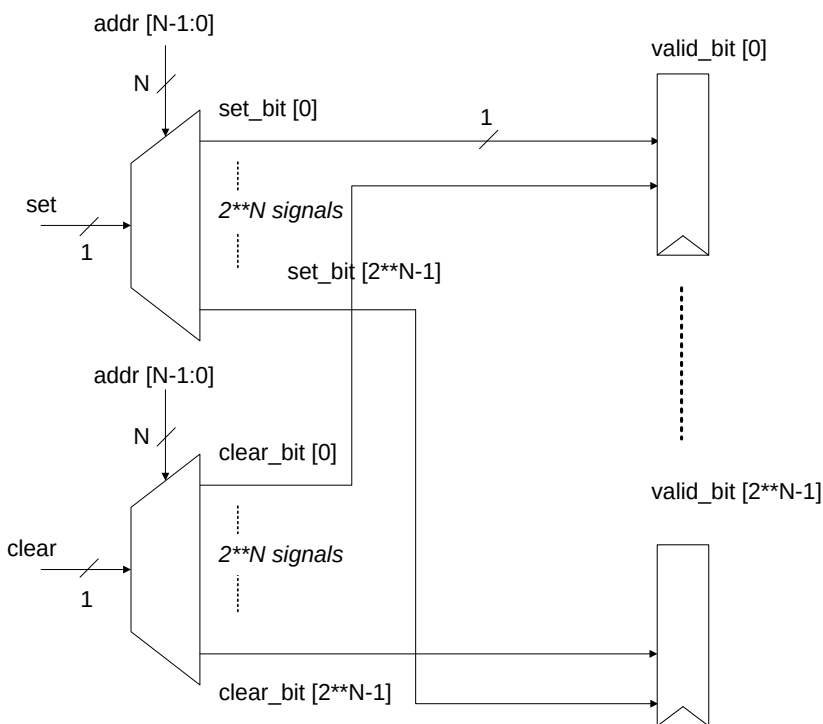
always @(*) begin
  set_bit = 'd0;
  set_bit[addr[N-1:0]] = set;

  clear_bit = 'd0;
  clear_bit[addr[N-1:0]] = clear;
```

```

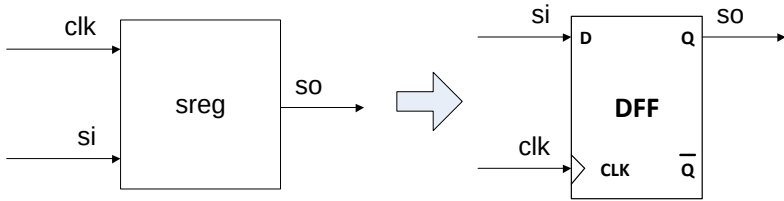
end
genvar i;
generate
  for(i=0; i<2**N; i=i+1) begin: valid_gen
    always @(posedge clk) begin
      if (!rst_n)
        valid_bit[i] <= 1'b0;
      else if (set_bit[i])
        valid_bit[i] <= 1'b1;
      else if (clear_bit[i])
        valid_bit[i] <= 1'b0;
    end
  end
endgenerate

```



Hình 3-46. Mạch nguyên lý của ví dụ 3-87

Tương tự, *generate-loop* có thể được dùng để gọi và ghép nối các khối chức năng theo quy luật. Giả sử, *sreg* là thanh ghi một bit có các tín hiệu ngõ vào *clk*, *si*, và ngõ ra là *so*. Ngõ ra *so* cập nhật giá trị từ ngõ vào *si* tại mỗi cạnh lên xung clock.



Hình 3-47. Sơ đồ tín hiệu giao tiếp thanh ghi một bit sreg

Một thanh ghi dịch có số lượng bit thay đổi trước khi tổng hợp bằng **parameter** *SIZE* thì thanh ghi dịch ghép nối các thành phần *sreg* như mô tả như ví dụ sau.

Ví dụ 3-88

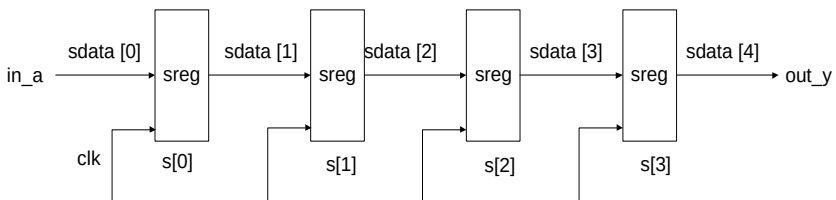
```

wire    [SIZE:0]  sdata;
//the width of sdata is SIZE+1 bits

assign  sdata[0] = in_a;
assign  out_y = sdata[SIZE];

genvar  i;
generate
    for(i=0; i<SIZE; i=i+1) begin: sreg_gen
        sreg s (.clk(clk), .si(sdata[i]), .so(sdata[i+1]));
    end
endgenerate

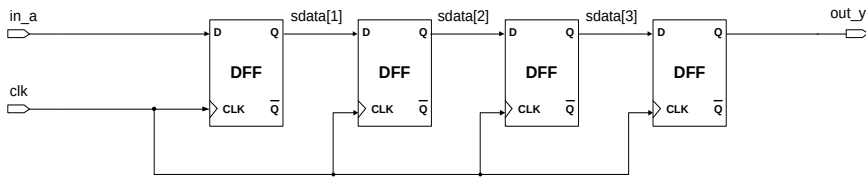
```



Hình 3-48. Sơ đồ nguyên lý của ví dụ 3-40 khi *SIZE* = 4

Đối với việc kết các khối lại với nhau, các khối con cần phải gán tên khác nhau. Khi kết nối các khối con bằng cách sử dụng phát biểu **generate** thì tên các khối con sẽ tự động thay đổi tên cho mỗi lần gọi theo giá trị của biến chạy của **generate** và thêm vào cuối

tên của khối con dạng mảng phần tử. Trong trường hợp này là $s[0]$, $s[1]$, $s[2]$, và $s[3]$.



Hình 3-49. Netlist sau tổng hợp của ví dụ 3-88 SIZE = 4

3.7.2.2 generate-conditional

generate-conditional thường sử dụng để tùy chọn tổng hợp ra các mạch chức năng khác nhau dựa trên các tham số cấu hình.

Ví dụ, tùy vào điều kiện ứng dụng cụ thể trong một thiết kế, mạch chức năng cho ngõ ra *output_y* là AND hoặc OR hai ngõ vào *input_x1* và *input_x2*. Yêu cầu này được thực hiện bằng phát biểu **generate** như sau:

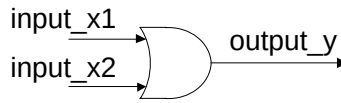
Ví dụ 3-89

```
parameter OR_GATE = 1'b1;

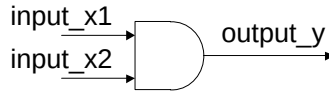
generate
  if (OR_GATE) begin: OR_GEN
    always @(*) begin
      output_y = input_x1 | input_x2;
    end
  end
  else begin
    always @(*) begin
      output_y = input_x1 & input_x2;
    end
  end
endgenerate
```

Mạch nguyên lý của ví dụ 3-89 được tạo ra khác nhau phụ thuộc vào giá trị của parameter *OR_GATE* như sau:

- Khi $OR_GATE = 1'b1$



- Khi $OR_GATE = 1'b0$



Ví dụ 3-90

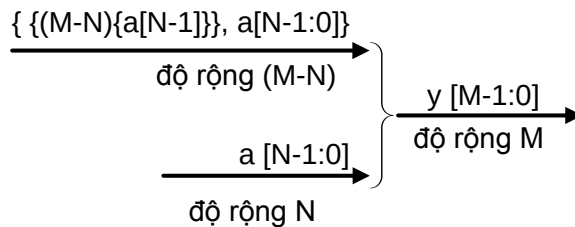
```

input    [N-1:0]    a;
output   [M-1:0]    y;

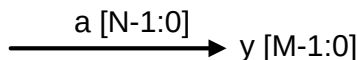
generate
  if (N < M)
    //the signed bit is the MSB bit
    assign y[M-1:0] = {{(M-N){a[N-1]}}, a[N-1:0]};
  else
    assign y[M-1:0] = a[N-1:0];
endgenerate

```

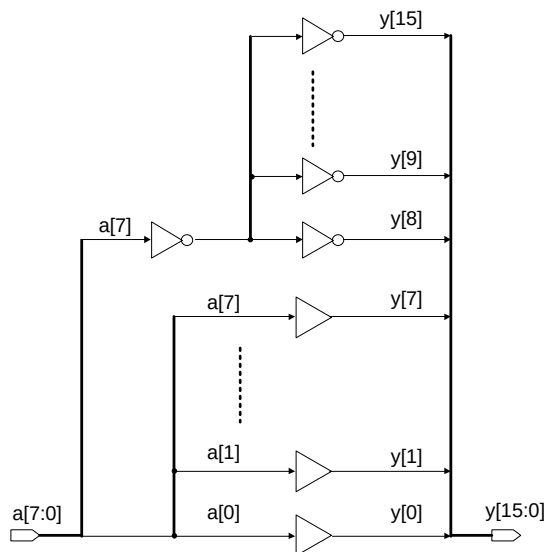
Ví dụ 3-90 sẽ tạo ra mạch mở rộng bit dấu của tín hiệu a khi độ rộng của tín hiệu a là N bit nhỏ hơn độ rộng tín hiệu y là M bit. Nếu độ rộng tín hiệu a nhỏ hơn hoặc bằng độ rộng tín hiệu y thì mạch tạo ra sẽ nối dây trực tiếp a bằng y .



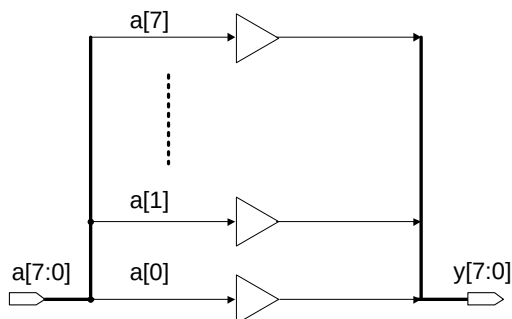
Hình 3-50. Mạch nguyên lý của ví dụ 3-91 trong trường hợp $N < M$



Hình 3-51. Mạch nguyên lý của ví dụ 3-91 trong các trường hợp khác



Hình 3-52. Netlist sau tổng hợp của ví dụ 3-90 khi $N = 8$, $M = 16$



Hình 3-53. Netlist sau tổng hợp của ví dụ 3-90 khi $N = 8$, $M = 8$

3.7.2.3 generate-case

Tương tự *generate-conditional*, với số điều kiện nhiều thì RTL code nên chuyển sang viết dạng *generate-case* để dễ quản lý và dễ quan sát.

Ví dụ 3-91

```
generate
case (WIDTH)
  8: begin : case8_name
    assign y[7:0] = a[7:0];
```

```

end
16: begin : case16_name
    assign y[15:0] = {b[7:0], a[7:0]};
end
32: begin : case32_name
    assign y[31:0] = {d[7:0], c[7:0], b[7:0], a[7:0]};
end
default: begin : default_case_name
    assign y[WIDTH-1:0] = { {(WIDTH-8){1'b0}}, a[7:0]};
end
endcase
endgenerate

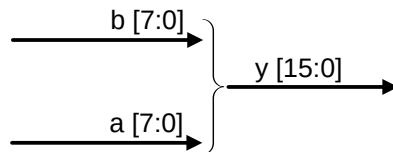
```

Mạch nguyên lý của từng trường hợp được mô tả sau đây

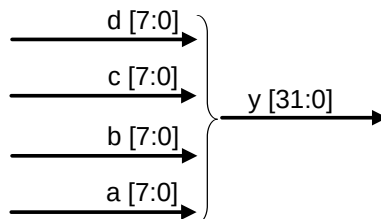
- Trường hợp WIDTH = 8



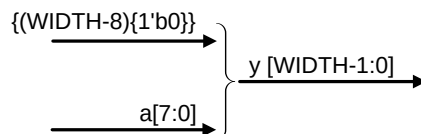
- Trường hợp WIDTH = 16

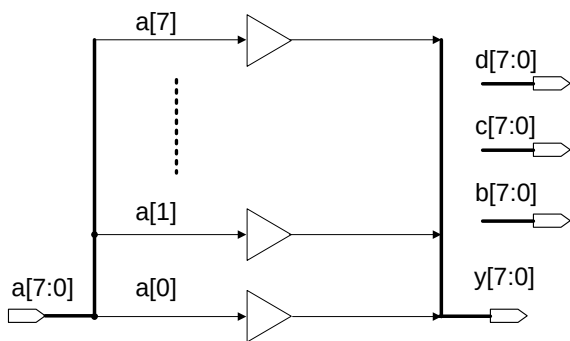


- Trường hợp WIDTH = 32

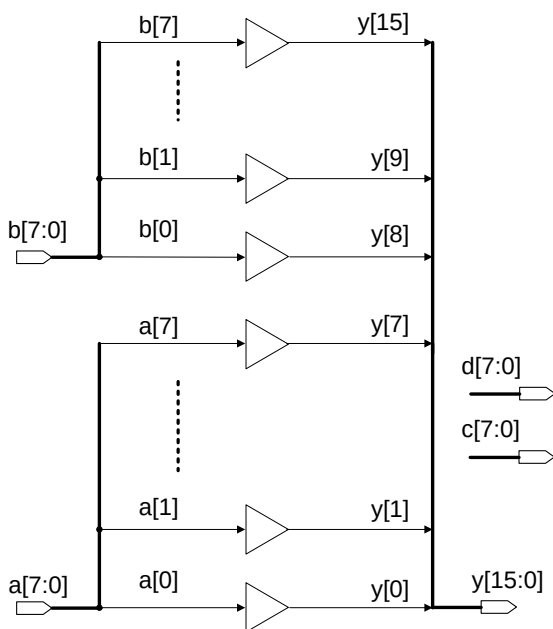


- Các trường hợp KHÁC





Hình 3-54. Netlist sau tổng hợp ví dụ 3-91 trường hợp WIDTH = 8



Hình 3-55. Netlist sau tổng hợp ví dụ 3-91 trường hợp WIDTH = 16

3.7.3 Lưu ý khi sử dụng phát biểu generate

3.7.3.1 Generate không phải là lặp một đoạn code

Xét mạch giải mã với số lượng đầu vào thay đổi theo tham số N và đầu ra nhiều hơn đầu vào 1 bit. Nếu giá trị ngõ vào được kiểm tra từ LSB đến MSB và có 1 bit là 1 thì ngõ ra là one-hot tương ứng với vị trí bit đó. Nếu ngõ vào là $N'b00...000$ thì ngõ ra là one-hot với bit MSB là 1 như mô tả ở bảng sau:

Bảng 3-6. Giá trị ngõ ra tương ứng với tham số N

in [N-1:0]	least_one [N:0]
N'bxx...xxx1	(N+1)'b00...0001
N'bxx...xx10	(N+1)'b00...0010
N'bxx...x100	(N+1)'b00...0100
.....	...
N'b10...000	(N+1)'b01...0000
N'b00...000	(N+1)'b10...0000

Giả sử người viết code mong muốn kiểm tra điều kiện từng bit với phát biểu **else if** nhiều lần như sau:

Ví dụ 3-92

```

if (in[0]==1'b1) least_one[N:0] = 1;
    // an expecting code of above code
    // but the following lines can't be generated
    else if (in[1]==1'b1) least_one[N:0] = 2**1;
    else if (in[2]==1'b1) least_one[N:0] = 2**2;
    else if (in[3]==1'b1) least_one[N:0] = 2**3;
    ...
    else if (in[N]==1'b1) least_one[N:0] = 2**N;

```

Với việc viết RTL code như trên sẽ không thực hiện được khi tham số N bị thay đổi. Vì vậy, để thực hiện đoạn code trên, phát biểu **for** được sử dụng để lặp lại đoạn RTL code kiểm tra điều kiện **else if**.

Ví dụ 3-93

```

genvar i;
generate
always @(in) begin
    if (in[0]==1'b1) least_one[N:0] = 1;
        for (i=1; i<N; i=i+1) begin: FIND_ONE_HOT
            //repeat following line
            else if (in[i]==1'b1) least_one[N:0] = 2**i;
        end
    else least_one[N:0] = 2**N;
endendgenerate

```

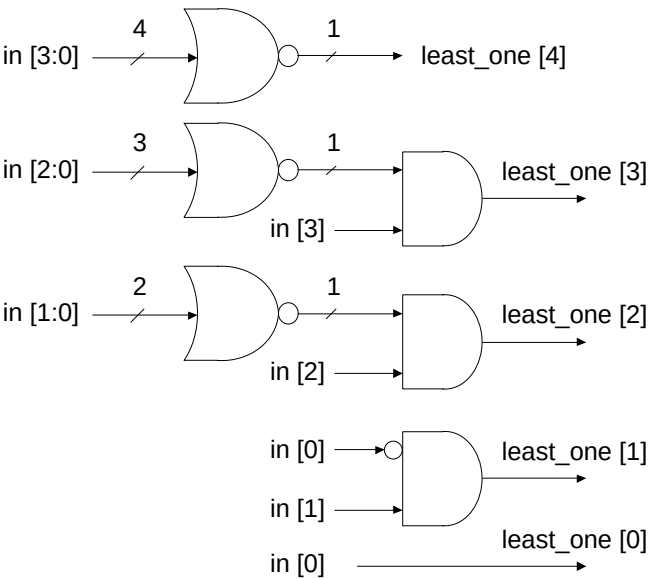


Nhưng thực tế, RTL code trong ví dụ 3-93 không tổng hợp được, vì đoạn RTL code trong vòng lặp **for** không được tạo ra nhiều lần như người viết mong muốn.

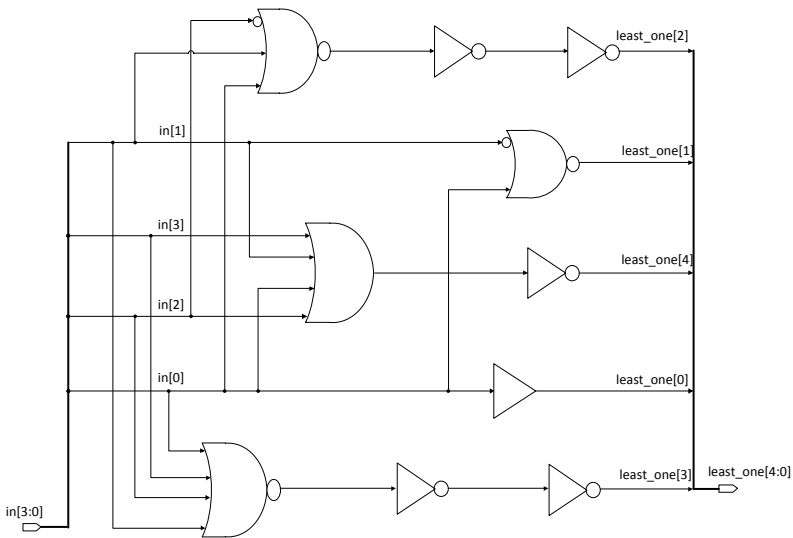
Để tổng hợp được RTL code trên, RTL code cần được viết lại và tạo ra đúng mạch chức năng là phát hiện vị trí của bit *1* từ LSB đến MSB.

Ví dụ 3-94

```
genvar i;  
generate  
    for (i = 1; i < N; i = i + 1) begin : U  
        assign least_one[i] = in[i] & ~|in[i - 1:0];  
    end  
endgenerate  
assign least_one[0] = in[0];  
assign least_one[N] = ~|in;
```



Hình 3-56. Mạch nguyên của ví dụ 3-94 khi N = 4



Hình 3-57. Netlist sau tổng hợp của ví dụ 3-94 khi N = 4

3.7.3.2 Phân biệt generate-loop với vòng lặp for trong always

Hai đoạn RTL code dưới đây tạo ra mạch có cùng chức năng, chỉ khác nhau là đoạn RTL code thứ nhất sử dụng **generate** và đoạn RTL code thứ hai thì không có. Các biến chạy trong hai trường hợp này phải khai báo khác nhau.

Ví dụ 3-95

RTL thứ nhất sử dụng generate-loop:

```
parameter ROWBITS = 4;
genvar i;
generate
  for (i = 0; i < ROWBITS; i = i + 1) begin: gen_order
    always @(posedge clk) begin
      temp[i] <= in[i];
    end
  end
endgenerate
```

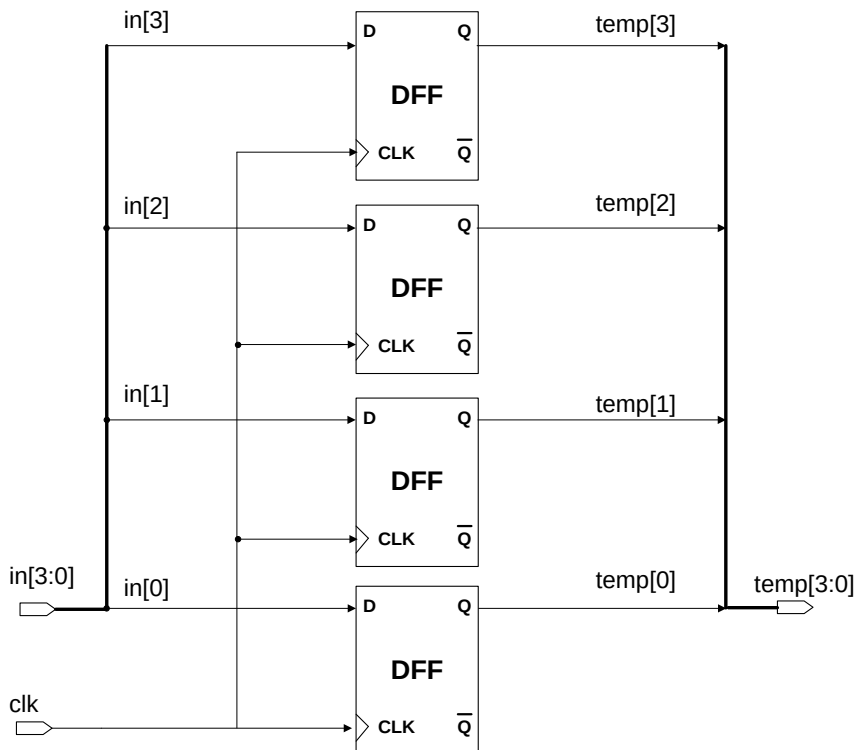
Đối với **generate** thì các vòng với biến chạy **genvar** phải được viết bên ngoài khối **always**. Bởi vì bên trong khối **always** các biến **genvar** sẽ không được hiểu là dùng cho **generate** nên sẽ sinh ra lỗi cú pháp khi tổng hợp.

Ví dụ 3-96

RTL code thứ hai chỉ sử dụng vòng lặp **for**:

```
parameter ROWBITS = 4;  
integer i;  
//generate  
  always @(posedge clk) begin  
    for (i = 0; i < ROWBITS; i = i + 1) begin: test  
      temp[i] <= in[i];  
    end  
  end  
//endgenerate
```

Đối với các vòng lặp bên trong khối **always** thì biến chạy cho **for...loop** phải được khai báo kiểu là **integer**.

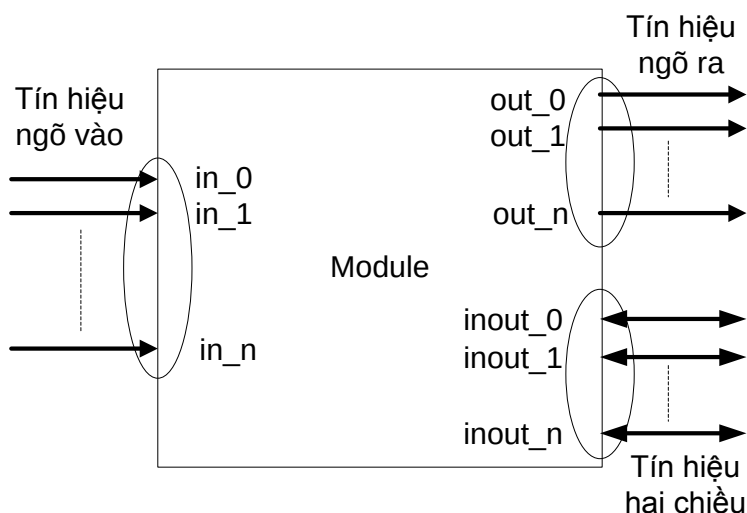


Hình 3-58. Netlist sau tổng hợp của ví dụ 3-95 và ví dụ 3-96

3.8 Module

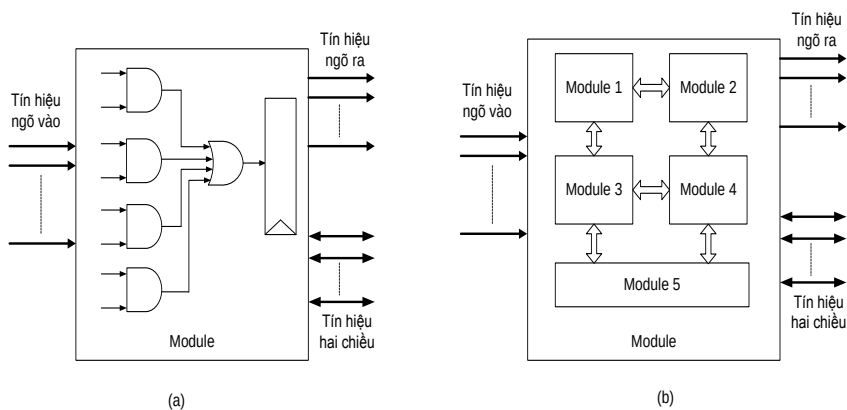
3.8.1 Định nghĩa module

Một module được định nghĩa bằng cặp từ khóa **module** và **endmodule**. Trong thiết kế tổng hợp được, module được dùng để mô hình hóa các khối chức năng.

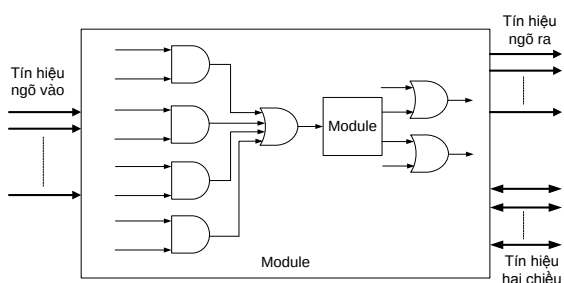


Hình 3-59. Mô hình module

Module có thể chứa thành phần RTL code mô tả trực tiếp mạch chức năng của khối thiết kế hoặc gọi các module khác bên trong nó.



Hình 3-60. Module mô tả mạch chức năng (a)
và module gọi các module khác (b)



Hình 3-61. Module kết chứa cả thành phần mô tả chức năng và gọi module khác

3.8.2 Cú pháp module

```
module module_name (<port_list>);
    port_declaration
    <signal_declaration>
    <module_description>
endmodule
```

Mô tả:

- *module_name* là tên **module** do người thiết kế đặt.
- *port_list* là danh sách liệt kê tất cả các tín hiệu ngõ vào, ngõ ra và tín hiệu hai chiều của **module**. Các tín hiệu trong danh sách được phân cách nhau bằng dấu phẩy “,”.
- *port_declaration* là phần khai báo các tín hiệu ngõ vào, ngõ ra và tín hiệu hai chiều của **module**.
- *signal_declaration* là phần khai báo các tín hiệu và biến sử dụng nội trong **module**.
- *module_description* là phần mô tả chức năng của **module**.

3.8.3 Cách sử dụng module

3.8.3.1 Khai báo port cho module

port_list và *port_declaration* của module được khai báo như sau:

```
module module_name (port_name0, port_name1, port_name2,
    ...);
input <type> <signed> <range> port_name0;
output <type> <signed> <range> port_name1;
inout <type> <signed> <range> port_name2;
    ...
endmodule
```

Port của module là những tín hiệu ngõ vào, tín hiệu ngõ ra hoặc tín hiệu hai chiều được sử dụng để kết nối với các thành phần bên ngoài module. Trong đó:

- **input** là từ khóa khai báo tín hiệu ngõ vào.
- **output** là từ khóa khai báo tín hiệu ngõ ra.
- **inout** là từ khóa khai báo tín hiệu hai chiều.
- *type* là kiểu dữ liệu của port.
- *range* là độ rộng của port.
- *port_name0*, *port_name1*, *port_name2*, ... là các tên của port. Trong một module, các tên port phải khác nhau và không trùng với bất kỳ từ khóa nào.
- **signed** chỉ ra loại dữ liệu của port là có dấu.

Ví dụ 3-97

```
module counter (clk, rst_n, load_dt, odd_val, data);  
    input wire clk;  
    input wire rst_n;  
    input wire load_dt;  
    output reg odd_val;  
    inout wire [3:0] data;  
    ...  
endmodule
```

Module *counter* có ba tín hiệu ngõ vào là *clk*, *rst_n* và *load_dt*; một tín hiệu ngõ ra là *odd_val*; một tín hiệu hai chiều là *data*. Tất cả các port của module đều là loại không dấu vì không có khai báo **signed**.

Kiểu dữ liệu của port có thể được khai báo riêng với phần khai báo port.

Ví dụ 3-98

```
module counter (sys_clk, rst_n, enable, load, odd_val,  
data);  
    // Declare all module ports  
    input sys_clk;  
    input rst_n;  
    input enable;  
    input load;
```

```

    output    odd_val;
    inout     [3:0] data;
    // Declare the data type of all ports
    wire sys_clk;
    wire rst_n;
    wire enable;
    wire load;
    reg odd_val;
    wire data;
    ...
endmodule

```

Tín hiệu kiểu **wire** có thể không cần phải khai báo vì đây là kiểu dữ liệu mặc định nhưng các phần mềm kiểm tra RTL code và trình tổng hợp có thể sẽ cảnh báo điều này.

Ví dụ 3-99

```

module counter (sys_clk, rst_n, enable, load, odd_val,
data);
// Declare all module ports
input sys_clk;
input rst_n;
input enable;
input load;
output odd_val;
inout [3:0] data;
// Declare the data type of all ports
reg odd_val;
...
endmodule

```

Các port cùng loại có kiểu dữ liệu và độ rộng giống nhau có thể được khai báo với một từ khóa và phân cách nhau bằng dấu phẩy “,” như các tín hiệu ngõ vào trong ví dụ sau đây.

Ví dụ 3-100

```

module counter (sys_clk, rst_n, enable, load, odd_val,
data);
// All input ports are declared on a line
input sys_clk, rst_n, enable, load;

```

```

output reg    odd_val;
inout [3:0] data;
...
endmodule

```

Ngoài cách khai báo port như đã giới thiệu, gồm phần liệt kê tín hiệu trên *port_list* và phân khai port, thì các port còn có thể được khai báo gộp chung phân định nghĩa port vào trong *port_list*.

Ví dụ 3-101

```

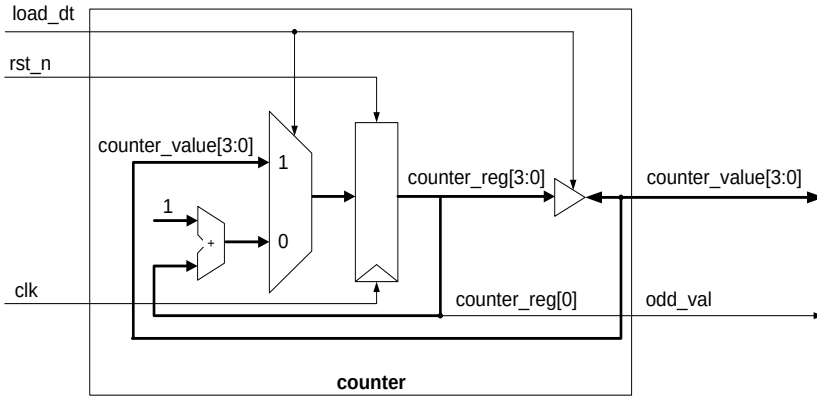
module counter (input    wire  sys_clk,
                  input    wire  rst_n,
                  input    wire  enable,
                  input    wire  load,
                  output reg   odd_val,
                  inout    wire [3:0] data);
...
endmodule

```

3.8.3.2 Mô tả chức năng module

Tất cả các thành phần có thể tổng hợp được của Verilog đều có thể được sử dụng để mô tả chức năng của một module. Một nguyên tắc quan trọng khi mô tả RTL code trong module là tất cả các tín hiệu và tham số phải được khai báo trước khi sử dụng. Tín hiệu kiểu **wire** có độ rộng một bit thì không cần phải khai báo trừ khi nó là port của module.

Giả sử, một module *counter* có chức năng đếm lên liên tục theo xung clock *clk* và được reset bất đồng bộ khi tín hiệu *rst_n* mức thấp. Một tín hiệu *load_dt* sử dụng để nạp giá trị cho *counter* từ port *data* khi nó tích cực mức 1. Đồng thời, *data* sẽ thể hiện giá trị bộ đếm khi *load_dt* bằng 0. Tín hiệu ngõ ra *odd_val* sẽ tích cực mức 1 nếu bộ đếm mang giá trị số lẻ.



Hình 3-62. Sơ đồ nguyên lý của module counter

Module *counter* được mô tả bằng RTL code như sau:

Ví dụ 3-102

```

module counter (clk, rst_n, load_dt, odd_val,
counter_value);
// Declare all module ports
input clk;
input rst_n;
input load_dt;
output wire odd_val;
inout wire [3:0] counter_value;
//Declare all internal signal
reg [3:0] counter_reg;
//Describe all circuits
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        counter_reg[3:0] <= 4'd0;
    else if (load_dt)
        counter_reg[3:0] <= counter_value[3:0];
    else
        counter_reg[3:0] <= counter_reg[3:0] + 4'd1;
end
//
assign counter_value[3:0] = load_dt? 4'bzzzz:
                                counter_reg[3:0];
//
assign odd_val = counter_reg[0];
endmodule

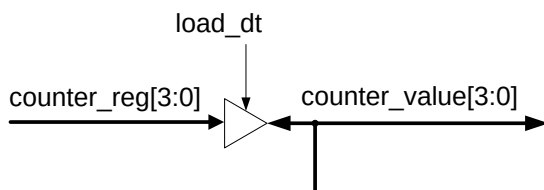
```

Trong module *counter*, tín hiệu *counter_reg* chính là bộ đếm và là tín hiệu nội. Phần thân module sử dụng một khối **always** và hai phát biểu

assign để mô tả chức năng của module. Vì port *counter_value* của module *counter* vừa được dùng làm ngõ ra cho bộ đếm, vừa được dùng để nạp giá trị cho bộ đếm nên *counter_value* là một tín hiệu hai chiều. Ngoài việc phải khai báo **inout** cho tín hiệu *counter_value* thì trong phần thân module, chức năng “hai chiều” của tín hiệu *counter_value* cũng phải được mô tả ở dòng RTL code:

```
assign counter_value[3:0] = load_dt? 4'bzzzz:
                                counter_reg[3:0];
```

Dòng RTL code trên sẽ mô hình hóa công ba trạng thái trong sơ đồ nguyên lý hình 3-62.



Hình 3-63. Công ba trạng thái trong sơ đồ nguyên lý hình 3-62

Mỗi khối **always** và phát biểu **assign** sẽ tạo ra một mạch chức năng khi tổng hợp. Thứ tự mô tả của các khối **always** và phát biểu **assign** trong phần thân module không ảnh hưởng đến chức năng mạch được tổng hợp.

Ví dụ 3-103

```
module counter (clk, rst_n, load_dt, odd_val, data);
...
assign counter_value[3:0] = load_dt? 4'bzzzz:
                                counter_reg[3:0];
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        counter_reg[3:0] <= 4'd0;
    else if (load_dt)
        counter_reg[3:0] <= counter_value[3:0];
    else
        counter_reg[3:0] <= counter_reg[3:0] + 4'd1;
```

```

end
assign odd_val = counter_reg[0];
endmodule

```

Việc đảo vị trí khối **always** và các phát biểu **assign** ở ví dụ 3-102 như ví dụ 3-103 cũng không ảnh hưởng đến chức năng mạch.

3.8.4 Gọi module

Gọi module là sử dụng lại một module có sẵn trong một module khác.

3.8.4.1 Cú pháp gọi module

```

module_name    <# (parameter_assignments)>
               <instance_name>
               <array_instantiation>
               (port_connections);

```

Mô tả:

- *module_name* là tên module được gọi
- *parameter_assignments* là các giá trị được sử dụng để gán cho các **parameter** có trong module được gọi.
- *instance_name* là tên được sử dụng để phân biệt với giữa các module được gọi. Tên này phải được đặt khác nhau nhưng không cần khác *module_name*.
- *array_instantiation* là khai báo tạo ra mảng module được gọi.
- *port_connections* là khai báo kết nối tín hiệu đến các port của module. Thành phần kết nối đến port module có thể là một tín hiệu hoặc một hằng số cố định. Có hai kiểu khai báo kết nối:
 - Kết nối theo thứ tự: chỉ khai báo tên các tín hiệu kết nối phân cách nhau bởi dấu phẩy “,”. Thứ tự tên tín hiệu kết nối phải theo đúng thứ tự tín hiệu trong *port_list* của module được gọi.
 - Kết nối theo tên: Khai báo đầy đủ tên port của module được gọi và tên tín hiệu kết nối tương ứng mà không cần quan tâm đến thứ tự port theo cú pháp:

```

.port_name (connection_name)

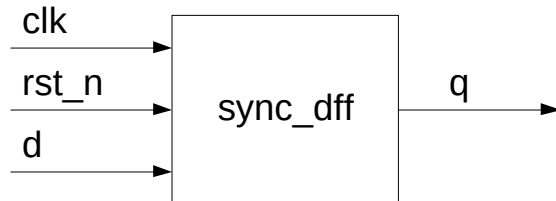
```

- o *port_name* là tên port của module được gọi.

- o *connection_name* là tên của tín hiệu sẽ kết nối đến port. Tên này có thể giống hoặc khác với *port_name*.

3.8.4.2 Cách gọi module cơ bản

Một module có thể được gọi nhiều lần trong một module khác hoặc được gọi trong nhiều module khác nhau. Giả sử, một module mô hình hóa Flip-Flop như sau:



Hình 3-64. Sơ đồ tín hiệu của *sync_dff*

Module *sync_dff* sẽ được gọi bốn lần để tạo ra module thanh ghi dịch bốn bit *shift_reg*.

Ví dụ 3-104

```

module shift_reg (clk, rst_n, d_in, q_out);
// Declare all module ports
input clk;
input rst_n;
input d_in;
outputwire q_out;
// Declare all internal signals
wire q_0, q_1, q_2;
// Describe module body
sync_dff    dff0 (clk, rst_n, d_in, q_0);

sync_dff    dff1 (clk, rst_n, q_0, q_1);

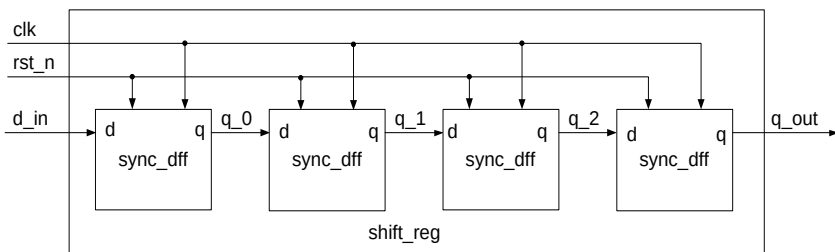
sync_dff    dff2 (clk, rst_n, q_1, q_2);

sync_dff    dff3 (clk, rst_n, q_2, q_out);

endmodule
  
```

Trong ví dụ trên, *sync_dff* là *module_name*. *dff0*, *dff1*, *dff2* và *dff3* là *instance_name*. Module *sync_dff* được gọi sử dụng phương pháp kết nối theo thứ tự. Các tín hiệu kết nối *clk*, *rst_n*, *d_in* và *q_0* của *dff0* sẽ

được kết nối theo thứ tự port trong module *sync_dff* là *clk*, *rst_n*, *d* và *q*. *dff1*, *dff2* và *dff3* cũng tương tự.



Hình 3-65. Sơ đồ nguyên lý của ví dụ 3-104

Nếu sử dụng phương pháp kết nối theo tên thì các tín hiệu ngõ vào và ngõ ra của *sync_dff* được khai báo đầy đủ. Các tín hiệu của module *shift_reg* được đặt trong các dấu ngoặc đơn theo sau các tín hiệu tương ứng muốn kết nối với module *sync_dff*.

Ví dụ 3-105

```

module shift_reg (clk, rst_n, d_in, q_out);
// Declare all module ports
input    clk;
input    rst_n;
input    d_in;
output   q_out;
// Declare all internal signals
wire    q_0, q_1, q_2;
// Describe module body
sync_dff  dff_0 (.clk(clk),
                .rst_n(rst_n),
                .d(d_in),
                .q(q_0));

sync_dff  dff_1 (.clk(clk),
                .rst_n(rst_n),
                .d(q_0),
                .q(q_1));

sync_dff  dff_2 (.clk(clk),
                .rst_n(rst_n),
                .d(q_1),
                .q(q_2));

```

```

sync_dff      dff_3 (.clk(clk),
                    .rst_n(rst_n),
                    .d(q_2),
                    .q(q_out));

```

endmodule

Phương pháp kết nối theo tên tín hiệu không yêu cầu kết nối theo thứ tự của các tín hiệu trong module *sync_dff*. Phương pháp này còn giúp việc quản lý các tín hiệu kết nối dễ hiểu và dễ gỡ lỗi hơn.

Nếu tín hiệu ngõ vào của module được gọi không kết nối đến bất kỳ tín hiệu hoặc giá trị logic cố định nào thì giá trị của tín hiệu đó là sẽ là *z* và có thể tạo ra giá trị không xác định ở các tín hiệu ngõ ra khi mô phỏng. Đồng thời, điều này có thể tạo ra mạch không mong muốn khi tổng hợp nên các module được gọi nên được khai báo và kết nối đầy đủ. Các tín hiệu ngõ vào không muốn sử dụng nên được kết nối với một giá trị xác định là *0* hoặc *1*.

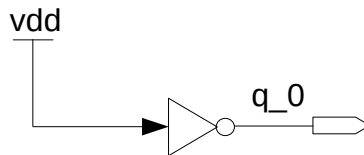
RTL code mô tả tín hiệu *rst_n* của module *sync_dff* không được kết nối.

Ví dụ 3-106

```

sync_dff      dff  (.clk(clk),
                    .d(d_in),
                    .q(q_0));

```



Hình 3-66. Mạch tổng hợp trên DC cho ví dụ 3-106

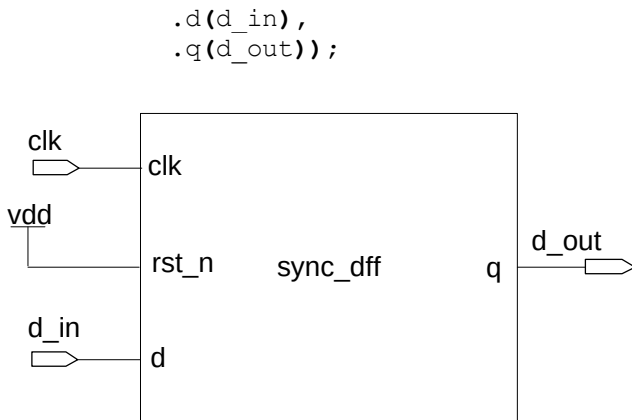
RTL code mô tả tín hiệu *rst_n* của module *sync_dff* được kết nối với một giá trị xác định là *1*.

Ví dụ 3-107

```

sync_dff      dff (.clk(clk),
                    .rst_n(1'b1),

```



Hình 3-67. Netlist sau tổng hợp cho ví dụ 3-107

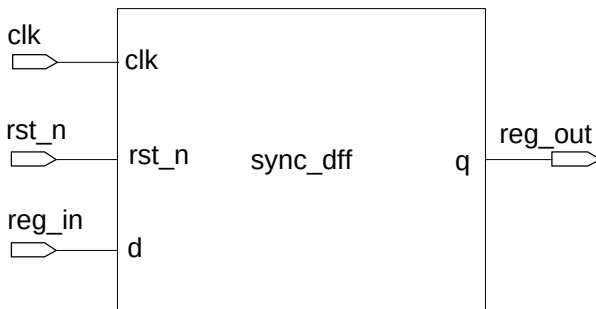
Ví dụ 3-108

RTL code mô tả tín hiệu của module *sync_dff* được kết nối đầy đủ.

```

sync_dff      dff (.clk(clk) ,
                  .rst_n(rst_n) ,
                  .d(d_in) ,
                  .q(d_out)) ;

```



Hình 3-68. Netlist sau tổng hợp cho ví dụ 3-108

Phép nối sẽ được sử dụng khi nhiều tín hiệu khác nhau kết nối đến các bit khác nhau của cùng một port. Ví dụ, bốn tín hiệu một bit a, b, c và d kết nối đến bốn bit ngõ vào *reg_in* của module *reg_4bit* thì phát biểu gọi module sẽ như sau:

Ví dụ 3-109

```

reg_4bit  reg_4bit( .clk(clk) ,
                   .rst_n(rst_n) ,
                   .reg_in({a,b,c,d}) ,
                   .reg_out(reg_out[3:0])) ;

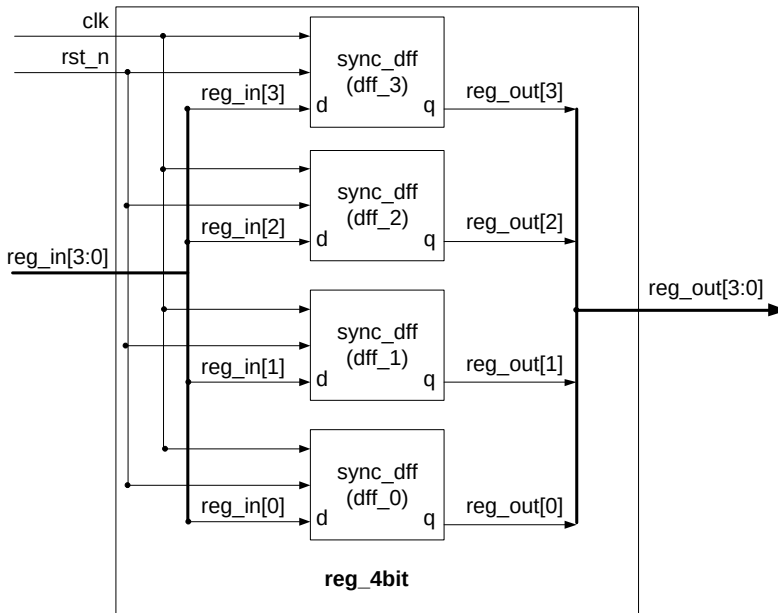
```

3.8.4.3 Cách gọi tạo mảng module

Module *sync_dff* có thể được dùng để tạo ra một thanh ghi bốn bit bằng bốn phát biểu gọi module.

Ví dụ 3-110

```
module reg_4bit (clk, rst_n, reg_in, reg_out);  
input          clk;  
input          rst_n;  
input  [3:0]    reg_in;  
output [3:0]    reg_out;  
  
sync_dff      dff_0(.clk(clk),  
                    .rst_n(rst_n),  
                    .d(reg_in[0]),  
                    .q(reg_out[0]));  
  
sync_dff      dff_1(.clk(clk),  
                    .rst_n(rst_n),  
                    .d(reg_in[1]),  
                    .q(reg_out[1]));  
  
sync_dff      dff_2(.clk(clk),  
                    .rst_n(rst_n),  
                    .d(reg_in[2]),  
                    .q(reg_out[2]));  
  
sync_dff      dff_3(.clk(clk),  
                    .rst_n(rst_n),  
                    .d(reg_in[3]),  
                    .q(reg_out[3]));  
  
endmodule
```



Hình 3-69. Sơ đồ nguyên lý ví dụ 3-110

Module *sync_dff* được gọi và kết nối song song trong module *reg_4bit*. Số lượng module kết nối càng nhiều thì số lượng phát biểu gọi cũng phải tăng lên tương ứng. RTL code đơn giản hóa trường hợp này bằng cách cho phép khai báo tạo mảng module đặt ngay sau tên định danh *instance_name*. Bốn phát biểu gọi trong ví dụ 3-110 được thay bằng một phát biểu đơn giản như sau:

Ví dụ 3-111

```
sync_dff dff[3:0] (.clk(clk),
                  .rst_n(rst_n),
                  .d(reg_in[3:0]),
                  .q(reg_out[3:0]));
```

Phát biểu gọi module kèm khai báo tạo mảng phải chú ý đến độ rộng các tín hiệu. Giả sử, module thanh ghi 16 bit được tạo ra từ module *sync_dff* có một ngõ vào *d* và một ngõ ra *q* thì phải dùng mười sáu phát biểu gọi đơn lẻ hoặc một phát biểu gọi mảng có độ rộng [15:0].

Ví dụ 3-112

```
sync_dff dff[15:0] (.clk(clk),
```



```

        .rst_n(rst_n),
        .d(reg_in[15:0]),
        .q(reg_out[15:0]));

```

Nếu module *reg_4bit* được sử dụng để tạo ra thanh ghi 16 bit thì chỉ phải dùng bốn phát biểu gọi đơn lẻ hoặc một phát biểu mảng có độ rộng *[3:0]* thay vì phải khai báo *[15:0]* khi sử dụng module *sync_dff*.

Ví dụ 3-113

```

reg_4bit dff[3:0] (.clk(clk),
                  .rst_n(rst_n),
                  .reg_in(reg_in[15:0]),
                  .reg_out(reg_out[15:0]));

```

Thiết kế module *reg_16* bit có 16 bit dữ liệu ngõ ra *q_out* và có 16 bit dữ liệu ngõ gồm tám bit *din_low* và tám bit *din_high*. Nếu module *reg_4bit* được sử dụng để thiết kế module *reg_16bit* thì module này được gọi lại bốn lần như ví dụ sau:

Ví dụ 3-114

```

module reg_16bit (clk, rst_n, din_low, din_high, q_out);

input          clk;
input          rst_n;
input  [7:0]    din_low;
input  [7:0]    din_high;
output [15:0]   q_out;
// din_low[3:0]
reg_4bit reg_4bit_0(.clk(clk),
                   .rst_n (rst_n),
                   .reg_in(din_low [3:0]),
                   .reg_out(q_out[3:0]));

// din_low[7:4]
reg_4bit reg_4bit_1(.clk(clk),
                   .rst_n (rst_n),
                   .reg_in(din_low[7:4]),
                   .reg_out(q_out[7:4]));

// din_high[3:0]
reg_4bit      reg_4bit_2(.clk(clk),
                       .rst_n(rst_n),
                       .reg_in(din_high[3:0]),
                       .reg_out(q_out[11:8]));

// din_high[7:4]

```

```

reg_4bit      reg_4bit_3(.clk(clk),
                        .rst_n(rst_n),
                        .reg_in(din_high[7:4]),
                        .reg_out(q_out[15:12]));

```

endmodule

Nếu sử dụng cách tạo mảng gồm bốn module *reg_4bit* để thiết kế module *reg_16* bit thì ngõ vào *reg_in* là 16 bit. Các tín hiệu *din_low* và *din_high* là tám bit nên cần phải ghép trước khi kết nối như ví dụ sau.

Ví dụ 3-115

```

module reg_16bit(clk, rst_n, din_low, din_high, q_out);
input      clk;
input      rst_n;
input  [7:0] din_low;
input  [7:0] din_high;
output [15:0] q_out;

```

```

reg_4bit      reg_4bit[3:0] (
                        .clk(clk),
                        .rst_n(rst_n),
                        .reg_in({din_high[7:0],din_low[7:0]}),
                        .reg_out(q_out[15:0]));

```

endmodule

3.8.4.4 Cách gọi module kèm thay đổi parameter

Khi gọi các module có sử dụng khai báo **parameter** bên trong, các giá trị **parameter** này có thể được gán lại giá trị mong muốn trước *instance_name* ở phát biểu gọi module. RTL code sau đây mô hình hóa một bộ nhớ có thể thay đổi được độ rộng bus địa chỉ và độ rộng bus dữ liệu thông qua hai **parameter** *AWIDTH* và *DWIDTH*.

Ví dụ 3-116

```

module mem_reg (clk, rd, wr, addr, datain, dataout);
// Declare parameters
parameter AWIDTH = 8;
parameter DWIDTH = 16;
// Declare all module ports
input      clk;

```

```

input                rd;
input                wr;
input      [AWIDTH-1:0]  addr;
input      [DWIDTH-1:0] datain;
output reg [DWIDTH-1:0] dataout;
...
endmodule

```

Nếu chỉ gọi và kết nối module *mem_reg* không khai báo **parameter** thì *AWIDTH* và *DWIDTH* sẽ lấy giá trị đã được gán trong module là 8 và 16.

Ví dụ 3-117

```

module mem_inst (clk, rd, wr, addr, d_in, d_out);
input      clk;
input      rd;
input      wr;
input  [7:0]  addr;
input  [15:0] d_in;
output [15:0] d_out;
mem_reg    mem_reg(.clk(clk),
                    .rd(rd),
                    .wr(wr),
                    .addr(addr[7:0]),
                    .datain(d_in[15:0]),
                    .dataout(d_out[15:0]));
endmodule

```

Nếu muốn gọi và sử dụng module này với độ rộng bus địa chỉ *AWIDTH* là 4 và độ rộng bus dữ liệu *DWIDTH* là 8 mà không phải sửa lại giá trị của chúng trong module thì phát biểu gọi module được viết bằng cách gán theo thứ tự như sau:

Ví dụ 3-118

```

mem_reg    #(4,8)mem_reg(.clk(clk),
                        .rd(rd),
                        .wr(wr),
                        .addr(addr[4:0]),
                        .datain(d_in[7:0]),
                        .dataout(d_out[7:0]));

```

Trong phát biểu gọi module trên, thứ tự trước sau của các giá trị gán tại phát biểu gọi module sẽ tương ứng với thứ tự khai báo của các

parameter trong module đó. Giá trị 4 sẽ được gán cho **parameter** *AWIDTH* vì nó được khai báo đầu tiên. Giá trị 8 được gán cho **parameter** kế tiếp là *DWIDTH* trong module *mem_reg*. Cách gán theo tên được thực hiện như sau:

Ví dụ 3-119

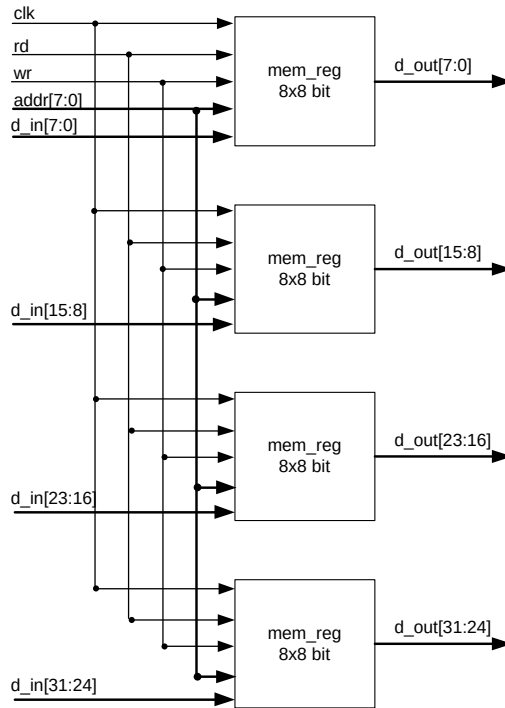
```
mem_reg    #(.AWIDTH(4), .DWIDTH(8))
           mem_reg ( .clk(clk),
                     .rd(rd),
                     .wr(wr),
                     .addr(addr[3:0]),
                     .datain(d_in[7:0]),
                     .dataout(d_out[7:0]));
```

Vì cách gán theo tên các **parameter** sẽ không phụ thuộc vào thứ tự gán nên có thể đảo thứ tự gán của hai **parameter** như sau:

Ví dụ 3-120

```
mem_reg    #(.DWIDTH(8), .AWIDTH(4))
           mem_reg ( .clk(clk),
                     .rd(rd),
                     .wr(wr),
                     .addr(addr[3:0]),
                     .datain(d_in[7:0]),
                     .dataout(d_out[7:0]));
```

Thiết kế một module *mem_top* có bus dữ liệu là 32 và địa chỉ là 8 sử dụng một mảng gồm bốn module *mem_reg*. Mỗi module *mem_reg* có bus dữ liệu là 8 và địa chỉ là 8 như mô tả ở hình sau:



Hình 3-70. Mảng gồm bốn module mem_reg

RTL code sử dụng khai báo **parameter** và tạo mảng bốn module *mem_reg* có bus dữ liệu là 8 và địa chỉ là 8.

Ví dụ 3-121

```

module mem_top (clk, rd, wr, addr, d_in, d_out);
input          clk;
input          rd;
input          wr;
input    [7:0]  addr;
input    [31:0] d_in;
output    [31:0] d_out;

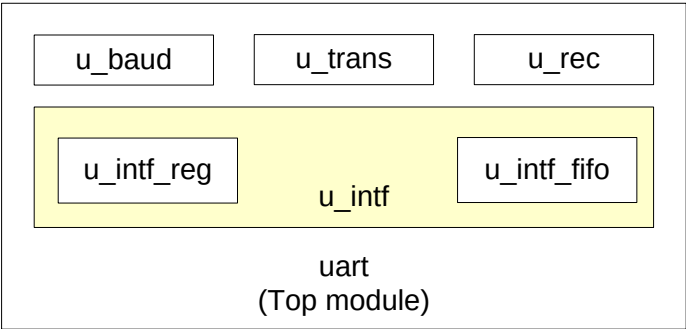
    mem_reg    #(.AWIDTH(8), .DWIDTH(8))
                mem_reg[3:0] (.clk(clk),
                              .rd(rd),
                              .wr(wr),
                              .addr(addr[7:0]),
                              .datain(d_in[31:0]),
                              .dataout(d_out[31:0]));

endmodule

```

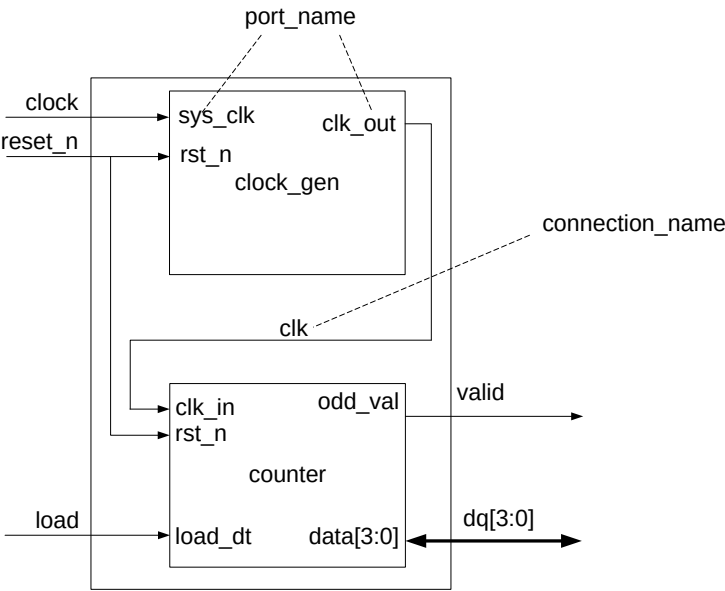
3.8.4.5 Ứng dụng gọi module

Việc gọi module cho phép kết nối các module với nhau và tạo ra các thiết kế có cấu trúc thứ bậc. Hình 3-71 mô tả một thiết kế có cấu trúc thứ bậc gồm bảy module. *uart* là module cấp một sẽ gọi bốn module cấp hai là *u_intf*, *u_baud*, *u_trans* và *u_rec*. Module cấp hai *u_intf* sẽ gọi hai module cấp 3 là *u_intf_reg* và *u_intf_fifo*. Trong thiết kế này, *uart* được gọi là top module vì nó không bị gọi bởi bất kỳ module nào khác.



Hình 3-71. Thiết kế có cấu trúc thứ bậc

Module *connection_module* gồm hai module con *clock_gen* và *counter* như hình 3-72.



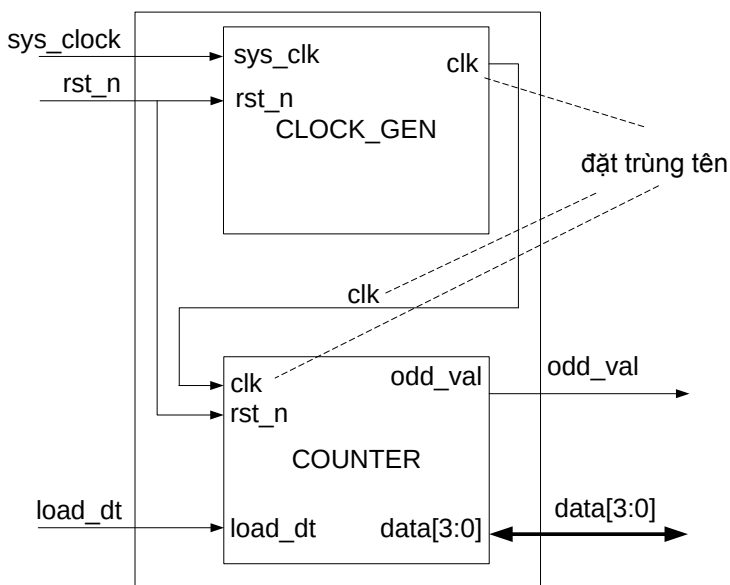
Hình 3-72. Minh họa kết nối giữa hai module

Ví dụ 3-122

```
module connection_module (clock, reset_n,  
load,valid,dq);  
// Declare all module ports  
input clock;  
input reset_n;  
input load;  
outputwire valid;  
inout wire [3:0] dq;  
// Declare all internal signals  
wire clk;  
// Call the modules  
clock_gen clk_gen (.sys_clk(clock),  
                    .rst_n(reset_n),  
                    .clk_out(clk));  
counter count (.clk_in(clk),  
               .rst_n(reset_n),  
               .load_dt(load),  
               .odd_val(valid),  
               .data(dq[3:0]));  
  
endmodule
```

connection_module gọi và kết nối *clock_gen* và *counter*. Tín hiệu nội *clk* là tín hiệu kết nối ngõ ra *clk_out* của khối *clock_gen* đến ngõ vào *clk_in* của khối *counter*. Tín hiệu ngõ vào *clk* kết nối đến ngõ vào *sys_clk* của module *clock_gen*. Tín hiệu ngõ vào *reset_n* kết nối đến ngõ vào *rst_n* của cả hai module. Tín hiệu ngõ vào *load* kết nối đến tín hiệu *load_dt* của module *counter*. Tín hiệu ngõ ra *valid* và tín hiệu hai chiều *dt[3:0]* kết nối trực tiếp từ tín hiệu *odd_val* và *data[3:0]*.

Trong cùng một thiết kế, để dễ kiểm soát các tín hiệu giao tiếp giữa các module thì tên tín hiệu ngõ vào của các module sẽ được đặt trùng với tên tín hiệu ngõ ra của module sẽ kết nối đến nó và trùng với tên tín hiệu kết nối. *clk_out*, *clk_in* và *clk* trong ví dụ 3-122 sẽ được đặt cùng một tên là *clk*.



Hình 3-73. Tên ngõ vào trùng với tên ngõ ra và tín hiệu kết nối

Module có thể vừa chứa các thành phần mô tả mạch chức năng như **always**, **assign**, **function**, các toán tử, ...; vừa gọi các module khác để sử dụng.

Ví dụ 3-123

```

module compound_module (sys_clk, rst_n, enable, load,
odd_val, data);
// Declare all module ports
input sys_clk;
input rst_n;
input enable;
input load;
outputwire odd_val;
inout wire [3:0] data;
// Declare all internal signals
wire clk;
wire load_dt;
// Call the modules
clock_gen ins1 (.sys_clk(sys_clk),
               .rst_n(rst_n),
               .clk(clk));
counter      ins2 (.clk(clk),
               .rst_n(rst_n),
               .load_dt(load_dt),

```

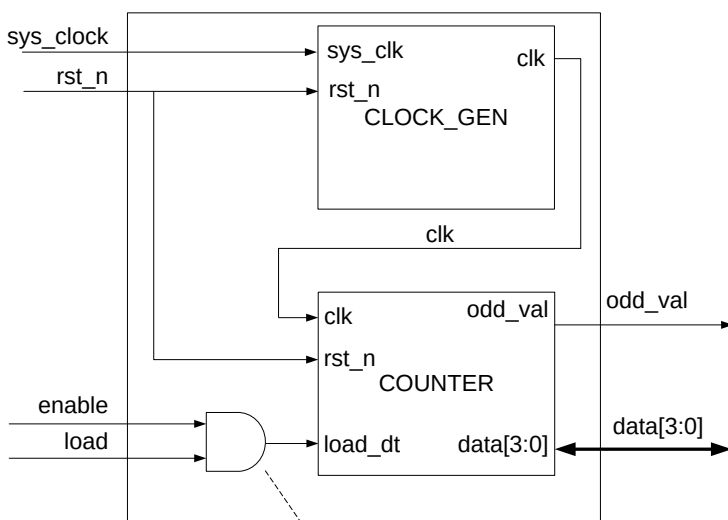


```

        .odd_val(odd_val),
        .data(data[3:0]));
// Describe all circuits
assign load_dt    = load & enable;
endmodule

```

compound_module gọi *clock_gen*, *counter* và chứa một cổng AND để tạo ra tín hiệu *load_dt*.



Hình 3-74. Cấu trúc module của ví dụ 3-123

4

PHƯƠNG PHÁP MÔ TẢ MẠCH

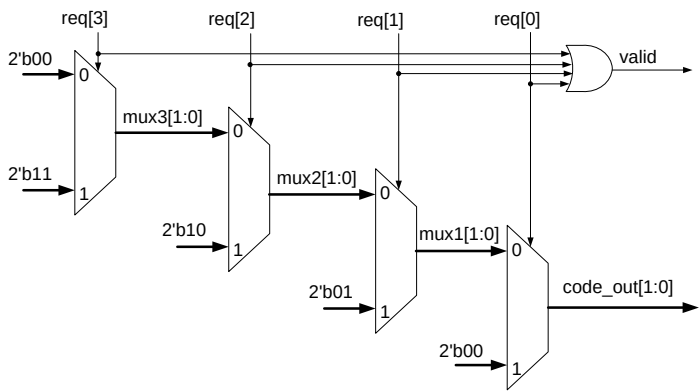
Nội dung chương này trình bày các phương pháp và những lưu ý khi mô tả mạch số bằng Verilog HDL. Các phương pháp mô tả mạch sẽ được trình bày dựa trên hai mô hình mạch cơ bản là mạch tổ hợp và mạch tuần tự. Thông qua các phân tích và ví dụ cụ thể, người sử dụng Verilog HDL sẽ lựa chọn phương pháp mô tả mạch đúng đắn và thích hợp.

4.1 Mạch tổ hợp

4.1.1 Phương pháp mô tả mạch tổ hợp

4.1.1.1 Giới thiệu chung

Verilog HDL hỗ trợ nhiều phương pháp khác nhau để mô tả mạch tổ hợp. Tùy vào từng trường hợp cụ thể, các phương pháp được lựa chọn phù hợp để tạo ra mạch chức năng mong muốn. Xét mạch mã hóa ưu tiên có sơ đồ nguyên lý như sau:



Hình 4-1. Sơ đồ nguyên lý mạch mã hóa ưu tiên

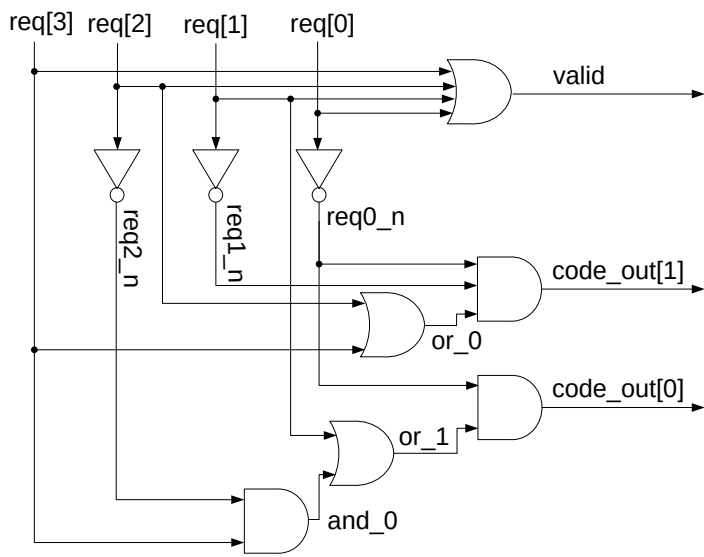
Ngõ ra *valid* sẽ tích cực mức 1 khi một trong các ngõ vào *req_0*, *req_1*, *req_2* và *req_3* tích cực mức 1. Thứ tự ưu tiên của các ngõ vào lần lượt là *req_0*, *req_1*, *req_2* và *req_3*. Ngõ ra *code_out[1:0]* sẽ thể hiện giá trị của ngõ vào tích cực có mức ưu tiên cao nhất.

Bảng 4-1. Bảng sự thật cho ngõ ra *valid* và *code_out*

req_3	req_2	req_1	req_0	valid	code_out[1:0]
0	0	0	0	0	00
1	0	0	0	1	11
x	1	0	0	1	10
x	x	1	0	1	01
x	x	x	1	1	00
x nghĩa là don't care					

Mạch nguyên lý được chuyển về dạng cổng logic bằng cách áp dụng phương pháp tổng các tích, tích các tổng hoặc bìa Karnaugh. Áp dụng phương pháp tổng các tích cho các ngõ ra *valid*, *code_out[1]* và *code_out[0]* dựa trên bảng 4-1, biểu thức các ngõ ra như sau:

$$\begin{aligned} \text{valid} &= \text{req}[3] + \text{req}[2] + \text{req}[1] + \text{req}[0] \\ \text{code_out}[1] &= (\text{req}[3] + \text{req}[2]).\overline{\text{req}[1]}. \overline{\text{req}[0]} \\ \text{code_out}[0] &= (\text{req}[3].\overline{\text{req}[2]} + \text{req}[1]).\overline{\text{req}[0]} \end{aligned}$$



Hình 4-2. Mạch nguyên lý mức cổng của bộ mã hóa ưu tiên hình 1-1

Các phương pháp mô tả RTL code cho mạch tổ hợp gồm:

- Mô tả mạch tổ hợp bằng phép gán **assign**
- Mô tả mạch tổ hợp bằng cấu trúc **always**
- Mô tả mạch tổ hợp bằng **function**
- Mô tả mạch tổ hợp bằng cách ghép nối các thành phần có sẵn
- Mô tả mạch tổ hợp bằng phương pháp kết hợp

4.1.1.2 Mô tả mạch tổ hợp bằng phép gán assign

Mạch tổ hợp có thể được mô tả một cách đơn giản bằng phép gán **assign**. Mạch nguyên lý hình 4-1 được mô tả bằng phép gán **assign** như sau.

Ví dụ 4-1

```
module p_encoder (req, code_out, valid);
input [3:0] req;
output wire [1:0] code_out;
output wire valid;
wire [1:0] MUX3, MUX2, MUX1;
//Circuit
assign MUX3[1:0] = (req[3])? 2'b11: 2'b00;
assign MUX2[1:0] = (req[2])? 2'b10: MUX3[1:0];
assign MUX1[1:0] = (req[1])? 2'b01: MUX2[1:0];
//the code_out port
assign code_out[1:0] = (req[0])? 2'b00: MUX1[1:0];
//the valid port
assign valid = |req[3:0];
endmodule
```

Bốn phép gán **assign** để tạo ra ngõ ra *code_out* có thể được thay bằng một phép gán **assign** duy nhất với các toán tử điều kiện lồng nhau.

Ví dụ 4-2

```
module p_encoder (req, code_out, valid);
input [3:0] req;
output [1:0] code_out;
output valid;
//the code_out port
assign code_out[1:0] = req[0]? 2'b00:
                      (req[1]? 2'b01:
                      (req[2]? 2'b10:
                      req[3]? 2'b11: 2'b00)));
//the valid port
assign valid = |req[3:0];
endmodule
```

Mạch nguyên lý mức cổng của bộ mã hóa ưu tiên ở hình 4-2 cũng có thể được mô tả bằng phép gán **assign** kết hợp với các toán tử bit-wise.

Ví dụ 4-3

```
module p_encoder (req, code_out, valid);
input [3:0] req;
output [1:0] code_out;
output valid;
//the code_out port
```

```

assign code_out[1] = (req[3]|req[2])& ~req[1]& ~req[0];
assign code_out[0] = (req[3]& ~req[2]|req[1])& ~req[0];
//the valid port
assign valid = |req[3:0];
endmodule

```

4.1.1.3 Mô tả mạch tổ hợp bằng cấu trúc **always**

Mạch tổ hợp được mô tả bằng cấu trúc **always** kết hợp với các phép gán blocking. mạch nguyên lý bộ mã hóa ưu tiên hình 4-1 được mô tả bằng cấu trúc **always** như sau.

Ví dụ 4-4

```

module p_encoder (req, code_out, valid);
input      [3:0] req;
output reg [1:0] code_out;
output reg valid;

    always @ (*) begin
        if (req[0])      code_out[1:0] = 2'b00;
        else if (req[1]) code_out[1:0] = 2'b01;
        else if (req[2]) code_out[1:0] = 2'b10;
        else if (req[3]) code_out[1:0] = 2'b11;
        else            code_out[1:0] = 2'b00;
    end

    always @ (*) begin
        valid = |req[3:0];
    end
endmodule

```

Phát biểu **if** trong RTL code trên có thể được thay bằng phát biểu **case**.

```

always @ (*) begin
    casez (req[3:0])
        4'b???1: code_out[1:0] = 2'b00;
        4'b???10: code_out[1:0] = 2'b01;
        4'b?100: code_out[1:0] = 2'b10;
        4'b1000: code_out[1:0] = 2'b11;
        default: code_out[1:0] = 2'b00;
    endcase
end

```

Mạch nguyên lý mức cổng hình 4-2 cũng có thể được mô tả bằng cấu trúc **always**.

Ví dụ 4-5

```
module p_encoder (req, code_out, valid);
input      [3:0] req;
output reg [1:0] code_out;
output reg  valid;
always @ (*) begin
//the code_out port
    code_out[1] = (req[3] | req[2]) & ~req[1] & ~req[0];
    code_out[0] = ((req[3]& ~req[2])|req[1]) & ~req[0];
//the valid port
    valid = |req[3:0];
end
endmodule
```

4.1.1.4 Mô tả mạch tổ hợp bằng function

Ví dụ sau sẽ sử dụng **function** để mô tả mạch mã hóa ưu tiên theo hình 4-1.

Ví dụ 4-6

```
module p_encoder (req, code_out, valid);
input  [3:0] req;
output [1:0] code_out;
output      valid;
//Call functions
assign code_out[1:0] = encoder(req[3:0]);
assign valid = or_gate(req[3:0]);
//code_out function
function [1:0] encoder;
    input [3:0] encoder_in;
    if (encoder_in[0]) encoder[1:0] = 2'b00;
    else if (encoder_in[1]) encoder[1:0] = 2'b01;
    else if (encoder_in[2]) encoder[1:0] = 2'b10;
    else if (encoder_in[3]) encoder[1:0] = 2'b11;
    else
        encoder[1:0] = 2'b00;
endfunction
//valid function
function or_gate;
    input [3:0] or_in;
    or_gate = |or_in[3:0];
endfunction
endmodule
```

function *encoder* mô tả mạch chức năng tạo ngõ ra *code_out*.
function *or_gate* mô tả mạch chức năng tạo ngõ ra *valid*. Hai

function được gọi trong hai phép gán **assign**. Hai phép gán này có thể thay bằng cấu trúc **always**.

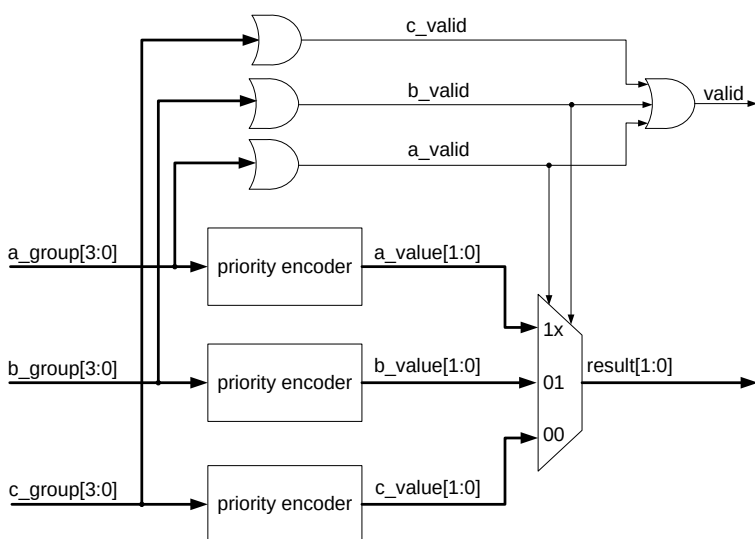
```
always @ (*) begin
    code_out[1:0] = encoder(req[3:0]);
    valid = or_gate(req[3:0]);
end
```

Tương tự, **function** cũng được dùng để mô tả mạch nguyên lý mức cổng ở hình 4-2.

Ví dụ 4-7

```
module p_encoder (req, code_out, valid);
input    [3:0] req;
output   [1:0] code_out;
output    valid;
//Call functions
assign code_out[1:0] = encoder(req[3:0]);
assign valid = or_gate(req[3:0]);
//code_out function
function [1:0] encoder;
    input [3:0] encoder_in;
    encoder[1] = (encoder_in[3]|encoder_in[2])&
                  ~encoder_in[1]& ~encoder_in[0];
    encoder[0] = (encoder_in[3]& ~encoder_in[2]
                  |encoder_in[1]) & ~encoder_in[0];
endfunction
//valid function
function or_gate;
    input [3:0] or_in;
    or_gate = |or_in[3:0];
endfunction
endmodule
```

function thường được sử dụng để mô tả các mạch chức năng dùng nhiều lần trong thiết kế nên các **function** được đặt trong một file riêng. File này sẽ được gọi trong các module sử dụng **function**. Một module phân xử ba nhóm tín hiệu có cấu trúc như sau:



Hình 4-3. Sơ đồ nguyên lý module phân xử

Module trên sẽ được thực hiện bằng cách sử dụng hai **function** *encoder* và *or_gate*.

Ví dụ 4-8

```

module arbiter (a_group, b_group, c_group,
                 valid, result);
    `include "com_function.h"
    //Ports
    input [3:0] a_group, b_group, c_group;
    output      valid;
    output reg [1:0] result;
    //The valid signal
    assign a_valid = or_gate(a_group[3:0]);
    assign b_valid = or_gate(b_group[3:0]);
    assign valid  = a_valid|b_valid|or_gate(c_group[3:0]);
    //The result signal
    always @ (*) begin
        casez ({a_valid, b_valid})
            2'b1?: result[1:0] = encoder(a_group[3:0]);
            2'b01: result[1:0] = encoder(b_group[3:0]);
            2'b00: result[1:0] = encoder(c_group[3:0]);
            default: result[1:0] = 2'b00;
        endcase
    end
endmodule

```

Hai function được đặt trong một file tên *com_function.h* nên khi gọi các function để sử dụng thì phải có dòng **include** file này.

4.1.1.5 Phương pháp ghép nối các thành phần có sẵn

Thành phần có sẵn là những thành phần mạch chức năng được hỗ trợ bởi Verilog HDL hoặc có trong thư viện tổng hợp hoặc các module do người sử dụng đã tạo ra trước đó.

Verilog HDL có sẵn các từ khóa mô tả cổng logic như **and**, **nand**, **nor**, **or**, **xnor**, **xor**, **not**, **buf**, **bufif1**, **bufif0**, **notif1** hay **notif0** gọi là các thành phần gốc. Ngoài ra, Verilog HDL còn hỗ trợ người dùng tự định nghĩa thêm các thành phần gốc để tái sử dụng gọi là thành phần gốc do người dùng định nghĩa. Để mô tả mạch tổ hợp sử dụng các thành phần gốc sẵn có của Verilog HDL, mạch nguyên lý phải được chuyển về dạng cổng logic.

Bộ mã hóa ưu tiên sẽ được mô tả bằng ba cổng NOT, một cổng OR bốn ngõ vào, hai cổng OR hai ngõ vào, hai cổng AND hai ngõ vào và một cổng AND ba ngõ vào như hình 4-2. Các tín hiệu *req0_n*, *req1_n*, *req2_n*, *or_0*, *or_1* và *and_0* là các tín hiệu nội.

Ví dụ 4-9

```
module p_encoder (req, code_out, valid);
input [3:0] req;
output wire [1:0] code_out;
output wire valid;
wire req0_n, req1_n, req2_n, or_0, or_1, and_0;

not not_inst0 (req0_n, req[0]);
not not_inst1 (req1_n, req[1]);
not not_inst2 (req2_n, req[2]);
or or_inst0 (or_0, req[3], req[2]);
and and_inst1 (and_0, req2_n, req[3]);
or or_inst1 (or_1, and_0, req[1]);
//the code_out port
and and_inst0 (code_out[1], req0_n, req1_n, or_0);
and and_inst2 (code_out[0], req0_n, or_1);
//the valid port
or or_inst2 (valid, req[3], req[2], req[1], req[0]);
endmodule
```

Phương pháp mô tả này ít được sử dụng vì rất khó đọc code, khó kiểm tra và khó gỡ lỗi nếu mạch thiết kế lớn.

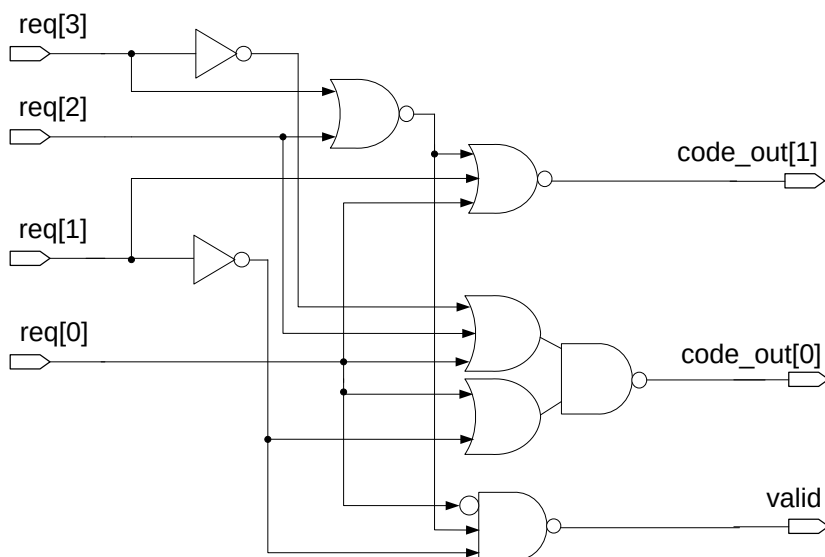
4.1.1.6 Mô tả mạch tổ hợp bằng phương pháp kết hợp

Phương pháp kết hợp là sử dụng hỗn hợp các cách khác nhau để mô tả mạch chính xác, ngắn gọn và dễ đọc. Cấu trúc **always** và **assign** được dùng để mô tả mạch nguyên lý bộ mã hóa ưu tiên hình 4-1.

Ví dụ 4-10

```
module p_encoder (req, code_out, valid);  
input      [3:0] req;  
output reg [1:0] code_out;  
output     valid;  
  
always @ (*) begin  
    if (req[0])    code_out[1:0] = 2'b00;  
    else if (req[1]) code_out[1:0] = 2'b01;  
    else if (req[2]) code_out[1:0] = 2'b10;  
    else if (req[3]) code_out[1:0] = 2'b11;  
    else          code_out[1:0] = 2'b00;  
end  
assign valid = |req[3:0];  
endmodule
```

Mạch tổ hợp có thể được mô tả bằng nhiều cách khác nhau nhưng kết quả tổng hợp là như nhau trong cùng điều kiện ràng buộc.



Hình 4-4. Netlist sau tổng hợp của bộ mã hóa ưu tiên

Bộ mã hóa ưu tiên là một ví dụ đơn giản nên việc mô tả mạch bằng phương pháp nào cũng cho RTL code ngắn gọn tương đương nhau nhưng phương pháp mô tả bằng cách ghép nối các mô hình cổng logic hỗ trợ bởi Verilog HDL rất khó đọc và gỡ lỗi.

Việc mô tả RTL code theo hành vi mạch nguyên lý như hình 4-1 giúp dễ đọc và dễ hình dung chức năng mạch hơn so với việc mô tả RTL code theo cấu trúc mức cổng logic như hình 4-2. Trong khi đó, mô tả RTL code theo cấu trúc nhưng sử dụng các từ khóa của Verilog HDL cũng không đảm bảo giữ được cấu trúc mạch mong muốn khi tổng hợp. Mạch nguyên lý mức cổng hình 4-2 rõ ràng khác với netlist sau tổng hợp hình 4-4 mặc dù chức năng mạch giống nhau.

Nếu muốn giữ nguyên cấu trúc thiết kế mà không bị phụ thuộc vào trình tổng hợp thì RTL code phải được mô tả bằng cách gọi và kết nối trực tiếp các thành phần của thư viện tổng hợp, gọi là các logic cell, nhưng điểm bất lợi của cách này là RTL code đó không thể tái sử dụng để tổng hợp với nhiều thư viện khác nhau.

4.1.2 Lưu ý khi mô tả mạch tổ hợp

4.1.2.1 Sử dụng đúng phép gán blocking

Verilog HDL định nghĩa hai phép gán có hành vi xử lý khác nhau là blocking và non-blocking nhưng chỉ nên sử dụng phép gán blocking khi mô tả mạch tổ hợp trong cấu trúc **always** để tránh những kết quả không mong muốn khi chạy mô phỏng mức RTL code.

Ví dụ 4-11

```
module test_blocking (a, b, c, y);
input a, b, c;
output reg y;
reg tmp;
always @ (*) begin
    tmp = a | b;
    y = c & tmp;
end
endmodule
```

Bảng 4-2. Hành vi của phép gán blocking trong ví dụ 4-11

Thứ tự	Mô tả	a	b	c	tmp	y	Ghi chú
1	Điều kiện đầu	0	0	1	0	0	
2	a thay đổi	1	0	1	0	0	khởi động khối always
3	$tmp = a \mid b$	1	0	1	1	0	tính toán $a \mid b$ và gán giá trị 1 cho tmp
4	$y = c \& tmp$	1	0	1	1	1	tính toán $c \& tmp$ và gán giá trị 1 cho y

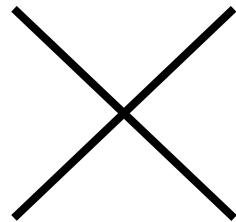
Hành vi của phép gán blocking phù hợp với hoạt động của mạch tổ hợp vì ngõ ra cập nhật giá trị khi có sự thay đổi của ngõ vào. Phép gán non-blocking được sử dụng thay cho phép gán blocking trong ví dụ sau để phân tích sự khác nhau trong hành vi của hai phép gán.

Ví dụ 4-12

```

module test_nonblocking (a, b, c, y);
input a, b, c;
output reg y;
reg tmp;
always @ (*) begin
    tmp <= a | b;
    y <= c & tmp;
end
endmodule

```

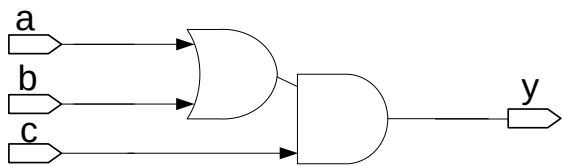


Bảng 4-3. Hành vi của phép gán non-blocking trong ví dụ 4-12

Thứ tự	Mô tả	a	b	c	tmp	y	Ghi chú
1	Điều kiện đầu	0	0	1	0	0	
2	a thay đổi	1	0	1	0	0	khởi động khối always
3	$tmp \leq a \mid b$	1	0	1	0	0	Tính toán $a \mid b$ bằng 1 nhưng không gán cho tmp
4	$y \leq c \& tmp$	1	0	1	1	0	Tính toán $c \& tmp$ bằng 0 và thực hiện gán các giá trị đã tính toán cho tmp và y

Ngõ ra y không cập nhật đúng giá trị mong muốn nên hành vi của phép gán non-blocking không phù hợp với hoạt động của một mạch tổ hợp. Tuy trình tổng hợp vẫn tạo ra mạch chính xác trong cả hai trường hợp ví dụ 4-11 và ví dụ 4-12 nhưng không nên sử dụng phép gán non-

blocking để mô tả mạch tổ hợp để tránh kết quả mô phỏng không mong muốn.



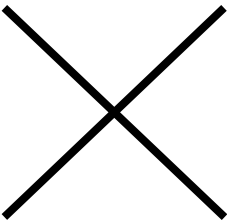
Hình 4-5. Netlist sau tổng hợp của ví dụ 4-11 và ví dụ 4-12

4.1.2.2 Mô tả đúng thứ tự các phép gán khi dùng always

Thứ tự gán cũng rất quan trọng khi mô tả mạch tổ hợp với cấu trúc **always** và cặp từ khóa **begin-end**. Thứ tự gán thay đổi có thể gây ra kết quả mô phỏng không mong muốn hoặc tạo ra mạch không đúng.

Ví dụ 4-13

```
module test_blocking (a, b, c, y);
input a, b, c;
output reg y;
reg tmp;
always @ (*) begin
    y = c & tmp;
    tmp = a | b;
end
endmodule
```

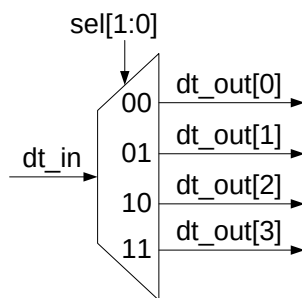


Đảo thứ tự hai phép gán của *y* và *tmp*, kết quả tổng hợp vẫn tạo ra mạch chức năng chính xác nhưng kết quả mô phỏng mức RTL lại không như mong muốn vì ngỡ ra *y* không có giá trị đúng với chức năng của một mạch tổ hợp.

Bảng 4-4. Kết quả mô phỏng không mong muốn của ví dụ 4-13

Thứ tự	Mô tả	a	b	c	tmp	y	Ghi chú
1	Bắt đầu	0	0	1	0	x	
2	<i>a</i> thay đổi	1	0	1	0	x	khởi động khối always
3	<i>y</i> = <i>c</i> & <i>tmp</i>	1	0	1	0	0	Tính toán <i>c</i> & <i>tmp</i> và gán giá trị 0 cho <i>y</i>
4	<i>tmp</i> = <i>a</i> <i>b</i>	1	0	1	1	0	Tính toán <i>a</i> <i>b</i> và gán giá trị 1 cho <i>tmp</i>
x là giá trị không xác định							

Thứ tự mô tả các phép gán trong cấu trúc **always** với **begin-end** còn ảnh hưởng đến kết quả tổng hợp mạch. Một bộ DEMUX có ngõ ra $dt_out[3:0]$ bốn bit.



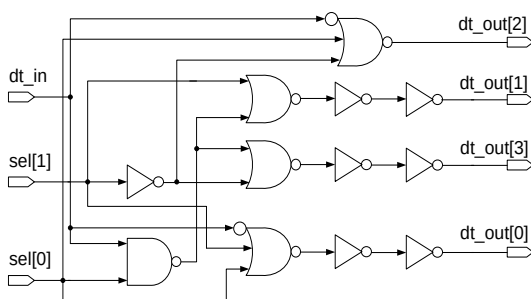
Hình 4-6. Sơ đồ nguyên lý của bộ DEMUX

Bộ DEMUX trên có thể được mô tả bởi đoạn RTL code sau:

Ví dụ 4-14

```
module demux (sel, dt_in, dt_out);
input  [1:0] sel;
input   dt_in;
output reg [3:0] dt_out;
always @ (*) begin
    dt_out[3:0] = 4'd0;
    case (sel[1:0])
        2'b00: dt_out[0] = dt_in;
        2'b01: dt_out[1] = dt_in;
        2'b10: dt_out[2] = dt_in;
        2'b11: dt_out[3] = dt_in;
    endcase
end
endmodule
```

Tín hiệu dt_out được gán bằng 0 trước. Sau đó, tùy vào giá trị của sel mà bit tương ứng trong tín hiệu dt_out được kết nối với ngõ vào dt_in . Cách mô tả này cho mạch chính xác khi tổng hợp.

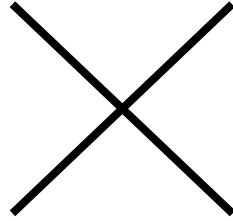


Hình 4-7. Netlist sau tổng hợp của ví dụ 4-14

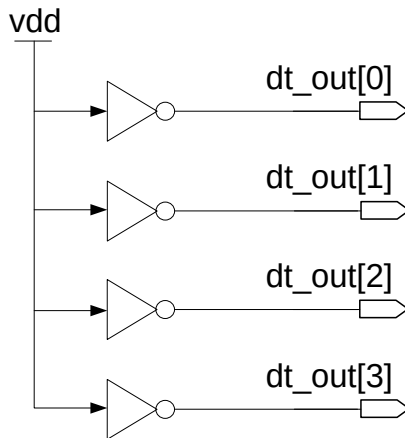
Nếu thay đổi thứ tự khai báo của ví dụ 4-14, hành vi các phép gán trong phát biểu **case** được thực thi trước phép gán ngõ ra $dt_out[3:0] = 4'd0$.

Ví dụ 4-15

```
module demux (sel, dt_in, dt_out);
input  [1:0] sel;
input  dt_in;
output reg [3:0] dt_out;
always @ (*) begin
    case (sel[1:0])
        2'b00: dt_out[0] = dt_in;
        2'b01: dt_out[1] = dt_in;
        2'b10: dt_out[2] = dt_in;
        2'b11: dt_out[3] = dt_in;
    endcase
    dt_out[3:0] = 4'd0;
end
endmodule
```



Mạch tạo ra sẽ chỉ mô tả chức năng của phép gán $dt_out[3:0] = 4'd0$.



Hình 4-8. Netlist sau tổng hợp của ví dụ 4-15

4.1.2.3 Tránh tạo Latch không mong muốn

Khi mô tả mạch tổ hợp, phát biểu **if** và **case** cần được khai báo đầy đủ các trường hợp của điều kiện ngõ vào. Đồng thời, ngõ ra phải được gán trên tất cả các nhánh. Phát biểu **if** thì phải có nhánh khai báo **else** và phát biểu **case** phải có nhánh **default** nếu các điều kiện ngõ vào không được liệt kê đầy đủ trường hợp.

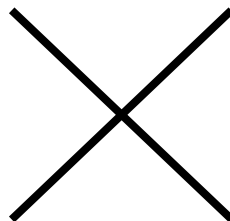
Việc khai báo đầy đủ tất cả các trường hợp là để tránh tạo Latch trong mạch tổ hợp khi dùng với cấu trúc **always** và tránh sai lệch kết quả mô phỏng giữa mức RTL và mức cổng khi dùng với **function**.

Khi một phát biểu **if** và **case** được sử dụng để mô tả nhiều tín hiệu thì phải đảm bảo tất cả các tín hiệu được gán đầy đủ trên tất cả các nhánh.

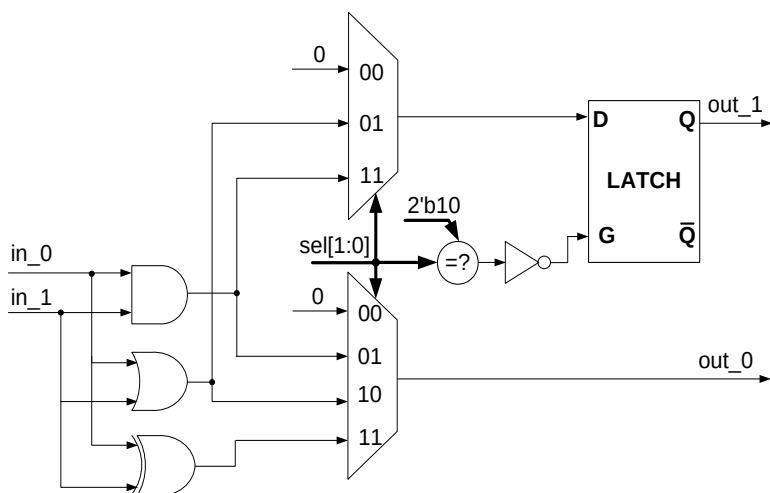
Ví dụ 4-16

```
module latch_gen (sel, in_0, in_1, out_0, out_1);
input [1:0] sel;
input in_0, in_1;
output reg out_0, out_1;

always @(*) begin
    if (sel == 2'b00) begin
        out_0 = 1'b0;
        out_1 = 1'b0;
    end
    else if (sel == 2'b01) begin
        out_0 = in_0 & in_1;
        out_1 = in_0 | in_1;
    end
    else if (sel == 2'b10) begin
        out_0 = in_0 | in_1;
        //out_1 is not assigned
    end
    else begin
        out_0 = in_0 ^ in_1;
        out_1 = in_0 & in_1;
    end
end
endmodule
```



Hai tín hiệu *out_0* và *out_1* có sự phụ thuộc khác nhau vào tín hiệu *sel*. Tại nhánh *sel == 2'b10*, tín hiệu *out_1* không được gán giá trị nên Latch sẽ được tạo ra cho tín hiệu *out_1*.



Hình 4-9. Mạch nguyên lý của ví dụ 4-16

Nếu không muốn tạo Latch tại tín hiệu *out_1*, giá trị của *out_1* ở nhánh *sel == 2'b10* phải được khai báo một giá trị mong muốn.

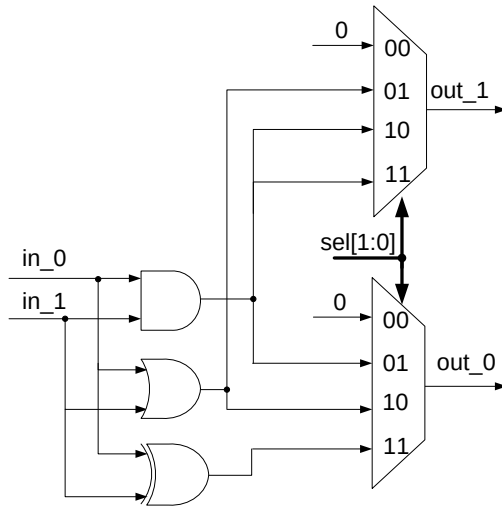
Ví dụ 4-17

```

module mux_dont_latch (sel, in_0, in_1, out_0, out_1);
input [1:0] sel;
input in_0, in_1;
output reg out_0, out_1;

always @(*) begin
    if (sel == 2'b00) begin
        out_0 = 1'b0;
        out_1 = 1'b0;
    end
    else if (sel == 2'b01) begin
        out_0 = in_0 & in_1;
        out_1 = in_0 | in_1;
    end
    else if (sel == 2'b10) begin
        out_0 = in_0 | in_1;
        out_1 = in_0 & in_1;
    end
    else begin
        out_0 = in_0 ^ in_1;
        out_1 = in_0 & in_1;
    end
end
endmodule

```



Hình 4-10. Mạch nguyên lý của ví dụ 4-17

Trường hợp có nhiều ngõ ra trong một cấu trúc **always** mạch tổ hợp thì việc mô tả riêng biệt từng tín hiệu giúp RTL code dễ đọc và kiểm tra hơn.

Ví dụ 4-18

```

module mux_dont_latch (sel, in_0, in_1, out_0, out_1);
input [1:0] sel;
input in_0, in_1;
output reg out_0, out_1;

always @(*) begin
    if (sel == 2'b00) out_0 = 1'b0;
    else if (sel == 2'b01) out_0 = in_0 & in_1;
    else if (sel == 2'b10) out_0 = in_0 | in_1;
    else out_0 = in_0 ^ in_1;
end
always @(*) begin
    if (sel == 2'b00) out_1 = 1'b0;
    else if (sel == 2'b01) out_1 = in_0 | in_1;
    else out_1 = in_0 & in_1;
end
endmodule

```

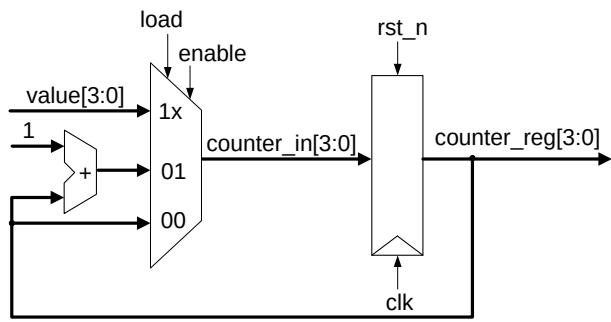
Khác với cấu trúc **always**, **function** không bao giờ tạo Latch cho dù các nhánh của phát biểu **if** hay **case** không được mô tả đầy đủ. Giá trị ngõ ra tại nhánh bị thiếu sẽ là x khi mô phỏng mức RTL code nhưng

sẽ là giá trị xác định 0 hoặc 1 sau khi tổng hợp. Điều này gây sai lệch kết quả mô phỏng giữa mức RTL code và mức cổng logic.

4.2 Mạch tuần tự

4.2.1 Phương pháp mô tả mạch tuần tự

Mạch tuần tự khác mạch tổ hợp ở một điểm quan trọng là có thêm Flip-Flop. Một trong những mạch tuần tự quen thuộc là bộ đếm. Giả sử, bộ đếm *counter* có sơ đồ nguyên lý như sau:



Hình 4-11. Sơ đồ nguyên lý của bộ đếm

Bộ đếm *counter_reg* được nạp giá trị *value* khi tín hiệu *load* tích cực mức 1. Khi tín hiệu *load* bằng 0, *counter_reg* sẽ đếm lên từng đơn vị nếu tín hiệu *enable* tích cực mức 1 và giữ nguyên giá trị nếu *enable* bằng 0. Tín hiệu reset *rst_n* tích cực mức thấp.

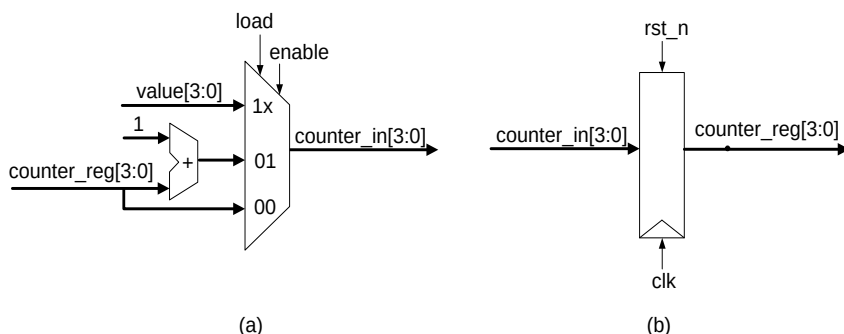
Bảng 4-5. Bảng sự thật của *counter_reg*

<i>rst_n</i>	<i>load</i>	<i>enable</i>	<i>counter_reg</i> [3:0]
0	x	x	0
1	0	0	<i>counter_reg</i> [3:0]
1	1	x	<i>value</i> [3:0]
1	0	1	<i>counter_reg</i> [3:0] + 1

Phương pháp mô tả RTL code mạch tuần tự có thể sử dụng gồm có:

- Mô tả riêng phần mạch tổ hợp và phần tạo Flip-Flop
- Mô tả kết hợp chung một phần hoặc toàn bộ phần mạch tổ hợp với Flip-Flop.

Mạch tuần tự được mô tả dễ dàng bằng cách tách riêng phần mạch tổ hợp và phần tạo Flip-Flop. Phần mô tả mạch tổ hợp có thể áp dụng một trong các phương pháp như đã trình bày. Phần mô tả tạo Flip-Flop là một khối **always** thể hiện các thành phần xung clock, reset, set và phép gán giá trị cho Flip-Flop.



Hình 4-12. Phần mạch tổ hợp (a) và Flip-Flop (b) của bộ đếm hình 4-11

Bộ đếm *counter* hình 4-11 được chia làm hai phần để mô tả. Phần mạch tổ hợp là bộ MUX có ngõ ra là *counter_in* được mô tả bằng một khối **always** với phát biểu **case**. Phần tạo Flip-Flop *counter_reg* được mô tả bằng một khối **always** khác thể hiện bộ đếm hoạt động theo cạnh lên xung clock *clk*, được reset bất đồng bộ bởi cạnh xuống *rst_n* và có ngõ vào là *counter_in*.

Ví dụ 4-19

```

module counter (clk, rst_n, load, enable, value,
counter_reg);
input      clk, rst_n;
input      load, enable;
input      [3:0] value;
output reg [3:0] counter_reg;
//internal signals
reg [3:0] counter_in;
//Combinational circuit
always @ (*) begin
    casez ({load, enable})
        2'b1?: counter_in[3:0] = value[3:0];
        2'b01: counter_in[3:0] = counter_reg[3:0] + 4'd1;
        default: counter_in[3:0] = counter_reg[3:0];
    endcase
end

```

```
//Flip-Flop generation
always @ (posedge clk, negedge rst_n) begin
    if (!rst_n) counter_reg[3:0] <= 4'd0;
    else counter_reg[3:0] <= counter_in[3:0];
end
endmodule
```

Tín hiệu *counter_in* trong bộ đếm *counter* là ngõ ra mạch tổ hợp và là ngõ vào Flip-Flop. Tín hiệu trung gian này có thể bỏ đi bằng cách gộp chung mô tả khối **always** mô tả mạch tổ hợp và khối **always** tạo Flip-Flop.

Ví dụ 4-20

```
module counter (clk, rst_n, load, enable, value,
counter_reg);
input          clk, rst_n;
input          load, enable;
input          [3:0] value;
output reg     [3:0] counter_reg;
//counter circuit
always @ (posedge clk, negedge rst_n) begin
    if (!rst_n) counter_reg[3:0] <= 4'd0;
    else begin
        casez ({load, enable})
            2'b1?: counter_reg[3:0] <=value[3:0];
            2'b01: counter_reg[3:0] <=counter_reg[3:0]+4'd1;
            default: counter_reg[3:0] <= counter_reg[3:0];
        endcase
    end
end
endmodule
```

Nhánh **default** của phát biểu **case** có ý nghĩa là giữ nguyên giá trị *counter_reg* nếu hai tín hiệu *load* và *enable* không bằng 1. Vì mạch tạo ra Flip-Flop nên nhánh này không cần phải mô tả.

Ví dụ 4-21

```
module counter (...);
...
//counter circuit
always @ (posedge clk, negedge rst_n) begin
    if (!rst_n) counter_reg[3:0] <= 4'd0;
    else begin
        casez ({load, enable})
```

```

        2'b1?: counter_reg[3:0] <= value[3:0];
        2'b01: counter_reg[3:0] <= counter_reg[3:0]
                +4'd1;

    endcase
end
end
endmodule

```

Phát biểu **case** có thể được thay bằng phát biểu **if** để mô tả bộ đếm *counter*.

Ví dụ 4-22

```

module counter (...);
...
//counter circuit
always @ (posedge clk, negedge rst_n) begin
    if (!rst_n) counter_reg[3:0] <= 4'd0;
    else if (load)
        counter_reg[3:0] <= value[3:0];
    else if (enable)
        counter_reg[3:0] <= counter_reg[3:0] + 4'd1;
end
endmodule

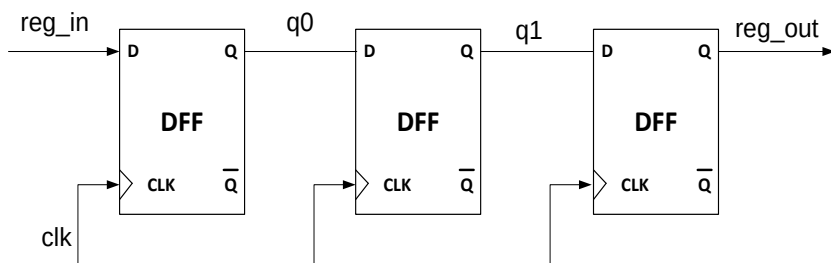
```

Tất cả các cách mô tả RTL code cho bộ đếm *counter* đã trình bày đều cho kết quả tổng hợp netlist giống nhau trong cùng điều kiện ràng buộc.

4.2.2 Lưu ý khi mô tả mạch tuần tự

4.2.2.1 Sử dụng đúng phép gán non-blocking

Hành vi của phép gán non-blocking thích hợp để sử dụng mô tả mạch tuần tự trong cấu trúc **always**. Mô hình thanh ghi dịch sau đây được sử dụng để phân tích tầm quan trọng của việc sử dụng đúng phép gán non-blocking khi mô tả mạch tuần tự.



Hình 4-13. Mạch nguyên lý thanh ghi dịch ba bit

Thanh ghi dịch được mô tả bằng đoạn RTL code sau:

Ví dụ 4-23

```

module shift_reg (clk, reg_in, reg_out);
input      clk, reg_in;
output reg reg_out;
reg q0, q1;
always @ (posedge clk) begin
    q0    <= reg_in;
    q1    <= q0;
    reg_out <= q1;
end
endmodule

```

Bảng 4-6. Hành vi của phép gán non-blocking trong ví dụ 4-23

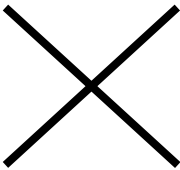
Thứ tự	Mô tả	reg_in	q0	q1	reg_out
1	Điều kiện đầu	1	0	1	0
2	cạnh lên <i>clk</i>	1	0	1	0
3	tính toán giá trị <i>reg_in</i> là 1 nhưng không gán cho <i>q0</i>	1	0	1	0
4	tính toán giá trị <i>q0</i> là 0 nhưng không gán cho <i>q1</i>	1	0	1	0
5	tính toán giá trị <i>q1</i> là 1 và thực thi gán tất cả các giá trị đã ước lượng từ vế phải sang vế trái	1	1	0	1

Vì việc gán giá trị chỉ xảy ra khi tất cả các biểu thức vế phải phép gán đã được tính toán giá trị xong nên tại cạnh lên xung clock *clk*, vế phải nhận giá trị cũ của vế trái. Điều này thích hợp với mô hình thanh ghi dịch đã nêu. Với phép gán non-blocking, thứ tự của các phép gán

trong khối **always** không ảnh hưởng đến kết quả mô phỏng cũng như mạch chức năng được tạo ra. Vì vậy, việc đảo thứ tự gán trong khối **always** vẫn cho kết quả mô phỏng và tổng hợp chính xác. Nếu thay phép gán non-blocking bằng phép gán blocking, kết quả mô phỏng và tổng hợp sẽ sai.

Ví dụ 4-24

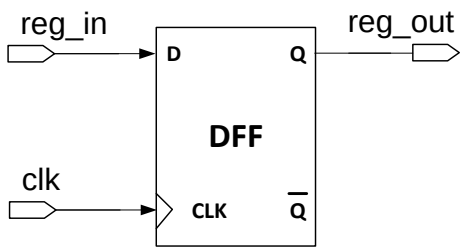
```
always @ (posedge clk) begin  
    q0 = reg_in;  
    q1 = q0;  
    reg_out = q1;  
end
```



Bảng 4-7. Hành vi của phép gán blocking trong ví dụ 4-24

Thứ tự	Mô tả	reg_in	q0	q1	reg_out
1	Điều kiện đầu	1	0	0	0
2	cạnh lên <i>clk</i>	1	0	0	0
3	tính toán giá trị <i>reg_in</i> là 1 và gán cho <i>q0</i>	1	1	0	0
4	tính toán giá trị <i>q0</i> là 1 và gán cho <i>q1</i>	1	1	1	0
5	tính toán giá trị <i>q1</i> là 1 và gán cho <i>reg_out</i>	1	1	1	1

Tại cạnh lên xung clock, *q0*, *q1* và *reg_out* cùng cập nhật giá trị của ngõ vào *reg_in*. Hành vi này không đúng với mô hình mạch ban đầu và kết quả tổng hợp cũng không như mong muốn.

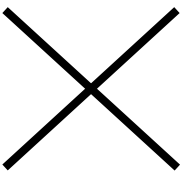


Hình 4-14. Netlist sau tổng hợp của ví dụ 4-24

Đồng thời, thứ tự của phép gán blocking khác nhau sẽ cho kết quả mô phỏng và tổng hợp khác nhau.

Ví dụ 4-25

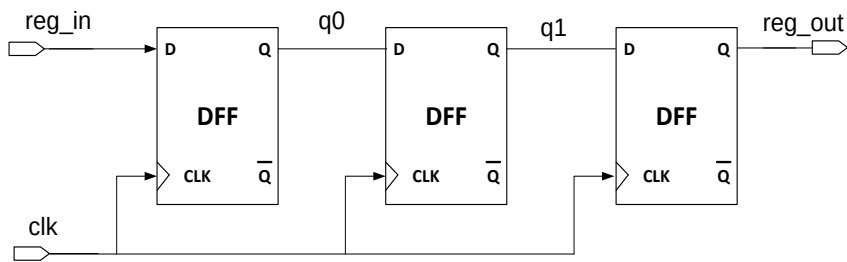
```
always @ (posedge clk) begin
    reg_out = q1;
    q1      = q0;
    q0      = reg_in;
end
```



Bảng 4-8. Hành vi của phép gán blocking trong ví dụ 4-25

Thứ tự	Mô tả	reg_in	q0	q1	reg_out
1	Điều kiện đầu	1	0	1	0
2	cạnh lên <i>clk</i>	1	0	1	0
3	tính toán giá trị <i>q1</i> là 1 và gán cho <i>reg_out</i>	1	0	1	1
4	tính toán giá trị <i>q0</i> là 0 và gán cho <i>q1</i>	1	0	0	1
5	tính toán giá trị <i>reg_in</i> là 1 và gán cho <i>q0</i>	1	1	0	1

Tuy hành vi của phép gán blocking khi đảo thứ tự cho kết quả mô phỏng chính xác và mạch tạo ra như mong muốn nhưng việc mô tả mạch tuần tự theo cách này là không nên.



Hình 4-15. Netlist sau tổng hợp của ví dụ 4-25

Để tránh sự phụ thuộc vào thứ tự phép gán trong cấu trúc **always** sử dụng **begin-end**, ba phép gán được tách thành ba khối **always** riêng biệt.

Ví dụ 4-26

```
always @ (posedge clk) q0      = reg_in;
always @ (posedge clk) q1      = q0;
always @ (posedge clk) reg_out = q1;
```



Cách mô tả trên tạo ra mạch chức năng chính xác vì mỗi khối **always** độc lập sẽ tạo ra một mạch tương ứng khi tổng hợp. Tuy nhiên, kết quả mô phỏng mức RTL code thì vẫn không đúng. Đối với trường hợp này, phép gán được thay bằng loại non-blocking thì cả mô phỏng và tổng hợp đều cho kết quả chính xác.

Ví dụ 4-27

```
always @ (posedge clk) q0      <= reg_in;  
always @ (posedge clk) q1      <= q0;  
always @ (posedge clk) reg_out <= q1;
```

Như vậy, việc sử dụng đúng phép gán non-blocking khi mô tả mạch tuần tự là rất quan trọng. Đồng thời, do sự khác nhau trong hành vi của phép gán blocking và non-blocking nên việc sử dụng cả hai loại phép gán này trong cùng một khối **always** là không nên.

4.2.2.2 Mô tả xung clock và reset

Một vấn đề quan trọng khác khi dùng cấu trúc **always** để mô tả mạch tuần tự là cách mô tả xung clock và reset². Trong thiết kế ASIC, vấn đề này càng phải được chú ý để đảm bảo việc tổng hợp mạch đúng đắn và tối ưu. Trong các thư viện tổng hợp ASIC, hầu hết các Flip-Flop thường chỉ có một tín hiệu xung clock duy nhất và hoạt động theo một cạnh duy nhất, cạnh lên hoặc cạnh xuống. Các mô hình Flip-Flop hoạt động theo cạnh lên xung clock thường được hỗ trợ đầy đủ hơn các mô hình Flip-Flop hoạt động theo cạnh xuống. Vì vậy, mạch tuần tự nên mô tả theo cạnh lên xung clock bằng từ khóa **posedge**.

Với reset đồng bộ, nguyên tắc quan trọng là phải mô tả chúng ở mức ưu tiên cao nhất trong mô tả mạch tuần tự. Tín hiệu reset sẽ được tổ hợp với ngõ vào trước khi kết nối đến ngõ vào D của Flip-Flop. Phần mạch tổ hợp này sẽ khác nhau tùy vào mức logic của tín hiệu reset, giá trị reset và điều kiện tổng hợp.

² Thuật ngữ reset được sử dụng với ý nghĩa là khởi tạo hoặc thiết lập lại giá trị cho Flip-Flop và giá trị này có thể là 0 hoặc 1. Nó có thể ứng với tín hiệu *reset* hoặc *set* của Flip-Flop.

Ví dụ 4-28

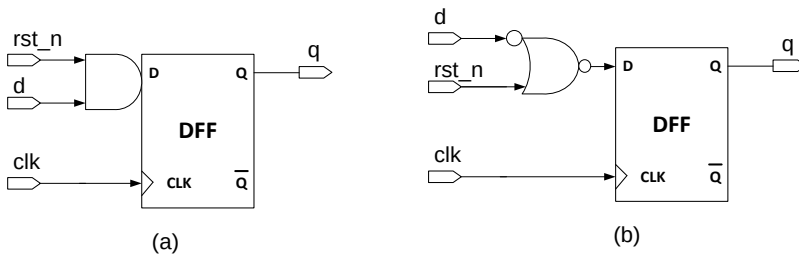
Reset đồng bộ tích cực mức thấp

```
always @ (posedge clk) begin
    if (!rst_n) dt_out <= 1'b0;
    else          dt_out <= dt_in;
end
```

Ví dụ 4-29

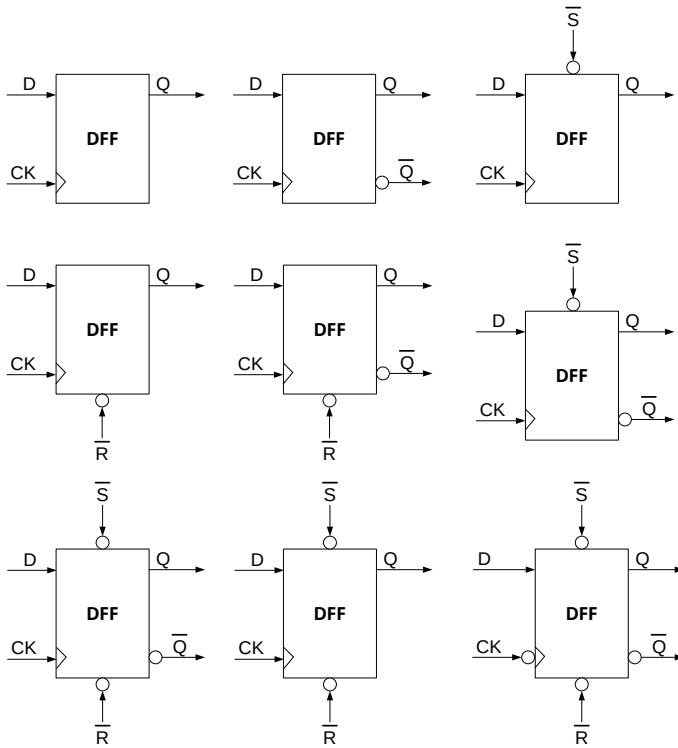
Reset đồng bộ tích cực mức cao

```
always @ (posedge clk) begin
    if (rst_n) dt_out <= 1'b0;
    else      dt_out <= dt_in;
end
```



Hình 4-16. Netlist sau tổng hợp của ví dụ 4-28 (a) và ví dụ 4-29 (b)

Khác với reset đồng bộ, tín hiệu reset bất đồng bộ sẽ kết nối đến chân *reset* hoặc *set* của Flip-Flop.



Hình 4-17. Một số Flip-Flop thường có trong các thư viện tổng hợp

Hầu hết tín hiệu *reset* và *set* của các loại Flip-Flop tích cực mức thấp nên tín hiệu *reset* và *set* bất đồng bộ thường được mô tả theo cạnh xuống bằng từ khóa **negedge**.

```
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        ...
end
```

Ví dụ 4-30

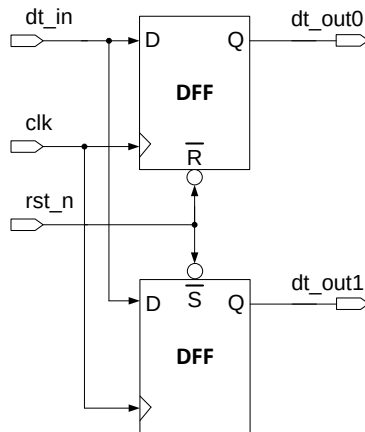
```
module async_reset_set (clk, rst_n, dt_in,
                        dt_out0, dt_out1);
    input      clk, rst_n;
    input      dt_in;
    output reg dt_out0, dt_out1;
    //asynchronous reset
```

```

always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) dt_out0 <= 1'b0;
    else        dt_out0 <= dt_in;
end
//asynchronous set
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) dt_out1 <= 1'b1;
    else        dt_out1 <= dt_in;
end
endmodule

```

Tín hiệu *rst_n* khởi tạo hai giá trị khác nhau cho *dt_out0* và *dt_out1*. Trong trường hợp này, *rst_n* được nối trực tiếp đến chân *reset* của Flip-Flop tạo ra *dt_out0* và chân *set* của Flip-Flop tạo ra *dt_out1*.



Hình 4-18. Netlist sau tổng hợp của ví dụ 4-25

Tín hiệu *reset* và *set* bất đồng bộ có thể được mô tả đồng thời nhưng cách mô tả này chỉ sử dụng khi thư viện tổng hợp có hỗ trợ loại Flip-Flop dạng này. Nếu thư viện tổng hợp không hỗ trợ loại Flip-Flop trên thì có thể gây ra lỗi tổng hợp hoặc mạch tổng hợp không chính xác.

Ví dụ 4-31

```

module async_reset_set (clk,rst_n,set_n,dt_in,dt_out);
input      clk, rst_n, set_n;
input      dt_in;
output reg dt_out;
//asynchronous set and reset
always @ (posedge clk, negedge set_n, negedge rst_n)
begin

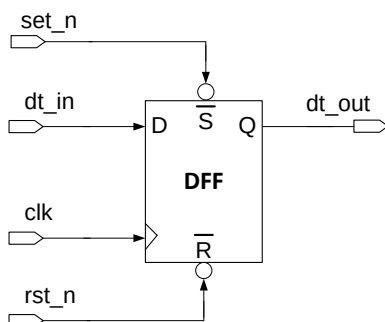
```

```

if (~set_n) dt_out <= 1'b1;
else if (~rst_n) dt_out <= 1'b0;
else dt_out <= dt_in;
end
endmodule

```

Mạch tạo ra đúng chức năng mong muốn nếu thư viện tổng hợp hỗ trợ loại Flip-Flop có đầy đủ cả hai tín hiệu reset và set bất đồng bộ.



Hình 4-19. Netlist sau tổng hợp của ví dụ 4-31

Việc mô tả RTL code tạo ra Flip-Flop có cả set và reset bất đồng bộ cần phải chú ý đến thứ tự ưu tiên của tín hiệu set và reset. Trong ví dụ 4-31, tín hiệu *set_n* được mô tả ưu tiên hơn tín hiệu *rst_n* và netlist tạo ra như hình 4-19 vì thư viện tổng hợp hỗ trợ loại Flip-Flop có chân set ưu tiên hơn chân reset. Nếu thứ tự ưu tiên được mô tả ngược lại mà thư viện tổng hợp không hỗ trợ loại này thì các thành phần logic sẽ được thêm vào để thực thi đúng chức năng mong muốn. Tóm lại, việc sử dụng cả set và reset bất đồng bộ cần phải thận trọng.

4.2.2.3 Mô tả giá trị khởi tạo của Flip-Flop

Giá trị khởi tạo của Flip-Flop phải được mô tả bằng thành phần tổng hợp được, reset đồng bộ hoặc reset bất đồng bộ. Việc sử dụng các thành phần không tổng hợp được để khởi tạo giá trị đầu cho Flip-Flop chỉ có ý nghĩa trong mô phỏng.

Ví dụ 4-32

```

module dff (clk,dt_in,dt_out);
input      clk, dt_in;
output reg dt_out;
//initial is not synthesised

```



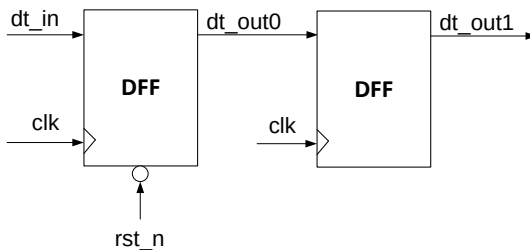
```

initial      dt_out = 1'b0;
always @ (posedge clk) begin
    dt_out <= dt_in;
end
endmodule

```

Ngõ ra *dt_out* được gán giá trị đầu là 0 bằng phát biểu **initial**. Khi mô phỏng mức RTL code, tại thời điểm bắt đầu mô phỏng, cạnh lên *clk* chưa xuất hiện thì giá trị *dt_out* sẽ bằng 0. Vì **initial** là thành phần không thể tổng hợp được nên trình tổng hợp bỏ qua phát biểu này. Khi mô phỏng mức cổng, tại thời điểm bắt đầu mô phỏng, cạnh lên *clk* chưa xuất hiện thì giá trị *dt_out* là x. Điều này gây ra sự sai lệch giữa kết quả mô phỏng mức RTL code và mức cổng.

Các Flip-Flop không cần khởi tạo giá trị đầu không nên mô tả chung với các Flip-Flop có reset trong cùng một khối **always**. Trường hợp này, mạch tạo ra khi tổng hợp có thể đúng chức năng nhưng sẽ có thêm các thành phần logic không mong muốn. Đặc biệt, đối với reset bất đồng bộ, trình tổng hợp có thể thực thi không chính xác khi mô tả gộp chung như trên. Giả sử, mạch tuần tự cần mô tả như sau:



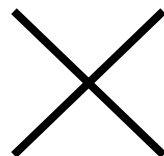
Hai Flip-Flop mắc nối tiếp nhau, Flip-Flop thứ nhất có reset bất đồng bộ nhưng Flip-Flop thứ hai thì không. Nếu hai Flip-Flop này được mô tả sai bằng cách gộp chung trong một khối **always** thì mạch tổng hợp sẽ không như mong muốn.

Ví dụ 4-33

```

module testcode (clk, rst_n, dt_in, dt_out1);
input clk, rst_n;
input dt_in;
output reg dt_out1;
reg dt_out0;
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) dt_out0 <= 1'b0;

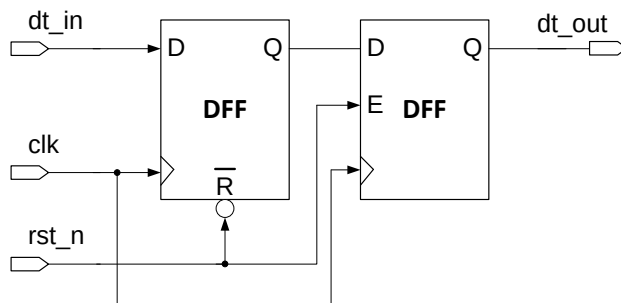
```




```

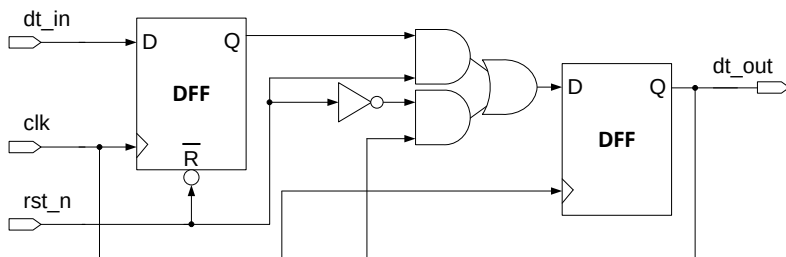
else begin
    dt_out0 <= dt_in;
    dt_out1 <= dt_out0;
end
end
endmodule

```



Hình 4-20. Netlist sau tổng hợp của ví dụ 4-33

Do mô tả gộp chung, loại Flip-Flop có chân *E* được sử dụng và tạo ra đường kết nối không mong muốn từ tín hiệu *rst_n* đến chân *E*. Nếu thư viện không hỗ trợ loại Flip-Flop có chân *E* này thì netlist có thể như sau:



Hình 4-21. Netlist sau tổng hợp của ví dụ 4-30 nếu không có Flip-Flop hỗ trợ chân *E*

Hai Flip-Flop được mô tả bằng hai khối **always** riêng biệt sẽ cho kết quả như mong muốn.

Ví dụ 4-34

```

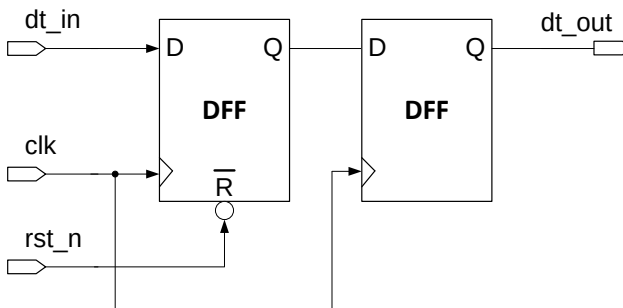
module testcode (clk, rst_n, dt_in, dt_out1);
input          clk, rst_n;

```

```

input          dt_in;
output reg     dt_out1;
reg dt_out0;
//Have the asynchronous reset
always @ (posedge clk,negedge rst_n) begin
    if (~rst_n) dt_out0 <= 1'b0;
    else        dt_out0 <= dt_in;
end
//Do not have the reset
always @ (posedge clk) begin
    dt_out1 <= dt_out0;
end
endmodule

```



Hình 4-22. Netlist sau tổng hợp của ví dụ 4-34

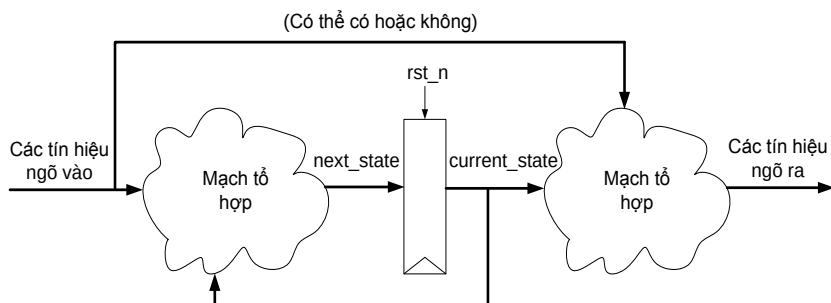
4.3 Máy trạng thái

4.3.1 Mô hình máy trạng thái

Trong thiết kế vi mạch số, máy trạng thái hữu hạn hay FSM, gọi tắt là máy trạng thái, là một dạng mạch tuần tự được sử dụng để mô hình hóa các hoạt động điều khiển theo từng bước. Việc sử dụng máy trạng thái giúp dễ dàng quản lý và hình dung luồng hoạt động của một thiết kế. FSM có ba thành phần cơ bản là mạch tạo trạng thái kế tiếp, thanh ghi trạng thái và mạch tạo ngõ ra.

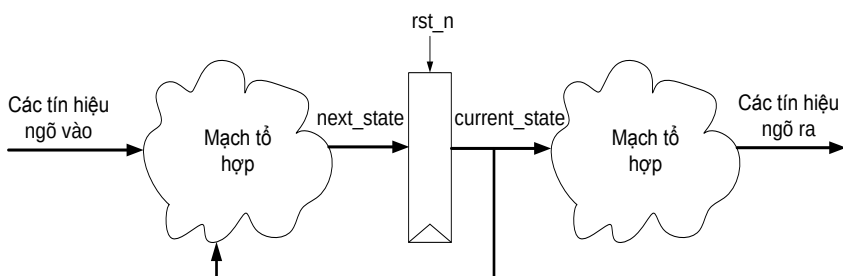
- Mạch tạo trạng thái kế tiếp là mạch tổ hợp phụ thuộc vào trạng thái hiện tại và ngõ vào.
- Thanh ghi trạng thái là mạch tuần tự lưu trạng thái hiện tại và có tín hiệu reset để khởi tạo trạng thái đầu trước khi FSM hoạt động. Số bit của thanh ghi trạng thái tùy thuộc vào số trạng thái và phương pháp mã hóa trạng thái.

- Mạch tạo ngõ ra là một mạch tổ hợp phụ thuộc vào trạng thái hiện tại và có thể phụ thuộc vào ngõ vào.



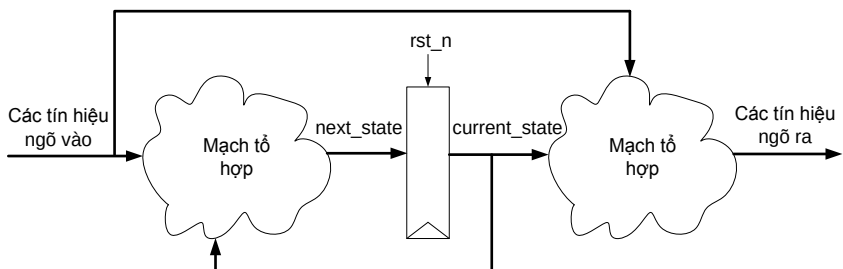
Hình 4-23. Mô hình máy trạng thái

Nếu ngõ ra FSM chỉ phụ thuộc vào trạng thái hiện tại thì FSM thuộc kiểu Moore.



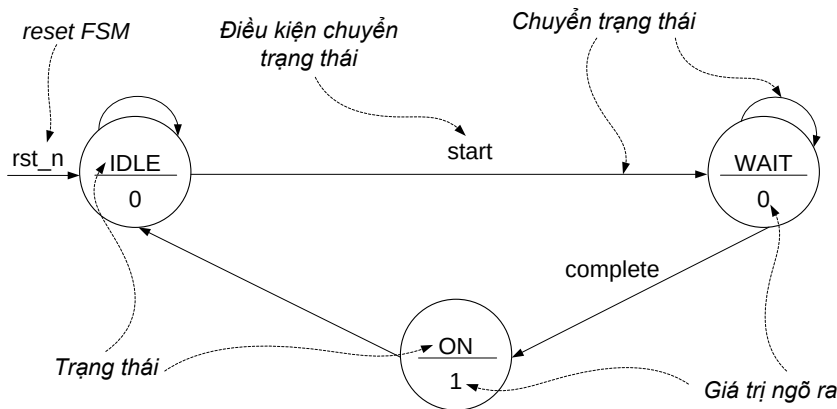
Hình 4-24. Mô hình máy trạng thái kiểu Moore

Nếu ngõ ra FSM vừa phụ thuộc vào trạng thái hiện tại, vừa phụ thuộc vào các tín hiệu ngõ vào thì FSM thuộc kiểu Mealy.



Hình 4-25. Máy trạng thái kiểu Mealy

FSM thường được mô tả bằng một sơ đồ thể hiện rõ tất cả các trạng thái, điều kiện chuyển trạng thái và giá trị ngõ ra gọi là sơ đồ trạng thái.



Hình 4-26. Minh họa một kiểu sơ đồ trạng thái

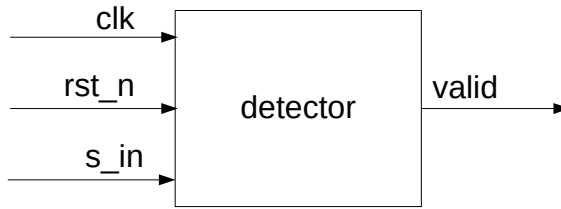
4.3.2 Phương pháp mô tả máy trạng thái

4.3.2.1 Giới thiệu chung

Dựa trên các thành phần của máy trạng thái, việc mô tả RTL code cho FSM được quy về mô tả hai mạch tổ hợp và một mạch tuần tự. Hai dạng mạch này có thể được mô tả bằng bất kỳ phương pháp nào như đã trình bày.

Ứng dụng nhỏ sau đây sẽ được dùng để phân tích cách thức mô tả RTL code cho máy trạng thái. Một hệ thống đồng bộ sử dụng FSM để phát hiện và báo hiệu nhóm bit 001 xuất hiện trong chuỗi bit ngẫu nhiên liên tục. Độ rộng của một bit bằng chu kỳ xung clock của bộ phát hiện nhóm bit. Các tín hiệu giao tiếp gồm:

- Tín hiệu xung clock hệ thống *clk*
- Tín hiệu reset bất đồng bộ tích cực mức thấp *rst_n*
- Tín hiệu ngõ vào của chuỗi bit nối tiếp *s_in*
- Tín hiệu ngõ ra *valid* tích cực một chu kỳ xung clock khi phát hiện được nhóm bit 001.

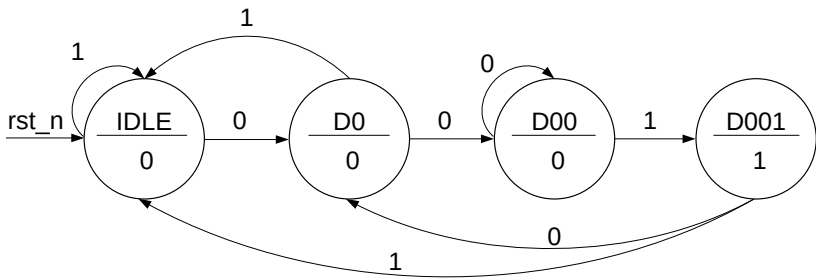


Hình 4-27. Sơ đồ tín hiệu giao tiếp của khối detector

4.3.2.2 Thực hiện máy trạng thái kiểu Moore

FSM kiểu Moore thực hiện chức năng phát hiện nhóm bit *001* gồm bốn trạng thái.

- IDLE là trạng thái khởi tạo ban đầu.
- D0 là trạng thái phát hiện được bit *0*.
- D00 là trạng thái phát hiện được bit *0* thứ hai liên tiếp.
- D001 là trạng thái đã phát hiện được nhóm bit *001*.

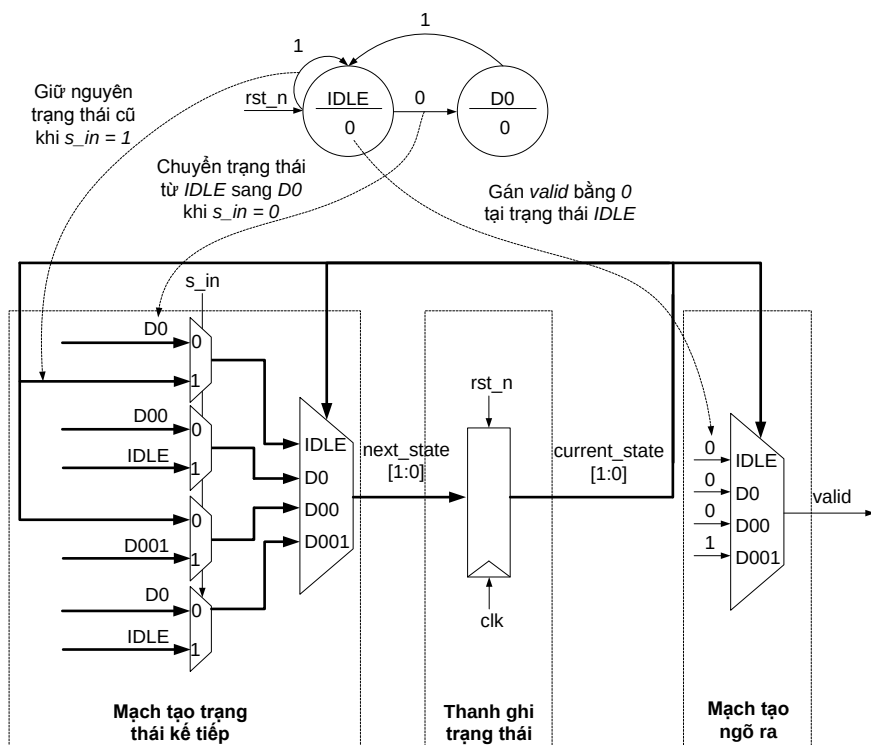


Điều kiện chuyển trạng thái là *s_in*

Ngõ ra FSM là *valid*

Hình 4-28. Sơ đồ FSM kiểu Moore phát hiện nhóm bit *001*

Sơ đồ trạng thái thể hiện rõ ràng hoạt động của một FSM nhưng lại không cho thấy cấu trúc FSM trong mạch số. Sơ đồ nguyên lý sau đây sẽ thể hiện rõ ràng hơn cấu trúc của FSM trong mạch số. Trong đó, hai bit thanh ghi trạng thái được sử dụng để phân biệt bốn trạng thái.



Hình 4-29. Sơ đồ nguyên lý FSM Moore trong tương quan với sơ đồ trạng thái

Đầu tiên, các trạng thái được mã hóa theo kiểu tối thiểu hóa số lượng Flip-Flop của thanh ghi trạng thái như sau:

```
parameter IDLE = 2'd0;
parameter D0 = 2'd1;
parameter D00 = 2'd2;
parameter D001 = 2'd3;
```

Tiếp theo, từng mạch của FSM được mô tả bằng một đoạn RTL code tương ứng.

Ví dụ 4-35

RTL code dành cho mạch tạo trạng thái kế tiếp.

```
always @ (*) begin
    case (current_state[1:0])
        IDLE: begin
```

```

        if (~s_in) next_state[1:0] = D0;
        else      next_state[1:0] = current_state[1:0];
    end
D0: begin
    if (~s_in) next_state[1:0] = D00;
    else      next_state[1:0] = IDLE;
end
D00: begin
    if (s_in)  next_state[1:0] = D001;
    else      next_state[1:0] = current_state[1:0];
end
D001: begin
    if (s_in)  next_state[1:0] = IDLE;
    else      next_state[1:0] = D0;
end
default:      next_state[1:0] = IDLE;
endcase
end

```

RTL code dành cho thanh ghi trạng thái.

```

always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[1:0] <= IDLE;
    else      current_state[1:0] <= next_state[1:0];
end

```

RTL code dành cho mạch tạo ngõ ra.

```

always @ (*) begin
    if (current_state[1:0] == D001)
        valid = 1'b1;
    else valid = 1'b0;
end

```

Tất cả các mô tả mạch trên được ghép lại và thêm đầy đủ khai báo tín hiệu sẽ tạo ra RTL code hoàn chỉnh cho FSM.

Ví dụ 4-36

```

module moore_fsm (clk,rst_n,s_in,valid);
//States
parameter IDLE = 2'd0;
parameter D0   = 2'd1;
parameter D00  = 2'd2;
parameter D001 = 2'd3;
//Inputs
input      clk, rst_n;
input      s_in;

```

```

//Outputs
output reg valid;
//Internal signals
reg [1:0] next_state, current_state;
//Combinational circuit of the next state
always @ (*) begin
    case (current_state[1:0])
        IDLE: begin
            if (~s_in) next_state[1:0] = D0;
            else      next_state[1:0] = current_state[1:0];
        end
        D0: begin
            if (~s_in) next_state[1:0] = D00;
            else      next_state[1:0] = IDLE;
        end
        D00: begin
            if (s_in) next_state[1:0] = D001;
            else      next_state[1:0] = current_state[1:0];
        end
        D001: begin
            if (s_in) next_state[1:0] = IDLE;
            else      next_state[1:0] = D0;
        end
        default:      next_state[1:0] = IDLE;
    endcase
end
//The state register
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[1:0] <= IDLE;
    else      current_state[1:0] <= next_state[1:0];
end
//Combinational circuit of the output
always @ (*) begin
    if (current_state[1:0] == D001)
        valid = 1'b1;
    else valid = 1'b0;
end
endmodule

```

Trong trường hợp này, mạch tạo ngõ ra là mạch đơn giản và có thể được mô tả bằng phép gán **assign** như sau:

```
assign valid = (current_state[1:0]==D001)? 1'b1: 1'b0;
```

hoặc:

```
assign valid = (current_state[1:0] == D001);
```


hoặc:

```
assign valid = &current_state[1:0];
```

Ở mạch tạo trạng thái kế tiếp, tín hiệu *current_state* trong các phép gán có thể được thay bằng tên trạng thái tương ứng. Trong trạng thái IDLE, *current_state* thay bằng tên IDLE. Trong trạng thái D00, *current_state* thay bằng tên D00.

Tín hiệu *next_state* chỉ là trung gian nên mạch tạo trạng thái kế tiếp và thanh ghi trạng thái có thể mô tả gộp chung.

Ví dụ 4-37

```
module moore_fsm (clk,rst_n,s_in,valid);  
  //States  
  parameter IDLE  = 2'd0;  
  parameter D0    = 2'd1;  
  parameter D00   = 2'd2;  
  parameter D001  = 2'd3;  
  //Inputs  
  input      clk, rst_n;  
  input      s_in;  
  //Outputs  
  output wire valid;  
  //Internal signals  
  reg [1:0] current_state;  
  //the next state and current_state  
  always @ (posedge clk, negedge rst_n) begin  
    if (~rst_n)      current_state[1:0] <= IDLE;  
    else begin  
      case (current_state[1:0])  
        IDLE: if (~s_in) current_state[1:0] <= D0;  
        D0: begin  
          if (~s_in) current_state[1:0] <= D00;  
          else      current_state[1:0] <= IDLE;  
        end  
        D00: if (s_in)  current_state[1:0] <= D001;  
        D001: begin  
          if (s_in)  current_state[1:0] <= IDLE;  
          else      current_state[1:0] <= D0;  
        end  
      endcase  
    end  
  end  
  //Combinational circuit of the output  
  assign valid = &current_state[1:0];  
endmodule
```

Vì mạch tạo ngõ ra FSM và mạch tạo trạng thái kế tiếp đều là mạch tổ hợp phụ thuộc vào *current_state* nên cũng có thể mô tả gộp chung nhưng phải gán đầy đủ giá trị cho các tín hiệu trên tất cả các nhánh để tránh tạo Latch.

Ví dụ 4-38

```

module moore_fsm (clk,rst_n,s_in,valid);
//States
parameter IDLE   = 2'd0;
parameter D0     = 2'd1;
parameter D00    = 2'd2;
parameter D001   = 2'd3;
//Inputs
input      clk, rst_n;
input      s_in;
//Outputs
output reg valid;
//Internal signals
reg [1:0] next_state, current_state;
//Combinational circuit of the next state
// and output
always @ (*) begin
    case (current_state[1:0])
        IDLE: begin
            valid = 1'b0;
            if (~s_in) next_state[1:0] = D0;
            else      next_state[1:0] = current_state[1:0];
        end
        D0: begin
            valid = 1'b0;
            if (~s_in) next_state[1:0] = D00;
            else      next_state[1:0] = IDLE;
        end
        D00: begin
            valid = 1'b0;
            if (s_in)  next_state[1:0] = D001;
            else      next_state[1:0] = current_state[1:0];
        end
        D001: begin
            valid = 1'b1;
            if (s_in)  next_state[1:0] = IDLE;
            else      next_state[1:0] = D0;
        end
        default: begin
            valid = 1'b0;
            next_state[1:0] = IDLE;
        end
    endcase
end

```

```

    endcase
end
//The state register
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[1:0] <= IDLE;
    else        current_state[1:0] <= next_state[1:0];
end
endmodule

```

Ngoài cách mô tả hành vi, tương tự các mạch số khác, FSM có thể chuyển về cấu trúc mức cổng và được mô tả RTL code dưới dạng này. Tuy cách này ít được sử dụng trong các thiết kế FSM có số lượng trạng thái hoặc ngõ ra nhiều nhưng phương pháp này được trình bày với mục đích giúp người đọc hiểu rõ hơn bản chất của FSM trong mạch số.

Bảng 4-9. Sự phụ thuộc của next_state vào current_state và ngõ vào

s_in	current_state[1:0]	next_state[1:0]	
0	IDLE	00	D0
1			01
1	D0	01	IDLE
0			00
1	D00	10	D001
0			11
0	D001	11	D00
1			10

Bảng 4-10. Sự phụ thuộc của valid vào current_state

current_state[1:0]	valid
IDLE	0
D0	0
D00	0
D001	1

Dựa trên bảng 4-9, biểu thức của từng bit trong tín hiệu next_state như sau:

$$\text{next_state}[1] = \overline{s_in} \cdot \overline{\text{current_state}[1]} \cdot \text{current_state}[0] + \text{current_state}[1] \cdot \overline{\text{current_state}[0]}$$

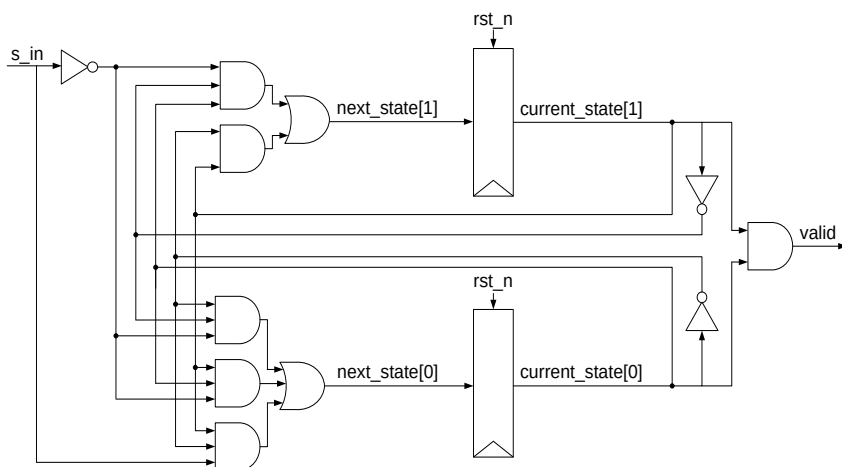
$$\begin{aligned} \text{next_state}[0] = & \overline{s_in} \cdot \overline{\text{current_state}[1]} \cdot \text{current_state}[0] + \\ & \overline{s_in} \cdot \text{current_state}[1] \cdot \text{current_state}[0] + \\ & s_in \cdot \overline{\text{current_state}[1]} \cdot \overline{\text{current_state}[0]} + \end{aligned}$$

	current_state[1:0]					current_state[1:0]			
	00	01	11	10		00	01	11	10
s_in	0	0	1	0	1	0	1	0	0
	1	0	0	0	1	0	0	0	1
	next_state[1]					next_state[0]			

Hình 4-30. Bìa Karnaugh để xác định biểu thức next_state

Dựa trên bảng 4-10, biểu thức của *valid* như sau:

$$\text{valid} = \text{current_state}[1] \cdot \text{current_state}[0]$$



Hình 4-31. Mạch nguyên lý mức cổng của FSM

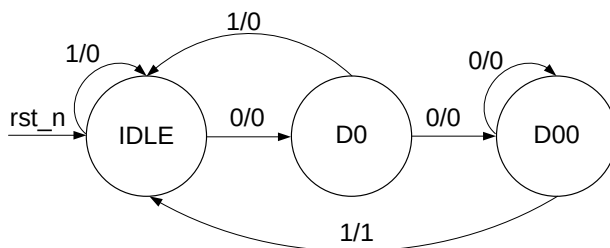
Ví dụ 4-39

RTL code theo mạch nguyên lý mức cổng của FSM

```
module moore_fsm (clk,rst_n,s_in,valid);
//Inputs
input    clk, rst_n;
input    s_in;
//Outputs
output wire valid;
//Internal signals
reg [1:0] current_state;
wire[1:0] next_state;
//the current_state
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[1:0] <= 2'b00;
    else        current_state[1:0] <= next_state[1:0];
end
//the next state
assign next_state[1] = (~s_in & ~current_state[1] &
    current_state[0]) |
    (current_state[1] &
    ~current_state[0]);
assign next_state[0] = (~s_in & ~current_state[1] &
    ~current_state[0]) |
    (~s_in & current_state[1] &
    current_state[0]) |
    (s_in & current_state[1] &
    ~current_state[0]);
//Combinational circuit of the output
assign valid = &current_state[1:0];
endmodule
```

4.3.2.3 Thực hiện máy trạng thái kiểu Mealy

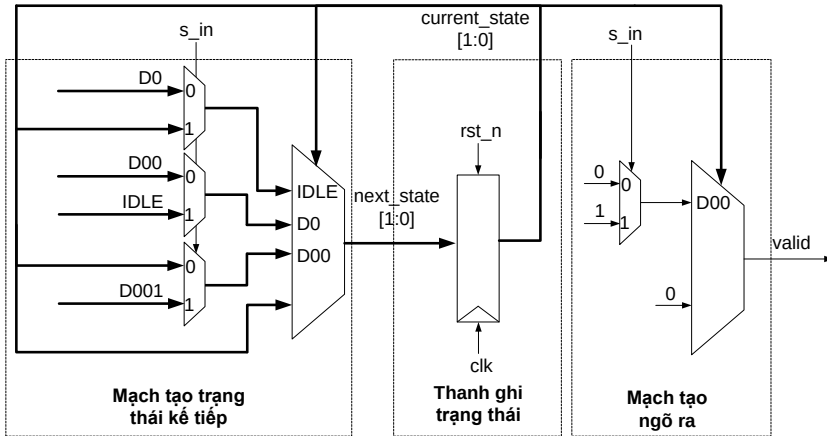
FSM kiểu Mealy thực hiện chức năng phát hiện nhóm bit 001 chỉ gồm ba trạng thái.



s_in / valid = điều kiện chuyển trạng thái / ngõ ra

Hình 4-32. FSM kiểu Mealy phát hiện nhóm bit 001

Tại trạng thái *D00* nếu phát hiện ngõ vào *s_in* mức 1 thì ngõ ra *valid* sẽ tích cực. Tín hiệu *valid* sẽ xuống 0 ngay khi chuyển trạng thái.



Hình 4-33. Mạch nguyên lý FSM Mealy phát hiện nhóm bit 001

Điểm khác biệt quan trọng giữa FSM Mealy và FSM Moore là ở mạch tạo giá trị ngõ ra. Mạch tạo giá trị ngõ ra của FSM Mealy phụ thuộc vào cả *current_state* và ngõ vào *s_in*. Phương pháp mô tả FSM Mealy cũng tương tự FSM Moore nên phần này chỉ trình bày một cách mô tả tổng quát. Các cách khác, người đọc có thể tự suy luận và thực hành để hiểu rõ thêm.

Ví dụ 4-40

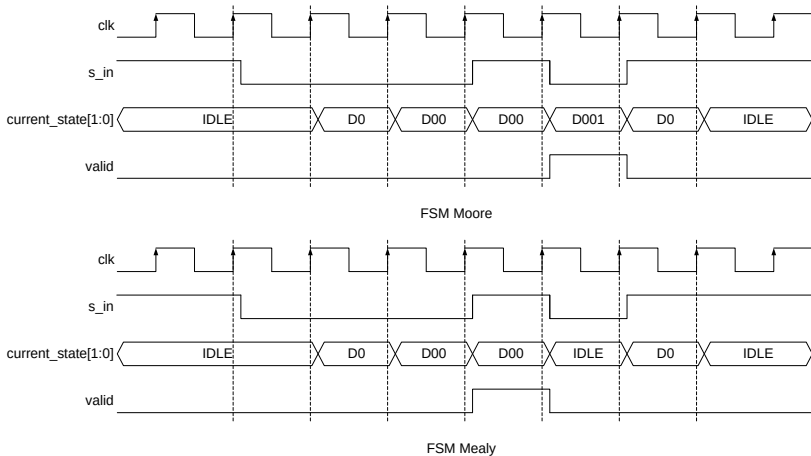
```
module mealy_fsm (clk,rst_n,s_in,valid);
//States
parameter IDLE = 2'd0;
parameter D0 = 2'd1;
parameter D00 = 2'd2;
//Inputs
input clk, rst_n;
input s_in;
//Outputs
output reg valid;
//Internal signals
reg [1:0] next_state, current_state;
//Combinational circuit of the next state
always @ (*) begin
    case (current_state[1:0])
        IDLE: begin
            if (~s_in) next_state[1:0] = D0;
            else next_state[1:0] = IDLE;
        end
    end
```

```

D0: begin
    if (~s_in) next_state[1:0] = D00;
    else      next_state[1:0] = IDLE;
end
D00: begin
    if (s_in)  next_state[1:0] = IDLE;
    else      next_state[1:0] = D00;
end
default:      next_state[1:0] = IDLE;
endcase
end
//The state register
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[1:0] <= IDLE;
    else        current_state[1:0] <= next_state[1:0];
end
//Combinational circuit of the output
always @ (*) begin
    if (current_state == D00)
        valid = s_in;
    else valid = 1'b0;
end
endmodule

```

FSM Mealy và FSM Moore cùng thực hiện một chức năng nhưng đáp ứng ngõ ra của hai FSM này khác nhau.



Hình 4-34. Đáp ứng ngõ ra của FSM phát hiện nhóm bit 001

Trường hợp này, nếu muốn đáp ứng ngõ ra của FSM Mealy tương tự như FSM Moore thì ngõ ra của FSM Mealy được chốt bằng một thanh ghi và RTL code ngõ ra được mô tả như sau:

```

always @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        valid <= 1'b0;
    else if (current_state == D00)
        valid <= s_in;
    else
        valid <= 1'b0;
end

```

4.3.3 Mã hóa trạng thái

Một điểm quan trọng trong mô tả FSM là lựa chọn mã hóa trạng thái. Cùng một FSM, việc lựa chọn mã hóa trạng thái khác nhau sẽ cho mạch tổng hợp có tài nguyên và tốc độ khác nhau. Nhiều loại mã có thể sử dụng để mã hóa trạng thái như binary tuần tự, Gray, one-hot, ... hoặc người dùng tự định nghĩa nhưng phần này chỉ trình bày về mã hóa trạng thái one-hot và phương pháp mô tả FSM one-hot.

Với mã hóa one-hot, mỗi bit trong thanh ghi trạng thái sẽ quy định một trạng thái. Mã hóa trạng thái one-hot thường tạo ra mạch có tốc độ cao hơn loại mã hóa tối thiểu số Flip-Flop thông thường. Tuy nhiên, tài nguyên tổng hợp và diện tích mạch FSM one-hot lại lớn hơn.

Các trạng thái của FSM kiểu Moore trong bộ phát hiện nhóm bit *001* được mã hóa one-hot như sau:

```

parameter IDLE   = 4'b0001;
parameter D0     = 4'b0010;
parameter D00    = 4'b0100;
parameter D001   = 4'b1000;

```

Lúc này, thanh ghi trạng thái sẽ có bốn bit tương ứng với bốn trạng thái của FSM.

Ví dụ 4-41

```

module moore_fsm_onehot (clk,rst_n,s_in,valid);
//State values
parameter IDLE   = 4'b0001;
parameter D0     = 4'b0010;
parameter D00    = 4'b0100;
parameter D001   = 4'b1000;
//Inputs
input    clk, rst_n;
input    s_in;
//Outputs

```



```

output    valid;
//Internal signals
reg [3:0] next_state, current_state;
//Combinational circuit of the next state
always @ (*) begin
    case (current_state[3:0])
        IDLE: begin
            if (~s_in) next_state[3:0] = D0;
            else       next_state[3:0] = IDLE;
        end
        D0: begin
            if (~s_in) next_state[3:0] = D00;
            else       next_state[3:0] = IDLE;
        end
        D00: begin
            if (s_in)  next_state[3:0] = D001;
            else       next_state[3:0] = D00;
        end
        D001: begin
            if (s_in)  next_state[3:0] = IDLE;
            else       next_state[3:0] = D0;
        end
        default:      next_state[3:0] = IDLE;
    endcase
end
//The state register
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[3:0] <= IDLE;
    else       current_state[3:0] <= next_state[3:0];
end
//Combinational circuit of the output
assign valid = current_state[3];
endmodule

```

Cách mô tả RTL code trên đây sử dụng *current_state* làm *case_expression* cho phát biểu **case** mô tả mạch tạo trạng thái kế tiếp. Giá trị của các tên trạng thái chính là giá trị của thanh ghi trạng thái. Một cách khác thường dùng để mô tả FSM one-hot là tên trạng thái được sử dụng làm chỉ số xác định bit quy định trạng thái. Giả sử, trạng thái IDLE được quy định bởi bit 0 trong thanh ghi trạng thái, D0 được quy định bởi bit 1, D00 được quy định bởi bit 2 và D001 được quy định bởi bit 3 thì giá trị các parameter được khai báo như trong ví dụ sau:

Ví dụ 4-42

```
module moore_fsm_onehot (clk,rst_n,s_in,valid);
//State Indexs
parameter IDLE   = 2'd0;
parameter D0     = 2'd1;
parameter D00    = 2'd2;
parameter D001   = 2'd3;
//Inputs
input    clk, rst_n;
input    s_in;
//Outputs
output   valid;
//Internal signals
reg [3:0] next_state, current_state;
//Combinational circuit of the next state
always @ (*) begin
    next_state[3:0] = 4'b0000;
    case (1'b1)
        current_state[IDLE]: begin
            if (~s_in) next_state[D0]   = 1'b1;
            else       next_state[IDLE] = 1'b1;
        end
        current_state[D0]: begin
            if (~s_in) next_state[D00]  = 1'b1;
            else       next_state[IDLE] = 1'b1;
        end
        current_state[D00]: begin
            if (s_in)  next_state[D001] = 1'b1;
            else       next_state[D00]  = 1'b1;
        end
        current_state[D001]: begin
            if (s_in)  next_state[IDLE] = 1'b1;
            else       next_state[D0]   = 1'b1;
        end
    endcase
end
//The state register
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) current_state[3:0] <= 4'b0001;
    else       current_state[3:0] <= next_state[3:0];
end
//Combinational circuit of the output
assign valid = current_state[D001];
endmodule
```

Trong ví dụ 4-42, *case_expression* là hằng số *1'b1*. Các *case_item* là giá trị của từng bit của thanh ghi trạng thái.

5

ỨNG DỤNG VERILOG HDL

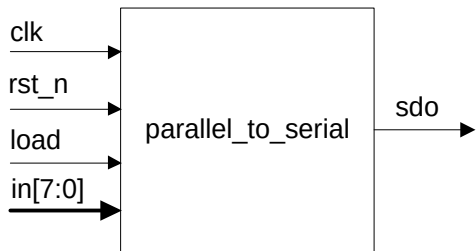
Chương này trình bày một số thiết kế phổ biến giúp người đọc hiểu rõ hơn về cách ứng dụng Verilog HDL trong mô tả mạch số. Mỗi vấn đề được nêu ra sẽ có nhiều cách thực hiện khác nhau nhưng chương này chỉ trình bày một cách tham khảo. Từ đó, người đọc có cái nhìn tổng quan và áp dụng mô tả các mạch số khác.

5.1 Mạch chuyển song song sang nối tiếp

5.1.1 Đặt vấn đề

Máy tính thường lưu và xử lý dữ liệu với định dạng song song như byte, halfword, word,... Nhưng khi dữ liệu được truyền đi một khoảng cách dài thì việc truyền song song không còn thích hợp vì tốn nhiều đường dây, hay bị nhiễu,... Vì vậy, dữ liệu truyền song song sẽ chuyển thành truyền nối tiếp.

Phần này thực hiện mạch chuyển tám bit song song thành nối tiếp như hình sau:

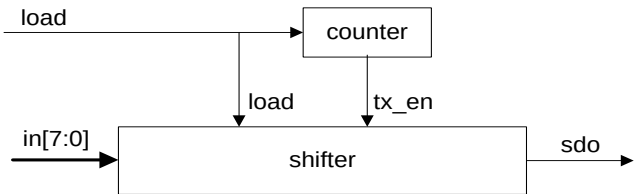


Hình 5-1. Sơ đồ tín hiệu mạch chuyển đổi dữ liệu song song sang nối tiếp

Tín hiệu *load* chỉ ra ngõ vào *in[7:0]* có giá trị mới cần phát đi, tín hiệu *sdo* là ngõ ra nối tiếp. Khi *load = 1* thì *in[7:0]* sẽ được lưu lại bên trong khối *parallel_to_serial* và dữ liệu sẽ được phát đi theo thứ tự từ bit 0 đến bit 7 mỗi khi có xung *clk*.

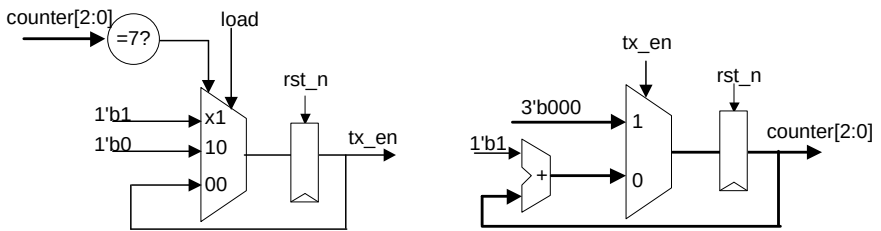
5.1.2 Phân tích vấn đề

Để thiết kế mạch chuyển đổi dữ liệu từ song song sang nối tiếp, thanh ghi dịch có thể sử dụng trong thiết kế như hình sau:



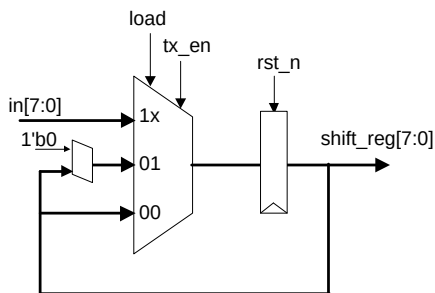
Hình 5-2. Sơ đồ khối mạch chuyển đổi dữ liệu song song sang nối tiếp

Khi có tín hiệu *load* thì tín hiệu *tx_en* bật lên để cho phép thanh ghi đếm hoạt động. Khi đếm lên 7 sẽ quay về 0 và tín hiệu *tx_en* sẽ xuống 0, thanh ghi đếm không hoạt động nữa.

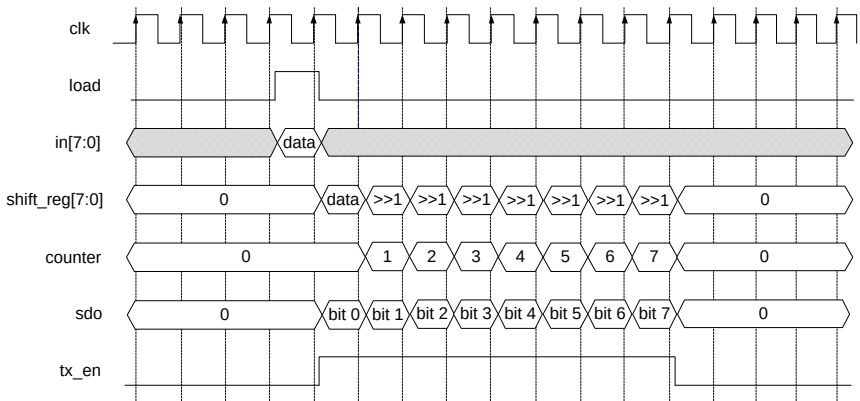


Hình 5-3. Sơ đồ mạch khối counter

Khi có tín hiệu *load* thì giá trị tại ngõ vào *in[7:0]* được lưu vào thanh ghi *shift_reg[7:0]*. Tín hiệu *tx_en* = 1'b1 cho phép thanh ghi *shift_reg[7:0]* dịch chuyển dữ liệu để đưa ra ngoài. Ngõ ra *sdo* là bit *shift_reg[0]*.



Hình 5-4. Sơ đồ mạch ghi dịch phát nối tiếp



Hình 5-5. Dạng sóng chuyển đổi dữ liệu từ song song sang nối tiếp

5.1.3 Mô tả RTL code

```
module parallel_to_serial(clk, rst_n, load, in, sdo);
input      clk;
input      rst_n;
input      load;
input [7:0] in;
output     sdo;

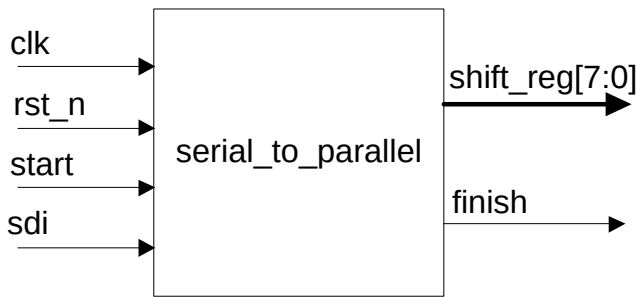
reg  [7:0] shift_reg;
reg      tx_en;
reg  [2:0] counter;
// serial output
assign sdo = shift_reg[0];
// tx_en
always @ (posedge clk) begin
    if (!rst_n) tx_en <= 1'b0;
    else if (load) tx_en <= 1'b1;
    else if (counter[2:0] == 3'd7) tx_en <= 1'b0;
end
// counter
always @ (posedge clk) begin
    if (!rst_n) counter[2:0] <= 3'd0;
    else if (load) counter[2:0] <= 3'd0;
    else if (tx_en) counter[2:0] <= counter[2:0] + 1'b1;
end
// transmit & shift data
always @ (posedge clk) begin
    if (!rst_n) shift_reg[7:0] <= 8'd0;
    else if (load) shift_reg[7:0] <= in[7:0];
    else if (tx_en) shift_reg[7:0] <= {1'b0, shift_reg[7:1]};
end
endmodule
```

5.2 Mạch chuyển nối tiếp sang song song

5.2.1 Đặt vấn đề

Khi dữ liệu truyền được chuyển đổi từ song song sang nối tiếp tại đầu phát thì tại đầu nhận cần phải chuyển đổi ngược lại vì các dữ liệu trong máy tính thường là dữ liệu song song: byte, word, haftword,...

Phần này thực hiện phân tích mạch chuyển tám bit nối tiếp thành song song như hình sau:

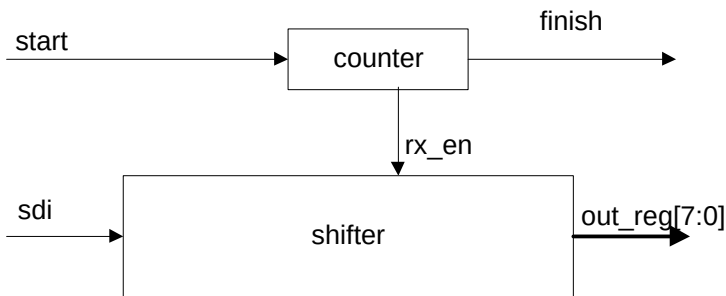


Hình 5-6. Sơ đồ tín hiệu mạch chuyển đổi dữ liệu nối tiếp sang song song

Tín hiệu *start* chỉ ra ngõ vào *sdi* bắt đầu có giá trị cần phải nhận, tín hiệu *finish* chỉ ra là ngõ ra *out_reg[7:0]* đã nhận đủ tám bit. Dữ liệu nhận được theo thứ tự từ *bit 0* đến *bit 7* mỗi khi có xung *clk*.

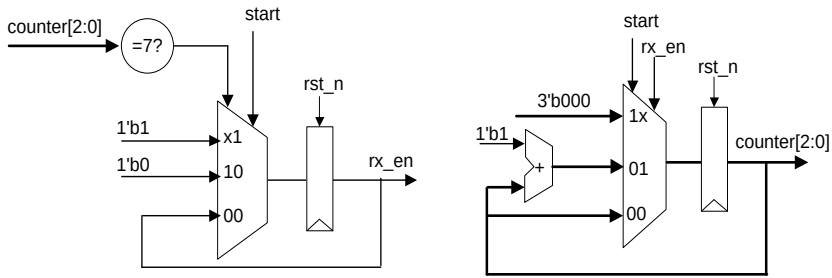
5.2.2 Phân tích vấn đề

Để thiết kế mạch chuyển đổi dữ liệu từ song song sang nối tiếp, thanh ghi dịch có thể sử dụng trong thiết kế như hình sau:



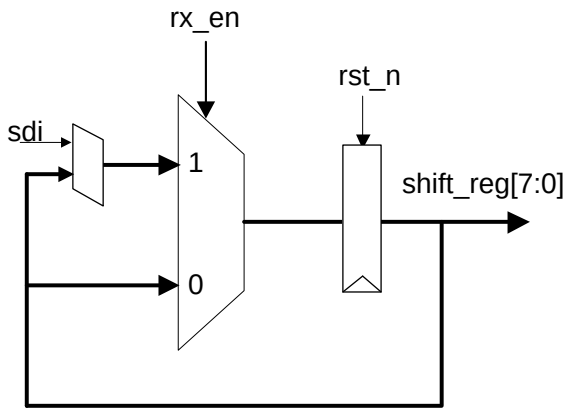
Hình 5-7. Sơ đồ khối mạch chuyển đổi dữ liệu từ nối tiếp sang song song

Thiết kế giống như phần 5.1, khi có tín hiệu *start = 1'b1* tín hiệu *rx_en* bật lên để cho phép thanh ghi *counter* hoạt động. Khi *counter* đếm lên 7 sẽ quay về 0, đồng thời tín hiệu *rx_en* sẽ xuống 0 và tín hiệu *finish = 1'b1* để chỉ ra đã nhận xong tám bit.

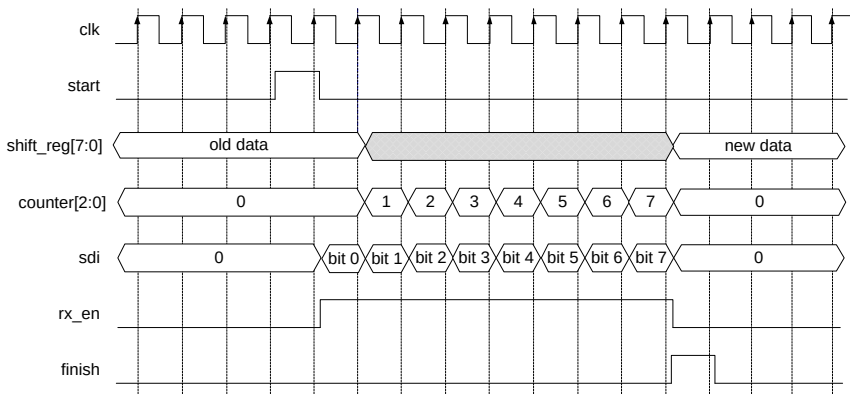


Hình 5-8. Sơ đồ mạch khối counter

Tín hiệu $rx_en = 1'b1$ cho phép thanh ghi $shift_reg[7:0]$ dịch chuyển và nhận dữ liệu từ ngõ vào sdi .



Hình 5-9. Sơ đồ mạch ghi dịch nhận nối tiếp



Hình 5-10. Dạng sóng chuyển đổi dữ liệu từ nối tiếp sang song song

5.2.3 Mô tả RTL code

```
module serial_to_parallel( clk, rst_n, start, sdi,
                          shift_reg, finish);

input      clk;
input      rst_n;
input      start;
input      sdi;
output [7:0] shift_reg;
output      finish;
reg [7:0] shift_reg;
reg      rx_en;
reg [2:0] counter;
reg      finish;

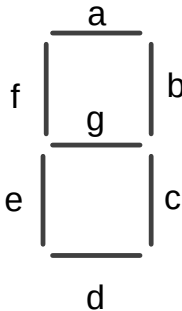
// rx_en
always @ (posedge clk) begin
    if (!rst_n) rx_en <= 1'b0;
    else if (start) rx_en <= 1'b1;
    else if (counter == 3'd7) rx_en <= 1'b0;
end
// counter
always @ (posedge clk) begin
    if (!rst_n) counter[2:0] <= 3'd0;
    else if (start) counter[2:0] <= 3'd0;
    else if (rx_en) counter[2:0] <= counter[2:0] + 1'b1;
end
// finish
always @ (posedge clk) begin
    if (!rst_n) finish <= 1'b0;
    else if (counter == 3'd7) finish <= 1'b1;
    else finish <= 1'b0;
end
// receive & shift data
always @ (posedge clk) begin
    if (!rst_n) shift_reg <= 8'd0;
    else if(rx_en) shift_reg[7:0] <= {sdi, shift_reg[7:1]};
end

endmodule
```

5.3 Giải mã hex sang mã LED bảy đoạn

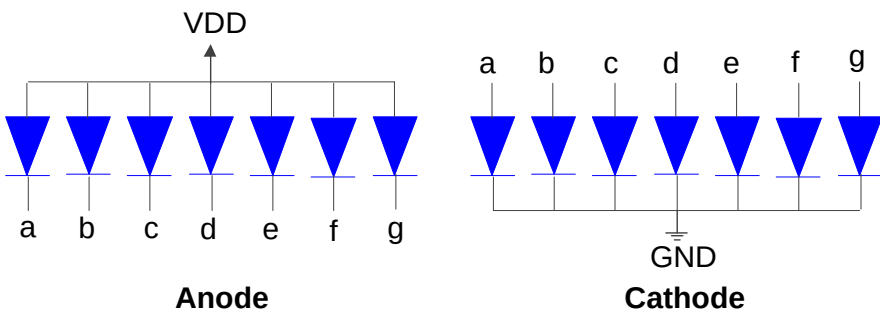
5.3.1 Đặt vấn đề

LED bảy đoạn được ứng dụng khá phổ biến khi cần hiển thị số tự nhiên hoặc vài chữ cái nhất định. LED bảy đoạn gồm có bảy đoạn LED nhỏ a, b, c, d, e, f và g, được quy ước các đoạn như hình sau:



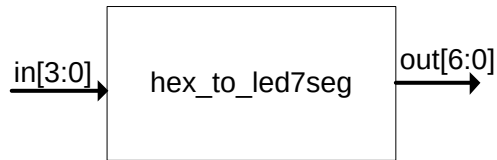
Hình 5-11. LED bảy đoạn

LED bảy đoạn có hai loại là anode và cathode. Loại anode có cực “+” được nối chung với *VDD*. Loại cathode có cực “-” được nối chung với *GND*. Các cực còn lại trên mỗi LED đơn được đưa ra ngoài để kết nối điều khiển.



Hình 5-12. Phân loại LED bảy đoạn

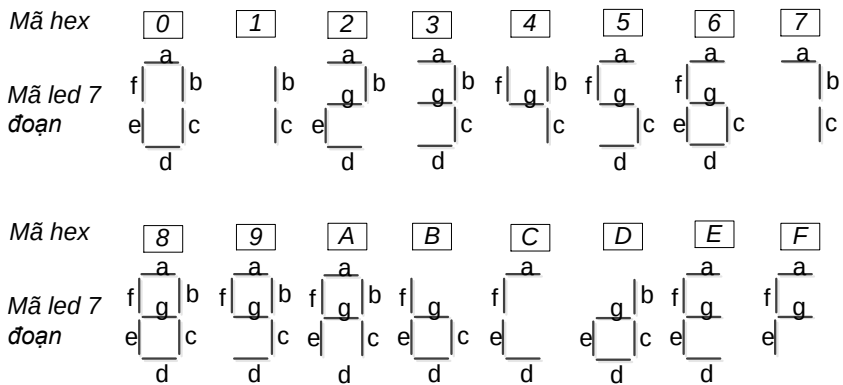
Phần này thực hiện phân tích mạch chuyển đổi bốn bit mã nhị phân sang mã hex trên LED bảy đoạn loại cathode. Tín hiệu điều khiển bảy đoạn LED tương ứng với 7 bit của ngõ ra *out[6:0]*.



Hình 5-13. Sơ đồ tín hiệu mạch chuyển đổi bốn bit mã hex sang LED bảy đoạn

5.3.2 Phân tích vấn đề

Mã hex gồm có 16 giá trị tương ứng từ 0 đến F



Hình 5-14. Giá trị hex trên là bảy đoạn

Các tín hiệu điều khiển $\{g,f,e,d,c,b,a\}$ tương đương với $out[6:0]$ thì ta có bảng mã điều khiển như sau:

Bảng 5-1. Bảng mã chuyển đổi mã hex sang LED bảy đoạn

Mã nhị phân (in[3:0])	Mã hex	Mã LED bảy đoạn (out[6:0])
0000	0	011_1111
0001	1	000_0110
0010	2	101_1011
0011	3	100_1111
0100	4	110_0110
0101	5	110_1101
0110	6	111_1101
0111	7	000_0111
1000	8	111_1111
1001	9	110_1111

Mã nhị phân (in[3:0])	Mã hex	Mã LED bảy đoạn (out[6:0])
1010	A	111_0111
1011	B	111_1100
1100	C	011_1001
1101	D	101_1110
1110	E	111_1001
1111	F	111_0001

Ở phần này, chúng ta không dùng bìa Karnaugh để tạo ra mạch chuyển đổi bốn bit mã nhị phân sang mã hex trên LED bảy đoạn mà sử dụng phát biểu **case** của Verilog HDL như là một bảng tra để tạo giá trị các ngõ ra để điều khiển LED bảy đoạn. Các tín hiệu điều khiển $\{g,f,e,d,c,b,a\}$ tương đương với $out[6:0]$.

5.3.3 Mô tả RTL code

```

module hex_to_led7seg (in, out);
input      [3:0]in;
output     [6:0]out;
reg        [6:0]out;
always @(*)begin
    case (in)
        4'h0:  out = 7'b011_1111;
        4'h1:  out = 7'b000_0110;
        4'h2:  out = 7'b101_1011;
        4'h3:  out = 7'b100_1111;
        4'h4:  out = 7'b110_0110;
        4'h5:  out = 7'b110_1101;
        4'h6:  out = 7'b111_1101;
        4'h7:  out = 7'b000_0111;
        4'h8:  out = 7'b111_1111;
        4'h9:  out = 7'b110_1111;
        4'hA:  out = 7'b111_0111;
        4'hb:  out = 7'b111_1100;
        4'hC:  out = 7'b011_1001;
        4'hd:  out = 7'b101_1110;
        4'hE:  out = 7'b111_1001;
        4'hF:  out = 7'b111_0001;
        default:  out = 7'b011_1111;
    endcase
end
endmodule

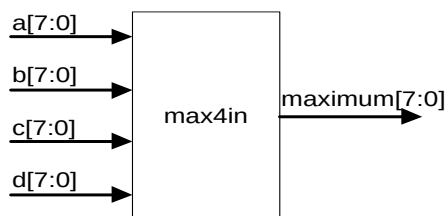
```

5.4 Mạch so sánh và tìm giá trị lớn nhất

5.4.1 Đặt vấn đề

Mạch so sánh có ứng dụng rất phổ biến trong việc tính toán, xử lý dữ liệu của hệ thống số. Bộ so sánh sẽ làm nhiệm vụ so sánh giá trị nhận được với giá trị ngưỡng đã cài đặt để ra quyết định điều khiển hợp lý.

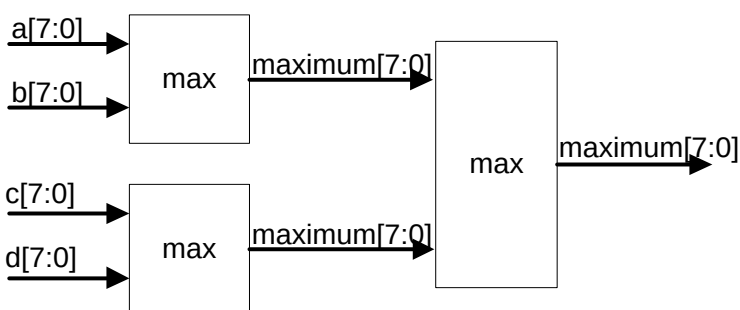
Phần này thực hiện phân tích mạch so sánh và tìm giá trị lớn nhất của bốn tín hiệu ngõ vào.



Hình 5-15. Sơ đồ tín hiệu mạch tìm giá trị lớn nhất

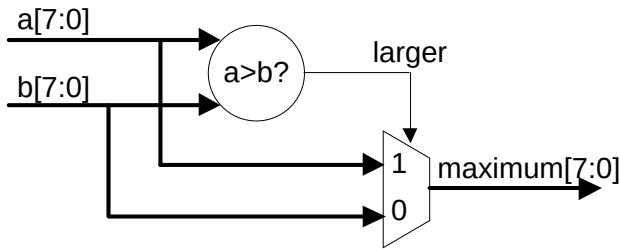
5.4.2 Phân tích vấn đề

Để tìm giá trị lớn nhất của bốn tín hiệu ngõ vào thì trước tiên chúng ta cần thiết kế mạch tìm giá trị lớn nhất của hai tín hiệu ngõ vào, sau đó ghép nối lại để tìm giá trị lớn nhất từ bốn tín hiệu ngõ vào như hình sau:



Hình 5-16. Sơ đồ khối ghép mạch tìm giá trị lớn nhất

Để tìm giá trị lớn nhất của hai tín hiệu ngõ vào thì chúng ta thiết kế mạch so sánh giá trị hai tín hiệu ngõ vào rồi mới lựa chọn giá trị ngõ ra như hình bên dưới.



Hình 5-17. Mạch tìm giá trị lớn nhất của hai tín hiệu ngõ vào

5.4.3 Mô tả RTL code

RTL code của mạch tìm giá trị lớn nhất của hai tín hiệu ngõ vào.

```

module max(a,b, maximum);
input    [7:0] a,b;
output   [7:0] maximum;
wire     sel;

assign sel = a[7:0] > b[7:0];
assign maximum[7:0] = sel ? a[7:0] : b[7:0];
endmodule

```

Sau khi thiết kế xong khối tìm giá trị lớn nhất của hai tín hiệu ngõ vào, thì chúng ta ghép các module *max* lại với nhau như hình 5-16. RTL code của mạch tìm giá trị lớn nhất của bốn tín hiệu ngõ vào như sau:

```

module max4in(a,b,c,d, maximum);
input    [7:0] a;
input    [7:0] b;
input    [7:0] c;
input    [7:0] d;
output   [7:0] maximum;
wire    [7:0] max2;
wire    [7:0] max1;

max      u0 ( .a(a[7:0]),
              .b(b[7:0]),
              .maximum(max1[7:0]) );

max      u1 ( .a(c[7:0]),
              .b(d[7:0]),
              .maximum(max2[7:0]) );

max      u2 ( .a(max1[7:0]),

```

```
.b (max2 [7:0] ) ,
.maximum (maximum) ) ;
```

endmodule

Tương tự như thiết kế ở trên, chúng ta có thể thiết kế mạch tìm giá trị nhỏ nhất từ các tín hiệu ngõ vào.

5.5 Mạch chia tần số

5.5.1 Đặt vấn đề

Trong máy tính có rất nhiều ngoại vi dùng để giao tiếp với các thiết bị bên ngoài như USB, Ethernet, I2S, SD Card, UART.... Mỗi ngoại vi thực hiện việc truyền nhận dữ liệu theo một giới hạn hay một tần số nhất định. Trong khi clock của hệ thống có thể là lớn hơn nhiều lần. Vì vậy, trong máy tính có bộ chia tần số từ clock hệ thống để tạo ra clock có tần số phù hợp cho từng ngoại vi.

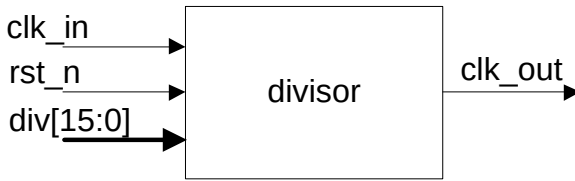
Phần này sẽ giới thiệu bộ chia tần số có thể thay đổi được hệ số chia để tạo ra các tần số clock khác nhau như công thức sau:

$$f_{out} = \frac{f_{in}}{2 \times (div + 1)}$$

Trong đó:

- f_{out} : tần số clock ngõ ra
- f_{in} : tần số clock ngõ vào
- div : hệ số chia. Ví dụ như: $f_{out} = f_{in}/10$ khi đó hệ số chia $div = 4$. Hệ số chia nhỏ nhất là $div = 0$ khi đó $f_{out} = f_{in}/2$. Nếu div là 16 bit thì hệ số chia lớn nhất là $div = 65535$ khi đó $f_{out} = f_{in}/131072$.

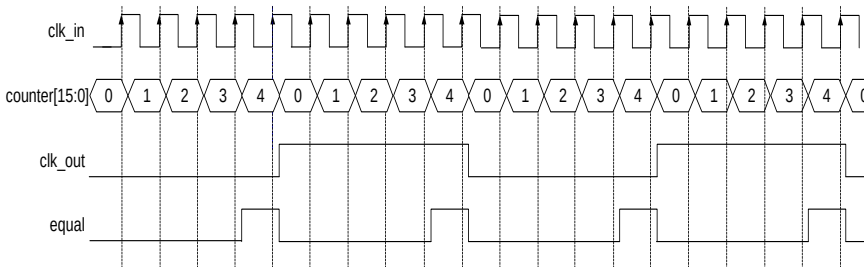
Để có được tần số clock ngõ ra theo hệ số chia như công thức trên chúng ta cần có mạch với các tín hiệu như sau:



Hình 5-18. Sơ đồ tín hiệu mạch chia tần số

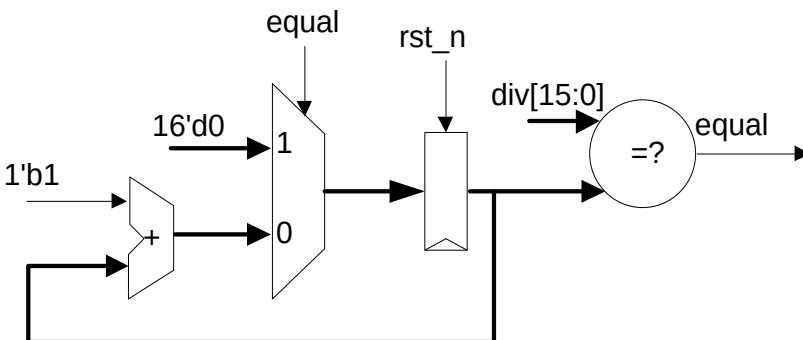
5.5.2 Phân tích và thiết kế

Giả sử, hệ số chia $div[15:0] = 4$ thì clk_out có dạng sóng được mô tả như sau:



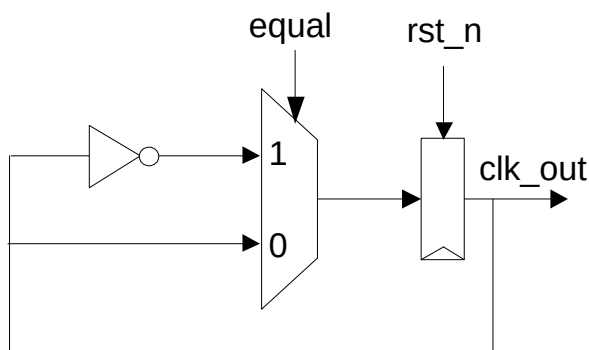
Hình 5-19. Dạng sóng của mạch chia tần số

Theo dạng sóng mô tả ở trên, *counter* sẽ đếm lên cho đến khi bằng với giá trị so sánh ($div = 4$) sẽ quay về 0 và tiếp tục đếm trở lại.



Hình 5-20. Sơ đồ mạch khối counter

Khi giá trị *counter* bằng với giá trị so sánh ($div = 4$) thì clk_out sẽ đổi trạng thái từ 1 về 0 hoặc từ 0 về 1.



Hình 5-21. Sơ đồ mạch khối tạo clk_out

5.5.3 Mô tả RTL code

```

module divisor(clk_in, rst_n, div, clk_out);
input      clk_in;
input      rst_n;
input [15:0] div;
output     clk_out;
reg        clk_out;
reg [15:0] counter;
wire       equal;

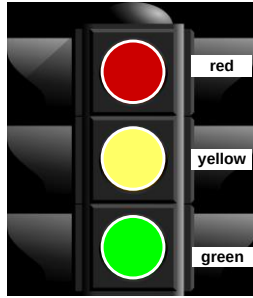
assign equal = (counter[15:0]==div[15:0])?1'b1:1'b0;
// counter
always @ (posedge clk_in) begin
    if (!rst_n) counter[15:0] <= 16'd0;
    else if (equal) counter[15:0] <= 16'd0;
    else counter[15:0] <= counter[15:0] + 1'b1;
end
//generate clk_out
always @ (posedge clk_in) begin
    if (!rst_n) clk_out <= 1'b0;
    else if (equal) clk_out <= ~clk_out;
end
endmodule

```

5.6 Mạch điều khiển đèn giao thông

5.6.1 Đặt vấn đề

Đèn giao thông là một thiết bị được dùng để điều khiển giao thông ở những giao lộ có lượng phương tiện lưu thông lớn (thường là ngã ba, ngã tư đông xe qua lại). Đèn giao thông phổ biến được dùng thường có ba trạng thái: xanh, vàng, đỏ.



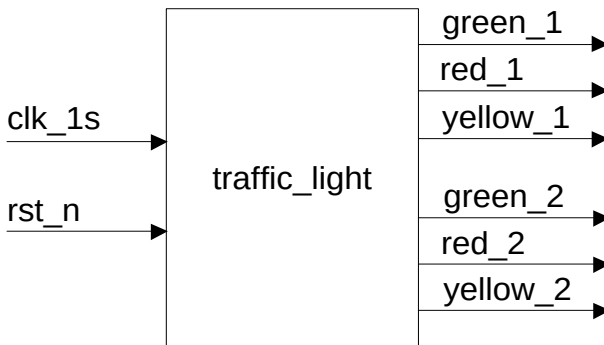
Hình 5-22. Các trạng thái đèn giao thông

Tại ngã tư, đèn giao thông đặt trên hai tuyến đường có hoạt động khác nhau. Nếu đèn giao thông trên tuyến đường thứ nhất là đèn xanh thì đèn giao thông trên tuyến đường thứ hai phải là đèn đỏ. Ví dụ các trạng thái hoạt động của đèn giao thông được mô tả như sau:

Bảng 5-2. Mô tả hoạt động đèn giao thông

Trạng thái	Thời gian (giây)	Đèn giao thông 1	Đèn giao thông 2
1	40	Xanh	Đỏ
2	5	Vàng	Đỏ
3	20	Đỏ	Xanh
4	5	Đỏ	Vàng

Để thực hiện mạch điều khiển giao thông như vậy, chúng ta sử dụng clock với tần số 1 giây. Nếu chưa có clock 1s, thì có thể sử dụng mạch chia tần số để tạo ra clock 1s.

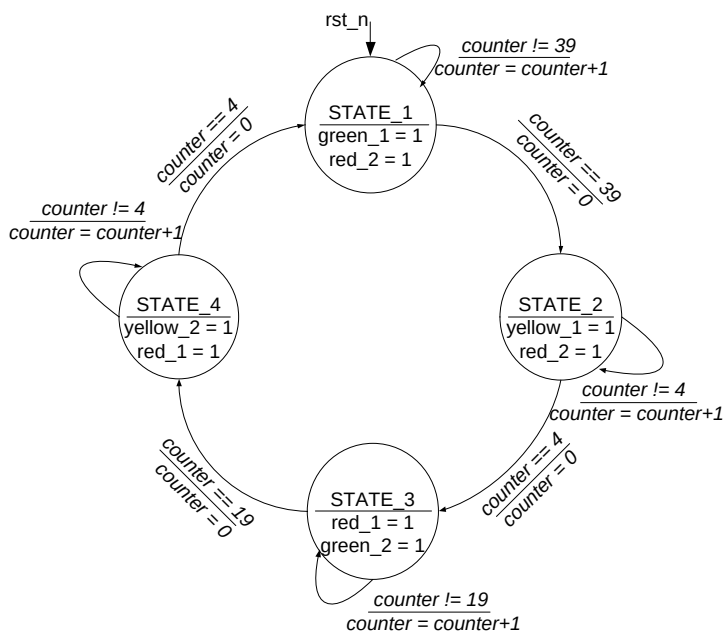


Hình 5-23. Sơ đồ tín hiệu mạch điều khiển đèn giao thông

Ngõ ra gồm sáu tín hiệu là *green_1*, *red_1*, *yellow_1*, *green_2*, *red_2*, *yellow_2* dùng để điều khiển các đèn giao thông.

5.6.2 Phân tích vấn đề

Máy trạng thái sẽ được sử dụng để thiết kế mạch điều khiển đèn giao thông. Máy trạng thái sử dụng một thanh ghi *counter* để định thời và làm điều kiện chuyển trạng thái như hình sau:



Hình 5-24. Sơ đồ máy trạng thái đèn giao thông

Máy trạng thái đèn giao thông có bốn trạng thái.

Trạng thái *STATE_1*: đây là trạng thái khi FSM bị reset. Khi *rst_n* = 1'b0 thì thanh ghi *counter* = 6'd0, tín hiệu *green_1* = 1'b1 và *red_2* = 1'b1. Khi *rst_n* = 1'b1, thanh ghi *counter* bắt đầu đếm. Khi *counter* = 39, FSM sẽ chuyển qua trạng thái *STATE_2* và thanh ghi *counter* bị xóa.

Trạng thái *STATE_2*: các tín hiệu *yellow_1* = 1'b1 và *red_2* = 1'b1. Thanh ghi *counter* đếm lên cho đến khi *counter* = 4. Khi đó, FSM sẽ chuyển qua trạng thái *STATE_3* và thanh ghi *counter* bị xóa.

Trạng thái *STATE_3*: các tín hiệu *red_1* = 1'b1 và *green_2* = 1'b1. Thanh ghi *counter* đếm lên cho đến khi *counter* = 19. Khi đó, FSM sẽ chuyển qua trạng thái *STATE_4* và thanh ghi *counter* bị xóa.

Trạng thái *STATE_4*: các tín hiệu *red_1* = 1'b1 và *yellow_2* = 1'b1. Thanh ghi *counter* đếm lên cho đến khi *counter* = 4. Khi đó, FSM sẽ chuyển qua trạng thái *STATE_1* và thanh ghi *counter* bị xóa.

5.6.3 Mô tả RTL code

```

module traffic_light(clk_1s, rst_n, green_1, red_1,
                    yellow_1, green_2, red_2, yellow_2);
parameter STATE_1    = 3'd0;
parameter STATE_2    = 3'd1;
parameter STATE_3    = 3'd2;
parameter STATE_4    = 3'd3;
input      clk_1s;
input      rst_n;
output     red_1;
output     green_1;
output     yellow_1;
output     red_2;
output     green_2;
output     yellow_2;
reg        red_1;
reg        green_1;
reg        yellow_1;
reg        red_2;
reg        green_2;
reg        yellow_2;
reg [5:0] counter;
reg [1:0] state, next_state;
// generate next_state
always @(*)begin
    green_1 = 1'b0;
    yellow_1 = 1'b0;
    red_1 = 1'b0;
    green_2 = 1'b0;
    yellow_2 = 1'b0;
    red_2 = 1'b0;
    next_state[1:0] = STATE_1;
    case (state[1:0])
        STATE_1: begin
            green_1 = 1'b1; // green_1 on
            red_2 = 1'b1;   // red_2 on
            if(counter==6'd39) next_state[1:0] = STATE_2;
            else next_state[1:0] = STATE_1;
        end
        STATE_2: begin
            yellow_1 = 1'b1; // yellow_1 on
            red_2 = 1'b1;    // red_2 on
            if(counter==6'd4) next_state[1:0] = STATE_3;
            else next_state[1:0] = STATE_2;
    endcase
end

```

```

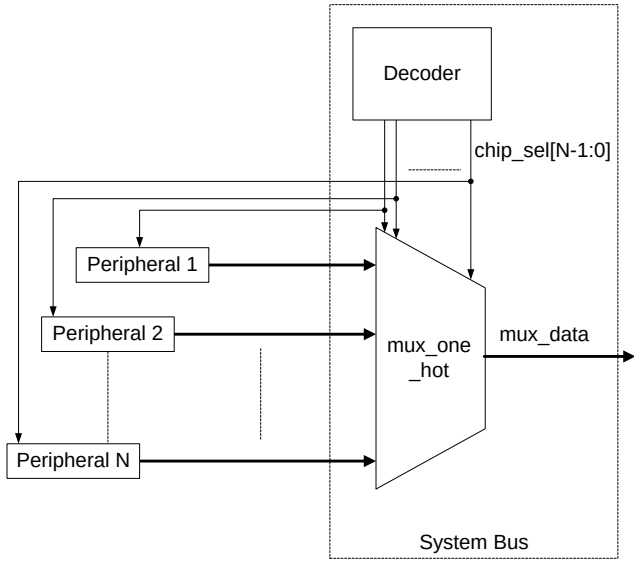
end
STATE_3: begin
    red_1 = 1'b1;    // red_1 on
    green_2 = 1'b1;  // green_2 on
    if(counter == 6'd19) next_state[1:0] = STATE_4;
    else next_state[1:0] = STATE_3;
end
STATE_4: begin
    red_1 = 1'b1;    // red_1 on
    yellow_2 = 1'b1; // yellow_2 on
    red_2 = 1'b0;
    if(counter == 6'd4) next_state[1:0]=STATE_1;
    else next_state[1:0] = STATE_4;
end
default: begin
    green_1 = 1'b1;  // green_1 on
    red_2 = 1'b1;    // red_2 on
    next_state[1:0] = STATE_1;
end
endcase
end
// current state
always @(posedge clk_1s) begin
    if(!rst_n) state[1:0] <= STATE_1;
    else state[1:0] <= next_state[1:0];
end
// control lights and counter
always @(posedge clk_1s) begin
    if(!rst_n) counter[5:0] <= 6'd0;
    else case(state)
        STATE_1: begin
            if(counter[5:0] == 6'd39) counter[5:0] <= 6'd0;
            else counter[5:0] <= counter[5:0] + 1'b1;
        end
        STATE_2: begin
            if(counter[5:0] == 6'd4) counter[5:0] <= 6'd0;
            else counter[5:0] <= counter[5:0] + 1'b1;
        end
        STATE_3: begin
            if(counter[5:0] == 6'd19) counter[5:0] <= 6'd0;
            else counter[5:0] <= counter[5:0] + 1'b1;
        end
        STATE_4: begin
            if(counter[5:0] == 6'd4) counter[5:0] <= 6'd0;
            else counter[5:0] <= counter[5:0] + 1'b1;
        end
        default: counter[5:0] <= 6'd0;
    endcase
end
endmodule

```

5.7 MUX one-hot tổng quát

5.7.1 Đặt vấn đề

Một bus hệ thống có số lượng ngoại vi kết nối đến bus được thay đổi tùy vào từng hệ thống. Bus có nhiều nguồn dữ liệu đầu vào, mỗi nguồn dữ liệu được lựa chọn tương ứng bằng một tín hiệu báo hợp lệ. Các tín hiệu báo hợp lệ có dạng là one-hot, nghĩa là chỉ có một tín hiệu tích cực tại một thời điểm xác định. Một tín hiệu hợp lệ tích cực thì dữ liệu của nguồn tương ứng sẽ đưa đến ngõ ra của MUX.



Hình 5-25. Minh họa một bus hệ thống sử dụng MUX one-hot

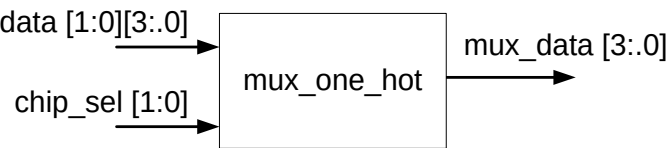
Vì số lượng nguồn đầu vào có thể thay đổi nên không thể dùng phát biểu **if** và phát biểu **case** để mô tả. Do đó, một cách mô tả RTL code tổng quát cho trường hợp này là cần thiết.

5.7.2 Phân tích vấn đề

Mạch thiết kế sẽ được phân tích trong trường hợp cụ thể có hai nguồn đầu vào bốn bit đưa cho khối MUX. Sau khi phân tích trường hợp cụ thể, mạch sẽ được phân tích với trường hợp các thông số thiết kế tổng quát.

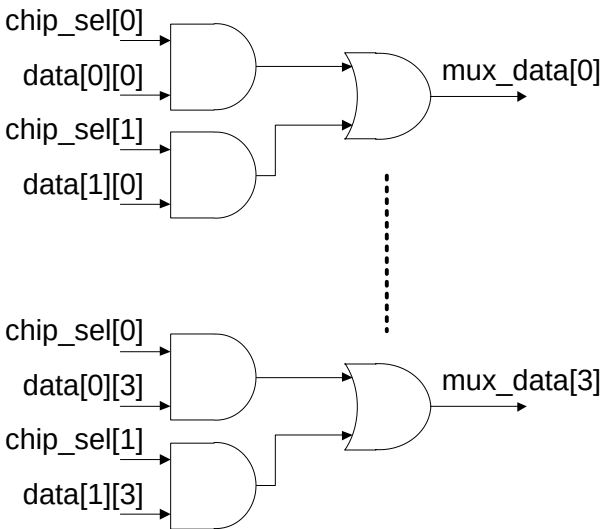
Ví dụ 5-1

```
wire [3:0] data [1:0]; // 2 data blocks of 2 input
                        // sources each of them is 4 bits
wire [1:0] chip_sel;   // the valid bits for 2 blocks
reg [3:0] mux_data;    // the output data of MUX
```



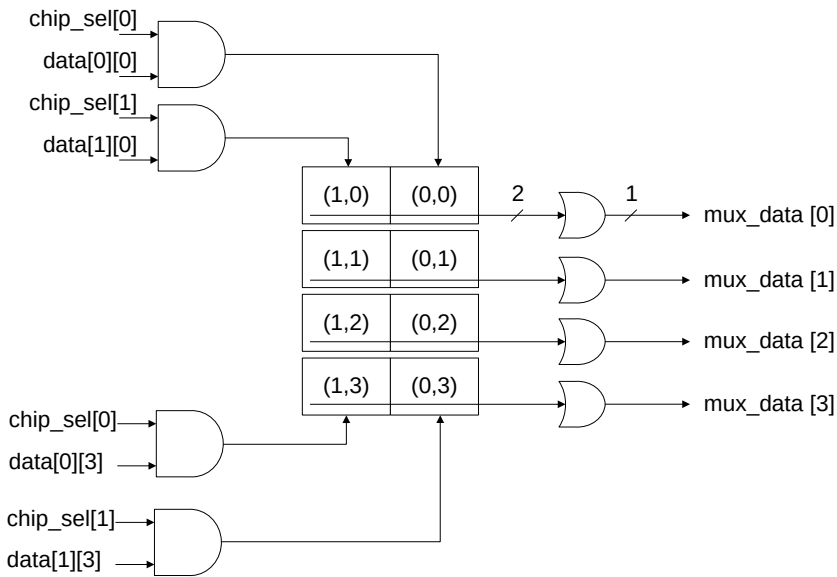
Hình 5-26. Sơ đồ tín hiệu của khối MUX có hai nguồn đầu vào bốn bit

chip_sel [1:0]	data[0][3:0]	data[1][3:0]	mux_data[3:0]
2'b00	4'h1	4'h2	4'h0
2'b01	4'h1	4'h2	4'h1
2'b10	4'h1	4'h2	4'h2
2'b11			không xác định



Hình 5-27. Mạch nguyên lý của ví dụ 5-1

Về hoạt động cơ bản, khi tín hiệu chọn *chip_sel* của nguồn tích cực thì dữ liệu đầu vào của nguồn tương ứng sẽ đưa đến ngõ ra của MUX.

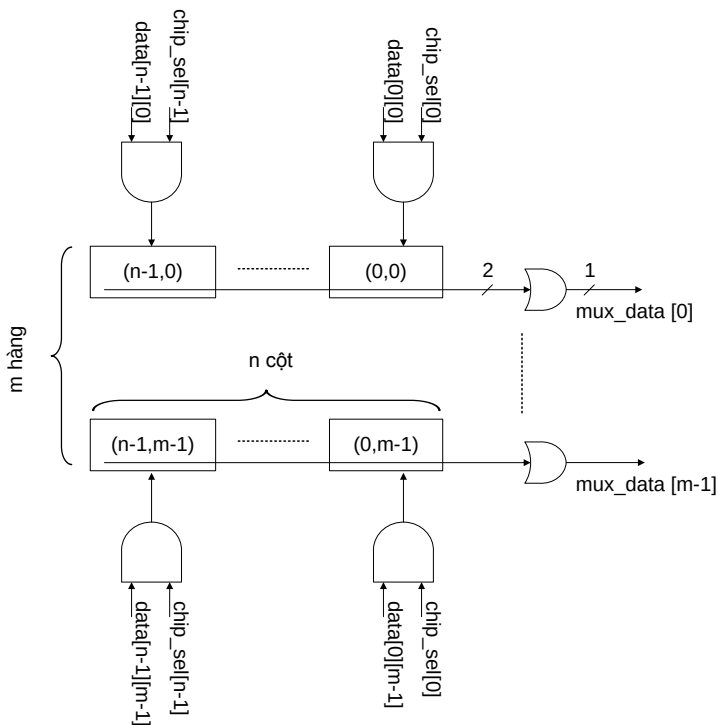


Hình 5-28. Ma trận AND cho hai nguồn, bốn bit dữ liệu

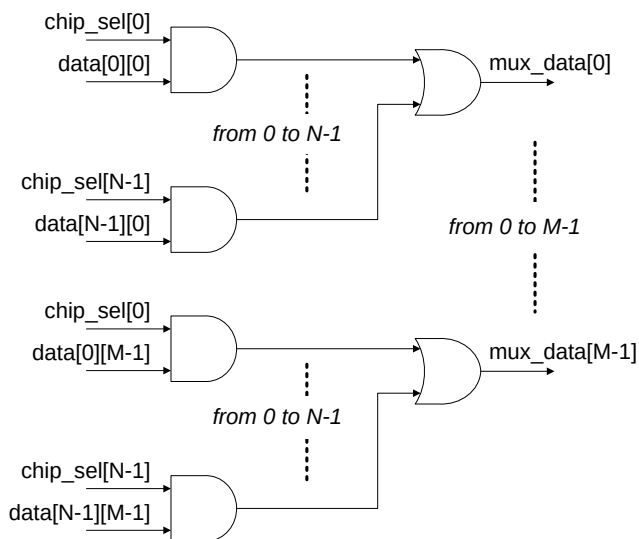
Để thực hiện cho trường hợp tổng quát, ta phân tích theo hướng sau:

- Trước tiên là lấy tín hiệu *chip_sel* của từng nguồn AND với từng bit dữ liệu của nó và xếp vào cột thứ tự tương ứng. Ví dụ như *chip_sel[0]* tích cực thì kết quả AND sẽ xếp vào cột 0. Mỗi cột sẽ có bốn bit tương ứng với độ rộng của bus dữ liệu.
- OR tất cả các bit của hàng thứ *i* để tạo ra dữ liệu ngõ ra bit thứ *i*, trong trường hợp này là có hai bit trên một hàng do có hai nguồn đầu vào.

Vì hai nguồn đầu vào có độ rộng dữ liệu là bốn bit nên kết quả sau khi AND là ma trận bốn hàng, hai cột. Tổng quát cho *n* nguồn đầu vào có *m* bit dữ liệu sẽ như sau:



Hình 5-29. Ma trận AND cho trường hợp tổng quát n nguồn, m bit



Hình 5-30. Mạch nguyên lý của trường hợp tổng quát N nguồn và M bit

5.7.3 Mô tả RTL code

```
module mux_one_hot(chip_sel, bus_data, mux_data);
parameter    N = 2, M = 4;

input    [N-1:0] chip_sel; //the select signals
                        //of N blocks
input    [N*M-1:0] bus_data; //all data bits
                        //of N sources
output    [M-1:0] mux_data; //the output data of MUX

wire    [M-1:0] data [N-1:0]; //N data blocks of
                        //N input sources
wire    [N-1:0] tmp [M-1:0];
// the AND-N matrix of chip_sel and data block.
// pay attention of the number of rows and columns

genvar i,j;

generate
    for (i=0; i<N; i=i+1) // divide group data
                        //of each source
    begin: gen_data_block
        assign data[i][M-1:0] = bus_data[(i+1)*M-1: i*M];
        //or data[i][M-1:0] = bus_data[i*M+:M];
    end

    for (i=0; i<N; i=i+1) // build AND matrix
    begin: gen_for_source_data
        for (j=0; j<M; j=j+1)
        begin: generate_for_block_data
            assign tmp[j][i] = chip_sel[i] & data[i][j];
        end
    end

    for (j=0; j<M; j=j+1)
    begin: gen_output_data
        assign mux_data[j] = |(tmp[j][N-1:0]);
    end
endgenerate

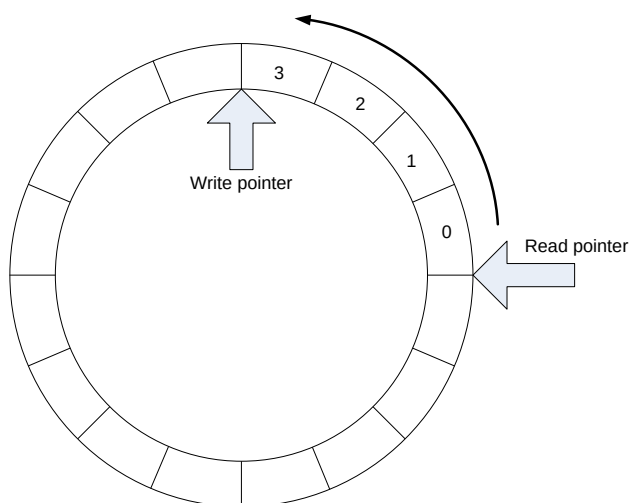
endmodule
```

5.8 FIFO đồng bộ

5.8.1 Đặt vấn đề

FIFO đồng bộ hoạt động theo một xung clock duy nhất và được sử dụng để làm bộ đệm dữ liệu theo thứ tự.

Nguyên tắc hoạt động của FIFO là thứ tự dữ liệu được đọc ra giống như thứ tự dữ liệu được ghi vào. Vị trí dữ liệu được ghi vào FIFO được xác định bằng một con trỏ ghi. Vị trí dữ liệu được đọc ra từ FIFO được xác định bằng một con trỏ đọc. Hai thông số đặc trưng của FIFO là số lượng ô nhớ và độ rộng ô nhớ. Hai thông số này sẽ cho biết dung lượng của FIFO.



Hình 5-31. Hoạt động của FIFO

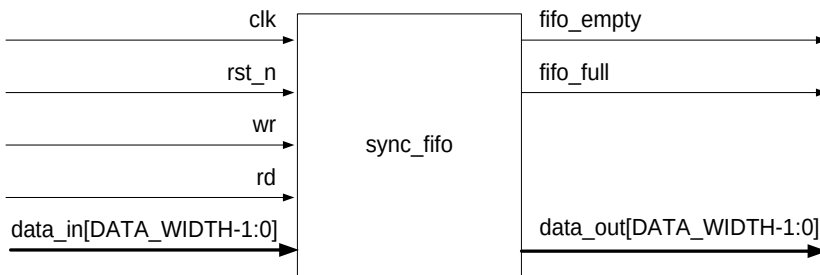
Trong một hệ thống đồng bộ, FIFO cần thiết kế có dung lượng thay đổi được trước khi tổng hợp theo các tham số được đặt tên như sau:

- DATA_WIDTH là độ rộng một ô dữ liệu.
- DATA_NUM là số ô lưu dữ liệu.
- POINTER_WIDTH là độ rộng của con trỏ đọc và con trỏ ghi. Độ rộng này có mối liên hệ với số ô dữ liệu như sau:

$$\text{DATA_NUM} = 2^{\text{POINTER_WIDTH}}$$

Các tín hiệu giao tiếp của FIFO gồm có:

- *clk*: xung clock hệ thống
- *rst_n*: tín hiệu reset bất đồng bộ tích cực mức thấp
- *wr*: tín hiệu cho phép ghi vào FIFO. Tín hiệu tích cực một chu kỳ xung clock tương ứng với một lần ghi FIFO.
- *rd*: tín hiệu cho phép đọc FIFO. Tín hiệu tích cực một chu kỳ xung clock tương ứng với một lần đọc FIFO.
- *data_in*: bus dữ liệu ghi
- *data_out*: bus dữ liệu đọc
- *fifo_empty*: tín hiệu báo FIFO rỗng. Tín hiệu này có thể được lựa chọn có hoặc không trước khi tổng hợp.
- *fifo_full*: tín hiệu báo FIFO đầy. Tín hiệu này có thể được lựa chọn có hoặc không trước khi tổng hợp.

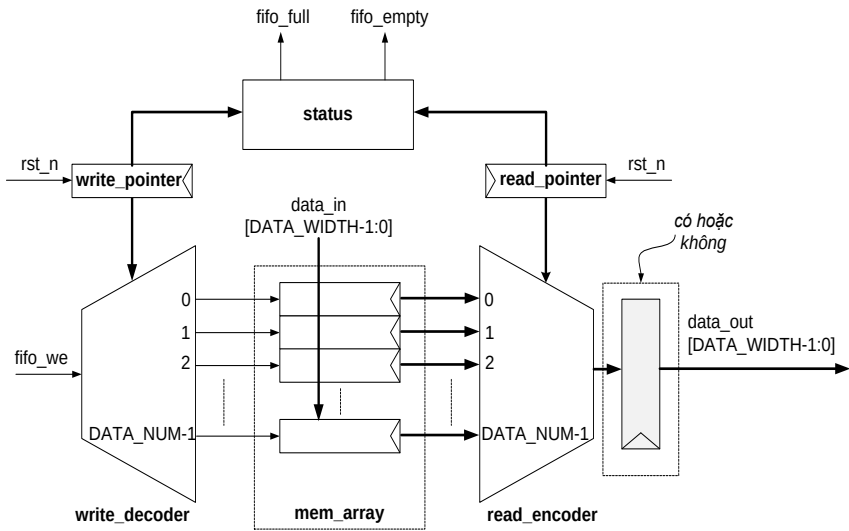


Hình 5-32. Sơ đồ tín hiệu giao tiếp của FIFO

5.8.2 Phân tích vấn đề

Một FIFO gồm các thành phần cơ bản như sau:

- *mem_array*: là mảng bộ nhớ có số lượng ô nhớ theo tham số DATA_NUM và độ rộng ô nhớ theo tham số DATA_WIDTH
- *write_pointer*: là con trỏ xác định vị trí ô nhớ ghi dữ liệu
- *read_pointer*: là con trỏ xác định vị trí ô nhớ đọc dữ liệu
- *write_decoder*: là mạch giải mã lựa chọn vị trí ghi dữ liệu
- *read_encoder*: là mạch giải mã lựa chọn vị trí đọc dữ liệu
- *status*: là mạch tạo tín hiệu báo trạng thái FIFO.

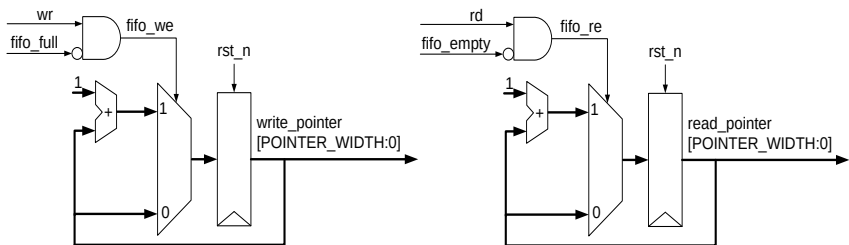


Hình 5-33. Sơ đồ nguyên lý của FIFO

Mạch giải mã ghi thực hiện giải mã dựa trên con trỏ ghi *write_pointer* để cấp tín hiệu cho phép ghi *fifo_we* đến ô nhớ tương ứng. Dữ liệu ngõ vào *data_in* đưa trực tiếp đến từng ô nhớ. Giá trị *data_in* được nạp vào ô nhớ khi tín hiệu cho phép ghi của ô nhớ đó tích cực.

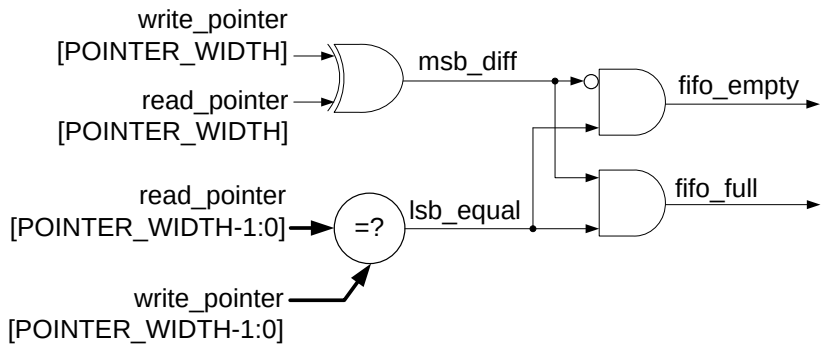
Mạch giải mã đọc thực hiện giải mã dựa trên con trỏ đọc *read_pointer* để lấy dữ liệu đọc tương ứng trong các ô nhớ của *mem_array*. Dữ liệu đọc *data_out* có thể tùy chọn được chốt bằng thanh ghi hoặc không.

Con trỏ ghi *write_pointer* đếm lên khi tín hiệu ghi *wr* tích cực và FIFO không đầy. FIFO không đầy được xác định dựa trên tín hiệu báo trạng thái *fifo_full*. Con trỏ đọc *read_pointer* đếm lên khi tín hiệu ghi *rd* tích cực và FIFO không rỗng. FIFO không rỗng được xác định dựa trên tín hiệu báo trạng thái *fifo_empty*.



Hình 5-34. Sơ đồ nguyên lý của *write_pointer* và *read_pointer*

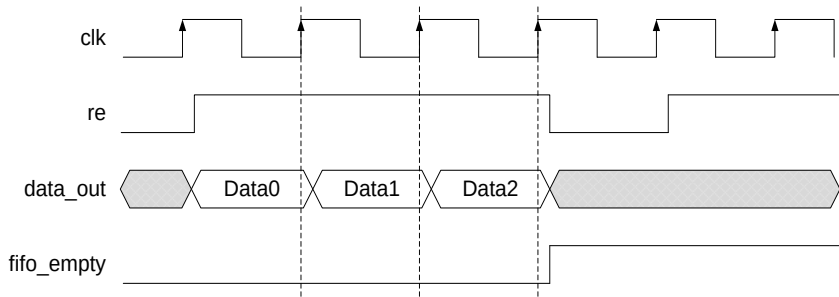
Một điểm quan trọng của thiết kế là độ rộng *write_pointer* và *read_pointer* được sử dụng nhiều hơn một bit so với yêu cầu giải mã địa chỉ *mem_array*. Mảng bộ nhớ *mem_array* chỉ yêu cầu các con trỏ có độ rộng là $[POINTER_WIDTH-1:0]$ để giải mã nhưng độ rộng được sử dụng là $[POINTER_WIDTH:0]$. Bit MSB của hai con trỏ sẽ được dùng để xác định trạng thái của FIFO.



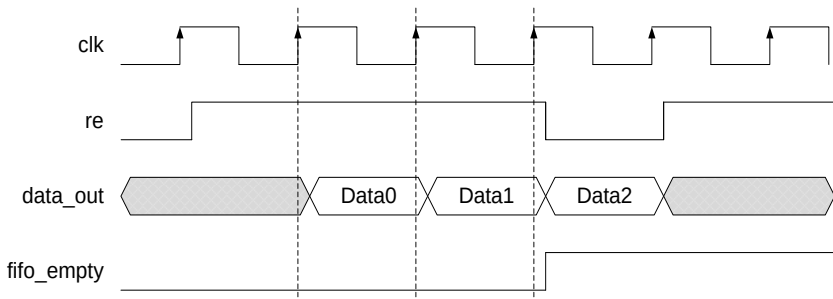
Hình 5-35. Sơ đồ nguyên lý của tín hiệu trạng thái

Vì các ngõ ra được tùy chọn tổng hợp nên define được sử dụng và đặt tên như sau:

- EMPTY_GEN: là chỉ dẫn cho phép tạo ra tín hiệu *fifo_empty*
- FULL_GEN: là chỉ dẫn cho phép tạo ra tín hiệu *fifo_full*
- OUT_REG: là chỉ dẫn cho phép tạo thanh ghi ở ngõ ra *data_out*



Hình 5-36. Dạng sóng quá trình đọc FIFO khi ngõ ra không phải là thanh ghi



Hình 5-37. Dạng sóng quá trình đọc FIFO khi ngõ ra là thanh ghi

5.8.3 Mô tả RTL code

Tất cả các thành phần chức năng của FIFO được viết chung trong một file RTL code.

```
`define EMPTY_GEN
`define FULL_GEN
`define OUT_REG
module sync_fifo (clk, rst_n, wr, rd,
    `ifdef EMPTY_GEN
        fifo_empty,
    `endif
    `ifdef FULL_GEN
        fifo_full,
    `endif
    data_in, data_out);
parameter DATA_WIDTH      = 8;
parameter POINTER_WIDTH    = 3;
parameter DATA_NUM        = 2**POINTER_WIDTH;
//inputs
input clk;
input rst_n;
input wr;
input rd;
input [DATA_WIDTH-1:0] data_in;
//outputs
output reg [DATA_WIDTH-1:0] data_out;
`ifdef EMPTY_GEN
    output fifo_empty;
`else
    wire    fifo_empty;
`endif
`ifdef FULL_GEN
    output fifo_full;
```

```

`else
    wire    fifo_full;
`endif
//internal signals
reg [POINTER_WIDTH:0] write_pointer;
reg [POINTER_WIDTH:0] read_pointer;
reg [DATA_WIDTH-1:0] mem_array [DATA_NUM-1:0];
wire    msb_diff;
wire    lsb_equal;
wire    fifo_re;
wire    fifo_we;
//write pointer
assign fifo_we = wr & (~fifo_full);
//
always @ (posedge clk or negedge rst_n) begin
    if (~rst_n) write_pointer <= {POINTER_WIDTH+1{1'b0}};
    else if (fifo_we) write_pointer <= write_pointer+1'b1;
end
//read pointer
assign fifo_re = rd & (~fifo_empty);
//
always @ (posedge clk or negedge rst_n) begin
    if (~rst_n) read_pointer <= {POINTER_WIDTH+1{1'b0}};
    else if (fifo_re) read_pointer <= read_pointer + 1'b1;
end
//memory array and write decoder
always @ (posedge clk) begin
    if (fifo_we)
        mem_array[write_pointer[POINTER_WIDTH-1:0]]
            <= data_in[DATA_WIDTH-1:0];
end
//read decoder
`ifndef OUT_REG
    always @ (posedge clk) begin
        data_out[DATA_WIDTH-1:0]
            <= mem_array[read_pointer[POINTER_WIDTH-1:0]];
    end
`else
    always @ (*) begin
        data_out[DATA_WIDTH-1:0]
            = mem_array[read_pointer[POINTER_WIDTH-1:0]];
    end
`endif
//status signal
assign msb_diff = write_pointer[POINTER_WIDTH]
                ^ read_pointer[POINTER_WIDTH];
//
assign lsb_equal = (write_pointer[POINTER_WIDTH-1:0]
                    == read_pointer[POINTER_WIDTH-1:0]);
//

```



```

assign  fifo_full = msb_diff & lsb_equal;
//
assign  fifo_empty = (~msb_diff) & lsb_equal;
endmodule

```

Việc có hay không các define sẽ cho kết quả tổng hợp khác nhau. Độ rộng dữ liệu và số ô nhớ được quy định trực tiếp bởi parameter *DATA_WIDTH* và *POINTER_WIDTH*. Với các thiết kế mà các define và parameter được sử dụng trong nhiều module khác nhau thì chúng sẽ được đặt trong các file riêng. RTL code trên sẽ được tách làm ba file như sau:

- File chứa các define *fifo_define.h*

```

`define EMPTY_GEN
`define FULL_GEN
`define OUT_REG

```

- File chứa các parameter *fifo_parameter.h*

```

parameter DATA_WIDTH      = 8;
parameter POINTER_WIDTH   = 3;
parameter DATA_NUM       = 2**POINTER_WIDTH;

```

- File chứa mô tả chức năng FIFO *sync_fifo.v*

```

`include "fifo_define.h"
module sync_fifo (...);
`include "fifo_parameter.v"
...
endmodule

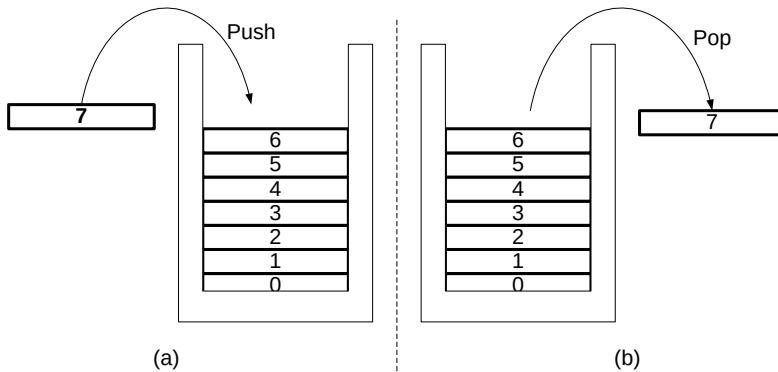
```

5.9 LIFO đồng bộ

5.9.1 Đặt vấn đề

LIFO đồng bộ hoạt động theo một xung clock duy nhất và được sử dụng để làm bộ đệm dữ liệu theo thứ tự khác với nguyên lý của FIFO.

Nguyên tắc hoạt động của LIFO là dữ liệu được ghi vào sau cùng sẽ được đọc ra trước tiên và ngược lại. Vị trí dữ liệu được ghi vào và vị trí dữ liệu được đọc ra từ LIFO được xác định tương ứng bằng một con trỏ ghi và một con trỏ đọc. Tương tự FIFO, hai thông số đặc trưng của LIFO là số lượng ô nhớ và độ rộng ô nhớ. Hai thông số này sẽ cho biết dung lượng của LIFO.



Hình 5-38. Nguyên tắc ghi (a) và đọc (b) của LIFO

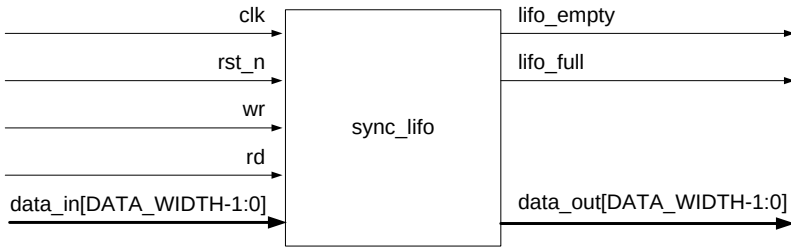
LIFO cần thiết kế được sử dụng trong một hệ thống đồng bộ. LIFO này không cho phép đọc, ghi đồng thời. Dung lượng LIFO thay đổi được trước khi tổng hợp theo các tham số được đặt tên như sau:

- *DATA_WIDTH*: là độ rộng một ô dữ liệu
- *DATA_NUM*: là số ô lưu dữ liệu
- *POINTER_WIDTH*: là độ rộng của con trỏ đọc và con trỏ ghi. Độ rộng này có mối liên hệ với số ô dữ liệu như sau:

$$DATA_NUM = 2^{POINTER_WIDTH}$$

Các tín hiệu giao tiếp của LIFO gồm có:

- *clk*: xung clock hệ thống
- *rst_n*: tín hiệu reset bất đồng bộ tích cực mức thấp
- *wr*: tín hiệu cho phép ghi vào LIFO. Tín hiệu tích cực một chu kỳ xung clock tương ứng với một lần ghi LIFO.
- *rd*: tín hiệu cho phép đọc LIFO. Tín hiệu tích cực một chu kỳ xung clock tương ứng với một lần đọc LIFO. Tín hiệu *rd* và *wr* không được phép tích cực cùng lúc.
- *data_in*: bus dữ liệu ghi
- *data_out*: bus dữ liệu đọc
- *lifo_empty*: tín hiệu báo LIFO rỗng. Tín hiệu này có thể được lựa chọn có hoặc không trước khi tổng hợp.
- *lifo_full*: tín hiệu báo LIFO đầy. Tín hiệu này có thể được lựa chọn có hoặc không trước khi tổng hợp.

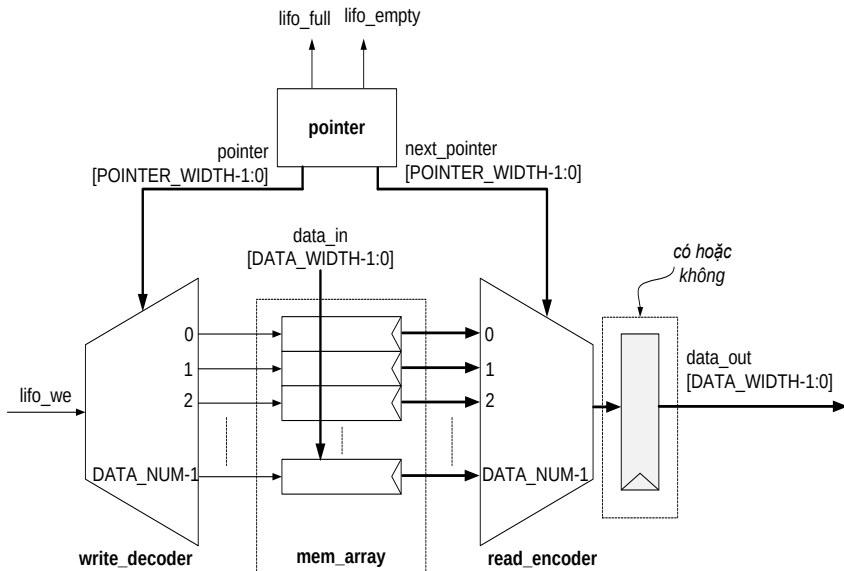


Hình 5-39. Sơ đồ tín hiệu giao tiếp của LIFO

5.9.2 Phân tích vấn đề

Một LIFO gồm các thành phần cơ bản như sau:

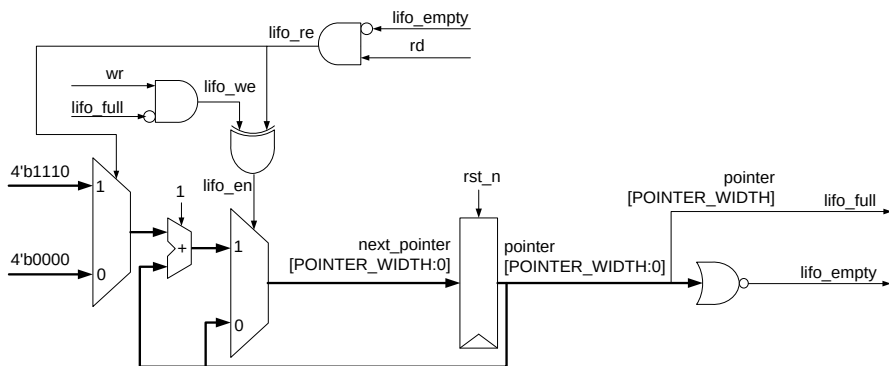
- *mem_array*: là mảng bộ nhớ có số lượng ô nhớ theo tham số *DATA_NUM* và độ rộng ô nhớ theo tham số *DATA_WIDTH*
- *pointer*: là con trỏ xác định vị trí ô nhớ ghi và đọc dữ liệu. Đồng thời, giá trị của *pointer* sẽ xác định trạng thái đầy và rỗng của LIFO
- *write_decoder*: là mạch giải mã lựa chọn vị trí ghi dữ liệu theo giá trị hiện tại của *pointer*
- *read_encoder*: là mạch giải mã lựa chọn vị trí đọc dữ liệu theo giá trị trước đó của *pointer*.



Hình 5-40. Sơ đồ nguyên lý của LIFO

Con trỏ *pointer* có độ rộng nhiều hơn một bit so với yêu cầu giải mã địa chỉ *mem_array*. Mảng bộ nhớ *mem_array* chỉ yêu cầu con trỏ có độ rộng là $[POINTER_WIDTH-1:0]$ để giải mã nhưng độ rộng được sử dụng là $[POINTER_WIDTH:0]$. Giá trị *pointer* sẽ tăng một khi tín hiệu ghi *wr* tích cực và LIFO không đầy. Giá trị *pointer* sẽ giảm một khi tín hiệu đọc *rd* tích cực và LIFO không rỗng. Chú ý, phép trừ một để giảm giá trị *pointer* khi đọc sẽ được thay thế tương đương bằng phép cộng bù hai.

LIFO đầy khi bit MSB của *pointer* bằng 1 và rỗng khi tất cả các bit của *pointer* bằng 0.



Hình 5-41. Sơ đồ nguyên lý của *pointer*

Vì LIFO có các ngõ ra được tùy chọn tổng hợp nên các define được sử dụng và đặt tên như sau:

- LIFO_EMPTY_GEN: là chỉ dẫn cho phép tạo ra tín hiệu *lifo_empty*
- LIFO_FULL_GEN: là chỉ dẫn cho phép tạo ra tín hiệu *lifo_full*
- LIFO_OUT_REG: là chỉ dẫn cho phép tạo thanh ghi ở ngõ ra *data_out*.

5.9.3 Mô tả RTL code

```
`define LIFO_EMPTY_GEN
`define LIFO_FULL_GEN
`define LIFO_OUT_REG
module sync_lifo (clk, rst_n, wr, rd,
                  `ifdef LIFO_EMPTY_GEN
                      lifo_empty,
```

```

        `endif
        `ifdef LIFO_FULL_GEN
            lifo_full,
        `endif
        data_in, data_out);
parameter DATA_WIDTH      = 8;
parameter POINTER_WIDTH    = 3;
parameter DATA_NUM        = 2**POINTER_WIDTH;
//inputs
input clk;
input rst_n;
input wr;
input rd;
input [DATA_WIDTH-1:0] data_in;
//outputs
output reg [DATA_WIDTH-1:0] data_out;
`ifdef LIFO_EMPTY_GEN
    output lifo_empty;
`else
    wire lifo_empty;
`endif
`ifdef LIFO_FULL_GEN
    output lifo_full;
`else
    wire lifo_full;
`endif
//Internal signals
reg [POINTER_WIDTH:0] pointer;
wire [POINTER_WIDTH:0] next_pointer, add_value;
reg [DATA_WIDTH-1:0] mem_array[DATA_NUM-1:0];
wire lifo_re, lifo_we, lifo_en;
//pointer
assign lifo_we = wr & ~lifo_full;
//
assign lifo_re = rd & ~lifo_empty;
//
assign lifo_en = lifo_re ^ lifo_we;
//
assign add_value[POINTER_WIDTH:0] = lifo_re?
                                     4'b11110: 4'b0000;
//
assign next_pointer[POINTER_WIDTH:0] =
    pointer[POINTER_WIDTH:0] +
    add_value[POINTER_WIDTH:0] + 1'b1;
//
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        pointer[POINTER_WIDTH:0] <=
            {(POINTER_WIDTH+1){1'b0}};
    else if (lifo_en)

```

```

        pointer[POINTER_WIDTH:0] <=
            next_pointer[POINTER_WIDTH:0];
    end
    //Status
    //
    assign lifo_full = pointer[POINTER_WIDTH];
    //
    assign lifo_empty= ~|pointer[POINTER_WIDTH:0];
    //memory array
    always @ (posedge clk) begin
        if (lifo_we)
            mem_array[pointer[POINTER_WIDTH-1:0]] <=
                data_in[DATA_WIDTH-1:0];
    end
    //
    `ifdef LIFO_OUT_REG
        always @ (posedge clk) begin
            data_out[DATA_WIDTH-1:0] <=
                mem_array[next_pointer[POINTER_WIDTH-1:0]];
        end
    `else
        always @ (*) begin
            data_out[DATA_WIDTH-1:0] =
                mem_array[next_pointer[POINTER_WIDTH-1:0]];
        end
    `endif
endmodule

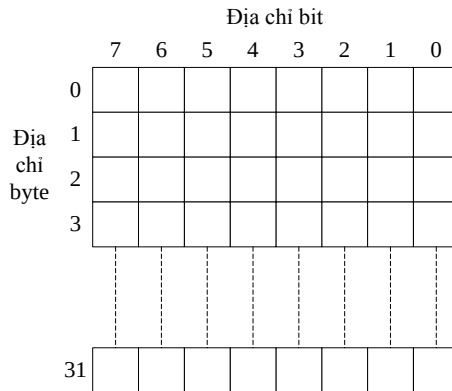
```

5.10 Truy xuất bộ nhớ NVM

5.10.1 Đặt vấn đề

Bộ nhớ NVM hay bộ nhớ không bốc hơi là loại bộ nhớ vẫn lưu dữ liệu khi không được cấp nguồn như Flash, EPROM, EEPROM, ...

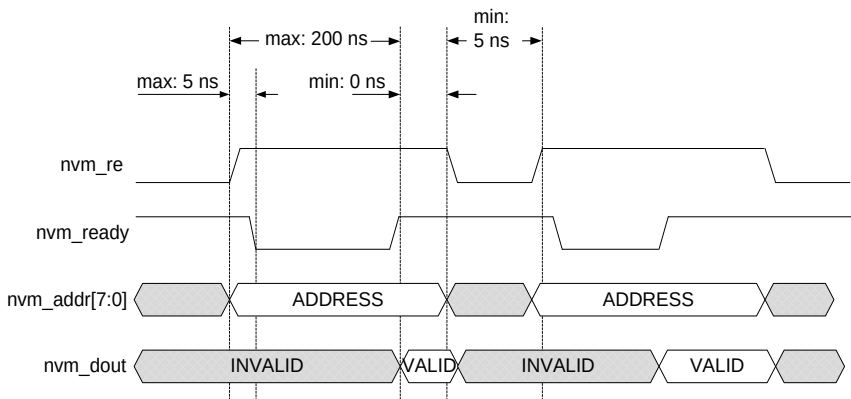
Giả sử, một hệ thống sử dụng bộ nhớ NVM có dung lượng 256 bit gồm 32 ô nhớ.



Hình 5-42. Địa chỉ bộ nhớ NVM

Bộ nhớ có giao tiếp đọc như sau:

- *nvm_re*: tín hiệu cho phép đọc tích cực mức cao
- *nvm_addr[7:0]*: địa chỉ từng bit trong bộ nhớ. Năm bit đầu định địa chỉ byte, ba bit cuối định địa chỉ bit
- *nvm_ready*: tín hiệu báo có dữ liệu hợp lệ trên đường *nvm_dout* khi bộ nhớ được đọc
- *nvm_dout*: tín hiệu trả dữ liệu đọc của bộ nhớ. Mỗi lần đọc, bộ nhớ sẽ trả giá trị một bit.

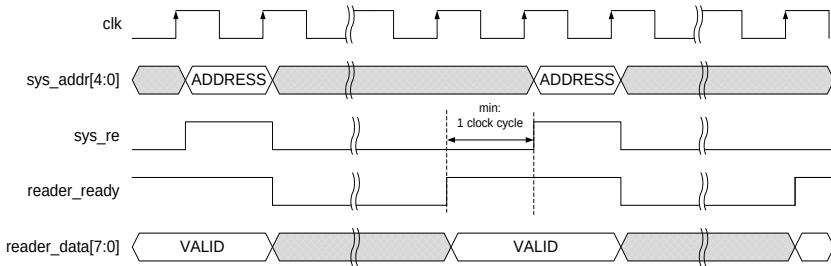


Hình 5-43. Định thời giao tiếp đọc bộ nhớ NVM

Một hệ thống đọc bộ nhớ NVM này có yêu cầu giao tiếp như sau:

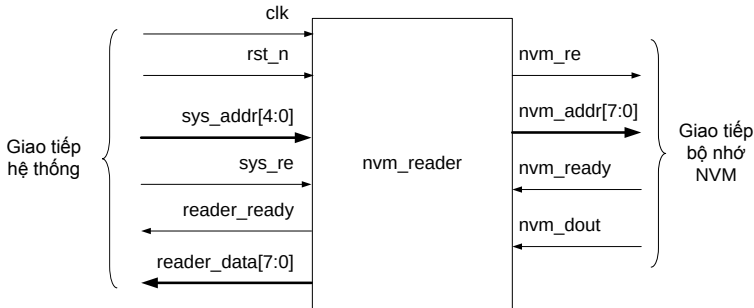
- *clk*: tần số hệ thống có giá trị 4MHz
- *rst_n*: reset bất đồng bộ tích cực mức thấp

- *sys_addr[4:0]*: địa chỉ của 1 byte dữ liệu
- *sys_re*: tín hiệu báo đọc dữ liệu của hệ thống chỉ tích cực một chu kỳ xung clock *clk*
- *reader_ready*: tín hiệu báo đã có byte dữ liệu hợp lệ ở *reader_data*
- *reader_data*: bus dữ liệu đọc của hệ thống.



Hình 5-44. Định thời giao tiếp hệ thống

Dựa trên những yêu cầu đặt ra, các tín hiệu giao tiếp của bộ truy xuất NVM được xác định.



Hình 5-45. Sơ đồ tín hiệu giao tiếp bộ truy xuất NVM

5.10.2 Phân tích vấn đề

Khi tín hiệu *sys_re* tích cực, khối *nvm_reader* có nhiệm vụ đọc từng bit dữ liệu nối tiếp từ NVM, ghép thành một byte hoàn chỉnh và trả về hệ thống qua tín hiệu *reader_data[7:0]*. Khối này có các thành phần cơ bản như sau:

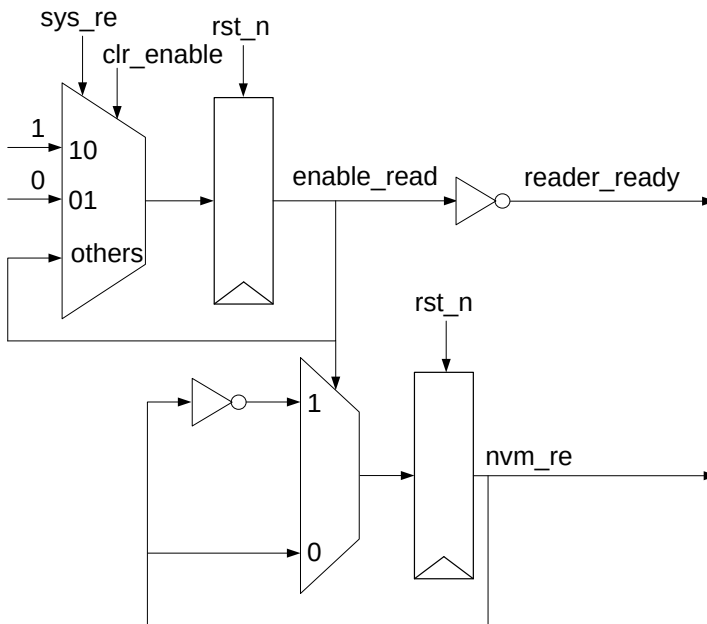
- Một thanh ghi dịch để lưu dữ liệu đọc từ NVM
- Một bộ đếm để xác định số lượng bit đã đọc được từ NVM. Vì một lần đọc tám bit nên bộ đếm này sẽ có ba bit.

- Một tín hiệu cho biết trạng thái hoạt động của *nvm_reader*
- Các mạch tạo tín hiệu ngõ ra

Bộ *nvm_reader* muốn đọc được dữ liệu từ NVM thì tín hiệu *nvm_re* phải đảm bảo định thời truy xuất của NVM. Theo định thời giao tiếp của NVM, thời gian trả dữ liệu đọc tối đa tính từ khi tín hiệu *nvm_re* tích cực là $200ns$. Bộ *nvm_reader* sử dụng tần số $4MHz$ tương ứng chu kỳ là $250ns$, với chu kỳ này thì chỉ cần tạo ra tín hiệu *nvm_re* kéo dài bằng một chu kỳ xung clock $4MHz$ là đảm bảo đọc được dữ liệu trên đường *dout* mà không cần quan tâm đến tín hiệu *nvm_eady*.

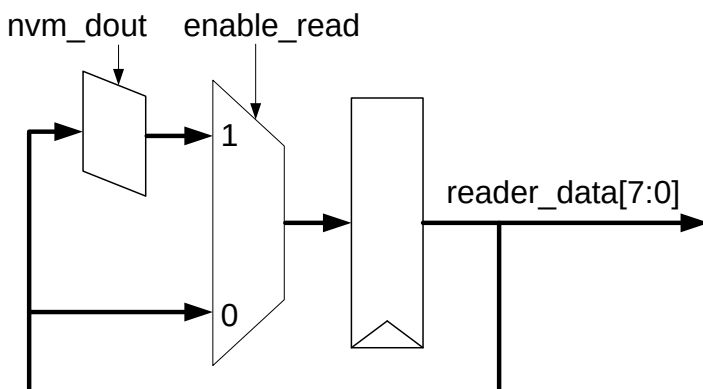
Để đọc bit tiếp theo, *nvm_re* phải kéo xuống 0 tối thiểu $5ns$ trước khi tích cực lại. Bộ *nvm_reader* hoạt động theo tần số $4MHz$ như vậy nếu kéo *nvm_re* xuống một chu kỳ xung clock này là đã đảm bảo.

Tín hiệu báo trạng thái đọc *enable_read* của *nvm_reader* được tích cực khi *sys_re* tích cực và bị xóa khi đã đọc xong tám bit. Tín hiệu *reader_ready* là đảo của tín hiệu này.



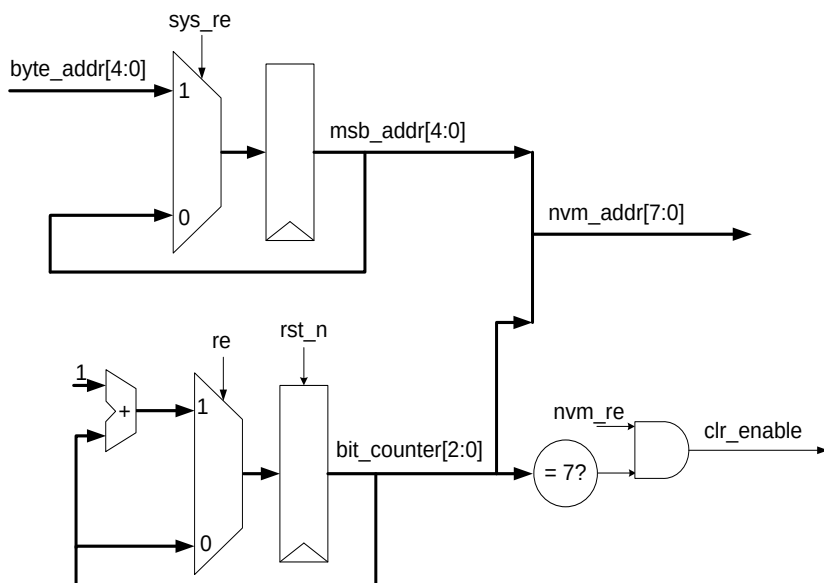
Hình 5-46. Tạo ngõ ra *reader_ready* và *nvm_re*

Thanh ghi dịch lưu giữ liệu đọc sẽ dịch từng bit dữ liệu từ ngõ vào *nvm_dout* khi tín hiệu báo trạng thái *enable_read* tích cực.



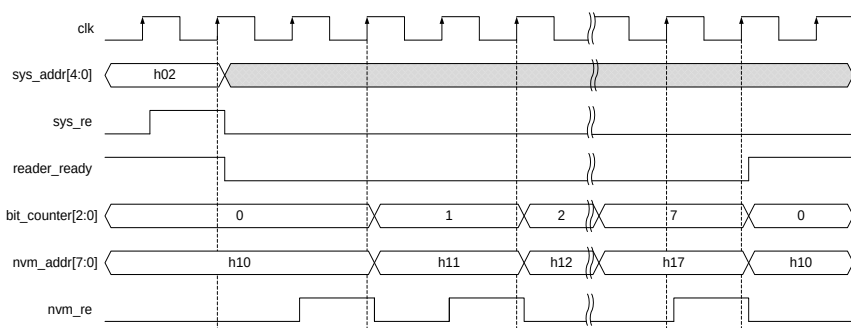
Hình 5-47. Thanh ghi dịch

Tín hiệu đọc NVM *nvm_re* được tạo ra bằng cách đảo liên tục theo mỗi chu kỳ clock *4MHz* và phải có tám mức logic cao để đọc đủ tám bit của một byte. Phần mạch cuối cùng là bộ đếm số bit đọc và mạch tạo địa chỉ truy xuất NVM.



Hình 5-48. Bộ đếm bit và mạch tạo địa chỉ

Mối liên hệ giữa giao tiếp hệ thống và giao tiếp bộ nhớ NVM của thiết kế được thể hiện trong dạng sóng sau đây.



Hình 5-49. Hoạt động truy xuất đọc của thiết kế

5.10.3 Mô tả RTL code

```

module nvm_reader (clk, rst_n, sys_addr, sys_re,
                    reader_ready, reader_data, nvm_re,
                    nvm_addr, nvm_dout);

//System interface
input          clk;
input          rst_n;
input          [4:0] sys_addr;
input          sys_re;
output         reader_ready;
output reg [7:0] reader_data;

//Memory interface
input          nvm_dout;
output reg     nvm_re;
output         [7:0] nvm_addr;

//internal signals
wire clr_enable;
reg   enable_rd;
reg [2:0] bit_counter;
reg [4:0] msb_addr;

//Shift register
always @ (posedge clk) begin
    if (nvm_re)
        reader_data[7:0] <= {nvm_dout, reader_data[7:1]};
end

//Read status bit
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) enable_rd <= 1'b0;
    else begin
        case ({sys_re, clr_enable})
            2'b10: enable_rd <= 1'b1;
            2'b01: enable_rd <= 1'b0;

```

```

        endcase
    end
end
//
assign reader_ready = ~enable_rd;
//NVM read enable
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) nvm_re <= 1'b0;
    else if (enable_rd) nvm_re <= ~nvm_re;
end
//Bit counter
always @ (posedge clk, negedge rst_n) begin
    if (~rst_n) bit_counter[2:0] <= 3'd0;
    else if (nvm_re)
        bit_counter[2:0] <= bit_counter[2:0] + 1'b1;
end
//
assign clr_enable = (&bit_counter) & nvm_re;
//memory address
always @ (posedge clk) begin
    if (sys_re) msb_addr[4:0] <= sys_addr[4:0];
end
//
assign nvm_addr[7:0] = {msb_addr[4:0], bit_counter[2:0]};
endmodule

```

Như đã phân tích, module *nvm_reader* sẽ không sử dụng tín hiệu *nvm_ready*.

Tài liệu tham khảo

1. *IEEE Standard Verilog® Hardware Description Language, IEEE Std 1364-2001*, 2001.
2. J. Bhasker, *Verilog HDL Synthesis – A Practical Primer*, Start Galaxy Publishing, 1998.
3. Douglas J. Smith, *HDL Chip Design: A Practical Guide for Designing, Synthesizing and Simulating ASICs and FPGAs Using VHDL or Verilog*, Doone Publications, 1998.
4. Samir Palnitkar, *Verilog HDL A guide to Digital Design and Synthesis*, SunSoft Press, 1996.
5. Donald E. Thomas, Philip R. Moorby, *The Verilog Hardware Description Language*, Kluwer Academic Publishers, 2001.
6. Clifford E. Cummings, Don Mills, *Synchronous Resets? Asynchronous Resets?*, Sunburst Design Inc, 2002.
7. Clifford E. Cummings, *New Verilog-2001 Techniques for Creating Parameterized Models*, Sunburst Design Inc, 2001.
8. Clifford E. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill*, Sunburst Design Inc, 2000.
9. Clifford E. Cummings, *State Machine Coding Styles for Synthesis*, Sunburst Design Inc, 1998.
10. *RTL Design Style Guide for Verilog HDL*, Semiconductor Technology Academic Research Center.